

Planning Under Uncertainty with Techniques from Operations Research

A thesis submitted for the degree of
Doctor of Philosophy of The Australian National University

Submitted May 2025

By:

Johannes Schmalz

Supervisor:

Dr. Felipe W. Trevizan



**Australian
National
University**

School of Computing
College of Systems and Society
The Australian National University

© Copyright by Johannes Schmalz 2025
All Right Reserved

Declaration:

I declare that this thesis is my own original work and has not been submitted for the award of any other degree. Except where otherwise indicated through proper reference or acknowledgement, the content presented herein is entirely my own.

Parts of this work have been published:

- Chap. 4: [Schmalz and Trevizan \(2024b, 2026\)](#).
- Chap. 7: [Schmalz and Trevizan \(2025\)](#).
- Chap. 8: [Schmalz and Trevizan \(2024c\)](#).

Johannes Schmalz, 2025 (revised 2026)

Acknowledgements

Thanks Felipe, for your support, and your insights which have shaped my understanding of the field, and given me an appreciation of it. Thanks to my peers and colleagues, especially at ANU, for interesting conversations that reminded that I enjoy my research, which I sometimes forgot after countless hours of fixing bugs and editing texts. Thanks to my friends and family who reminded me to enjoy things outside my work, especially after countless hours of fixing bugs and editing texts. Last but not least, thanks Alma for tolerating me throughout.

Abstract

Planning under uncertainty is the task of making decisions to achieve a goal, subject to some element of uncertainty. In particular, this thesis considers Stochastic Shortest Path problems (SSPs), where an agent must move between states using actions with a predefined probability distribution of next states. SSPs are a well-established problem that can be used to model real-world problems, e.g., penetration testing in cybersecurity, ecological management, and power management.

The planning community specialises in this type of problem, and has produced the state-of-the-art algorithms for solving them. Orthogonally, Operations Research (OR) is a broader, more established discipline whose scope also includes planning under uncertainty. Thus, there is an intersection of these fields at planning under uncertainty, and at this intersection there are deep links between the formulations, techniques, and fundamental algorithms of planning and OR. This connection bridges the two disciplines, allowing the two communities to share results and insights.

This thesis focuses on SSPs and some of its generalisations under the lens of OR, with the aim of strengthening the connections between the two disciplines, and improving state-of-the-art planning algorithms with techniques from OR. With this approach, this thesis presents four contributions to the study of SSPs, three of which result in novel algorithms that achieve state-of-the-art performance.

The first contribution strengthens the connection between guided search and constraint generation and building on this, defines CG-iLAO*, the first algorithm that is able to use heuristics to avoid actions as well as states, resulting in state-of-the-art performance. The second contribution is a new way to solve SSPs via a sequence of deterministic planning problems, which gives a better understanding of SSPs and opens the door to solving more difficult variants of SSPs. The last two contributions relate to Constrained SSPs, a generalisation of SSPs with additional constraints over secondary costs. These apply Lagrangian relaxation and constraint generation, respectively, producing state-of-the-art algorithms for their contexts.

Table of Contents

1	Introduction	1
1.1	Research Statement	1
1.2	Thesis Structure and Contributions	2
1.3	Notation	3
2	Related Work	5
2.1	Models	5
2.2	OR Techniques in Probabilistic Planning	7
I	Stochastic Shortest Path Problems	11
3	Background	13
3.1	The Problem and its Solutions	13
3.2	Bellman Backups and Value Iteration	16
3.3	Heuristic Search	17
3.4	Linear Programs	19
3.5	Constraint and Column Generation	21
3.6	Determinisation	23
4	CG-iLAO*: Solving SSPs with Constraint Generation	25
4.1	Motivation	25
4.2	Background: iLAO*	27
4.3	iLAO* under the lens of OR	29
4.4	CG-iLAO*	31
4.5	Properties of CG-iLAO* and Proof of Correctness	34
4.6	Different Expansion Methods for CG-iLAO*	39
4.7	Related Work: Action Elimination	42
4.8	Related Work: Partial Expansion A*	43
4.9	Experiments	46
4.10	Summary	66
5	CoGNe: Solving SSPs via Plans and Cycles	67
5.1	Background: Trajectories, Plans, and Cycles	67
5.2	Expressing Policies as Plans and Cycles	68
5.3	CoGNe	74
5.4	Evaluating CoGNe	80
5.5	Related Work: Solving SSPs via Determinisation	85
5.6	Summary	86
II	Constrained Stochastic Shortest Path Problems	87
6	Background	89
6.1	The Problem and its Solutions	89

6.2	Deterministic and Stochastic Policies	90
6.3	Algorithms	93
7	CARL: Solving CSSPs with Scalarisation	103
7.1	Motivation	103
7.2	Constrained Value Function LP	104
7.3	CARL	105
7.4	Correctness of CARL	111
7.5	Related Work: LAny and Multi-Objective Planning	112
7.6	Empirical Evaluation	113
7.7	Summary	118
8	I²-dual-det: Finding Deterministic Policies for CSSPs	119
8.1	Background: Finding Deterministic Policies with MIPs	119
8.2	Technical Gap in MIPs: Selecting M	120
8.3	I ² -dual-det	124
8.4	How Interesting Are a Problem's Constraints?	126
8.5	Evaluation of i ² -dual-det	128
8.6	Summary	134
9	Conclusion	135
9.1	Summary	135
9.2	New High-level Insights	135
9.3	Future Work	136

Introduction

1.1 Research Statement

This thesis investigates planning under uncertainty using results from Operations Research (OR). The motivation for this research comes from the well-established connections between planning and OR, as well as the valuable results gained from previous efforts to bridge the two disciplines. This work extends these existing connections, produces new insights into planning under uncertainty, and uses these to develop new planning algorithms. A brief overview of planning and OR in the context of planning under uncertainty is provided, motivating how deep the connections are between these two fields. Finally, the research statement of this thesis is precisely formulated.

Planning is the study of problems which model an agent that must traverse states with actions that move it through the state space. This is a general problem that is ubiquitous throughout theory and practice, and has been used to encode many real-world problems, e.g., finding the shortest route on a map, coordinating warehouse robots to fulfil orders in an efficient and collaborative way (Koenig et al., 2023), and producing fuel-efficient flight routes that take into account bad weather and complex aviation restrictions (Geißer et al., 2020). In the classical planning problem, actions move the agent between states in a deterministic way. We focus on probabilistic planning: when the agent applies an action, it moves into its next state according to a probability distribution that is known a priori. An important development from the planning community is that compact encodings of planning problems (e.g., PPDDL (Younes et al., 2005)) can be exploited to produce useful information about the full problem, called a heuristic. Heuristics can be used to guide search, and in practice this approach can improve the speed and scalability of algorithms by orders of magnitude. This technique was pioneered by A* (Hart, Nilsson, and Raphael, 1968), and has led to state-of-the-art algorithms for probabilistic planning, e.g., iLAO* (Hansen and Zilberstein, 2001) and LRTDP (Bonet and Geffner, 2003b).

Operations Research (OR) is an older and broader area of research than planning, and one of its facets considers planning under uncertainty. While the problems being considered are fundamentally the same, OR research does not generally assume the compact encodings used by the planning community, and therefore does not make extensive use of guided search. This has forced the community to develop other approaches for solving large problems efficiently, e.g., column generation (Desrosiers and Lübbecke, 2005). By exploiting connections between planning and OR, it is possible to combine the powerful framework of guided search with the orthogonal improvements from OR. In some cases, this does not only yield efficient algorithms but also a better understanding of both fields.

The connections between planning and OR are deep, and link the fundamental algorithms of both disciplines. As an example of this, Dijkstra’s algorithm and A*, cornerstone algorithms of planning, are equivalent to OR’s primal-dual algorithm (Papadimitriou and Steiglitz, 1998; Ye, Han, and Lin, 2010). Stochastic Shortest Path problems (SSPs) are a particular flavour of probabilistic planning, which can be solved by a system of equations called the Bellman equations. Both planning and OR have algorithms for solving SSPs via the Bellman equations, one using variants of Value Iteration (Bellman, 1957) that are guided by heuristics, e.g., iLAO* (Hansen and Zilberstein, 2001), LRTDP (Bonet and Geffner, 2003b); and the other with LPs. This suggests that any algorithm that builds on these fundamentals will inherit a connection to the other discipline, thanks to the strong coupling between the fundamental algorithms.

Previous work bridging planning and OR has led to several key results in the planning community. Recently, LP techniques have played a pivotal role for heuristics in planning, both in the deterministic setting, e.g., (Pommerening et al., 2014, 2015), and in the probabilistic one, e.g., (Trevizan, Thiébaux, and Haslum, 2017). OR techniques have also been used to develop new algorithms in the probabilistic setting, e.g., for Constrained Stochastic Shortest Path Problems (Trevizan et al., 2016; Trevizan, Thiébaux, and Haslum, 2017; Geißer et al., 2020) and Partially Observable Markov Decision Processes (Lee et al., 2018).

This motivates that planning and OR have their own techniques for planning under uncertainty, which are distinct but compatible. The deep and fundamental connection between planning and OR has been exploited by previous works to yield theoretically satisfying and practical insights into planning. It is clear that this connection has not been exhausted, and there is a lot more that can be learned about planning using the tools and theory from OR. This thesis explores these connections further, focussing on probabilistic planning:

I will study and extend connections between probabilistic planning and operations research. The aim is to deepen the understanding of probabilistic planning, produce new insights, and exploit this to develop competitive probabilistic planning algorithms.

1.2 Thesis Structure and Contributions

The thesis starts with an overview of existing work at the intersection of probabilistic planning and OR

Chap. 2. Related Work. This chapter gives an overview of existing work in the intersection of probabilistic planning and OR in order to contextualise the work of this thesis.

The remainder of the thesis is broken into two parts. Part I is dedicated to Stochastic Shortest Path problems (SSPs), and its chapters are:

Chap. 3. Background for SSPs.

Chap. 4. Contribution: this chapter introduces CG-iLAO*, a novel algorithm for solving SSPs optimally. CG-iLAO*'s novelty is that it can use heuristics to avoid unpromising actions, in contrast to existing guided-search approaches that only use heuristics to avoid unpromising states. This technique lets CG-iLAO* save computation and endows it with state-of-the-art performance. The central technique is developed using insights from constraint generation. This work was published in (Schmalz and Trevizan, 2024b), and an extended version is under review at the *Artificial Intelligence* journal.

Chap. 5. Contribution: this chapter introduces CoGNe, another algorithm for solving SSPs optimally. The key insight is that policies can be expressed by a combination of plans and cycles. CoGNe uses this idea to solve SSPs by producing the relevant plans and cycles and combining them, departing from existing algorithmic approaches for solving SSPs. This paradigm shift leaves open questions, and in its current state the algorithm is not competitive in terms of performance.

Part II addresses Constrained Stochastic Shortest Path problems (CSSPs). This is an extension of SSPs that introduces secondary costs for actions, and constraints over these secondary costs, e.g., the monetary costs associated with actions must not exceed a given budget. It has the following chapters:

Chap. 6. Background for CSSPs.

Chap. 7. Contribution: This chapter presents CARL, a novel algorithm for solving CSSPs optimally. It uses the Lagrangian dual, an OR technique, to solve CSSPs as a sequence of unconstrained SSPs, which is done efficiently with techniques from part I. CARL outperforms the state-of-the-art by orders of magnitude on many problems. This work was published in (Schmalz and Trevizan, 2025).

Chap. 8. Contribution: This chapter focuses on the problem of finding deterministic-optimal policies for CSSPs (there may be cheaper stochastic ones); it addresses a minor technical gap in existing MIP approaches, and presents the novel algorithm i²-dual-det. I²-dual-det implements MIPs for finding deterministic policies into the i²-dual framework (Trevizan, Thiébaux, and Haslum, 2017), and competes with the state-of-the-art. This work was published in (Schmalz and Trevizan, 2024c).

Then, tying the two parts together,

Chap. 9 Conclusion of the thesis. Summarises the thesis and outlines directions for future work.

1.3 Notation

- $\mathbb{R}_{>0}$ are the positive real numbers $\{x \in \mathbb{R} : x > 0\}$ and $\mathbb{R}_{\geq 0}$ are the non-negative real numbers $\{x \in \mathbb{R} : x \geq 0\}$.
- $\mathbb{N}$ is the set of natural numbers including 0, and $\mathbb{N}_{>0}$ is the set of positive natural numbers $\{n \in \mathbb{N} : n > 0\}$.
- $\llbracket \cdot \rrbracket$ are Iverson brackets. For some boolean expression ϕ , the Iverson brackets evaluate to

$$\llbracket \phi \rrbracket = \begin{cases} 1 & \text{if } \phi \\ 0 & \text{if } \neg \phi. \end{cases}$$

- We write vectors and vector functions in boldface, e.g., $\mathbf{x} \in \mathbb{R}_{\geq 0}^n$ and $\mathbf{V} : \mathbf{A} \rightarrow \mathbb{R}_{\geq 0}^n$.
- Given two real number vectors of the same dimension $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$, we write $\mathbf{u} \cdot \mathbf{v}$ to denote the standard dot product over real numbers:

$$\mathbf{u} \cdot \mathbf{v} = \mathbf{u}^T \mathbf{v} = \sum_{i=1}^n u_i \cdot v_i.$$

- Given a finite set X and a function $f : X \rightarrow \mathbb{R}$, the argmin and argmax are subsets of X that consist of elements that minimise or maximise f respectively, i.e.,

$$\begin{aligned} \operatorname{argmin}_{x \in X} f(x) &= \left\{ x \in X : f(x) = \min_{x' \in X} f(x') \right\} \\ \operatorname{argmax}_{x \in X} f(x) &= \left\{ x \in X : f(x) = \max_{x' \in X} f(x') \right\}. \end{aligned}$$

We emphasise that argmin and argmax are sets, and not individual representative elements.

Related Work

2.1 Models

We start with a high-level explanation of the models we will use throughout the thesis, and position them with respect to other models in probabilistic planning. Their formal definitions are presented in the background chapters [chap. 3](#) and [chap. 6](#), respectively.

Stochastic Shortest Path problems (SSPs) ([Bertsekas and Tsitsiklis, 1991](#)). This thesis focuses on SSPs and some of its generalisations. SSPs model an agent that must move through states to reach a specified goal state. It can move through the state space by applying actions, where applying an action incurs a cost and leads to a successor state according to a probability distribution that is known a priori. Solutions for SSPs are functions that map states onto which action to apply, and they are called policies. We consider the standard optimisation objective for SSPs, where a policy is optimal if it reaches the goal with probability 1 and minimises the expected cost ([Bertsekas, 1995](#)), but other objectives are also possible, e.g., maximise the probability of success (MAX-PROB) ([Kolobov et al., 2011](#)), or to first maximise the probability of reaching goals and then minimise the expected cost (S³P) ([Teichteil-Königsbuch, 2012](#)) and (MCMP) ([Trevizan, Teichteil-Königsbuch, and Thiébaux, 2017](#)). The SSP originates from OR ([Bellman, 1957](#)), is well-established in the optimal control community ([Bertsekas, 1995](#)), and has been used extensively by the planning community, e.g., ([Hansen and Zilberstein, 2001](#); [Bonet and Geffner, 2003b](#); [Trevizan et al., 2016](#); [Klöbner and Hoffmann, 2021](#)). SSPs have also been framed in terms of polyhedral combinatorics ([Guillot and Stauffer, 2020](#)). For solving SSPs, this thesis considers Value Iteration (VI) ([Bellman, 1957](#)) as the fundamental algorithm. VI works by solving the *Bellman equations* to produce a unique value function that maps states to their expected cost to reach the goal; this value function induces optimal policies for the SSP. Building on VI, we are interested in optimal guided-search algorithms such as iLAO* ([Hansen and Zilberstein, 2001](#)) and LRTDP ([Bonet and Geffner, 2003b](#)), which use a heuristic function to estimate states’ values to focus on promising states, and avoid unpromising ones. There are also algorithms based on sampling such as Monte Carlo Tree Search, e.g., PROST ([Keller and Eyerich, 2012](#)), but these are generally outperformed by guided search in the problems we consider. Policy Iteration (PI) ([Howard, 1960](#)) is an alternative to VI, but it is not as useful in the context of guided search for SSPs.

Constrained SSPs (CSSPs) ([Altman, 1995](#); [Trevizan et al., 2016](#)). This thesis also focuses on CSSPs. CSSPs extend SSPs by replacing the scalar cost function with a vector cost function, where the first term is called the primary cost, and other terms are called secondary costs. CSSPs also have an upper bound over the secondary-costs, which defines secondary-cost constraints with the semantic that the expected secondary costs must not exceed the upper bound. Thus, the agent’s task is still to reach the goal with probability 1, but its expected secondary costs must be lower than the relevant constraint’s upper bound, and under these restrictions the agent must minimise the expected primary cost.

Now, we present other probabilistic planning models. They are related to SSPs and CSSPs, but they will not be used explicitly in this thesis.

Markov Decision Processes (MDPs) (Puterman, 1990). MDPs are the main alternative to SSPs. MDPs are very similar to SSPs, and most algorithms that can be used to solve one can also be used to solve the other with minimal modification, e.g., the algorithms VI, LRTDP, iLAO*, and PI can all be applied to MDPs as well as SSPs. For this reason, we use SSP and MDP interchangeably in this section, and throughout the thesis we may refer to an SSP algorithm that was originally developed for MDPs. There are, however, a few differences between the problems. The first difference is superficial: MDPs maximise rewards rather than minimising cost. These are clearly equivalent and only affect terminology. The second difference is important: MDPs do not have goal states. For now, we focus on infinite-horizon MDPs, where the agent moves through the state space with no horizon, *ad infinitum*. Such MDPs require a discount factor $\gamma \in [0, 1)$, with the semantic that immediate rewards are worth more than rewards in the future. To represent this, a reward that is t steps into the future is penalised by a multiplicative discount of γ^t . Thus, MDPs maximise the discounted reward over expectation, assuming that the agent moves through the MDP forever with no termination condition. It turns out that the goals and lack of discount factors make SSPs more general than MDPs, and MDPs can be transformed into equivalent SSPs (Bertsekas, 2019). This connection is another motivation for why we focus on SSPs, and do not make a distinction between SSPs and MDPs (except in chap. 8 where the difference is important). Constrained MDPs (Altman, 1995) extend MDPs in the same way that CSSPs extend SSPs; these two constrained problems inherit the similarities from SSPs and MDPs, and we will not make distinctions (again, except in chap. 8 where the difference is important). An alternative to the infinite-horizon MDPs we have discussed are finite-horizon MDPs where the agent must accumulate maximal reward over expectation (with or without a discount), but its execution and reward is stopped after finitely many steps. Finite-horizon MDPs can be converted into infinite-horizon ones (Geffner and Bonet, 2013).

Partially Observable MDPs (POMDPs) (Kaelbling, Littman, and Cassandra, 1998). With MDPs, the agent always knows which state it is in. This knowledge is lost in the partial observability setting of POMDPs, and instead of being given the agent’s state after every step, the agent only receives observation tokens, which it can use to infer what state it might be in. Thus, an intelligent agent must maintain a probability distribution over the states, representing the probability with which the agent is at the relevant state, given the information it has seen. This probability distribution is called an agent’s belief. Tracking belief makes POMDPs significantly more difficult to solve than MDPs. To relate the two, a POMDP can be transformed into an equivalent MDP, called a belief MDP (Kaelbling, Littman, and Cassandra, 1998), where the belief MDP has a state for each POMDP belief, and its transitions depend on the observations made in the POMDP. Importantly, the space of beliefs is continuous, which translates into a continuous space of states in the belief MDP. The change from MDPs’ finite state-space to POMDPs’ (and belief MDPs’) continuous state-space eliminates many of the algorithms for MDPs and represents a big jump in difficulty. POMDP algorithms are more general than MDP ones, but they are not generally useful for MDPs because the requirement to handle a continuous state-space makes them significantly slower than dedicated MDP algorithms that assume finite state-spaces. Nevertheless, some insights can be shared between POMDP and MDP algorithms.

Non-deterministic Planning. This thesis concerns itself with probabilistic planning, where uncertainty is quantified with concrete probabilities. An alternative approach is to describe an action’s uncertainty as a set of outcomes that might occur, without specifying the concrete probabilities. One formalisation of this model is called Fully Observable Non-Deterministic (FOND) planning (Daniele, Traverso, and Vardi, 2000). The absence of concrete probabilities completely changes the problem, because the notion of expected cost becomes undefined. Instead, problems are solved by strong solutions which reach the goal for all possible outcomes or weak solutions which reach the goal for some possible outcome. Strong solutions generally do not permit cycles in the solution, i.e., the agent must reach a goal in a bounded number of steps; and solutions that do permit cycles are called strong cyclic solutions. Due to this fundamental difference, this thesis will not consider FOND or its algorithms. It is however worth mentioning that finding strong cyclic solutions is equivalent to finding a policy that reaches the goal with probability 1, and finding weak solutions is equivalent to finding a policy that reaches the goal with non-zero probability. Thus, SSP techniques such as FF-Replan (Yoon, Fern, and Givan, 2007) and Robust-FF (Teichteil-Königsbuch, Kuter, and Infantes, 2010), which focus on maximising the probability to reach a goal, can be applied to FOND. Conversely, state-of-the-art FOND planners such as PRP Rebooted (Muise, McIlraith, and Beck, 2024) can be used for finding MAX-PROB policies for SSPs as long as a policy that reaches the goal with probability 1 exists, and work has been done in trying to adapt FOND planners to a specific subset of SSP problems (Camacho, Muise, and McIlraith, 2016).

Probabilistic Planning with no Observability (Kushmerick, Hanks, and Weld, 1995). In this variant of probabilistic planning, the agent is given an initial belief for which state it is in, and during execution the agent does not receive any information about which state it is in and must progress “blindly,” in contrast to POMDPs where typically some information is given about the current state. A solution for this type of problem is a sequence of actions that is executed regardless of which of the initial possible states the agent is in, and regardless of which states it encounters during execution. This is completely different from the SSPs we consider, where the agent is fully aware of which state it is in, and it is required to react to the specific state that it is in with a *policy*. The best-known approach for solving these problems is Probabilistic-FF (Domshlak and Hoffmann, 2007), which performs a guided forward search on the space of probabilistic belief states. This model and the techniques for solving it are completely different from the topic considered in this thesis, so they will not be discussed further.

Encoding. We use the Probabilistic Planning Domain Definition Language (PPDDL) (Younes et al., 2005) to encode our SSPs in a compact form. This is an important detail because this compact encoding gives us access to informative domain-independent heuristics such as Landmark Cut (LMCut) (Helmert and Domshlak, 2009), Regrouped Operator Counting (ROC) (Trevizan, Thiébaux, and Haslum, 2017), and Pattern Databases (PDBs) (Klößner and Hoffmann, 2021). State-of-the-art guided-search algorithms rely on such heuristic methods. For this reason, we do not consider SSPs presented in a non-compact form, and we also do not consider RDDDL (Samner, 2010), for which similar heuristics do not exist. Note that nearly all algorithms discussed and introduced in this thesis can be applied to any encoding of SSPs, but the guided-search algorithms can not be expected to perform well without access to informative heuristic functions.

2.2 OR Techniques in Probabilistic Planning

Now, we give an overview of OR techniques that have been applied to probabilistic planning.

Linear Programming (LP). The task of finding an optimal policy for SSPs can be encoded into an LP (d’Epenoux, 1963; Mausam and Kolobov, 2012; Trevizan et al., 2016). Solving these LPs directly is impractical because the number of columns and constraints corresponds to the number of states and actions in the SSP, which can grow exponentially with our compact problem encoding. These LP formulations are important because they are used as a starting point for other OR techniques. Interestingly, under certain conditions, the simplex method applied to the LP formulation behaves the same as PI (Ye, 2011).

LP Heuristic Functions. In the context of SSPs, heuristic functions estimate each state’s expected cost to reach the goal so that guided-search algorithms can use this estimate to focus on promising states, and avoid unpromising ones. An active area of research in the planning community is to develop domain-independent heuristics that can be efficiently generated for any valid input problem in PPDDL. Two important heuristic functions, the Regrouped Operator Counting (ROC) and Projection Occupation Measure (POM) (Trevizan, Thiébaux, and Haslum, 2017), are constructed in terms of LPs. They leverage insights from network flow to relax SSPs into simpler problems. POM was generalised to the setting of SSPs with probabilistic LTL constraints (Baumgartner, Thiébaux, and Trevizan, 2018). Note that operator counting has also been used for heuristics in the context of deterministic planning (Pommerening et al., 2014), numerical planning (Piacentini et al., 2018), and others. The dual of the operator counting LP induces the cost partitioning heuristic, where an LP combines different heuristics into a stronger one (Pommerening et al., 2015). This dual perspective has also been applied to SSPs (Klößner et al., 2022).

Approximate LP (ALP) (Guestrin et al., 2003). ALP approximates an SSP’s optimal value function by constructing an LP with columns and constraints over a compact SSP representation, rather than constructing an LP over the SSP’s whole state-and-action space, which lets the LP scale to significantly larger problems. This formulation requires a factored SSP representation (Boutilier, Dearden, and Goldszmidt, 2000), which is a compact representation that gives a lot of structure. Additionally, the ALP requires a basis function that maps state features onto the values of those features. The canonical example is the sysadmin domain, where the agent tries to maximise the number of running computers; each state of this problem is defined by which computers are running (e.g., computer 1 is running, computer

2 is down), which are the state’s features. An appropriate set of basis functions for the problem with n computers is $b_1, \dots, b_n$ where b_i maps a state onto 1 if the i^{th} computer is running, and 0 otherwise. Then, the ALP approximates the value function with a sum of the basis terms and attempts to optimise this. The quality of the approximation depends on the basis functions. There are efficient schemes for generating basis functions automatically, e.g., (Sanner and Boutilier, 2006). An alternative approach to ALP for solving SSPs in the compact representation directly is to represent the compact value function and Bellman operators with Algebraic Decision Diagrams (ADD), which can be used to define a compact version of VI called SPUDD (Hoey et al., 1999). Such methods are not used in PPDDL planning any more, because it is usually faster to use guided-search on the non-compact SSP.

Primal-dual Algorithm. The primal-dual algorithm (Papadimitriou and Steiglitz, 1998) is a scheme for solving LPs via a sequence of simpler subproblems. Its subproblems determine whether the candidate solution can satisfy complementary slackness, which is sufficient to prove optimality; if it does not satisfy complementary slackness this gives information on how to update the candidate solution. The primal-dual algorithm applied to deterministic shortest path problems behaves the same as Dijkstra’s algorithm (Papadimitriou and Steiglitz, 1998). Ye, Han, and Lin (2010) extended this insight to show that A^* with a consistent heuristic H is equivalent to the primal-dual algorithm if its initial solution is constructed from H . Although this connects deterministic planning with OR, not probabilistic planning, this equivalence motivates that there are connections between guided search and OR.

Constraint Generation. Constraint generation (Bertsimas and Tsitsiklis, 1997) is a technique from OR that makes it possible to solve LPs with impractically many constraints. It works in the following steps:

1. Start with an LP that has only a subset of the original LP’s constraints. This is called a relaxed LP.
2. Solve the relaxed LP.
3. If the relaxed LP’s solution satisfies the original problem’s constraints, then it is immediately an optimal solution for the original problem and the algorithm can terminate. If one (or more) of the original problem’s constraints is violated, then the constraint is added to the relaxed LP and the algorithm loops to step 2.

This procedure is guaranteed to find an optimal solution; in the worst case, constraint generation will add all the original problem’s constraints, at which point it trivially yields an optimal solution for the original problem. Often, only a small subset of the original problem’s constraints are needed to describe an optimal solution, in which case constraint generation speeds up the search process significantly by omitting unnecessary constraints.

The paradigm of constraint generation lends itself well to complex planning problems where a relaxation can be solved efficiently. An example that demonstrates this very clearly is Multi-Agent Path-Finding (MAPF) (Stern et al., 2021), where the aim is to coordinate multiple agents to achieve their objectives as efficiently as possible, making sure that the agents do not interfere with each, e.g., they should not crash into each other. This problem is notoriously difficult, especially as the number of agents increases; finding optimal solutions is NP-hard (Yu and LaValle, 2013), and in some cases even finding a feasible solution is NP-hard (Nebel, 2020). On the other hand, if we focus on each agent in isolation, then finding their individual optimal paths is a classical planning problem, which can be solved efficiently. If the isolated optimal paths do not interfere, then this immediately gives an optimal solution for the MAPF problem, but in general there will be interferences that must be addressed. We can impose constraints that block such interferences; for example, suppose that following the isolated paths causes agents A and B to crash into each other in location (x, y) at time t , then we can prevent the crash by disallowing A from entering (x, y) at time t . Then, a viable algorithmic loop is to find paths for the agents in isolation, determine where they interfere, add constraints that block the undesired behaviour, then find updated plans for the agents in isolation that respect the added constraints, and repeat until no interferences remain. The MAPF community calls this technique Conflict Based Search (CBS) (Sharon et al., 2012), and it has seen a reasonable amount of success (Boyarski et al., 2015). Constraint generation has also been used explicitly (by name) in MAPF with Mixed Integer Programs (Calliess and Roberts, 2021). The same paradigm applies similarly to other complex planning problems, e.g., in metric hybrid factored planning in nonlinear domains (Say and Sanner, 2019).

Counter-Example Guided Abstraction Refinement (CEGAR) (Seipp and Helmert, 2013) can also be considered a form of constraint generation, which has seen success in constructing heuristics (Rovner, Sievers, and Helmert, 2019) and policy verification (Vinzent, Sharma, and Hoffmann, 2023). In the context of classical planning, the idea is to use abstraction to compress the original problem’s state-space, so that the ensuing problem is smaller and easier to solve, but still captures some of the original problem’s structure. Then, we can solve the smaller problem to obtain a plan. A plan for the smaller problem may immediately be a solution for the original problem, in which case the problem is solved; more likely, the plan has some *flaw* when we try to apply it to the original problem. This is called a counter-example, and is used to refine the abstraction so that this particular flaw can not occur again, effectively adding constraints prohibiting it. Consequently, we get the algorithmic loop that solves an abstraction, finds flaws, refines the abstraction to prohibit these flaws, resolves the abstraction, and then repeats until no more flaws can be found. Again, this fits the framework of constraint generation.

There is also existing work in the planning literature that applies constraint generation to various value functions. For POMDPs, the value function is encoded by a set of vectors, and an LP with constraint generation can prune unneeded vectors (Walraven and Spaan, 2017). The ALP similarly uses a linear value function to give an approximation of the value function for factored SSPs, and Schuurmans and Patrascu (2001) apply constraint generation to solve it efficiently.

Column Generation. Column generation (Desrosiers and Lübbecke, 2005), also known as variable generation, is analogous to constraint generation; it is an OR technique for solving LPs with impractically many variables (columns) by starting with a subset and adding more as they are required. It works in the following steps:

1. Start with an LP that has only a subset of the original LP’s variables. This is called a Reduced Master Problem (RMP).
2. Solve the RMP.¹ The RMP’s solution is feasible for the original problem, but may not be optimal.
3. Solve the *pricing problem*, which determines if a variable exists so that its addition to the RMP will improve the RMP’s objective. If no improving variable is found, then the RMP’s optimal solution is also optimal for the original problem; if an improving variable is found, then it is added to the RMP and the algorithm loops to step 2.

Similarly to constraint generation, this procedure is guaranteed to find the optimal solution eventually, and can be significantly more efficient than solving the original LP if only a small subset of variables is required to encode the optimal solution.

Column generation has been applied in various ways to planning under uncertainty. One use-case is to solve constrained problems with column generation, so that the new algorithm can solve simpler unconstrained problems as subproblems, and combine their solutions into a solution for the original problem. This approach has been applied to black-box CSSPs (Geißer et al., 2020) and Constrained POMDPs (Walraven and Spaan, 2018); these approaches solve unconstrained SSPs and POMDPs as their pricing problems, respectively, and treat their policies as columns in an LP that combines these policies into a policy for the constrained problem. In terms of pruning unneeded vectors for POMDPs, Hansen and Bowman (2020) extended the constraint generation algorithm of Walraven and Spaan (2017) by additionally applying column generation on top of constraint generation. Column generation has also been used to solve complex models such as Correlated Simple Temporal Network (Murray et al., 2023a) and Virtual Network Function Placement and Routing Problems (Murray et al., 2023b); an overview of these is given by Murray (2024).

I-dual (Trevizan et al., 2016) and i²-dual (Trevizan, Thiébaux, and Haslum, 2017) are important algorithms in part II of this thesis. They solve CSSPs by simultaneously applying constraint and column generation: they solve a sequence of smaller CSSP LPs, iteratively adding constraints and columns as they are needed until they produce a solution for the original CSSP. These are described in more detail in chap. 6. Steinmetz et al. (2024) extended these methods to a more general problem.

Both constraint and column generation are used extensively throughout this thesis. Chap. 4 reformulates guided search in terms of constraint and column generation, and uses insights from constraint generation to solve problems more efficiently. Chap. 5 applies column generation to decompose SSPs into a sequence

¹For now, we assume that RMPs are always solvable. We address the issue of infeasible RMPs later.

of deterministic problems, and chap. 8 uses both constraint and column generation by building on the ideas of i-dual and i²-dual.

Constraint and column generation are well-established techniques in OR, and have been used to solve difficult real-world problems, e.g. the Vehicle Routing Problem (Archetti and Speranza, 2014) and Crew Rostering (Kohl and Karisch, 2004). Consequently, there is a lot of OR literature about improving constraint and column generation, which may be applicable to the use cases we have discussed. For example, column generation often suffers from slow convergence, which can be mitigated with stabilised column generation (du Merle et al., 1999). Such techniques are beyond the scope of this thesis but may be promising directions for future work.

Lagrangian dual. Given a constrained planning problem, a Lagrangian relaxation removes the constraints and places them into the problem’s optimisation objective, so that any constraint violations penalise the solution’s objective. Each Lagrangian relaxation’s optimal objective provides a lower bound for the original problem’s optimal objective. Then, the Lagrangian dual searches over penalty terms for the constraint violations in the objective to find the largest lower bound. Under certain conditions, the Lagrangian dual induces optimal solutions for the original problem. Solving the Lagrangian dual requires the solving of multiple Lagrangian relaxations, which are unconstrained versions of the original problem, and these unconstrained problems are typically easier to solve. This framework has been used successfully to solve constrained deterministic shortest path problems (Handler and Zang, 1980), Constrained POMDPs (Lee et al., 2018), and to find deterministic policies for CSSPs (Hong and Williams, 2023). It has not been used previously to find optimal (potentially stochastic) policies for CSSPs, which is the topic of chap. 7.

Hindsight Optimisation (HOP). HOP is a technique from stochastic control that approximates a state’s value in the probabilistic problem by solving a set of deterministic relaxations from that state and combining their values. FF-Hindsight (Yoon et al., 2008) applies this technique to SSPs. For a state s , FF-Hindsight approximates s ’s value by constructing W -many deterministic problems, where each deterministic problem resolves the probabilistic effects by picking one outcome. Importantly, for each deterministic problem the probabilistic actions’ effects are picked randomly during the construction of the problem. An additional important detail is that the effect selected for an action is allowed to be different over different timesteps, i.e., action a may give the effect s' if it is applied in the first step, and $s'' \neq s'$ if it is applied in the second. Effectively, each deterministic solution can be treated as a sample, and then these samples are combined to give an estimate of s ’s value in the original problem. HOP is not used in the thesis, but it can be used to derive informative heuristics for problems without domain-independent heuristics, so it could be used to bridge the gap between the algorithms in this thesis and non-PPDDL problems.

Model Predictive Control (MPC). Given a state s , MPC determines a good action for state s by constructing a simplification of the original problem, solving this simplified problem from s , and applying this solution’s optimal action for s . As the agent moves to a new state s' , a new simplified problem is constructed for s' , solved from s' , and its optimal action for s' is applied; this process is repeated for each new state the agent encounters. The key idea is that the simplified problems are significantly easier to solve while giving a sufficiently good approximation of the original problem so that its optimal action for s has a good chance to be optimal for the original problem’s s . Importantly, MPC acknowledges that the simplified problem is an imperfect approximation that will deviate from the original problem as more actions are applied, so it (generally) recomputes its actions for each state it encounters and only applies the first action from each simplified problem. The simplified problem is often constructed by setting a small horizon, so MPC is also known as Receding Horizon Control. Short-Sighted SSPs (Trevizan and Veloso, 2012, 2014) apply MPC to SSPs by generalising the concept of horizons, allowing subproblems to be generated by restrictions to SSP-specific metrics, e.g., policy size and failure probability. Algorithms such as FF-Replan (Yoon, Fern, and Givan, 2007) and Robust-FF (Teichteil-Königsbuch, Kuter, and Infantes, 2010) also apply MPC, but their subproblems approximate SSPs with determinisations, which remove probabilities.

Part I

Stochastic Shortest Path Problems

Background

3.1 The Problem and its Solutions

Stochastic Shortest Path Problems (SSPs) represent problems where an agent must move between states using actions with probabilistic effects that are known a priori. SSPs are generalisations of Markov Decision Processes (MDPs) (Bertsekas, 2019), and these two have been used to solve many real-world problems where there is an element of uncertainty, for example:

- penetration testing (pentesting) (Hoffmann, 2015),
- ecological management (Péron et al., 2017),
- managing hydroelectric and thermal power generation (White, 1985).

We now give the formal definition of SSPs.

Definition 1 (Stochastic Shortest Path Problem (SSP) (Bertsekas and Tsitsiklis, 1991)). *An SSP is defined by the tuple $\mathbb{S} = \langle S, s_I, G, A, P, C \rangle$ where*

- S is the finite set of states that the agent can encounter;
- $s_I \in S$ is the agent's initial state;
- $G \subsetneq S$ is the set of goal states, where the agent's task is to reach one of these. We assume that $s_I \notin G$ and $G \neq \emptyset$;
- A is the finite set of actions that the agent can encounter, and we write $A(s) \subseteq A$ to denote the set of applicable actions at state $s \in S$;
- P is the probability transition function, where $P(s'|s, a)$ gives the probability of reaching s' after applying action a in state s ;
- $C : A \rightarrow \mathbb{R}_{>0}$ is the cost function and $C(a)$ is the cost of applying action a .

We treat the goals as sink states, so that whenever the agent reaches a goal, it terminates, applies no more actions, and incurs no more costs. Some authors instead define a goal as a regular state s with a zero-cost action that loops back to s . These two formulations are equivalent. The cost function can also be defined as $C : S \times A \rightarrow \mathbb{R}_{>0}$ where $C(s, a)$ denotes the cost of applying a in s ; or even $C : S \times A \times S \rightarrow \mathbb{R}_{>0}$ where $C(s, a, s')$ denotes the cost of applying a in state s and then transitioning to s' . Most of our work is compatible with all formulations, but some domain-independent heuristics require $C : A \rightarrow \mathbb{R}_{>0}$, so we focus on this case.

SSPs are solved by policies, which are functions that inform the agent what to do in a given state. There are many types of policies, we focus on the following two:

Definition 2 (Deterministic Policy). *A deterministic policy is a function $\pi : S \rightarrow A$. Semantically, the agent inputs its current state s and applies the action $\pi(s)$.*

Definition 3 (Stochastic Policy). *A stochastic policy is a function $\pi : \mathcal{S} \rightarrow \text{distr}(\mathcal{A})$, where $\text{distr}(\mathcal{A})$ refers to a probability distribution over the actions $\mathcal{A}$. Semantically, the agent inputs its current state s , randomly selects an action a according to the probability distribution $\pi(s)$, and then applies a .*

For stochastic policies, we write $\pi(s, a)$ to denote the probability of a in the probability distribution $\pi(s)$, i.e., the probability with which a is applied in state s when following π . Deterministic policies are a special case of stochastic policies where the probability distribution is degenerate, i.e., each probability distribution has precisely one entry with probability 1, and all others have 0. Thus, we also write $\pi(s, a)$ for deterministic policies, bearing in mind that $\pi(s, a) = 1$ for a single action, and $\pi(s, a') = 0$ for all the others. By abuse of notation, if we know the policy to be deterministic, we may also write $\pi(s)$ to indicate the action with unit probability, rather than the whole distribution.

The policies we have defined are Markovian and stationary, because the decision depends only on the agent’s current state, with no additional information. This is sufficient for the problems we consider in this thesis, and more general policies do not yield better solutions. In fact, it turns out that deterministic policies are sufficient for SSPs, which we will formalise later after we define policy costs.

We need the ability to discuss what states are reachable by applying certain actions or policies, which leads to the following definitions of successors and the policy envelope:

Definition 4 (Successors). *Given a state $s \in \mathcal{S}$ and an applicable action $a \in \mathcal{A}(s)$ in the SSP $\mathbb{S}$, the states that might be the result of applying a to s are called the successors of (s, a) ,*

$$\text{succ}(s, a) \stackrel{\text{def}}{=} \{s' \in \mathcal{S} : P(s'|s, a) > 0\}.$$

Definition 5 (Policy Envelope). *Given a policy π for an SSP $\mathbb{S}$, the policy envelope rooted at state $s \in \mathcal{S}$ is $S^{\pi, s} \subseteq \mathcal{S}$ denotes the set of states that can be reached by following π from s .*

In this thesis, we mostly consider policy envelopes rooted at s_I , so we write $S^\pi = S^{\pi, s_I}$ and call S^π the policy envelope of π . We have defined policies to over the entire state space $\mathcal{S}$ (called complete policies), but in practice, policies do not need to be complete because policies do not typically encounter the whole state space, i.e., $S^\pi \subsetneq \mathcal{S}$. So, we allow policies to be partial, i.e., defined over a subset of states. However, it is desirable that the agent knows what to do in each state that it does encounter, and that the agent eventually reaches the goal, which leads to the following definitions:

Definition 6 (Closed and Open Policies). *A policy π is closed w.r.t. s if for all states in the policy envelope $s' \in S^\pi$, either $\pi(s')$ is defined or s' is a goal state. It is open otherwise.*

Definition 7 (Proper and Improper Policies). *A policy π is proper w.r.t. s if, by following π from s , the agent will reach a goal with probability 1. It is improper otherwise.*

For both definitions, if s is not specified we assume that it is s_I . Properness implies closedness. For closed policies π , their expected cost is the total expected cost incurred by the agent following π from s_I .

Definition 8 (Policy Cost). *The cost of a closed policy π , written $C(\pi)$, is the expected cost incurred by the agent by following π from s_I .*

We require closedness because it is otherwise unclear how to define the cost of encountering a state where the policy is undefined.

In this thesis, we make two standard assumptions for SSPs:

Assumption 1 (Reachability). *There exists at least one proper policy w.r.t. s_I .*

Assumption 2. *All improper policies have infinite expected cost.*

Note that $\mathcal{A}(s) \neq \emptyset$ for all $s \in \mathcal{S} \setminus \mathcal{G}$ as a consequence of assumption 2. If this were not the case, then there could be a state $s \in \mathcal{S} \setminus \mathcal{G}$ with $\mathcal{A}(s) = \emptyset$ and we could construct an example where a policy π reaches s with probability 1 — such π ’s expected cost is finite even though π is improper. In our experiments, we consider SSPs that violate assumption 1, i.e., SSPs with dead ends, which we address by applying the fixed-penalty transformation:

Definition 9 (Fixed-penalty Transformation (Trevizan, Teichteil-Königsbuch, and Thiébaux, 2017)). *Given an SSP $\mathbb{S} = \langle \mathcal{S}, s_I, \mathcal{G}, \mathcal{A}, P, C \rangle$ and a penalty term $d \in \mathbb{R}_{>0}$, its fixed-penalty transformation is the new SSP $\mathbb{S}' = \langle \mathcal{S}, s_I, \mathcal{G}, \mathcal{A}', P', C' \rangle$ with:*

- $\mathcal{S}, s_I, \mathcal{G}$ identical;
- $\mathcal{A}'(s) = \mathcal{A}(s) \cup \{\text{give-up}\}$, i.e., $\mathcal{A}$ is augmented with the artificial give-up action;

$$\begin{aligned}
\bullet \quad C'(a) &= \begin{cases} C(a) & \text{if } a \in A(s) \\ d & \text{if } a = \text{give-up}; \end{cases} \\
\bullet \quad P'(s'|s, a) &= \begin{cases} 1 & \text{if } s' = g \text{ and } a = \text{give-up} \\ 0 & \text{if } s' \neq g \text{ and } a = \text{give-up} \\ P(s'|s, a) & \text{else} \end{cases}
\end{aligned}$$

where $g \in G$ is some representative goal state.

Such transformed SSPs S' always satisfy reachability (assumption 1), because the agent can always reach the goal with probability 1 by applying the give-up action. For sufficiently large d , the optimal policy of S' is what we intuitively want: it maximises the probability of reaching goals, and then minimises cost.

Other approaches for relaxing our assumptions on SSPs, such as S^3P (Teichteil-Königsbuch, 2012), can also be implemented without significant changes.

With these assumptions, we can define optimal policies:

Definition 10 (Optimal Policy). A policy π is optimal, often written π^* , if it is proper and $C(\pi) \leq C(\pi')$ for all proper policies π' .

Note that this definition requires reachability (assumption 1), otherwise we would have to define how to balance the probability of reaching goals with policy cost, e.g., maximise probability first and then minimise cost (Teichteil-Königsbuch, 2012).

Any SSP that satisfies our assumptions will have one or more optimal policies, and one of these optimal policies is guaranteed to be deterministic (Bertsekas and Tsitsiklis, 1991). Thus, the focus of part I is on deterministic policies.

One approach for computing optimal policies is via *costs-to-go* and *value functions*. Given a policy π and a state s , the cost-to-go from s is the agent's expected incurred cost of following π from s (assuming that π is proper w.r.t. s). The optimal cost-to-go is the cost-to-go associated with an optimal policy π^* . This can be expressed formally in terms of value functions:

Definition 11 (Value Function). A value function $V : S \rightarrow \mathbb{R}_{\geq 0}$ maps states onto non-negative real numbers.

We use the following notations for value functions:

- Given V , the Q -value of s and a is a common shorthand

$$Q(s, a) \stackrel{\text{def}}{=} C(a) + \sum_{s' \in A(s)} P(s'|s, a) V(s).$$

- Whenever we have a value function V^x we will refer to its associated Q -values as Q^x .
- We will write $V \leq V'$ to compare value functions element-wise, i.e., $V \leq V'$ iff $V(s) \leq V'(s) \forall s \in S$, and similarly for other comparators, e.g., $=, \geq$.

The *optimal value function* describes the optimal cost-to-go, and we may use the terms interchangeably:

Definition 12 (Optimal Value Function). The optimal value function $V^* : S \rightarrow \mathbb{R}_{\geq 0}$ maps each state s onto the expected cost of following an optimal policy π^* from s (assuming that π^* is defined for all states). V^* is defined as the unique fixed-point solution to the following set of equations, called the Bellman equations:

$$V(s) = \begin{cases} 0 & \text{if } s \in G \\ \min_{a \in A(s)} Q(s, a) & \text{if } s \in S \setminus G. \end{cases} \quad (3.1)$$

Although an SSP may have multiple optimal policies, the optimal value function is unique.

For V^* , each $Q^*(s, a)$ can be interpreted as the optimal cost-to-go after we apply a to s . In general, value functions do not have built-in semantics, but we use them to approximate V^* , or as intermediate steps to obtain V^* , so it can be helpful to interpret them as costs-to-go. Now, we show how to extract policies from value functions.



Figure 3.1: Invalid SSPs that have optimal value functions that induce suboptimal policies. One SSP is invalid due to a zero-cost cycle, and the other SSP violates assumption 2 by having a state with no applicable actions.

Definition 13 (Greedy Policy). *Given a value function V , its greedy policy is*

$$\pi_V(s) = a_{\min} \quad \text{for some} \quad a_{\min} \in \underset{a \in A(s)}{\operatorname{argmin}} Q(s, a) \quad \forall s \in \mathcal{S}$$

where any ties between $\underset{a \in A(s)}{\operatorname{argmin}} Q(s, a)$ can be broken arbitrarily.

Importantly, for the optimal value function V^* , its greedy policy π_{V^*} is an optimal policy. Moreover, the set of all greedy policies for V^* , obtained by breaking ties in all possible ways, is precisely the set of optimal policies. To avoid dealing with multiple greedy policies we make the following tie-breaking assumption:

Assumption 3 (Tie-breaking for Greedy Policies). *Given V , we ensure the greedy policy π_V is unique by breaking ties in $\underset{a \in A(s)}{\operatorname{argmin}} Q(s, a)$ with arbitrary but reproducible tie-breaking rules. An example of such a rule is lexicographical tie-breaking.*

In practice, a greedy policy for V does not need to be defined for all $s \in \mathcal{S}$, and it can be constructed for $s \in \mathcal{S}^{\pi_V}$ by starting at s_I and determining $\pi_V(s)$ as required.

Now that we have defined value functions and greedy policies, we briefly justify the assumptions we have made about SSPs. In fig. 3.1 we present two SSPs that violate assumptions: in fig. 3.1a, the state s_1 has a zero-cost action to itself which contradicts the requirement of strictly positive costs $C : \mathcal{S} \rightarrow \mathbb{R}_{>0}$ and violates assumption 2; and in fig. 3.1b the state s_1 has no applicable actions which violates assumption 2. Both SSPs have optimal value functions V^* with $V^*(s_I) = 1, V^*(s_1) = 0, V^*(s_g) = 0$ (arguably $V^*(s_1) = -\infty$ for fig. 3.1b, depending on how we define the min operator over the empty set), and the greedy policies π_{V^*} have $\pi_{V^*}(s_I) = a'_0$, making them improper, even though a proper policy exists. Thus, the optimal value function V^* does not induce an optimal policy. In general, for SSPs that violate assumption 2, an optimal value function V^* need not induce an optimal policy with π_{V^*} , and moreover, V^* need not even be unique, e.g., $V^*(s_I) = 3, V^*(s_1) = 2, V^*(s_g) = 0$ satisfies the Bellman equations for fig. 3.1a. Defining SSPs over strictly positive costs $C : \mathcal{S} \rightarrow \mathbb{R}_{>0}$ avoids the issue of zero-cost cycles and prohibits fig. 3.1a; and assumption 2 avoids such issues in general and prohibits both examples in fig. 3.1.

3.2 Bellman Backups and Value Iteration

Value Iteration (VI) (Bellman, 1957) is the foundational algorithm for finding a solution to the Bellman equations (3.1), thereby solving SSPs, that modern algorithms build on. VI starts with an arbitrary value function V^0 , and computes a sequence of value functions $V^1, V^2, \dots$ which in the limit converge to V^* . These value functions are computed by applying *Bellman backups*:

Definition 14 (Bellman Backup). *Given value functions V and V' , a Bellman backup applied to state s populates $V'(s)$ with*

$$V'(s) \leftarrow \min_{a \in A(s)} Q(s, a).$$

and does not update $V'(s')$ for any other state $s' \neq s$.

To compute V^{t+1} for $t \in \mathbb{N}_{>0}$, the classical version of VI applies Bellman backups on V^t and V^{t+1} for each $s \in \mathcal{S}$. Asynchronous VI (also called Gauss-Seidel VI) computes V^{t+1} by initialising it with $V^{t+1} = V^t$ and then it only performs a Bellman backup for a single state in each t ; but importantly, it ensures

that all states are backed up fairly, i.e., if the sequence $V^0, V^1, \dots$ is infinite then each state is updated infinitely often. In both versions, for any choice of V^0 , the sequence of value functions produced by VI will asymptotically converge to the optimal value function, i.e., $\lim_{t \rightarrow \infty} V^t = V^*$ (Bertsekas, 1995). In practice, VI is terminated in finite time by considering the difference between successive value functions, measured by the *Bellman residual*.

Definition 15 (Bellman Residual). *Given a value function V , its Bellman residual at state s is*

$$\text{RES}(s) \stackrel{\text{def}}{=} \left| V(s) - \min_{a \in A(s)} Q(s, a) \right|.$$

VI terminates when the residual between successive value functions is small over all states. Formally, given a value $\epsilon \in \mathbb{R}_{>0}$, VI terminates upon the following condition:

Definition 16 (Global ϵ -consistency). *V is globally ϵ -consistent for an SSP $\mathbb{S}$ when $\text{RES}(s) \leq \epsilon \forall s \in \mathbb{S}$.*

Importantly, when VI is stopped with this condition its value function is only an approximate solution and for any ϵ it is possible to construct an SSP so that VI terminates with V whose greedy policy is not an optimal policy. In practice, for reasonably small ϵ , e.g., 0.0001, the greedy policy for globally ϵ -consistent V will be optimal or close to optimal. Moreover, there is a bound on how much such a V can deviate from V^* :

Definition 17. *Let $N(s, V, \mathbb{S}) = \max\{N^*(s), N^{\pi_V}(s)\}$ where $N^{\pi_V}(s)$ denotes the expected number of steps to reach some goal from s by following the greedy policy π_V , and $N^*(s)$ gives the maximum expected number of steps to reach a goal from s over $\mathbb{S}$'s optimal policies.*

Theorem 1 (VI Error (Mausam and Kolobov, 2012)). *Consider globally ϵ -consistent V (defn. 16). Then,*

$$|V(s) - V^*(s)| \leq \epsilon N(s, V, \mathbb{S}) \forall s \in \mathbb{S}.$$

Note that the error term $\epsilon N(s, V, \mathbb{S})$ can get significantly larger than ϵ . Nevertheless, $N(s, V, \mathbb{S})$ can be bounded from above, and as $\epsilon \rightarrow 0$ the error disappears and VI's V converges to V^* .

3.3 Heuristic Search

A heuristic function $H : \mathbb{S} \rightarrow \mathbb{R}_{\geq 0}$ is a value function with the semantic that it estimates V^* and is either known a priori or computed efficiently. Heuristic-search (a.k.a. guided-search) algorithms use these heuristic functions to guide their search: they focus on states that look promising, i.e., have small $H(s)$, and avoid unpromising states with large $H(s)$. This is a powerful technique that can enable algorithms to consider a small subset of states. In comparison, algorithms such as VI must consider every state $s \in \mathbb{S}$ without any guidance, so they are called blind-search algorithms. Blind-search algorithms do not scale to large problems, especially for compactly encoded problems such as PPDDL (Younes et al., 2005), where the state space can grow exponentially with respect to the problem size. The technique of heuristic search was pioneered by A* (Hart, Nilsson, and Raphael, 1968) in the context of deterministic planning, and has been applied in the context of SSPs to produce state-of-the-art optimal algorithms such as iLAO* (Hansen and Zilberstein, 2001) and LRTDP (Bonet and Geffner, 2003b).

The key insight for heuristic-search on SSPs is that we do not require V^* for all states to define an optimal policy, and we only need V^* for the optimal policy's envelope. Then, instead of considering global ϵ -consistency, we consider ϵ -consistency:

Definition 18 (ϵ -consistency (Bonet and Geffner, 2003a)). *A value function V is ϵ -consistent for an SSP $\mathbb{S}$ if $\text{RES}(s) \leq \epsilon \forall s \in \mathbb{S}^{\pi_V}$.*

Note that ϵ -consistency relies on π_V , which is ambiguous when V has multiple greedy policies; so, we fall back on assumption 3 and assume that ties are broken such that V has a single greedy policy. This definition of ϵ -consistency requires the residual to be small for states in the greedy policy's envelope, and states outside can have arbitrarily large residuals. Importantly, to ensure that ϵ -consistent value functions produce optimal policies as $\epsilon \rightarrow 0$, we need additional properties over the value functions. In particular, we require *admissibility*:

Definition 19 (Admissible Value Function). *V is admissible if $V \leq V^*$.*

An admissible value function is simply a lower bound on the optimal value function. Without admissibility, ϵ -consistency does not produce optimal policies. For example, consider fig. 3.2: the value function

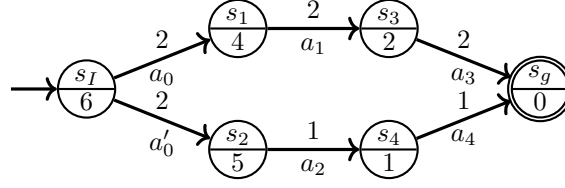


Figure 3.2: An SSP with a value function that is ϵ -consistent but inadmissible, and its greedy policy is suboptimal.

shown in the bottom of nodes is ϵ -consistent w.r.t. the greedy policy with $\pi_V(s_I) = a_0$, but it is inadmissible and fails to capture the optimal policy with $\pi^*(s_I) = a'_0$.

Another useful condition for value functions, which is stronger and implies admissibility, is *monotonicity*:

Definition 20 (Monotonic Value Function). *V is monotonic if $V(s) = 0 \forall s \in G$ and*

$$V(s) \leq \min_{a \in A(s)} Q(s, a) \quad \forall s \in S \setminus G.$$

An alternative name for monotonic value functions is consistent value functions, and it generalises the notion of consistent heuristics in deterministic planning. Monotonic value functions get their name from the property that Bellman backups will never decrease a value function, i.e., the sequence $V^0, V^1, V^2, \dots$ obtained by applying Bellman backups is monotonically non-decreasing with $V^i \leq V^{i+1} \forall i \in \mathbb{N}$. This property holds for any V^{i+1} obtained from V^i via Bellman backups, for any arbitrary number of Bellman backups applied to an arbitrary set of states. Conveniently, Bellman backups preserve both admissibility and monotonicity. So, an admissible value function with arbitrarily many Bellman backups applied to an arbitrary set of states will yield another admissible value function, and similarly for monotonicity (Barto, Bradtke, and Singh, 1995; Bertsekas, 1995; Bonet and Geffner, 2003b). Thus, an algorithm that is initialised with an admissible value function and only uses Bellman backups to modify V , has $V \leq V^*$ as an invariant.

Heuristic-search algorithms that are initialised with an admissible value function $V^0 \leq V^*$ and apply Bellman backups until ϵ -consistency is satisfied, produce optimal policies. Note that this contrasts to VI, which does not have any requirements over its initial value function V^0 . We present this result formally, via the FIND-AND-REVISE framework:

Definition 21 (FIND-AND-REVISE Algorithm (Bonet and Geffner, 2003a)). *A FIND-AND-REVISE algorithm applied to SSP S uses a value function V and consists of the steps*

- FIND a state s in the greedy policy envelope S^{π_V} s.t. $\text{RES}(s) > \epsilon$,
- REVISE the found state by applying a Bellman backup to s ,
- repeat these steps until no such state is found and V is ϵ -consistent w.r.t. S .

The FIND-AND-REVISE schema describes many heuristic-search algorithms, including iLAO* and LRTDP. FIND-AND-REVISE algorithms are optimal under the following conditions:

Theorem 2 (Optimality of FIND-AND-REVISE (Bonet and Geffner, 2003a; Mausam and Kolobov, 2012)). *Consider a FIND-AND-REVISE algorithm initialised with an admissible heuristic and terminated when its V is ϵ -consistent for any given $\epsilon \in \mathbb{R}_{>0}$. As $\epsilon \rightarrow 0$, its output $V(s)$ approaches $V^*(s)$ for $s \in S^{\pi_V}$.*

Other termination conditions are possible in place of ϵ -consistency, e.g., ϵ -optimality (Hansen and Zilberstein, 2001; Hansen and Abdoulahi, 2015). We do not consider these alternatives because they build on top of ϵ -consistency, and are essentially automatic methods for selecting the parameter for ϵ -consistency so that stronger guarantees can be provided. Consequently, our ϵ -consistent algorithms can be adapted to their termination conditions.

3.4 Linear Programs

In this section, we present two Linear Programs (LPs) for solving SSPs, which represent the core connection between planning and OR. Every contribution in this thesis relates to these LPs.

3.4.1 Basics of LPs

We give a brief overview of the properties of LPs that will be used throughout this thesis, and refer the reader to (Bertsimas and Tsitsiklis, 1997) for details. LPs are optimisation problems over a set of real-valued variables where a linear expression of variables must be maximised or minimised subject to linear constraints over the variables. The linear expression to be maximised or minimised is called the LP's objective. Fig. 3.3 gives an example of a minimisation and a maximisation LP. For an LP, a feasible solution is an assignment for the variables that satisfies all the linear constraints. Geometrically, each linear inequality describes a half-space and each linear equality describes a plane; the intersection of the constraints' half-spaces and planes produces a convex polyhedron, and a feasible solution must lie within the convex polyhedron. An optimal solution must maximise or minimise the linear objective while remaining feasible. Thus, an optimal solution must be on an extreme point of the polyhedron, i.e., a vertex, edge, or facet (assuming that the polyhedron is bounded and non-empty). Every LP has a *dual LP*, which has the opposite optimisation direction (maximisation becomes minimisation and vice versa) but still solves the same problem. The original LP is called the primal LP, and its dual is called the dual LP. In fig. 3.3, the two LPs are dual to each other. By taking the dual twice we recover the primal LP, i.e., a dual LP's dual is the primal LP. LPs enjoy strong duality which means that any dual LP's optimal solution has the same objective as the primal's optimal solution.

$$\begin{array}{ll}
 \min_{\mathbf{x} \in \mathbb{R}^n} \mathbf{c} \cdot \mathbf{x} \text{ s.t.} & \max_{\mathbf{y} \in \mathbb{R}^m} \mathbf{b} \cdot \mathbf{y} \text{ s.t.} \\
 \mathbf{A}\mathbf{x} = \mathbf{b} & \mathbf{A}^T \mathbf{y} \leq \mathbf{c} \\
 \mathbf{x} \geq 0 &
 \end{array}$$

$$\mathbf{A} \in \mathbb{R}^{m \times n}, \mathbf{b} \in \mathbb{R}^m, \mathbf{c} \in \mathbb{R}^n$$

Figure 3.3: A minimisation and a maximisation LP. They are dual to each other.

The standard method for solving LPs is the *simplex algorithm*, and we will assume that a variant of the simplex algorithm is used to solve all LPs in this thesis. Geometrically, the simplex algorithm relies on the observation that any LP has some optimal solution on a vertex, and the algorithm moves along edges to improving vertices until it finds an optimal vertex. The solutions corresponding to vertices are called *basic feasible solutions*. Algebraically, basic feasible solutions are feasible solutions that partition the variables into *basic variables* and *non-basic variables*, where non-basic variables are forced to be zero and basic variables are free. The simplex algorithm works by selecting a non-basic variable whose inclusion in the basic variables may improve the objective, and then this variable is exchanged with a current basic variable, i.e., the non-basic variable becomes basic and one of the basic variables becomes non-basic.

3.4.2 Value Function LP (VLP)

The VLP (d'Epenoux, 1963) is given in LP 1. Its variables V encode the value function with V_s for each $s \in \mathbf{S}$, and its objective and constraints work together to encode the Bellman equations (3.1). Consequently, its optimal solution V^* represents the optimal value function V^* , and thus VLP solves SSPs via the optimal value function. To emphasise that the VLP's variables encode a value function we write it in non-bold and treat it as a scalar value function $V : \mathbf{S} \rightarrow \mathbb{R}_{\geq 0}$, rather than a vector $V \in \mathbb{R}^{|\mathbf{S}|}$.¹ Furthermore, we write $V(s)$ and V_s interchangeably, and we will refer to the right-hand side of (s, a) 's constraint C1 as $Q(s, a)$. Since this LP optimises over the value function, we call it the **primal formulation**.

To see that VLP solves the Bellman equations (3.1), observe the differences: the Bellman equations have an equality and a min operator, whereas the LP has inequalities and a maximisation objective. Initially,

¹Note that $V_g = 0$ for $g \in \mathbf{G}$. So, we can treat V_g as non-free variables, or as constants so that $V \in \mathbb{R}^{|\mathbf{S} \setminus \mathbf{G}|}$.

these changes may seem surprising, but recall that the minimum of a set of numbers can be defined as the largest lower bound (infimum) on that set, as long as the infimum is contained within the set; each constraint C1 ensures that V_s is a lower bound over all of s 's Q -values, and the maximisation pushes V to be the largest lower bound, i.e., VLP precisely encodes the Bellman equations. Another way to interpret VLP is that its constraints force feasible solutions to be monotonic value functions, and it is looking for a monotonic value function that maximises V_{s_I} , i.e., it is looking for the perfect heuristic V^* .

Value Function LP (VLP) $\begin{aligned} \max_V \quad & V_{s_I} && \text{(LP 1)} \\ \text{s.t.} \quad & V_s \leq C(a) + \sum_{s' \in \mathcal{S}} P(s' s, a) V_{s'} && \forall s \in \mathcal{S} \setminus \mathcal{G}, a \in \mathcal{A}(s) \text{ (C1)} \\ & V_g = 0 && \forall g \in \mathcal{G} \text{ (C2)} \end{aligned}$	
--	--

Similar to VI, the VLP can be initialised with any vector. An LP solver will make the vector feasible and then improve its quality until it is optimal. VLP's constraints require V to be a monotonic value function, so an admissible value function may not describe a feasible solution, e.g., the following admissible heuristic is not monotonic and therefore infeasible:

$$H(s) = \begin{cases} V^*(s) & \text{if } s = s_I \\ 0 & \text{if } s \neq s_I. \end{cases}$$

So, given a non-monotonic value function, an LP solver solving the VLP will first make it monotonic, and then work towards making the value function optimal.

In contrast to VI, the solution produced by VLP only needs to be optimal for the greedy policy envelope, and not over the whole state-space, reflecting the optimality condition of heuristic-search algorithms. For example, consider fig. 3.4, where an optimal solution for VLP is given in the bottom of each node. This solution induces the optimal policy with $\pi^*(s_I) = a'_0$, and over its policy $\mathcal{S}^{\pi^*} = \{s_I, s_2, s_4, s_g\}$, its values are V^* . However, at s_1 , the solution does not match $V^*(s_1) = 4$.

3.4.3 Occupation Measure LP (OMLP)

The dual LP of VLP is called the OMLP (d'Epenoux, 1963; Trevizan et al., 2016), and is presented in LP 2. Thanks to the strong duality of linear programming, this LP also solves SSPs optimally. Its variables are the occupation measures $\mathbf{x}$ where $x_{s,a}$ represents the expected number of times that s is encountered and then a is applied when following the policy $\pi_{\mathbf{x}}$ induced by $\mathbf{x}$. We consider the VLP as the primal formulation, so the OMLP is the **dual formulation**.

The OMLP can be interpreted as a network flow problem, where the problem of routing probability mass through states and actions turns into a problem of routing flow (e.g., water) through a system of pipes. Under this interpretation, the constraints C3 are called conservation-of-flow constraints because they ensure that the amount of flow entering a non-goal state is equal to the amount of flow exiting it. We use the macros $in(s)$ and $out(s)$ to emphasise this semantic. There is a special case at s_I where a unit of flow is injected, and this represents the unit of probability that must be routed from s_I to the goals. The LP also has non-negativity constraints C6 to ensure that the flow through a state-action pair is non-negative.

A feasible solution $\mathbf{x}$ induces the stochastic policy $\pi_{\mathbf{x}}$ with

$$\pi_{\mathbf{x}}(s, a) = \frac{x_{s,a}}{out(s)} \text{ for } s \in \mathcal{S} \setminus \mathcal{G} \text{ s.t. } out(s) > 0, a \in \mathcal{A}(s).$$

For optimal $\mathbf{x}^*$, we have the following properties:

Theorem 3. (Puterman, 2005, pp. 227–228)

- Suppose $\mathbf{x}^*$ is an optimal solution for the OMLP, then $\pi_{\mathbf{x}^*}$ is an optimal policy.
- Suppose $\mathbf{x}^*$ is an optimal basic solution to the OMLP, then $\pi_{\mathbf{x}^*}$ is an optimal deterministic policy.

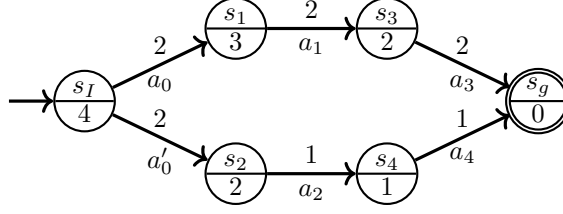


Figure 3.4: An SSP with a VLP solution that encodes an optimal policy, but does not give the optimal value function at s_1 .

Occupation Measure LP (OMLP)

$$\begin{aligned}
 \min_{\mathbf{x}} \quad & \sum_{s \in \mathbf{S}, a \in \mathbf{A}(s)} x_{s,a} \cdot C(a) & (\text{LP 2}) \\
 \text{s.t.} \quad & \text{out}(s) - \text{in}(s) = \mathbb{I}[s = s_I] & \forall s \in \mathbf{S} \setminus \mathbf{G} \quad (\text{C3}) \\
 & \text{out}(s) = \sum_{a \in \mathbf{A}(s)} x_{s,a} & \forall s \in \mathbf{S} \quad (\text{C4}) \\
 & \text{in}(s) = \sum_{s' \in \mathbf{S}, a' \in \mathbf{A}(s')} x_{s',a'} \cdot P(s|s', a') & \forall s \in \mathbf{S} \quad (\text{C5}) \\
 & x_{s,a} \geq 0 & \forall s \in \mathbf{S} \quad (\text{C6})
 \end{aligned}$$

The second point is convenient because we are interested in deterministic policies in the context of SSPs. Recall that the simplex algorithm, which is the algorithm we consider for solving LPs, iterates between different basic feasible solutions, and is guaranteed to output an optimal basic feasible solution, i.e., an optimal basic solution. Thus, solving the OMLP with the simplex method produces the optimal basic solution $\mathbf{x}^*$ which induces an optimal deterministic policy $\pi_{\mathbf{x}^*}$. More generally, for feasible $\mathbf{x}$ we have the following properties:

Theorem 4. (*Puterman, 2005, pp. 224–227*)

- Suppose $\mathbf{x}$ is a feasible solution for the OMLP, then $\pi_{\mathbf{x}}$ is a proper policy.
- Suppose $\mathbf{x}$ is a basic feasible solution for the OMLP, then $\pi_{\mathbf{x}}$ is a proper deterministic policy.

In fact, there is a bijection between the OMLP's feasible solutions and the SSP's proper policies so that the LP's objective is equal to the corresponding policy's expected cost. Thus, any feasible $\mathbf{x}$ induces a proper policy, and any proper policy can be encoded as a feasible solution to the OMLP.

Often, the OMLP also includes the following constraint, called the sink constraint, to emphasise that the entire unit of flow injected into s_I must reach the goals:

$$\sum_{g \in \mathbf{G}} \text{in}(g) = 1 \quad (\text{C7})$$

However, the sink constraint does not need to be specified explicitly for SSPs because the other constraints and the restriction to strictly positive costs $C : \mathbf{S} \rightarrow \mathbb{R}_{>0}$ already force this equality to hold.

Theorem 5. Suppose $\mathbf{x}$ is a feasible solution for the OMLP, then $\sum_{g \in \mathbf{G}} \text{in}(g) = 1$.

This follows from the results of *Puterman (2005)*. In particular, if $\mathbf{x}$ does not satisfy the sink constraint, then $\pi_{\mathbf{x}}$ can not be a proper policy.

3.5 Constraint and Column Generation

Constraint generation is a technique from OR that lets us solve LPs with impractically many constraints, by relaxing some of them, and re-adding constraints as they are needed. Similarly, column generation (a.k.a. variable generation) lets us solve LPs with impractically many variables (columns), again by starting with a subset and adding them as required. These two techniques will be used repeatedly

throughout this work, and are responsible for the connection to heuristic search – note the similarity that heuristic search does not consider all states, but only focuses on those deemed promising by the heuristic function.

We will explain both techniques in terms of the general LP $\min_{\mathbf{x}} \mathbf{c} \cdot \mathbf{x}$ s.t. $\mathbf{Ax} = \mathbf{b}$ which we rewrite as

$$\min_{\mathbf{x}} \sum_{j \in J} c_j x_j \text{ s.t. } \sum_{j \in J} A_{ij} x_j = b_i \quad \forall i \in I \quad (\text{LP } 3)$$

where J describes the set of active variables, i.e., the variables that can be non-zero; and similarly I describes the set of active constraints, i.e., the constraints that need to be satisfied. It is useful to keep in mind the OMLP (LP 2) because it has the same form, and we will apply constraint and column generation to it later. It is equally possible to consider an LP with a maximisation objective, such as the VLP (LP 1), where we will also apply constraint and column generation later.

3.5.1 Constraint Generation

Constraint generation, also known as the cutting plane method, solves LPs with excessively many constraints (Bertsimas and Tsitsiklis, 1997). The original (complete) LP, is called the Master Problem (MP). Constraint generation applied to LP 3 as the MP works in the following steps:

1. Start with an LP that has a subset of constraints $\hat{I} \subset I$, which induces a copy of LP 3 with only the constraints $\sum_{j \in J} A_{ij} x_j = b_i \quad \forall i \in \hat{I}$. This is called a *relaxed LP*, because some of the MP's constraints have been relaxed.
2. Solve the relaxed LP. Assuming that the MP is feasible, then the relaxed LP must also be feasible. Since it is relaxed, we know that the relaxed LP's optimal solution $\bar{\mathbf{x}}$ compared to the MP's optimal solution $\mathbf{x}^*$ must have $\mathbf{c} \cdot \bar{\mathbf{x}} \leq \mathbf{c} \cdot \mathbf{x}^*$, remembering that we have a minimisation objective. However, it is possible that the relaxed LP's solution $\bar{\mathbf{x}}$ does not satisfy all the MP's constraints, that is, $\sum_{j \in J} A_{ij} \bar{x}_j \neq b_i$ for some $i \in I \setminus \hat{I}$, i.e., $\mathbf{A}\bar{\mathbf{x}} \neq \mathbf{b}$.
3. Identify constraints violated by $\bar{\mathbf{x}}$. The mechanism for detecting the MP's constraints that are violated by $\bar{\mathbf{x}}$ is called the *separation oracle*. A naïve separation oracle simply checks whether $\bar{\mathbf{x}}$ satisfies all the MP's constraints, but it is often possible to exploit problem structure and find constraint violations more efficiently (as we do in sec. 4.4). An efficient implementation of constraint generation relies on an efficient separation oracle.
4. If the separation oracle finds violated constraints, then these should be added to the relaxed LP, and then we loop back to step 2. If no violated constraints are found, then we know that $\bar{\mathbf{x}}$ is feasible solution for the MP, and we know it is optimal because $\mathbf{c} \cdot \bar{\mathbf{x}} \leq \mathbf{c} \cdot \mathbf{x}^*$. In this case, we terminate and return $\bar{\mathbf{x}}$. The point is that we add the constraints that are currently violated so that the next relaxed LP will not violate them, and this process is guaranteed to yield a feasible solution for the MP, which is immediately optimal due to the relation of objectives.

Observe that in the worst case, constraint generation has to add every constraint from the MP. In practice, it is common that only a small subset of constraints is needed to describe the MP's optimal solution, and so constraint generation usually pays off by ignoring the unnecessary ones.

3.5.2 Column Generation

Column generation is analogous to constraint generation but addresses LPs with impractically many variables (Desrosiers and Lübbecke, 2005). This technique is also called variable generation, because it generates variables as they are required; the name column generation comes from the fact that each variable corresponds to a column in the constraint matrix A . Applying column generation to LP 3 works in the following steps:

1. Start with a subset of variables $\hat{J} \subset J$, which induces the LP

$$\min_{\mathbf{x}} \sum_{j \in \hat{J}} c_j x_j \text{ s.t. } \sum_{j \in \hat{J}} A_{ij} x_j = b_i \quad \forall i \in I,$$

noting that x_j does not affect the objective nor constraints for $j \in J \setminus \hat{J}$, and should be interpreted as “missing” from the LP. This LP with reduced variables is called the *Reduced Master Problem* (RMP).

2. Solve the RMP. If the RMP's solution is $\bar{\mathbf{x}}$, then we know that $\bar{\mathbf{x}}$ must be a feasible solution for the MP. However, if $\mathbf{x}^*$ is the MP's optimal solution, then we only have $\mathbf{c} \cdot \bar{\mathbf{x}} \geq \mathbf{c} \cdot \mathbf{x}^*$, and it may not be optimal for the MP. Note that this is dual to constraint generation, where the relaxed LP's solution may not be feasible, but has a higher objective. We are essentially doing the opposite of relaxation and adding additional constraints in the RMP (namely, setting some $\mathbf{x}$ to zero) in order to make the problem easier to solve. One inconvenient consequence is that the RMP may be infeasible even if the MP is feasible, which needs to be addressed when implementing column generation.
3. Identify variables that can improve the RMP with the *pricing problem*.
4. If the *pricing problem* returns a variable, then it has the potential to decrease (i.e., improve) the RMP, so it is added to the RMP and we loop to step 2. If the *pricing problem* finds no variables, then this is proof that the RMP's objective can not be improved, i.e., $\bar{\mathbf{x}}$ is an optimal solution for the MP, so we stop and return it.

We give some details about the pricing problem. Suppose we have solved an RMP, giving the solution $\bar{\mathbf{x}}$ and a dual solution $\bar{\mathbf{y}}$. We consider each missing variable x_j 's *reduced cost*, given by $rc(x_j, \bar{\mathbf{y}}) \stackrel{\text{def}}{=} c_j - \bar{\mathbf{y}} \cdot \mathbf{a}_j$, where c_j is x_j 's coefficient in the objective and $\mathbf{a}_j$ is the variable's column in the constraint matrix A . The reduced cost gives a bound on how much x_j can improve the RMP's objective if it is added; so if $rc(x_j, \bar{\mathbf{y}}) < 0$ then x_j has the potential to improve the RMP's objective, and if $rc(x_j, \bar{\mathbf{y}}) \geq 0$ then it can not possibly improve the objective and variable x_j should not be added. Thus, the pricing problem is to find any variables with $rc(x_j, \bar{\mathbf{y}}) < 0$; if such a variable is found it is added to the RMP, and if no such variable is found it is proof that the RMP's solution is optimal for the MP. Generally, the pricing problem is easier to solve than the problem encoded by the LP, e.g., a constrained shortest path problem has an unconstrained shortest path problem as its pricing problem (Desrosiers and Lübbecke, 2005). Recall that the simplex method works by traversing the vertices of the LP's polyhedron, which algebraically means the variables are partitioned into non-basic variables (where the variables are set to zero) and basic variables (where the variables are free), and in each step of the simplex algorithm it determines which non-basic variable should be made basic. This selection is made with the same reduced cost that we use in column generation. The difference is that the simplex algorithm computes the reduced cost for every non-basic variable, and selects the minimiser, whereas column generation determines some negative reduced cost with the pricing problem, which is generally easier to solve and does not require enumerating each basic variable. This gives valuable insight into why column generation is complete (it is essentially an implementation of the simplex algorithm) and into why it is often more effective than solving the MP directly (we do not need to enumerate all basic variables). The performance of column generation relies on an efficient pricing problem.

3.6 Determinisation

A determinisation of an SSP is a relaxed, non-equivalent problem that removes all the SSP's probabilistic effects. Importantly, determinisations are deterministic shortest path problems whose solutions are sequences of actions called plans. Consequently, deterministic shortest path algorithms such as A^* (Hart, Nilsson, and Raphael, 1968) can be used to solve determinisations, which is usually much faster than solving the original problem with probabilities.

The canonical determinisation, which is the one we will be mostly using, relaxes an SSP into a deterministic problem by splitting each action's probabilistic effects into separate deterministic actions:

Definition 22 (All-outcomes Determinisation (Bonet and Geffner, 2003b; Yoon, Fern, and Givan, 2007)). *Given SSP $\mathbb{S} = \langle S, s_I, G, A, P, C \rangle$, its all-outcomes determinisation is $\mathbb{S}_{det} = \langle S, s_I, G, A_{det}, P_{det}, C_{det} \rangle$ with*

- S, s_I, G identical
- $A_{det} = \{a_{s \rightarrow s'} : s \in S, a \in A(s), s' \in \text{succ}(s, a)\}$ with $A_{det}(s) = \{a_{s' \rightarrow s''} \in A_{det} : s' = s\}$
- $P_{det}(\sigma' | \sigma, a_{s \rightarrow s'}) = \begin{cases} 1 & \text{if } \sigma' = s' \text{ and } \sigma = s \\ 0 & \text{else.} \end{cases}$
- $C_{det}(a_{s \rightarrow s'}) = C(a) \forall s \in S, a \in A(s), s' \in \text{succ}(s, a)$.

Importantly, P_{det} is degenerate, i.e., the probabilities are either 0 or 1, and $\mathbb{S}_{det}$ is indeed deterministic.

The all-outcomes determinisation is a proper relaxation in the sense that, if the original SSP $\mathbb{S}$ has a solution (a proper policy), then the determinisation $\mathbb{S}_{\text{det}}$ must also have a solution (a plan). An alternative determinisation is the most-likely-outcomes determinisation, which only produces one deterministic action for each probabilistic one:

Definition 23 (Most-likely Outcomes Determinisation (Yoon, Fern, and Givan, 2007)). *Given SSP $\mathbb{S} = \langle \mathbf{S}, s_I, \mathbf{G}, \mathbf{A}, P, C \rangle$, its most-likely-outcomes determinisation is $\mathbb{S}_{\text{det}} = \langle \mathbf{S}, s_I, \mathbf{G}, \mathbf{A}_{\text{det}}, P_{\text{det}}, C_{\text{det}} \rangle$, defined similarly to the all-outcomes determinisation except $\mathbf{A}_{\text{det}} = \{a_{s \rightarrow \phi(s,a)} : s \in \mathbf{S}, a \in \mathbf{A}(s)\}$ where the state $\phi(s, a) \in \mathbf{S}$ is a maximiser of probability $\phi(s, a) \in \text{argmax}_{s' \in \text{succ}(s,a)} P(s'|s, a)$, breaking ties arbitrarily.*

The most-likely-outcomes determinisation is not a proper relaxation because the removal of less probable outcomes can make the determinisation unsolvable, even if the original SSP was solvable.

CG-iLAO*: Solving SSPs with Constraint Generation

In this chapter, we present the CG-iLAO* algorithm. It is the first optimal heuristic-search algorithm that is able to use the heuristic function to ignore actions, as well as states, enabling it to consider a smaller search space and save Q -value computations. Thanks to these savings, CG-iLAO* outperforms the previous state-of-the-art on our benchmark problems. To derive CG-iLAO*, we start by building on existing connections between planning and OR, and presenting iLAO* (Hansen and Zilberstein, 2001), a state-of-the-art heuristic search algorithm, in terms of LPs with constraint and variable generation. Under this lens, we refine iLAO*'s constraint generation procedure. Bringing this back into a dynamic programming implementation yields CG-iLAO*.

We give a breakdown of this chapter:

- Sec. 4.1 motivates that existing approaches for solving SSPs consider unneeded actions, thereby wasting computation.
- Sec. 4.2 describes iLAO*, a state-of-the-art heuristic-search algorithm for SSPs.
- Sec. 4.3 reframes iLAO* in terms of LPs with constraint and variable generation.
- Sec. 4.4 introduces our novel algorithm CG-iLAO*, building on the insights developed from iLAO*.
- Sec. 4.5 presents some unintuitive properties of CG-iLAO* that distinguish it from other algorithms, and proves CG-iLAO*'s correctness.
- Sec. 4.6 describes different expansions methods that can be used for CG-iLAO*.
- Sec. 4.7 discusses action elimination as an alternative way to cope with unneeded actions.
- Sec. 4.8 describes PEA*, an algorithm for deterministic shortest path problems that uses a similar approach of partial expansion, and then discusses how it relates to CG-iLAO*.
- Sec. 4.9 describes the setup and results of our experiments.
- Sec. 4.10 summarises the chapter.

Parts of this chapter were published in AAAI 2024 (Schmalz and Trevizan, 2024b), and the whole chapter was published at the *Artificial Intelligence* journal (Schmalz and Trevizan, 2026).

4.1 Motivation

The state-of-the-art algorithms for solving SSPs, namely iLAO* (Hansen and Zilberstein, 2001) and LRTDP (Bonet and Geffner, 2003b), build on VI and can be orders of magnitude faster because they use heuristics to guide their search. VI is a blind-search algorithm, which means it considers all states in the problem. In contrast, iLAO* and LRTDP have access to heuristic functions that estimate the optimal costs-to-go for all states, which lets the algorithms focus on promising states with small heuristic values,

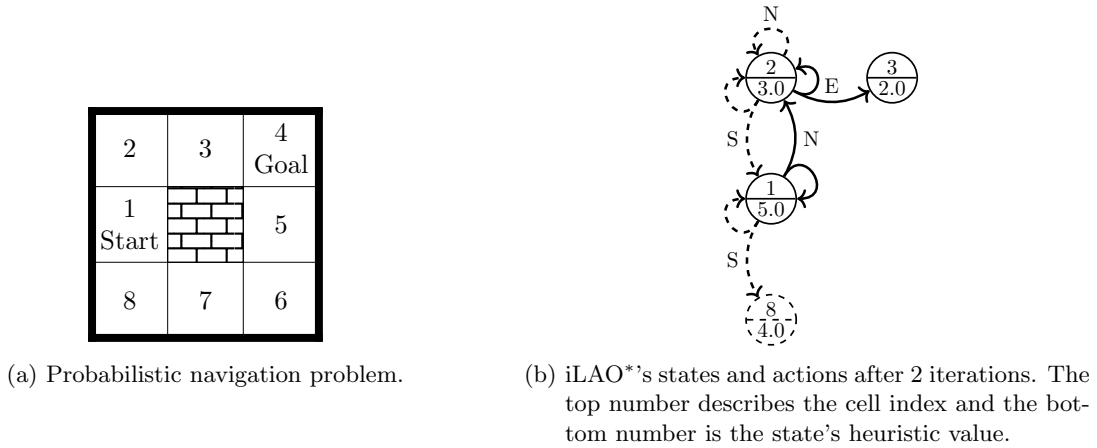


Figure 4.1: A gridworld probabilistic navigation problem and the states and actions that iLAO* considers on this problem after 2 iterations. This example was taken from Hansen and Zilberstein (2001).

and avoid unpromising states with large values. Thus, heuristic-search algorithms apply Bellman backups only to a promising subset of states. In practice, heuristic-search algorithms scale to much larger problems than blind-search algorithms because the promising subset of states is usually orders of magnitude smaller than the total reachable state space.¹ So, these heuristic-search algorithms enjoy a significant performance advantage by restricting the number of states considered, but there is a gap because they do not restrict the actions to be considered. These state-of-the-art algorithms are fundamentally built on Bellman backups, so they always consider all applicable actions for their states, but it is not clear that heuristic-search algorithms need to do this — they can use their heuristics to avoid not only expensive states but also actions that lead to expensive states. This is an important issue because considering unneeded actions is not a one-time expense: in probabilistic planning, states must often be backed up many times before the algorithm converges. In particular, cycles require many backups before converging to the correct cost-to-go. Also, the search may reconsider the same states in many of their iterations, for example, s_1 is backed up in every iteration of LRTDP and iLAO*. Each time the state is backed up we compute all its actions' costs-to-go, thus incurring many unneeded cost-to-go computations.

To make this concept of unneeded actions that waste computation more concrete, consider the gridworld example in fig. 4.1a, which was used as a demonstration by Hansen and Zilberstein (2001). The agent must navigate from the starting cell to the goal cell using the movement actions North (N), East (E), and South (S), each of which, assuming that the destination is unblocked, has a probability 0.5 of moving the agent to the expected cell, and a 0.5 probability of remaining in the same cell. Fig. 4.1b shows which states and actions are considered by iLAO* after 2 iterations.² Observe that there are unneeded actions:

- the heuristic (shown in the bottom of each node) determines that cell 8 is too expensive and it will never be expanded, but nevertheless, the action S from cell 1 needs to be considered and its cost-to-go re-evaluated in each Bellman backup for cell 1; and
- actions such as S and N in cell 2 do not progress the agent and will never have the lowest costs-to-go, but their costs-to-go are still evaluated in each Bellman backup on cell 2.

These actions are not useful for finding the optimal solution, and the heuristic values make this clear — iLAO* is wasting computational resources by considering these actions and re-evaluating their costs-to-go each time the relevant states are backed up.

There is an existing approach for removing such unneeded actions called *action elimination* (Bertsekas, 1995). Action elimination enables an algorithm to prove that an action can never be part of the optimal solution, so that the action can be subsequently permanently removed from search, i.e., eliminated. Action elimination requires a lower bound and an upper bound on an SSP's costs-to-go, and then determines that an action $\tilde{a}$ may be eliminated if the lower bound on $\tilde{a}$'s cost-to-go is larger than the upper bound

¹There are situations where heuristic search is not faster, e.g., if the heuristic gives misleading estimates of the optimal costs-to-go. However, with the heuristics and problems we consider, heuristic search is always significantly faster.

²In the original paper (Hansen and Zilberstein, 2001) this figure is used as a demonstration of LAO*, but in this case LAO* behaves the same as iLAO*.

on another action a 's cost-to-go, i.e., $\text{LB}(\tilde{a}) > \text{UB}(a)$. Domain-independent lower bounds (a.k.a. admissible heuristics) can be automatically computed efficiently; however, we are not aware of any efficient ways to automatically derive informative domain-independent upper bounds. Consequently, action elimination has not found much use in domain-independent planning, and FTVI (Dai, Mausam, and Weld, 2009) is the only algorithm we know that uses action elimination in the context of optimal heuristic planning for SSPs. There are other algorithms that use upper bounds, but they do not explicitly use action elimination to remove actions from their search, e.g., BRTDP (McMahan, Likhachev, and Gordon, 2005), FRTDP (Smith and Simmons, 2006), VPI-RTDP (Sanner et al., 2009), and IBLAO* (Warnquist, Kvarnström, and Doherty, 2010).

So, the state-of-the-art algorithms for SSPs suffer from considering all applicable actions, even when it is clear that some actions are not needed to find the optimal solution, and action elimination does not satisfactorily address this gap. We address this open problem by exploiting connections between planning and operations research, and use the technique of constraint generation to intelligently select promising actions. More concretely: our algorithm initially considers none of the actions, and only adds actions lazily when they are deemed promising by the current costs-to-go, which is determined with theory from constraint generation. This contrasts with action elimination, which initially considers all actions and only removes them when it can prove they are suboptimal. Moreover, action elimination requires lower and upper bounds, whereas we only need lower bounds. Our approach improves upon state-of-the-art algorithms by saving on cost-to-go computations, which action elimination has not been able to do. This is an important step in heuristic search, which lets algorithms use heuristics to select actions as well as states.

4.2 Background: iLAO*

In this section, we explain iLAO* (Hansen and Zilberstein, 2001) in detail since it will be necessary for the insights and definition of CG-iLAO*.

In each step, iLAO* considers a subproblem called a *partial SSP*, which is a copy of the original SSP with a subset of states and actions.³ To define them, it is convenient to use SSPs with terminal costs:

Definition 24 (SSP with Terminal Costs). *An SSP with terminal costs is given by the tuple $\mathbb{S}_T = \langle \mathbb{S}, s_I, \mathbb{G}, \mathbb{A}, P, C, T \rangle$. All terms except T appear in the definition of SSPs, and are defined identically. The new term $T : \mathbb{G} \rightarrow \mathbb{R}_{\geq 0}$ is a terminal-cost function that applies the cost $T(g)$ once, whenever the agent reaches the goal $g \in \mathbb{G}$.*

SSPs with terminal costs are not more expressive than regular SSPs, and they are in fact equivalent: to convert an SSP with terminal costs into a regular SSP, add a new artificial goal state $\bar{g}$, make all old goal states non-goals, and encode the one-time terminal cost for each old goal state g with a deterministic action that leads to $\bar{g}$ with cost $T(g)$. We use SSPs with terminal cost only because they are more convenient for the definition of partial SSPs:

Definition 25 (Partial SSP). *Consider an SSP $\mathbb{S} = \langle \mathbb{S}, s_I, \mathbb{G}, \mathbb{A}, P, C \rangle$ and a heuristic H . A partial SSP for $\mathbb{S}$ with H is an SSP with terminal costs $\hat{\mathbb{S}} = \langle \hat{\mathbb{S}}, s_I, \hat{\mathbb{G}}, \hat{\mathbb{A}}, P, C, H \rangle$ where*

- $\hat{\mathbb{S}}$ and $\hat{\mathbb{A}}$ are subsets of their counterparts in $\mathbb{S}$, i.e., $\hat{\mathbb{S}} \subseteq \mathbb{S}$ and $\hat{\mathbb{A}}(s) \subseteq \mathbb{A}(s) \forall s \in \hat{\mathbb{S}}$,
- s_I is identical to $\mathbb{S}$,
- $\hat{\mathbb{G}}$ is a set of goals that satisfies $\hat{\mathbb{G}} \subseteq \mathbb{G}, \mathbb{G} \cap \hat{\mathbb{S}} \subseteq \hat{\mathbb{G}}$,
- P and C are the same as $\mathbb{S}$ but restricted to $\hat{\mathbb{S}}$ and $\hat{\mathbb{A}}$ (we abuse notation and reuse the same symbols as in $\mathbb{S}$ to emphasise that these functions are otherwise unchanged), and
- H is the terminal cost.

In $\hat{\mathbb{S}}$, whenever the agent reaches an artificial goal $\hat{g} \in \hat{\mathbb{G}}$, it incurs a cost of $H(\hat{g})$ once, and then terminates.

The Bellman equations for partial SSPs are similar to (3.1), but the equations only consider the partial

³Partial SSPs are called explicit graphs in the original paper (Hansen and Zilberstein, 2001).

Algorithm 1: iLAO*

```

1 Function iLAO* ( $\mathbb{S}, H, \epsilon$ )
2    $\widehat{\mathbb{S}} \leftarrow$  partial SSP  $\langle \{s_I\}, s_I, \{s_I\}, \emptyset, P, C, H \rangle$ 
3    $V \leftarrow$  Value Function initialised by  $H$ 
4    $\widehat{\pi}_{\text{curr}} \leftarrow$  candidate policy initialised as undefined everywhere (used to approximate  $\widehat{\pi}_V$ )
5   repeat
6      $\mathcal{E} \leftarrow$  post-order DFS traversal of  $\widehat{\pi}_{\text{curr}}$  from  $s_I$ 
7      $\widehat{\mathbb{S}}, \widehat{\pi}_{\text{curr}} \leftarrow \text{EXPAND-FRINGES}(\mathbb{S}, \widehat{\mathbb{S}}, \widehat{\pi}_{\text{curr}}, \mathcal{E})$ 
8      $F \leftarrow \mathbb{S}^{\widehat{\pi}_{\text{curr}}} \cap (\widehat{\mathbb{G}} \setminus \mathbb{G})$ 
9      $V, \text{RES}, \widehat{\pi}_{\text{curr}}, \widehat{\pi}_{\text{old}} \leftarrow \text{BACKUPS}(\widehat{\mathbb{S}}, \widehat{\pi}_{\text{curr}}, \mathcal{E}, V, F)$ 
10    until  $F = \emptyset$  and  $\widehat{\pi}_{\text{old}} = \widehat{\pi}_{\text{curr}}$  and  $\text{RES} \leq \epsilon$ 
11    return  $V$ 

12 Function EXPAND-FRINGES( $\mathbb{S}, \widehat{\mathbb{S}}, \widehat{\pi}_{\text{curr}}, \mathcal{E}$ )
13    $F \leftarrow \mathcal{E} \cap (\widehat{\mathbb{G}} \setminus \mathbb{G})$ 
14   for  $s_f \in F$  do
15      $\widehat{\mathbb{S}} \leftarrow \text{ADD-ACTIONS}(\mathbb{S}, \widehat{\mathbb{S}}, s_f, \mathbb{A}(s_f))$ 
16      $\widehat{\pi}_{\text{curr}}(s_f) \leftarrow \text{argmin}_{a \in \widehat{\mathbb{A}}(s)} Q(s, a)$ 
17   return  $\widehat{\mathbb{S}}, \widehat{\pi}_{\text{curr}}$ 

18 Function ADD-ACTIONS( $\mathbb{S}, \widehat{\mathbb{S}}, s, \mathbb{A}'$ )
19    $\widehat{\mathbb{G}} \leftarrow \widehat{\mathbb{G}} \setminus \{s_f\}$ 
20    $\widehat{\mathbb{A}}(s) \leftarrow \widehat{\mathbb{A}}(s) \cup \mathbb{A}'$ 
21   for  $a \in \mathbb{A}'$  do
22      $\widehat{\mathbb{G}} \leftarrow \widehat{\mathbb{G}} \cup (\text{succ}(s, a) \setminus \widehat{\mathbb{S}})$ 
23      $\widehat{\mathbb{S}} \leftarrow \widehat{\mathbb{S}} \cup \text{succ}(s, a)$ 
24   return  $\widehat{\mathbb{S}}$ 

25 Function BACKUPS( $\widehat{\mathbb{S}}, \widehat{\pi}_{\text{curr}}, \mathcal{E}, V, F$ )
26    $\widehat{\pi}_{\text{old}} \leftarrow \widehat{\pi}_{\text{curr}}$ 
27   repeat
28      $\text{RES} \leftarrow 0$ 
29     for  $s \in \mathcal{E} \setminus \widehat{\mathbb{G}}$  do
30        $Q_{\min} \leftarrow \min_{a \in \widehat{\mathbb{A}}(s)} Q(s, a)$ 
31        $\text{RES} \leftarrow \max(|V(s) - Q_{\min}|, \text{RES})$ 
32        $V(s) \leftarrow Q_{\min}$ 
33      $\widehat{\pi}_{\text{curr}} \leftarrow \widehat{\pi}_V$ 
34   until  $F \neq \emptyset$  or  $\widehat{\pi}_{\text{curr}} \neq \widehat{\pi}_{\text{old}}$  or  $\text{RES} \leq \epsilon$ 
35   return  $V, \text{RES}, \widehat{\pi}_{\text{curr}}, \widehat{\pi}_{\text{old}}$ 

```

SSP's subset of states and actions, and the goals incur terminal costs:

$$V(s) = \begin{cases} H(s) & \text{if } s \in \widehat{\mathbb{G}} \\ \min_{a \in \widehat{\mathbb{A}}(s)} Q(s, a) & \text{if } s \in \widehat{\mathbb{S}} \setminus \widehat{\mathbb{G}}. \end{cases} \quad (4.1)$$

We assume that $s \in \widehat{\mathbb{S}}, a \in \widehat{\mathbb{A}}(s)$ implies that $\text{succ}(s, a) \subseteq \widehat{\mathbb{S}}$, and therefore $Q(s, a)$ is always defined in (4.1). For a partial SSP $\widehat{\mathbb{S}}$, we will use the following terminology

- $\widehat{\mathbb{G}} \setminus \mathbb{G}$ is called the set of *artificial goals*,
- $\widehat{\mathbb{S}} \setminus \widehat{\mathbb{G}}$ are called *internal states*,
- $\widehat{\mathbb{A}}$ are *internal actions*,
- $\widehat{\pi}_V$ is the greedy policy over V restricted to $\widehat{\mathbb{S}} \setminus \widehat{\mathbb{G}}$, the non-goal states of $\widehat{\mathbb{S}}$.

iLAO* (alg. 1) is an iterative algorithm and in each iteration, it works towards two opposing objectives:

1. **Optimise:** iLAO* tries to find the optimal policy for $\widehat{\mathbb{S}}$ by working towards solving the partial SSP $\widehat{\mathbb{S}}$'s Bellman equations (4.1).
2. **Close:** iLAO* expands any artificial goals reachable by its candidate policy $\widehat{\pi}_{\text{curr}}$ (called *fringe states*) so that eventually $\widehat{\pi}_{\text{curr}}$ has no artificial goals and is closed w.r.t. the original SSP.

To achieve a closed candidate policy (objective 2), iLAO* performs a Depth-First Search (DFS) on $\widehat{\pi}_{\text{curr}}$ to find all fringe states (alg. 1 line 6) and then expands them (alg. 1 line 7). To expand fringe state s , iLAO* adds $A(s)$ to the partial SSP, and any states reachable from s , i.e., $\bigcup_{a \in A(s)} \text{succ}(s, a)$ are added to the partial SSP as new artificial goals, if they are not already in $\widehat{\mathbb{S}}$. Expanding the fringe states in each iteration ensures that the candidate policy $\widehat{\pi}_{\text{curr}}$ will eventually be closed w.r.t. s_I on the original SSP.

Simultaneously, iLAO* works towards finding an optimal policy on its partial SSP (objective 1), by applying Bellman backups to all states in $\widehat{\pi}_{\text{curr}}$'s envelope $\mathcal{E}$ (alg. 1 line 9). The aim is to make V ϵ -consistent in order to show that $\widehat{\pi}_{\text{curr}}$ is optimal (w.r.t. ϵ). There is a subtlety regarding how $\widehat{\pi}_{\text{curr}}$ approximates $\widehat{\pi}_V$: during EXPAND-FRINGES and BACKUPS an update to $\widehat{\pi}_{\text{curr}}$ can cause it to reach internal states from which $\widehat{\pi}_{\text{curr}}$ leads to unaccounted fringe states, and these fringes are not tracked in $\mathcal{E}$. iLAO* recomputes F in alg. 1 line 8, immediately after EXPAND-FRINGES, so there are no unaccounted fringe states there. BACKUPS addresses the issue by recomputing $\widehat{\pi}_{\text{curr}} \leftarrow \widehat{\pi}_V$ after each pass of Bellman backups; recall that the issue only appears if $\widehat{\pi}_{\text{curr}}$ is updated so that it reaches unaccounted fringe states, so the main loop of BACKUPS stops as soon as $\widehat{\pi}_{\text{curr}} \neq \widehat{\pi}_{\text{old}}$, and F will be recomputed in the main loop at alg. 1 line 8. Thus, this implementation of iLAO* handles the issue by ensuring $\widehat{\pi}_V$'s fringe states are in fact tracked in F , and requires the policy to be recomputed in each loop of BACKUPS (alg. 1 line 33). Our algorithm CG-iLAO* deviates from this, as we explain in sec. 4.4.

The objectives of optimising and closing interfere with each other, in the sense that expanding fringe states can break V 's ϵ -consistency and create more work for BACKUPS; and applying backups to the policy envelope can make $\widehat{\pi}_{\text{curr}}$ change in a way that introduces new fringe states that need to be expanded. Arguably, this is what makes iLAO* so efficient, because it is able to abandon working towards objective 1 if it conflicts with objective 2 and avoid redundant work, and vice versa. Once iLAO* achieves both objectives, it has a value function V that is ϵ -consistent w.r.t. the original SSP, since $\widehat{\pi}_{\text{curr}}$ has no artificial goals and $\text{RES} \leq \epsilon$. We require that iLAO* is initialised with an admissible heuristic, and since it only modifies V with Bellman backups it preserves the invariant $V \leq V^*$. Consequently, we can refer to thm. 2, and conclude that iLAO* is optimal.

4.3 iLAO* under the lens of OR

We present a new way to interpret iLAO* in terms of LPs with constraint and variable generation. Recall that SSPs can be solved with the VLP (LP 1), and that *constraint and variable generation* (sec. 3.5) solve large LPs efficiently by starting with a subset of constraints and variables, and adding them as required. The key insight is that iLAO* and its incremental growing of partial SSPs can be reinterpreted as an algorithm that solves LP 1 with *variable and constraint generation*.

The original SSP's VLP (LP 1) is the Master Problem (MP), and iLAO*'s partial SSPs induce LP 4, which are simultaneously Reduced Master Problems (RMPs) and relaxations of LP 1 because they have the subset of variables $\{V_s : s \in \widehat{\mathbb{S}} \setminus \widehat{\mathbb{G}}\}$ and only require monotonicity constraints for actions in $\widehat{\mathbb{A}}$ (C8). Recall that the VLP encodes the Bellman equations (3.1), and its variables V correspond to a value function, so we use V_s and $V(s)$ interchangeably, and write $Q(s, a)$ for the right-hand side of LP's constraint for (s, a) (C1 in LP 1 and C8 in LP 4). In terms of this LP, iLAO* generates new variables for its RMP (partial SSP) by expanding $\widehat{\pi}_{\text{curr}}$'s fringe states s_f , and inserting them as variables V_{s_f} , replacing the previous constant $V_{s_f} = H(s_f)$. Note that this procedure is guided by the heuristic H in the sense that $\widehat{\pi}_{\text{curr}}$ takes H into account. The separation oracle for iLAO*'s constraint generation, i.e., the mechanism for detecting constraints in the MP that are violated, is very simple: whenever a variable V_s is added, all constraints associated with s are immediately added. Adding all constraints trivially ensures that the relaxed LP will not violate any of the MP's constraints, but iLAO* does not get any savings from leaving out non-violated constraints.

Observe that the VLP for partial SSPs includes the constraints C9 for artificial goals, in order to capture the terminal costs. Here, V_g are constants for $g \in \widehat{\mathbb{G}}$, similarly to LP 1's $V_g = 0$ for $g \in \mathbb{G}$. Strictly, due

VLP for Partial SSP

$$\begin{aligned}
& \max_{\mathbf{V}} V_{s_I} & (\text{LP } 4) \\
& \text{s.t. } V_s \leq C(s, a) + \sum_{s' \in S} P(s'|s, a) V_{s'} & \forall s \in \widehat{S} \setminus \widehat{G}, a \in \widehat{A}(s) \quad (\text{C8}) \\
& V_g = H(g) & \forall g \in \widehat{G} \quad (\text{C9})
\end{aligned}$$

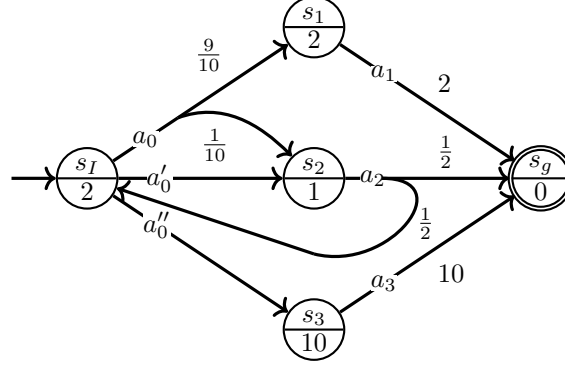


Figure 4.2: Example SSP that we solve with iLAO* under the lens of constraint and variable generation. $H(s)$ are given in the bottom of each node.

to these artificial goal constraints, **LP 4** is not a true relaxation of **LP 1**, since these constraints do not appear in **LP 1**. However, it can still be considered a relaxation because these constraints only affect the “boundary condition” of constants, and since we use an admissible heuristic H , we know that an optimal solution for **LP 4** remains a lower bound on an optimal solution for **LP 1**.

We illustrate iLAO* as a constraint and variable algorithm with a small example. Consider the SSP in fig. 4.2 with the heuristic values shown in the bottom of each node. iLAO*’s iterations will look like this:

Iter. 1 $\widehat{S} = \{s_I\}, \widehat{G} = \{s_I\} - \widehat{\pi}_{\text{curr}} = \{\} - \text{expand } s_I;$

Iter. 2 $\widehat{S} = \{s_I, s_1, s_2, s_3\}, \widehat{G} = \{s_1, s_2, s_3\} - \widehat{\pi}_{\text{curr}} = \{s_I \rightarrow a'_0\} - \text{expand } s_2;$

Iter. 3 $\widehat{S} = \{s_I, s_1, s_2, s_3, s_g\}, \widehat{G} = \{s_1, s_3, s_g\} - \widehat{\pi}_{\text{curr}} = \{s_I \rightarrow a_0\} - \text{expand } s_1;$

Iter. 4 $\widehat{S} = \{s_I, s_1, s_2, s_3, s_g\}, \widehat{G} = \{s_3, s_g\} - \widehat{\pi}_{\text{curr}} = \{s_I \rightarrow a_0, s_1 \rightarrow a_1, s_2 \rightarrow a_2\} - \text{done.}$

We give the VLPs for iLAO*’s partial SSPs at the start of iterations 2–4:

Iter. 2	Iter. 3	Iter. 4
$\max_{V_{s_I}} V_{s_I}$	$\max_{V_{s_I}, V_{s_2}} V_{s_I}$	$\max_{V_{s_I}, V_{s_1}, V_{s_2}} V_{s_I}$
$V_{s_I} \leq Q(s_I, a_0)$	$V_{s_I} \leq Q(s_I, a_0)$	$V_{s_I} \leq Q(s_I, a_0)$
$V_{s_I} \leq Q(s_I, a'_0)$	$V_{s_I} \leq Q(s_I, a'_0)$	$V_{s_I} \leq Q(s_I, a'_0)$
$V_{s_I} \leq Q(s_I, a''_0)$	$V_{s_I} \leq Q(s_I, a''_0)$	$V_{s_I} \leq Q(s_I, a''_0)$
	$V_{s_2} \leq Q(s_2, a_2)$	$V_{s_1} \leq Q(s_1, a_1)$
$V_{s_1} = H(s_1) = 2$	$V_{s_1} = H(s_1) = 2$	$V_{s_2} \leq Q(s_2, a_2)$
$V_{s_2} = H(s_2) = 1$		
$V_{s_3} = H(s_3) = 10$	$V_{s_3} = H(s_3) = 10$	$V_{s_3} = H(s_3) = 10$
	$V_{s_g} = 0$	$V_{s_g} = 0$

For each partial SSP $\widehat{\mathbb{S}}$, its VLP has variables for each internal state $s \in \widehat{\mathbb{S}} \setminus \widehat{\mathbb{G}}$, and the goals $\widehat{\mathbb{G}}$ correspond to constants, e.g., $V_{s_g} = 0$. Thus, when iLAO* expands a state, we generate the corresponding variable for the VLP, as well as the constraints for its outgoing actions. Note that the constants for $s \in \widehat{\mathbb{G}}$ disappear when s gets expanded, and they are replaced with variables.

We point out that the solution to the final partial SSP has

- $V_{s_I} = 3.05$
- $V_{s_1} = 2.0$
- $V_{s_2} = 2.52$.

Importantly, notice that the constraint $V_{s_I} \leq Q(s_I, a_0'')$ is trivially satisfied because $3.05 \leq 11$; i.e., the constraint $V_{s_I} \leq Q(s_I, a_0'')$ is not needed, and corresponds to an unneeded action that could be left out of the partial SSPs because $H(s_3)$ is large and deems s_3 unpromising. This is a consequence of iLAO*'s naïve separation oracle that adds constraints for all outgoing actions as states get expanded.

The interpretation of iLAO* under the lens of constraint and variable generation highlights the shortcoming of Bellman backups that consider all actions because these correspond to a naïve separation oracle that adds all constraints. In the next section, we address this shortcoming, and introduce an efficient separation oracle that only adds violated constraints.

4.4 CG-iLAO*

All previous algorithms based on VI update a state's value function $V(s)$ by considering all applicable actions. In iLAO*, this happens because all its states are either unexpanded and have no available actions $\widehat{\mathbb{A}}(s) = \emptyset$, or fully expanded with all applicable actions available $\widehat{\mathbb{A}}(s) = \mathbb{A}(s)$; iLAO* is not able to partially expand states. We address this shortcoming by allowing states to be partially expanded so that backups in the partial SSP only consider a subset of $\mathbb{A}(s)$. To this end, we first define which actions can be left out of the partial SSP without affecting optimality.

Definition 26 (Inactive Action). *Consider an SSP $\mathbb{S}$, its partial SSP $\widehat{\mathbb{S}}$, a value function V , and a state $s \in \widehat{\mathbb{S}}$. An action $a \in \mathbb{A}(s) \setminus \widehat{\mathbb{A}}(s)$ is inactive in state s if $\min_{\widehat{a} \in \widehat{\mathbb{A}}(s)} Q(s, \widehat{a}) < Q(s, a)$.*

If an action a is inactive for s , then $Q(s, a)$ can not reduce $V(s)$, and we can still obtain V^* if a is ignored by our Bellman backups. Of course a may become active later, but while it is inactive, it can be safely left out of the partial SSP. To understand inactive actions in terms of linear programming, consider the LP 4 associated with $\widehat{\mathbb{S}}$ and let V be a solution for it. Each state-action pair (s, a) corresponds to the consistency constraint C8. An inactive action a for state s does not have an associated constraint in the LP since $a \notin \widehat{\mathbb{A}}(s)$, and importantly, if the constraint were added it would be inactive (also called loose). Therefore, adding the constraint does not affect the solution and only introduces redundant work.

To exploit this insight, we generalise iLAO* so it can ignore inactive actions in each of its partial SSPs and partially expand states, i.e., states may have a non-empty strict subset of applicable actions available in the partial SSP, $\emptyset \neq \widehat{\mathbb{A}}(s) \subsetneq \mathbb{A}(s)$. Then, we introduce a mechanism that efficiently adds actions that are not inactive as they appear. In terms of linear programming, we are using the constraint generation framework to leave unneeded constraints out of the LP, and use a separation oracle to identify constraints that may be needed to encode the optimal solution. This yields a new dynamic programming algorithm, which, in honour of this tight connection to constraint generation, we call Constraint-Generation iLAO* (CG-iLAO*).

CG-iLAO* (alg. 2) is similar to iLAO* (alg. 1), but with the key difference that in its expansion phase (alg. 2 line 8), CG-iLAO* calls PARTLY-EXPAND-FRINGES and expands a state with only the greedy actions w.r.t. V , and not all applicable actions. This introduces two issues that can affect CG-iLAO*'s correctness.

1. Partial expansion may ignore an optimal action if V is inaccurate. This is clearly problematic because CG-iLAO* can not possibly find the optimal policy if its partial SSP does not contain the optimal actions. So, CG-iLAO* requires a mechanism that can detect and add optimal actions if the relevant state's partial expansion missed them.
2. V is evaluated w.r.t. the partial SSP, so if an optimal action a is missing, then it is not taken into account by V (this can cause $V \geq V^*$). If the missing action is later added, then V does not

Algorithm 2: CG-iLAO*

```

1 Function CG-iLAO* ( $\mathbb{S}, H, \epsilon, \eta$ )
2    $\widehat{\mathbb{S}} \leftarrow$  partial SSP  $\langle \{s_I\}, s_I, \{s_I\}, \emptyset, P, C, H \rangle$ 
3    $V \leftarrow$  value function initialised by  $H$ 
4    $\widehat{\pi}_{\text{curr}} \leftarrow$  candidate policy initialised as undefined everywhere (used to approximate  $\widehat{\pi}_V$ )
5    $\Gamma \leftarrow \emptyset$ 
6   repeat
7      $\mathcal{E} \leftarrow$  post-order DFS traversal of  $\widehat{\pi}_{\text{curr}}$  from  $s_I$ 
8      $\widehat{\mathbb{S}}, \widehat{\pi}_{\text{curr}}, \mathcal{E} \leftarrow$  PARTLY-EXPAND-FRINGS( $\mathbb{S}, \widehat{\mathbb{S}}, \widehat{\pi}_{\text{curr}}, \mathcal{E}, V$ )
9      $\mathbf{F} \leftarrow \mathbf{S}^{\widehat{\pi}_{\text{curr}}} \cap (\widehat{\mathbf{G}} \setminus \mathbf{G})$ 
10     $V, \text{RES}, \widehat{\pi}_{\text{curr}}, \widehat{\pi}_{\text{old}}, \Gamma \leftarrow$  CG-BACKUPS( $\mathbb{S}, \widehat{\mathbb{S}}, \widehat{\pi}_{\text{curr}}, \mathcal{E}, V, \mathbf{F}, \Gamma, \epsilon, \eta$ )
11     $V, \text{RES}, \Gamma, \widehat{\mathbb{S}} \leftarrow$  FIX-CONSTRS( $\mathbb{S}, \widehat{\mathbb{S}}, \widehat{\pi}_{\text{curr}}, V, \Gamma, \text{RES}, \epsilon$ )
12  until  $\mathbf{F} = \emptyset$  and  $\widehat{\pi}_{\text{old}} = \widehat{\pi}_{\text{curr}}$  and  $\text{RES} \leq \epsilon$ 
13  return  $V$ 

14 Function PARTLY-EXPAND-FRINGS( $\mathbb{S}, \widehat{\mathbb{S}}, \widehat{\pi}_{\text{curr}}, \mathcal{E}, V$ )
15  while  $\exists s_f \in \mathcal{E} \cap (\widehat{\mathbf{G}} \setminus \mathbf{G})$  do
16     $\mathbf{A}' \leftarrow \{a \in \mathbf{A}(s_f) : Q(s_f, a) = \min_{a' \in \mathbf{A}(s_f)} Q(s_f, a')\}$  /* Select greedy actions
17     $\text{argmin}_{a \in \mathbf{A}} Q(s, a)$  */
18     $\widehat{\mathbb{S}} \leftarrow$  ADD-ACTIONS( $\mathbb{S}, \widehat{\mathbb{S}}, s_f, \mathbf{A}'$ ) /* Note:  $s_f \notin \widehat{\mathbf{G}}$  after ADD-ACTIONS */
19     $\widehat{\pi}_{\text{curr}}(s_f) \leftarrow$  some greedy action  $a \in \widehat{\mathbf{A}}(s)$ 
20    for  $s_{\text{int}} \in \text{succ}(s_f, \widehat{\pi}_{\text{curr}}(s_f)) \setminus \widehat{\mathbf{G}}$  do
21       $\mathcal{E} \leftarrow \mathcal{E}$  updated with post-order DFS traversal of  $\widehat{\pi}_{\text{curr}}$  from  $s_{\text{int}}$  (no duplicate states)
22  return  $\widehat{\mathbb{S}}, \widehat{\pi}_{\text{curr}}, \mathcal{E}$ 

23 Function CG-BACKUPS( $\mathbb{S}, \widehat{\mathbb{S}}, \widehat{\pi}_{\text{curr}}, \mathcal{E}, V, \mathbf{F}, \Gamma, \epsilon, \eta$ )
24   $\widehat{\pi}_{\text{old}} \leftarrow \widehat{\pi}_{\text{curr}}$ 
25  repeat
26     $\text{RES} \leftarrow 0$ 
27    for  $s \in \mathcal{E} \setminus \widehat{\mathbf{G}}$  do
28       $Q_{\min} \leftarrow \min_{a \in \widehat{\mathbf{A}}(s)} Q(s, a)$ 
29      if  $Q_{\min} - V(s) > \eta$  then
30         $\Gamma \leftarrow \Gamma \cup \text{EXT-SUCCS}(s, \mathbb{S}, \widehat{\mathbb{S}})$ 
31      else if  $V(s) - Q_{\min} > \eta$  then
32         $\Gamma \leftarrow \Gamma \cup \text{PREDS}(s, \mathbb{S}, \widehat{\mathbb{S}})$ 
33       $\text{RES} \leftarrow \max(|V(s) - Q_{\min}|, \text{RES})$ 
34       $V(s) \leftarrow Q_{\min}$ 
35       $\widehat{\pi}_{\text{curr}}(s) \leftarrow$  some greedy action  $a \in \widehat{\mathbf{A}}(s)$  s.t.  $Q(s, a) = Q_{\min}$ 
36  until  $\mathbf{F} \neq \emptyset$  or  $\widehat{\pi}_{\text{curr}} \neq \widehat{\pi}_{\text{old}}$  or  $\text{RES} \leq \epsilon$ 
37  return  $V, \text{RES}, \widehat{\pi}_{\text{curr}}, \widehat{\pi}_{\text{old}}, \Gamma$ 

38 Function FIX-CONSTRS( $\mathbb{S}, \widehat{\mathbb{S}}, \widehat{\pi}_{\text{curr}}, V, \Gamma, \text{RES}, \epsilon$ )
39   $\Gamma' \leftarrow \emptyset$ 
40  for  $(s, a) \in \Gamma$  s.t.  $V(s) > Q(s, a) + \epsilon$  do
41    if  $a \notin \widehat{\mathbf{A}}(s)$  then
42       $\widehat{\mathbf{A}}(s) \leftarrow \widehat{\mathbf{A}}(s) \cup \{a\}$ 
43       $\widehat{\mathbf{G}} \leftarrow \widehat{\mathbf{G}} \cup (\text{succ}(s, a) \setminus \widehat{\mathbf{S}})$ 
44       $\widehat{\mathbf{S}} \leftarrow \widehat{\mathbf{S}} \cup \text{succ}(s, a)$ 
45     $\text{RES} \leftarrow \max(V(s) - Q(s, a), \text{RES})$ 
46     $V(s) \leftarrow Q(s, a)$ 
47     $\widehat{\pi}_{\text{curr}}(s) \leftarrow a$ 
48     $\Gamma' \leftarrow \Gamma' \cup \text{PREDS}(s, \mathbb{S}, \widehat{\mathbb{S}})$ 
49  return  $V, \text{RES}, \Gamma', \widehat{\mathbb{S}}$ 

```

automatically reflect that a has been made available, so we must ensure that a 's improvements to V are fully propagated.

Elegantly, it turns out that both of these problems are instances of constraint violation, and can therefore be addressed by detecting constraint violations with a separation oracle and then enforcing that the constraints become satisfied.

How can we find constraint violations efficiently? A naïve separation oracle considers all constraints missing in the partial SSP, i.e., the constraints for $s \in \widehat{S}$, $a \in A(s) \setminus \widehat{A}(s)$, and evaluates whether they are violated by checking if $V(s) > Q(s, a)$. This means we compute Q -values for each external action and therefore save no work compared to adding them to the partial SSP in the first place. The key realisation that lets us save Q -value computations is that non-violated constraints remain non-violated unless V is updated in a particular way. Our efficient separation oracle exploits this by computing a set of constraints that might be violated, and then checking violations only in this set. Suppose there are currently no constraint violations and $V(s)$ is assigned $\min_{a \in \widehat{A}(s)} Q(s, a)$. Then, the separation oracle determines that the constraints associated with s may become violated according to the following three cases:

1. **$V(s)$ stays the same.** There can be no new constraint violations.
2. **$V(s)$ increases.** The constraints $V(s) \leq Q(s, a')$ may be violated for external successors $(s, a') \in \text{EXT-SUCCS}(s, \widehat{S}, \widehat{S})$ where

$$\text{EXT-SUCCS}(s, \widehat{S}, \widehat{S}) \stackrel{\text{def}}{=} \{(s, a) : a \in A(s) \setminus \widehat{A}(s)\}.$$

Note that we only have to consider external successor actions, and not internal ones (s, a') with $a' \in \widehat{A}(s)$, because $V(s) \leftarrow \min_{a \in \widehat{A}(s)} Q(s, a) \leq Q(s, a')$, and therefore the constraint associated with (s, a') must be satisfied already.

3. **$V(s)$ decreases.** The only constraints that may be violated are $V(s') \leq Q(s', a')$ for predecessors $(s', a') \in \text{PREDS}(s, \widehat{S}, \widehat{S})$ where

$$\text{PREDS}(s, \widehat{S}, \widehat{S}) \stackrel{\text{def}}{=} \{(s', a') : s' \in \widehat{S}, a' \in A(s'), s \in \text{succ}(s', a')\}.$$

In this case, we must consider internal actions as well as external ones. However, we do not consider external predecessor states $s' \in \widehat{S} \setminus \widehat{S}$, because such states will eventually be expanded and handled that way if their constraint violations persist.

In the pseudocode, these checks are performed whenever V is updated, and all potential violations are stored in Γ :

- if $V(s)$ increases then $\Gamma \leftarrow \Gamma \cup \text{EXT-SUCCS}(s, \widehat{S}, \widehat{S})$ (alg. 2 line 29);
- if $V(s)$ decreases, then $\Gamma \leftarrow \Gamma \cup \text{PREDS}(s, \widehat{S}, \widehat{S})$ (alg. 2 line 31 and line 47).

At the end of each iteration of its main loop, CG-iLAO* calls FIX-CONSTRS to check all potentially violated constraints Γ . If a constraint in Γ is not violated, it is simply discarded. If a constraint (s, a) is indeed violated, then FIX-CONSTRS fixes it by setting $V(s) \leftarrow Q(s, a)$ (alg. 2 line 45), and then removes (s, a) from Γ . Importantly, this update to V may create new constraint violations, so we must apply the same rules as before and track any new potential constraint violations in Γ for the next iteration. This ensures that Γ is always a superset of CG-iLAO*'s constraint violations, and the algorithm only terminates once they have all been addressed.

A subtle technical difference that follows from this construction is that CG-iLAO* requires the additional error parameter $\eta \in \mathbb{R}_{>0}$ that is used to decide whether a state's value function has changed sufficiently to warrant updating Γ . For now, it is convenient to use $\eta = \epsilon$, and we explain the effects of this parameter in page 38.

There are also two subtle differences in the implementations of iLAO* and CG-iLAO* as presented in alg. 1 and alg. 2, which will turn out to be significant in our experiments (sec. 4.9). These changes are orthogonal to CG-iLAO*'s partial expansion mechanism, so it is possible to define four similar algorithms: (i) full expansion with implementation 1, (ii) full expansion with implementation 2, (iii) partial expansion with implementation 1, (iv) partial expansion with implementation 2. In previous work we only considered

(i) and (iv) (Schmalz and Trevizan, 2024b), which is problematic for comparing the effect of using partial expansions versus full expansions. We address this by comparing (ii) with (iv), and we also include (i) as a way to compare to the previous work. The two subtle implementation differences are:

1. PARTLY-EXPAND-FRINGS deviates from EXPAND-FRINGS in the case where the expansion adds an internal state $s \in \hat{S}$ where $\hat{\pi}_{\text{curr}}$ is defined: iLAO* stops there and does not update $\mathcal{E}$; CG-iLAO* follows $\hat{\pi}_{\text{curr}}$ from s , updating $\mathcal{E}$, and expands any new fringe states that are encountered this way. Our pseudocode does this by updating $\mathcal{E}$ during the `while` loop, and in practice it is implemented with recursion. iLAO* (alg. 1)’s implementation follows Hansen and Zilberstein (2001), whereas CG-iLAO* (alg. 2)’s implementation follows the recursive definition from Hansen and Abdoulahi (2016). Thus, per call, CG-iLAO*’s PARTLY-EXPAND-FRINGS can potentially expand more states than iLAO*’s EXPAND-FRINGS if it keeps expanding states that lead to internal states where $\hat{\pi}_{\text{curr}}$ is defined.
2. The second change concerns how $\hat{\pi}_{\text{curr}}$ is tracked, and how to handle its deviation from $\hat{\pi}_V$. Recall from sec. 4.2, that iLAO*’s EXPAND-FRINGS or BACKUPS may encounter an internal state whose policy leads to a fringe state that is not accounted for in $\mathcal{E}$. There, it is handled by making sure $\hat{\pi}_{\text{curr}} = \hat{\pi}_V$ during BACKUPS. CG-iLAO* embraces the property that $\hat{\pi}_{\text{curr}}$ approximates $\hat{\pi}_V$, and only updates $\hat{\pi}_{\text{curr}}$ at states s where $V(s)$ changes (alg. 2 line 34 and line 46). This is more efficient, but can have the effect that the inaccurate $\hat{\pi}_{\text{curr}}$ has no fringe states, even though $\hat{\pi}_V$ does. This is not an issue because in such situations $\hat{\pi}_{\text{curr}}$ must be updated at some point, setting $\hat{\pi}_{\text{curr}} \neq \hat{\pi}_{\text{old}}$, and thereby ensuring that CG-iLAO* does not terminate until it has indeed closed all fringes. Thus, CG-iLAO* saves the overhead of computing $\hat{\pi}_{\text{curr}}$ explicitly, but may require additional iterations to discover that its greedy policy still contains fringes.

We will see in our experiments that these differences can have a significant performance impact.

In summary, CG-iLAO* generalises iLAO* by allowing actions to be left out of its partial SSPs. CG-iLAO* ignores inactive actions and only adds actions as they are required. Under the lens of linear programming, the inactive actions correspond to inactive constraints, and CG-iLAO* refines iLAO* by making the separation oracle more selective, so that only constraints that are actively violated are added to the partial SSP. We note that CG-iLAO*’s separation oracle incurs an overhead of Q -value computations, since it must determine whether previously non-violated constraints have become violated. In our experiments (sec. 4.9), we show that the Q -value savings from ignoring expensive actions outweigh the separation oracle’s Q -value overhead, and CG-iLAO* reliably computes fewer Q -values than the state-of-the-art.

4.5 Properties of CG-iLAO* and Proof of Correctness

In this section, we show some properties of CG-iLAO* that distinguish it from previous approaches. Then, we prove that CG-iLAO* is correct.

CG-iLAO* is not monotonic. In all previous algorithms based on Bellman backups, such as LRTDP and iLAO*, if they are initialised with a monotonic heuristic, their backups are guaranteed to be monotonically non-decreasing, i.e., $V^{t+1} \geq V^t$ for all timesteps t . CG-iLAO* does not have this guarantee because it ignores inactive actions and may evaluate a suboptimal policy before recognising and inserting the missing optimal action, which decreases the value function. As an example of this, consider the SSP in fig. 4.3 where $H(s)$ is given in the bottom of the respective node. H is monotonic, but as we step through the iterations of CG-iLAO*, a cost decrease occurs:

Iter. 1 Expands s_I .

Iter. 2 Partly expands s_1 with a_1 .

Iter. 3 Expands s_2 with a_2 and, after CG-BACKUPS, we have $V(s_2) = 3$, $V(s_1) = 4$, $V(s_0) = 5$ and $\Gamma = \{(s_1, a'_1)\}$. Since $\Gamma \neq \emptyset$, FIX-CONSTRS verifies that (s_1, a'_1) is currently better than the existing action a_1 for s_1 , so a'_1 is added to $\hat{A}(s_1)$ and $V(s_1)$ is changed from 4 to 3. Recall that $V(s_0) = 5$, so when $V(s_1)$ is updated to 3, $\{s_0, a_0\}$ is inserted into Γ , and no further changes are made in this iteration.

Iter. 4 Expands s_3 and CG-BACKUPS reduces $V(s_I)$ from 5 to 4, so V has been decreased by CG-BACKUPS.

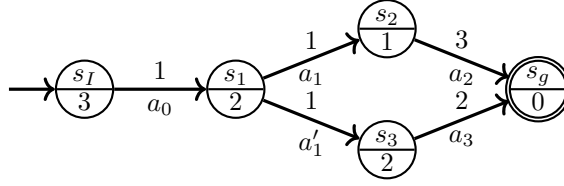


Figure 4.3: An SSP equipped with a monotonic heuristic where CG-iLAO*'s value function is not monotonically non-decreasing.

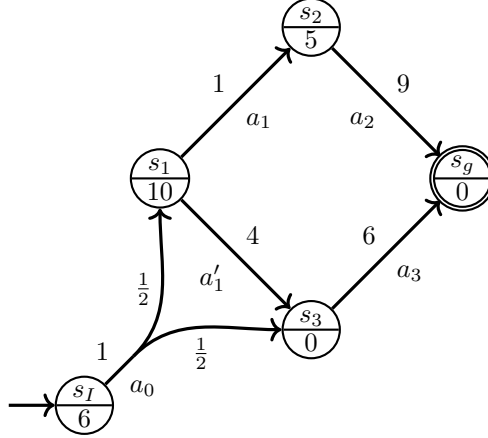


Figure 4.4: An SSP where CG-iLAO* does not return an ϵ -consistent w.r.t. the whole SSP.

CG-iLAO* is ϵ -consistent on $\hat{\mathbb{S}}$, not $\mathbb{S}$. Recall the definition of ϵ -consistency as given in defn. 18. For FIND-AND-REVISE algorithms, the output V is ϵ -consistent w.r.t. the original SSP. This is not the case for CG-iLAO*: it guarantees ϵ -consistency over its partial SSP by construction, but may produce a solution that is not ϵ -consistent w.r.t. $\mathbb{S}$. This is not an issue and does not prevent CG-iLAO* from finding optimal solutions, as we explain later, but it is important to understand. Consider CG-iLAO*'s steps on the pathological SSP in fig. 4.4:

Iter. 1 Expands s_I .

Iter. 2 Expands s_3 and partly expands s_1 with a'_1 since $Q(s_1, a'_1) = 4$ and $Q(s_1, a_1) = 6$. After CG-BACKUPS, $V(s_3) = 6, V(s_1) = 10, V(s_0) = 9$. Importantly, $V(s_1)$ did not increase, so its successors are not added to Γ .

Iter. 3 There are no fringes left to expand, so CG-iLAO* does one pass of BACKUPS which leaves V unchanged and exits with $\text{RES} = 0$ and $\Gamma = \emptyset$. The algorithm terminates.

The algorithm has terminated, but $V(s_1) - Q(s_1, a_1) = 4 > \epsilon$, so V is not ϵ -consistent. Nevertheless, the algorithm has found an optimal policy. The upshot is that we must be careful and specify the SSP for which the algorithm is ϵ -consistent. This is analogous to FIND-AND-REVISE algorithms not being globally ϵ -consistent, and requiring that the greedy policy is specified.

Thus, CG-iLAO* has different behaviours to previous algorithms based on Bellman backups. Consequently, we can not immediately apply previous proof techniques, and we must be careful with our definition of optimality. We take a moment to formalise the notion of an optimal algorithm.

Definition 27 (Optimal Algorithm). *An algorithm parameterised by the error term $\epsilon \in \mathbb{R}_{>0}$ is called optimal if, as $\epsilon \rightarrow 0$, it outputs an optimal policy.*

Observe that this definition was used implicitly before, and we are just making it explicit now. Thm. 1 and thm. 2 demonstrate that VI and FIND-AND-REVISE algorithms are optimal, respectively.

It turns out that ϵ -consistency over partial SSPs $\hat{\mathbb{S}}$ is sufficient to prove optimality over the original SSP $\mathbb{S}$, provided that V is admissible w.r.t. $\mathbb{S}$, and the greedy policy $\hat{\pi}_V$ does not terminate in one of $\hat{\mathbb{S}}$'s artificial goal states $\hat{\mathbb{G}} \setminus \mathbb{G}$. The first condition is inherited from regular ϵ -consistency, and the second condition ensures that the greedy policy $\hat{\pi}_V$ induces a closed policy for $\mathbb{S}$. To formalise and prove this claim, we start by introducing an upper bound on the expected number of steps to reach the goal.

Definition 28. Let $\mathbb{S}_{G'}$ denote an SSP that is identical to $\mathbb{S}$ but has different goal states $G' \subseteq S$, and let $N(s, \pi, \mathbb{S}_{G'})$ denote the expected number of steps to reach the goals G' by following the proper policy π from s . Then, $\bar{N}(s, \mathbb{S})$ is the smallest number that satisfies $\bar{N}(s, \mathbb{S}) \geq N(s, \pi, \mathbb{S}_{G'})$ for all $\mathbb{S}_{G'}$ and all policies π that are proper for $\mathbb{S}_{G'}$ from s , omitting any $\mathbb{S}_{G'}$ that have no proper policies from s . Importantly, for a fixed $\mathbb{S}$, the number of ways to pick $\mathbb{S}_{G'}$ is finite, and for each $\mathbb{S}_{G'}$ the number of proper policies from s is also finite. Therefore, we can indeed pick $\bar{N}(s, \mathbb{S})$ to be the minimal number that satisfies our constraints, so it is well-defined and finite.

This term will be useful because it lets us use the same term over many variants of the SSP $\mathbb{S}$. Without it, we would have a different $N(s, V, \mathbb{S}')$ for each SSP $\mathbb{S}'$, and we would have to be careful how they relate. The fact that $\bar{N}(s, \mathbb{S})$ can get large is unimportant for our proofs as long as $\epsilon \bar{N}(s, \mathbb{S}) \rightarrow 0$ as $\epsilon \rightarrow 0$. It is possible to sharpen our proofs with tighter upper bounds, but this does not contribute towards proving that CG-iLAO* is correct.

We are now equipped to formalise and prove our claim that ϵ -consistency over partial SSPs $\hat{\mathbb{S}}$ is sufficient to prove optimality over the original SSP $\mathbb{S}$, provided that V is admissible w.r.t. $\mathbb{S}$, and the greedy policy $\hat{\pi}_V$ does not terminate in one of $\mathbb{S}$'s artificial goal states $\hat{G} \setminus G$. Note that we weaken the requirement of $V \leq V^*$, which will be useful for later proofs.

Theorem 6. Consider an algorithm that for each ϵ outputs $\hat{\mathbb{S}}$ and V such that

- V is ϵ -consistent w.r.t. $\hat{\mathbb{S}}$,
- V satisfies $V(s) \leq V^*(s) + \epsilon \bar{N}(s, \mathbb{S}) \forall s \in \mathcal{S}^{\hat{\pi}_V}$, and
- $\hat{\pi}_V$ does not reach any artificial goal states, i.e., $\mathcal{S}^{\hat{\pi}_V} \cap (\hat{G} \setminus G) = \emptyset$.

As $\epsilon \rightarrow 0$, the greedy policy $\hat{\pi}_V$ becomes an optimal policy, i.e., the algorithm is optimal.

Proof. First, ϵ -consistency over $\hat{\mathbb{S}}$ implies that $\hat{\pi}_V$ is accurately evaluated by V , i.e.,

$$|V(s) - V_{\hat{\pi}_V}(s)| \leq \epsilon \bar{N}(s, \mathbb{S}) \forall s \in \mathcal{S}^{\hat{\pi}_V}.$$

To get this bound, we construct a copy of $\mathbb{S}$ called $\mathbb{S}'$ with the actions changed thus: $A'(s) = \{\pi_V(s)\} \forall s \in S \setminus G$. The optimal value function of $\mathbb{S}'$ is $V_{\hat{\pi}_V}$ (defined over $\mathcal{S}^{\hat{\pi}_V}$), where $V_{\hat{\pi}_V}(s)$ is the cost-to-go of $\hat{\pi}_V$ from s . Our ϵ -consistent V must be globally ϵ -consistent for $\mathbb{S}'$, and we can apply thm. 1 to obtain $|V(s) - V_{\hat{\pi}_V}(s)| \leq \epsilon N(s, V, \mathbb{S}') \forall s \in \mathcal{S}^{\hat{\pi}_V}$. We get the desired bound by observing that $N(s, V, \mathbb{S}') \leq \bar{N}(s, \mathbb{S})$ for all $s \in \mathcal{S}^{\hat{\pi}_V}$ by defn. 28.

Second, we have $V_{\hat{\pi}_V}(s) \geq V^*(s) \forall s \in \mathcal{S}^{\hat{\pi}_V}$ because $\hat{\pi}_V$ is a closed policy for $\mathbb{S}$ which can not be cheaper than an optimal one. We combine this with the previous observation to get

$$V(s) + \epsilon \bar{N}(s, \mathbb{S}') \geq V_{\hat{\pi}_V}(s) \geq V^*(s) \forall s \in \mathcal{S}^{\hat{\pi}_V}$$

which can be rearranged into

$$V(s) \geq V^*(s) - \epsilon \bar{N}(s, \mathbb{S}) \forall s \in \mathcal{S}^{\hat{\pi}_V}.$$

On the other hand, we have assumed the upper bound $V(s) \leq V^*(s) + \epsilon \bar{N}(s, \mathbb{S}) \forall s \in \mathcal{S}^{\hat{\pi}_V}$, which lets us squeeze V thus:

$$V^*(s) - \epsilon \bar{N}(s, \mathbb{S}) \leq V(s) \leq V^*(s) + \epsilon \bar{N}(s, \mathbb{S}) \forall s \in \mathcal{S}^{\hat{\pi}_V}.$$

As $\epsilon \rightarrow 0$, the errors disappear, so $V(s) = V^*(s) \forall s \in \mathcal{S}^{\hat{\pi}_V}$ and $V(s) \leq V^*(s) \forall s \in S$. Thus, the greedy policy $\hat{\pi}_V$ is greedy w.r.t. V^* since $V(s) = V^*(s)$ over $\hat{\pi}_V$'s envelope, i.e., $\hat{\pi}_V$ must be an optimal policy. $\square$

Thus, thm. 6 lets us relax the assumption that algorithms must be ϵ -consistent w.r.t. the original SSP, and shows that ϵ -consistency w.r.t. partial SSPs still provides optimality guarantees, provided that some additional requirements are satisfied. We briefly point out that ϵ -consistency w.r.t. partial SSPs can be relaxed even further by specifying a closed policy π , and inferring a partial SSP from π 's envelope. Algorithms that are in this sense ϵ -consistent w.r.t. a specified policy are still optimal under similar assumptions to ϵ -consistency w.r.t. partial SSPs by similar arguments. The notion of ϵ -consistency w.r.t. partial SSPs accommodates CG-iLAO*, and equips us to prove that CG-iLAO* is sound and complete, i.e., it always returns some solution and it is optimal. We start by showing that CG-iLAO* always terminates:

Lemma 1. PARTLY-EXPAND-FRINGS *terminates*.

Proof. In each pass of PARTLY-EXPAND-FRINGS's main loop (alg. 2 lines 15–20), a new state is removed from the artificial goals $\widehat{G} \setminus G$ and added as an internal state. But the SSP S has finitely-many states, so eventually PARTLY-EXPAND-FRINGS must run out of new states to add, and will therefore exit its main loop and terminate. $\square$

Lemma 2. CG-BACKUPS *terminates*.

Proof. The main loop of CG-BACKUPS (alg. 2 lines 24–35) runs VI on $\mathcal{E} \setminus \widehat{G}$ until ϵ -consistency, with bookkeeping (alg. 2 lines 28–31) that does not affect VI. Given that $\widehat{S}$ and $\mathcal{E}$ are fixed, this VI eventually reduces the residual to $\text{RES} \leq \epsilon$, so CG-BACKUPS exits the main loop and terminates. Note that the other conditions for termination $F \neq \emptyset$ or $\widehat{\pi}_{\text{curr}} \neq \widehat{\pi}_{\text{old}}$ may cause CG-BACKUPS to terminate sooner. $\square$

Theorem 7. CG-iLAO* *terminates*.

Proof. For contradiction, suppose CG-iLAO* does not terminate. By lem. 1 and lem. 2 we know that the functions that CG-iLAO* calls must terminate, so if CG-iLAO* does not terminate, it must be because it gets stuck in its main loop (alg. 2 lines 6–12). Suppose that $F \neq \emptyset$ for infinitely-many steps. In each pass of the main loop where $F \neq \emptyset$, PARTLY-EXPAND-FRINGS changes at least one of $\widehat{\pi}_{\text{curr}}$'s fringe states to an internal state. But this is impossible, because S only has finitely-many states and we only allow fringe states to be expanded once, so CG-iLAO* must eventually run out of states to partially expand, and after finitely-many steps F remains empty. Moreover, after finitely-many steps $\widehat{S}$ remains constant by a similar argument, since S only has finitely-many actions that can be added to $\widehat{S}$. Then there must be a finite set of states $X \subseteq \widehat{S}$ that are updated with Bellman backups infinitely often by CG-BACKUPS or FIX-CONSTRS. But X induces a new partial SSP, and applying Bellman backups infinitely often to all X solves this new partial SSP with VI. Consequently, V converges to a fixed point and the residual will be less than ϵ in finite time. Since V will not be updated, $\widehat{\pi}_{\text{curr}}$ will also no longer be updated. At this point all the termination conditions are satisfied (alg. 2 line 12), forcing CG-iLAO* to terminate, giving us the desired contradiction. $\square$

We now work towards proving that CG-iLAO* is optimal by showing that it satisfies ϵ -consistency w.r.t. its partial SSPs, and that it satisfies the extra conditions required by thm. 6. To help with this, we prove some simpler properties of CG-iLAO* first:

Lemma 3. *At the end of CG-iLAO*'s main loop (alg. 2 after line 11), if there is $s \in \widehat{S} \setminus \widehat{G}$ and $a \in A(s)$ such that $V(s) > Q(s, a) + \epsilon$, then $(s, a) \in \Gamma$ or $V(s) \leq V^*(s)$.*

Proof. We prove by induction over n , the number of iterations of CG-iLAO*. The base case is $n = 0$, i.e., the partial SSP $\widehat{S}$ has been initialised and never updated, so $\widehat{S} \setminus \widehat{G} = \emptyset$, making the claim is vacuously true. Our inductive hypothesis is that the lemma holds after n iterations, and we now prove that this makes the lemma hold after $n + 1$ iterations. We step through the possible ways that the lemma's precondition can be made true, and how in those cases the algorithm ensures that the relevant column is in Γ :

1. If $V(s) > Q(s, a) + \epsilon$ at the start of iteration $n + 1$, then by the inductive hypothesis, $(s, a) \in \Gamma$ or $V(s) \leq V^*(s)$. So, we only concern ourselves with any constraint violations that are introduced in this iteration.
2. The post-order DFS traversal (alg. 2 line 7) does not affect $\widehat{S} \setminus \widehat{G}$ nor V , and therefore can not introduce any new constraint violations.
3. PARTLY-EXPAND-FRINGS (alg. 2 line 8) does not affect V , but may introduce new states into $s \in \widehat{S} \setminus \widehat{G}$. It expands fringe states s in $S^{\widehat{\pi}_{\text{curr}}}$ by bringing s into $\widehat{S} \setminus \widehat{G}$, adding some action $\widehat{a} \in A(s)$ into $\widehat{A}(s)$, and setting $\widehat{\pi}_{\text{curr}}(s) \leftarrow \widehat{a}$. This may introduce constraint violations but s was until now a fringe state, so $V(s) = H(s) \leq V^*(s)$.
4. In CG-BACKUPS (alg. 2 line 10), a new $V(s) > Q(s, a) + \epsilon$ may occur if $V(s)$ is increased or $Q(s, a)$ is decreased. CG-BACKUPS tracks the appropriate EXT-SUCCS or PREDS for V increases and decreases respectively.
5. FIX-CONSTRS (alg. 2 line 11) ensures for each $(s, a) \in \Gamma$ that $V(s) \leq Q(s, a) + \epsilon$ before removing (s, a) from Γ . $V(s)$ can not increase in FIX-CONSTRS, since we are setting $V(s) \leftarrow \min\{V(s), Q(s, a)\}$ for $(s, a) \in \Gamma$. However, new instances of $V(s) > Q(s, a) + \epsilon$ can be introduced if some $Q(s, a)$ is decreased, which is tracked by FIX-CONSTRS.

So indeed, any constraint violations $V(s) > Q(s, a) + \epsilon$ introduced by CG-iLAO* in lines 7–11 are tracked in Γ or $V(s) \leq V^*(s)$. $\square$

Importantly, this lemma applies only to internal states $s \in \widehat{\mathbf{S}} \setminus \widehat{\mathbf{G}}$, but in terms of actions applies to all applicable actions $a \in \mathbf{A}(s)$, not only internal ones. Equipped with this invariant, we prove two important properties of CG-iLAO* when $\Gamma = \emptyset$.

Lemma 4. *At the end of CG-iLAO*'s main loop (alg. 2 after line 11), $\Gamma = \emptyset$ implies V is admissible up to an error, i.e., $V(s) \leq V^*(s) + \epsilon \overline{N}(s, \mathbf{S}) \forall s \in \mathbf{S}$.*

Proof. All fringe, external, and true goal states $s \in \widehat{\mathbf{G}} \cup (\mathbf{S} \setminus \widehat{\mathbf{S}})$, are initialised with $V(s) \leftarrow H(s) \leq V^*(s)$ and never have their $V(s)$ updated. For the remaining states, i.e., the internal states $\widehat{\mathbf{S}} \setminus \widehat{\mathbf{G}}$, we can use lem. 3. We ignore the internal states s with $V(s) \leq V^*(s)$, because they immediately satisfy the requirement of the lemma, and we focus on the internal states with $V(s) > V^*(s)$, which we call $\tilde{\mathbf{S}}$. We assume that $\Gamma = \emptyset$, so by the contrapositive of lem. 3 it must be the case that $V(s) \leq Q(s, a) + \epsilon \forall s \in \tilde{\mathbf{S}}$. It follows that $V(s) \leq V^*(s) + \epsilon \overline{N}(s, \mathbf{S}) \forall s \in \tilde{\mathbf{S}}$. To see this: if we apply VI to $\tilde{\mathbf{S}}$, the $\mathbf{S}$ restricted to the states $\tilde{\mathbf{S}}$, V will remain unaffected because $\text{RES}(s) \leq \epsilon$ for these states, and then we can apply thm. 1 to get $V(s) \leq V^*(s) + \epsilon N(s, V, \tilde{\mathbf{S}}) \forall s \in \tilde{\mathbf{S}}$. Then, $N(s, V, \tilde{\mathbf{S}}) \leq \overline{N}(s, \mathbf{S}) \forall s \in \tilde{\mathbf{S}}$ (defn. 28), giving the desired bound. Thus, we have addressed all states in $\mathbf{S}$, proving our claim. $\square$

Lemma 5. *Upon termination, CG-iLAO* has $\Gamma = \emptyset$.*

Proof. For contradiction, suppose CG-iLAO* does terminate with $\Gamma \neq \emptyset$. Then, Γ must have been populated by the final call to FIX-CONSTRS, but constraints are only added to Γ in FIX-CONSTRS when $V(s) > Q(s, a) + \epsilon$, and therefore $\text{RES} \geq V(s) - Q(s, a) > \epsilon$. This yields the desired contradiction because CG-iLAO* can not terminate until $\text{RES} \leq \epsilon$. $\square$

Finally, we are ready to prove that CG-iLAO* is optimal.

Theorem 8. *CG-iLAO* is optimal.*

Proof. Suppose that CG-iLAO* terminates with the value function V , the partial SSP $\widehat{\mathbf{S}}$, and the candidate policy $\hat{\pi}_{\text{curr}}$. Upon termination, we know that $\hat{\pi}_{\text{curr}} = \hat{\pi}_V$, otherwise a policy change must have occurred. Then, V satisfies ϵ -consistency w.r.t. $\widehat{\mathbf{S}}$, since $\text{RES} \leq \epsilon$ is one of CG-iLAO*'s termination conditions (alg. 2 line 12). Now, it suffices to show $V(s) \leq V^*(s) + \epsilon \overline{N}(s, \mathbf{S}) \forall s \in \mathbf{S}$ and that $\hat{\pi}_V$ does not encounter any artificial goals $\widehat{\mathbf{G}} \setminus \mathbf{G}$, and then we can apply thm. 6 to conclude that CG-iLAO* is optimal. Indeed, upon termination, CG-iLAO* has $\Gamma = \emptyset$ by lem. 5, so we can apply lem. 4 to get $V(s) \leq V^*(s) + \epsilon \overline{N}(s, \mathbf{S}) \forall s \in \mathbf{S}$. We also know that $\hat{\pi}_V$ does not encounter any artificial goals thanks to CG-iLAO*'s termination condition $\mathbf{F} \neq \emptyset$ (alg. 2 line 12). $\square$

In these proofs, we have assumed that $\eta = 0$ in order to guarantee that all cost increases and decreases are tracked. In practice, we use $\eta > 0$ due to the inherent numerical instability issues of floating-point number representations, and in particular it is convenient to choose $\eta = \epsilon$, to minimise the number of parameters. This is not problematic in our benchmarks, but one must be careful because $\eta > 0$ can introduce arbitrarily large errors into the value function. For example, consider the SSPs in fig. 4.5, with initial value functions shown in the bottom of each node. For the SSP on the left, CG-iLAO* with $\eta = \epsilon$ does the following:

Iter. 1 Partially expands s_I with a_0 .

Iter. 2 Expands s_1 with a_1 and updates $V(s_1) = 3 + \frac{2\epsilon}{3}, V(s_I) = 4 + \frac{2\epsilon}{3}$. The increases to $V(s_1)$ and $V(s_I)$ are $\frac{2\epsilon}{3} < \eta = \epsilon$, so nothing is added to Γ .

Iter. 3 Expands s_2 with a_2 and updates $V(s_2) = 2 + \frac{2\epsilon}{3}, V(s_1) = 3 + \frac{4\epsilon}{3}, V(s_I) = 4 + \frac{4\epsilon}{3}$. Again, the increases to V are $\frac{2\epsilon}{3} < \eta = \epsilon$, so nothing is added to Γ .

Iter. 4 Expands s_3 with a_3 and updates $V(s_3) = 1 + \frac{2\epsilon}{3}, V(s_2) = 2 + \frac{4\epsilon}{3}, V(s_1) = 3 + \frac{6\epsilon}{3}, V(s_I) = 4 + \frac{6\epsilon}{3}$. For the last time, the increases to V are $\frac{2\epsilon}{3} < \eta = \epsilon$, so nothing is added to Γ .

Iter. 5 There are no fringes left to expand, so CG-iLAO* does one pass of BACKUPS which leaves V unchanged and exits with $V(s_I) = 4 + 2\epsilon$. The algorithm terminates.

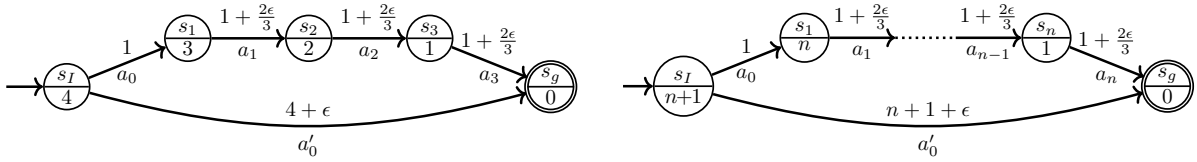


Figure 4.5: SSPs where CG-iLAO* with a large η ($\eta \geq \epsilon$) accumulates a large error. The left SSP ends up with an error of ϵ and $V(s_I) = 4 + 2\epsilon$, and the right SSP can have an arbitrarily large error of $(2k - 1)\epsilon$ with $V(s_I) = n + 1 + 2k\epsilon$ for $k = 3n$.

Thus, CG-iLAO* finds a policy for the SSP on the left, which is suboptimal by ϵ . CG-iLAO* behaves similarly on the right SSP, accumulating an error of $(2k - 1)\epsilon$ which gets arbitrarily large as the number of states n grows. Clearly, as $\eta \rightarrow 0$ this error disappears, so in particular $\eta = \epsilon$ works in that sense, but one has to be careful. Regardless of the choice of ϵ , we now have the additional theoretical issue that for any η it is possible to construct a pathological SSP that makes CG-iLAO*'s policy arbitrarily bad.

We note that although CG-iLAO* is not ϵ -consistent w.r.t. $\mathbb{S}$ in general, it becomes ϵ -consistent w.r.t. $\mathbb{S}$ if H is monotonic. Also, we can make CG-iLAO* ϵ -consistent w.r.t. $\mathbb{S}$ by modifying PARTLY-EXPAND-FRINGS so that for each expanded fringe $s_f \in \mathcal{E} \cap (\widehat{\mathbb{G}} \setminus \mathbb{G})$, we add $A(s_f)$ to Γ . This addresses the edge case in step 3. of lem. 3's proof. As explained before, it is not important that CG-iLAO* is ϵ -consistent w.r.t. $\mathbb{S}$ for the normal use-case of CG-iLAO*.

4.6 Different Expansion Methods for CG-iLAO*

In this section we explore different methods that CG-iLAO* can use to expand fringe states. For the correctness of CG-iLAO*, PARTLY-EXPAND-FRINGS($\mathbb{S}, \widehat{\mathbb{S}}, \widehat{\pi}_V, \mathcal{E}, V$) has two requirements:

1. at least one fringe state $s \in \mathcal{E} \cap (\widehat{\mathbb{G}} \setminus \mathbb{G})$ is expanded, i.e., afterwards $s \notin \widehat{\mathbb{G}} \setminus \mathbb{G}$ and $\widehat{A}(s) \neq \emptyset$ or $s \in \mathbb{G}$,
2. after expansion, $\mathcal{E}$ is up to date and new fringe states are accurately recorded in $\widehat{\mathbb{G}}$.

In this work, PARTLY-EXPAND-FRINGS expands all fringe states $\mathcal{E} \cap (\widehat{\mathbb{G}} \setminus \mathbb{G})$, i.e., we do not consider a method that expands a subset of fringe states. This is based on the empirical results of Hansen and Zilberstein (2001) that suggest expanding all reachable fringes is more efficient than expanding subsets. We now describe different methods for expanding each fringe state.

4.6.1 One-step-look-ahead

Here, we consider three methods for expanding a state s that only look one step ahead and greedily select actions based on their Q -values.

SINGLE-GREEDY-ACTION expands s by computing the Q -values of all applicable actions, and selects a single greedy action, breaking ties arbitrarily but in a reproducible way (assumption 3). Formally, SINGLE-GREEDY-ACTION(s) sets

$$\widehat{A}(s) \leftarrow \{\widehat{a}\} \text{ for some } \widehat{a} \in A(s) \text{ s.t. } Q(s, \widehat{a}) = \min_{a \in A(s)} Q(s, a). \quad (4.2)$$

In the best case, this method adds precisely the actions that define an optimal policy, and consequently CG-iLAO*'s partial SSP is minimal in the sense that it only contains the states and actions that are necessary to define the relevant optimal policy.

Theorem 9. *If initialised with V^* , CG-iLAO* with SINGLE-GREEDY-ACTION as its expansion mechanism will have a minimal partial SSP. That is, given CG-iLAO*'s output π^* , its partial SSP will consist precisely of $\widehat{\mathbb{S}} = \mathbb{S}^{\pi^*}$ and $\widehat{A}(s) = \{\pi^*(s)\}$ for each $s \in \widehat{\mathbb{S}} \setminus \mathbb{G}$.*

Proof. In each expansion, CG-iLAO* will insert the greedy action according to $V = V^*$. Then, Bellman updates will not change V since the greedy actions' $Q(s, a)$ are precisely $V^*(s)$ and $V = V^*$. Therefore Γ remains empty and the greedy policy is never updated (it breaks ties in favour of its current actions). So, CG-iLAO* will perform a greedy best-first search using $V = V^*$, which results in expanding precisely the states in the greedy policy envelope $\mathbb{S}^{\pi^*}$ and the corresponding actions. $\square$

Note that if there are multiple optimal policies, there is no guarantee that CG-iL^{AO}* returns the one with the smallest envelope, i.e., it does not guarantee the fewest possible expansions over all optimal policies.

In practice, it is usually worthwhile to add all tied-greedy actions upon expansion. So, we define the expansion ALL-GREEDY-ACTIONS(s) which sets

$$\widehat{A}(s) \leftarrow \left\{ \widehat{a} \in A(s) : Q(s, \widehat{a}) = \min_{a \in A(s)} Q(s, a) \right\}. \quad (4.3)$$

To motivate that it often pays off to add tied-greedy actions, suppose CG-iL^{AO}* expands state s with the single greedy action $a_{\min}$. If $V(s)$ increases, then CG-iL^{AO}* is forced to add all remaining external successor actions $A(s) \setminus \{a_{\min}\}$ to Γ , and from these must add all actions with $Q(s, a) = Q(s, a_{\min})$ to $\widehat{S}$. Such increases to $V(s)$ are common, e.g., an increase is guaranteed when $H(s) < \min_{a \in A(s)} Q(s, a)$, which is typical for practical heuristics. Thus, in this common situation all tied-greedy actions get added to $\widehat{S}$ anyway (assuming their own Q -values did not increase significantly), so it makes sense to avoid the overhead and add them immediately. This is the expansion presented in CG-iL^{AO}*’s pseudocode (alg. 2).

The final option we consider is to simply add all applicable actions. With this expansion, CG-iL^{AO}* implements iL^{AO}*, noting the minor differences in implementation discussed in sec. 4.4.⁴

4.6.2 Rollout

We now consider expansion methods that use rollouts. A rollout from state s is obtained by simulating the execution of a *base policy*: we start in s , apply the base policy, randomly select the next state according to P , apply the base policy to the new state, and so on. This produces a trajectory through the SSP from s , which we can use to inform our expansion. Rollouts have a long history in value-iteration style algorithms, and they have enjoyed success in planning, e.g., PROST (Keller and Eyerich, 2012), LRTDP (Bonet and Geffner, 2003b), and Robust-FF (Teichteil-Königsbuch, Kuter, and Infantes, 2010). The primary benefit of rollouts is that they enable information from the goal to be propagated sooner. Observe that the value functions of CG-iL^{AO}* with one-step-lookahead expansions are informed exclusively by the heuristic for most of their lifetimes, and only start propagating information from goal states when a goal enters the partial SSP. To try to address this, we implement rollouts into CG-iL^{AO}* during the expansion phase: when expanding state s , we perform a rollout from s to generate a trajectory $\langle s^0, a^0, s^1, a^1, \dots, s^n \rangle$, and then add all the state-action pairs (s^i, a^i) to the partial SSP. Although this may insert suboptimal actions into the partial SSP, this expansion satisfies the requirements of PARTLY-EXPAND-FRINGES, and therefore does not affect CG-iL^{AO}*’s correctness. We do not consider adding rollouts to iL^{AO}*’s expansion, because iL^{AO}* would have to add all applicable actions for all the states in the trajectory, which makes its partial SSP significantly larger than it needs to be. We now present two base policies to use for CG-iL^{AO}*’s rollout expansions: trials and FF.

Trial rollouts. Following RTDP (Barto, Bradtke, and Singh, 1995), we roll out with a trial by selecting the greedy action w.r.t. V at each step. The pseudocode for our trials is given in alg. 3.

SSPs can have cycles, and a naïve implementation of trial is susceptible to getting stuck inside a cycle indefinitely. RTDP deals with cycles by updating $V \leftarrow \min_{a \in A(s)} Q(s, a)$ for each state s it encounters; suppose RTDP’s simulation is stuck inside a cycle, then the relevant V values increase until the Q -values to remain inside the cycle get so large that the simulation leaves the cycle. In practice, to handle SSPs that violate reachability (assumption 1), we use the fixed-penalty transform (Trevizan, Teichteil-Königsbuch, and Thiébaux, 2017) that allows the agent to “give-up” with a large cost $d \in \mathbb{R}_{>0}$ — consequently, if RTDP is stuck within a cycle, the simulation “gives up” as soon as $V(s) \geq d$. Similarly to RTDP, we update V in our trial, but, importantly, in order for CG-iL^{AO}* to remain correct, it must preserve that $V(s) \leq V^*(s)$ for all external states $s \in S \setminus \widehat{S}$. Recall that CG-iL^{AO}* may have $V(s) \not\leq V^*(s)$ for internal states $s \in \widehat{S}$. To preserve $V(s) \leq V^*(s)$ for external states, we force the trial to terminate whenever it can reach an internal state from its current state s , i.e., the trial is stopped whenever there is an action $a \in A(s)$ such that $\text{succ}(s, a) \cap \widehat{S} \neq \emptyset$. Consequently, $V(s)$ is only updated by the trial if all effects of all applicable actions are external. External states are in this way only updated with full Bellman

⁴There is a minor exception apart from the implementation changes where CG-iL^{AO}* with EXPAND-FRINGES does not behave like iL^{AO}*: if H is not monotonic, then a decrease to V can occur, causing constraints to be added to Γ , which CG-iL^{AO}* has to deal with. Though possible, we have not observed this edge case in our experiments.

Algorithm 3: trial

```

1 Function trial( $s_{init}, V, t_{max}$ )
2    $s \leftarrow s_{init}$ 
3   trajectory  $\leftarrow \langle s \rangle$ 
4    $t \leftarrow 0$ 
5   while  $s \notin \hat{S}$  and  $s \notin G$  do
6     // Return failure if the trial reaches a dead end or times out
7     if  $t > t_{max}$  or  $V(s) > d$  or  $A(s) = \emptyset$  then
8       return failure
9     // If there is an action that reaches  $\hat{S}$ : take the cheapest one
10    if  $\exists a \in A(s) : succ(s, a) \cap \hat{S} \neq \emptyset$  then
11       $a_{min} \leftarrow \operatorname{argmin}_{a \in A(s) : succ(s, a) \cap \hat{S} \neq \emptyset} Q(s, a)$ 
12       $s \leftarrow \text{some state } \in succ(s, a_{min}) \cap \hat{S}$ 
13    // Otherwise continue RTDP trial as normal
14    else
15       $a_{min} \leftarrow \operatorname{argmin}_{a \in A(s)} Q(s, a)$ 
16       $V(s) \leftarrow Q(s, a_{min})$ 
17       $s \leftarrow \text{state randomly chosen from } P(\cdot | s, a_{min})$ 
18    append  $\langle a_{min}, s \rangle$  to trajectory
19     $t \leftarrow t + 1$ 
20  remove cycles from trajectory
21  return trajectory

```

backups using other external states. Recall that external states are initialised with $V(s) = H(s) \leq V^*(s)$, so by updating $V(s)$ for external states only by applying Bellman backups to other external states, we guarantee that $V(s) \leq V^*(s)$ for external states. Terminating the trial whenever an internal state is reachable also has the intuitive interpretation that we stop the rollout as soon as it reaches a state where “we know what to do already.”

A trial from s can either “succeed” by reaching a goal or an internal state, or it can “fail” by reaching a dead end or by its number of steps exceeding t_{max} .

- If the trial succeeded: we first remove any cycles from the trial, then we expand the remaining acyclic trajectory by adding all its state-action pairs to $\hat{S}$.⁵
- If the trial failed: it does not make sense to expand the entire trial; the only fact we know about this sequence of states and actions is that they do not result in a goal or an internal state. Instead, we expand s with ALL-GREEDY-ACTIONS on the updated V , with the idea that the updated V will inform the one-step-lookahead to avoid the action that lead to failure.

Our trials are parameterised by t_{max} . A low value of t_{max} serves as another mechanism to escape cycles, and more generally as a mechanism to avoid wasting time on a simulation that ends up in an expensive part of the state space that CG-iLAO* does not benefit from expanding. However, this is at the cost of finding fewer successful trajectories that end in goals or internal states.

FF rollouts. Inspired by the success of the algorithms FF-Replan (Yoon, Fern, and Givan, 2007) and Robust-FF (Teichteil-Königsbuch, Kuter, and Infantes, 2010), we use the FF (Hoffmann and Nebel, 2001), an algorithm for deterministic shortest path problems, to find a plan for the SSP’s determinisation and use this plan as a rollout. A determinisation removes the SSP’s probabilistic effects, and produces a deterministic shortest path problem. We consider two determinisations:

1. **All-outcomes determinisation.** The all-outcomes determinisation splits each probabilistic action $a \in A(s)$ into multiple deterministic ones. For each $s' \in succ(s, a)$, the determinisation produces the actions $a_{s \rightarrow s'}$ that lead to s' deterministically. This is proper relaxation of the SSP: if a proper policy exists for the SSP, then a plan must exist for the all-outcomes determinisation.

⁵To remove cycles from the trial we identify the first and last occurrence of each state in the trial, and if these are not equal, remove the subsequence between them (as well as one of the copies of the state).

2. **Most-likely-outcome determinisation.** The most-likely-outcomes determinisation transforms each probabilistic action $a \in A(s)$ into a single deterministic action. It selects s' for some $s' \in \operatorname{argmax}_{s''} P(s''|s, a)$ (breaking ties arbitrarily), and then produces the single action $a_{s \rightarrow s'}$ that leads to s' deterministically. This determinisation has the benefit that it has fewer actions and maintains some information about probability distributions by picking the most likely; however, it is not a proper relaxation, because it can happen that the single most-likely outcomes that we selected do not allow a plan to be constructed.

We run FF rollouts by running FF on the respective determinisation. Then, FF returns the plan $\langle s^0, a^0, \dots, s^n \rangle$ where $s^0 = s$ and $s^n \in G$, or returns that no plan exists. If it finds a plan, we expand it. If no plan exists on the all-outcomes determinisation, then it is proof that the state we are expanding is a dead end and we mark it as such. If no plan exists on the most-likely-outcomes determinisation we can not conclude that it is a dead end, so we run FF again, but on the all-outcomes determinisation, to construct a plan or prove a dead end.

4.7 Related Work: Action Elimination

In this section, we give more details about the technique of action elimination. It is particularly relevant to CG-iLAO*, because it addresses the same issue of unneeded actions being considered during search. Action elimination lets us prove that certain actions can not be part of the optimal policy, and can therefore be permanently removed from search without affecting optimality. This technique, similarly to CG-iLAO*'s mechanism for ignoring actions, allows us to avoid computing Q -values for actions that will not help with the solution. However, action elimination approaches the issue from the opposite direction, in the sense that it starts with all actions and permanently removes them when they are proved to be outside optimal policies, compared to CG-iLAO*'s mechanism which initially ignores all actions and adds them as required.

Action elimination requires upper and lower bounds on V^* , called V_{ub} and V_{lb} respectively, and determines which actions can be eliminated using the following theorem.

Theorem 10 (Action Elimination (Bertsekas, 1995)). *Given a state $s \in S$ and action $\tilde{a} \in A(s)$, if $\exists a \in A(s)$ such that $Q_{lb}(s, \tilde{a}) > Q_{ub}(s, a)$, then action $\tilde{a}$ cannot be optimal for state s , and can therefore be eliminated.*

We have already discussed how to maintain V_{lb} as a lower bound. Note that outside this section we call such value functions V , but here we call it V_{lb} to emphasise the distinction between upper and lower bounds. The main drawback of action elimination, and the reason its use is not widespread in planners, is that obtaining the upper bound V_{ub} is difficult. There are methods for deriving an upper bound V_{ub} from only a lower bound and its Bellman residual (Hansen and Zilberstein, 2001; Hansen and Abdoulahi, 2015), but they require the expensive computation of the expected number of steps to reach the goal or that the policy is proper, which is impractical for our purposes. The standard way to track V_{ub} is to initialise V_{ub} s.t. $V_{ub} \geq V^*$, and then apply Bellman backups to it, which are guaranteed to preserve $V_{ub} \geq V^*$. This is done in addition to tracking V_{lb} , so the algorithm tracks two value functions, and each partial backup must be applied both to V_{lb} and V_{ub} . Thus, if all else is the same, then tracking the two value functions exactly doubles the number of computed Q -values. An additional difficulty for tracking V_{ub} , on top of the additional maintenance cost, is that we are not aware of any efficient and domain-independent algorithms to compute an initial upper bound. The only domain-independent upper bound we are aware of, is the trivial upper bound

$$V_{ub}(s) = \begin{cases} 0 & \text{if } s \in G \\ d & \text{if } s \notin G \end{cases}$$

where $d = \infty$ or the penalty term, if we are using a fixed-penalty transformation.

Action elimination has not been used by optimal heuristic-search algorithms to remove actions during search. There are variants of RTDP (Barto, Bradtke, and Singh, 1995) that track both V_{ub} and V_{lb} , such as BRTDP (McMahan, Likhachev, and Gordon, 2005), FRTDP (Smith and Simmons, 2006), VPI-RTDP (Sanner et al., 2009); and variants for LAO* (Hansen and Zilberstein, 2001) such as IBLAO* (Warnquist, Kvarnström, and Doherty, 2010). However, none of these use action elimination: they use lower and upper bounds to define different ways to prioritise expansion and backups, and to define alternative termination conditions.

The only algorithm we are aware of that explicitly uses action elimination is FTVI (Dai et al., 2011). FTVI consists of a computation stage where it runs Topological Value Iteration (TVI), and a preprocessing stage, where it tries to make the search space more amenable to TVI. TVI works well when the state space has small strongly connected components (SCCs), and degenerates to standard VI as the SCCs get bigger. The preprocessing stage attempts to break any large SCCs into smaller ones by removing suboptimal actions with action elimination. This step iteratively performs a Depth-First Search on the state space, applies Bellman backups on V_{lb} and V_{ub} in post-order, and uses action elimination to remove actions when it can. Note that FTVI is not a heuristic search algorithm because its search step does not use a heuristic and considers the whole state space.

Action elimination has another disadvantage w.r.t. to CG-iLAO*: it initially considers all actions, and only eliminates them later in its lifetime. In comparison, CG-iLAO* starts with no actions, and adds them as they are deemed necessary. This means that CG-iLAO* starts with a small partial SSP, and grows it over time, enjoying a relatively small partial SSP during its entire execution; whereas action elimination has a large partial SSP for most of its execution, only shrinking it when V_{lb} and V_{ub} become sufficiently accurate to prove actions are suboptimal.

To summarise, action elimination has two key disadvantages compared to CG-iLAO*'s mechanism for ignoring actions:

1. Action elimination requires both a lower bound V_{lb} and an upper bound V_{ub} , whereas CG-iLAO* only needs V_{lb} . Maintaining V_{ub} incurs an overhead of Q -value computations.
2. Action elimination is only able to shrink its partial SSPs late in its lifetime, whereas CG-iLAO* enjoys smaller partial SSPs from the start.

We show experimentally in sec. 4.9.7 that CG-iLAO*'s mechanism for ignoring actions is indeed more efficient than action elimination.

4.8 Related Work: Partial Expansion A*

The concept of ignoring unneeded actions has been considered in the deterministic shortest path setting by the Partial Expansion A* (PEA*) algorithm (Yoshizumi, Miura, and Ishida, 2000). Here, we explain their algorithm and compare it to CG-iLAO*. Similarly to CG-iLAO*, PEA*'s states are partially expanded: the algorithm only considers a greedy subset of applicable actions, and efficiently tracks the other actions to be reconsidered when they may become needed. We assume that the reader is familiar with A* (Hart, Nilsson, and Raphael, 1968; Russell and Norvig, 2016), and we only give a brief reminder of how the algorithm works. We use similar notation to Yoshizumi, Miura, and Ishida (2000), but talk about states instead of nodes. Then, for a state s

- $g(s)$ denotes the lowest path cost to s , i.e., the currently best-found cost to move from s_I to s ;
- $h(s)$ is an admissible heuristic function evaluated at s (defined similarly to heuristic functions in our setting);
- $f(s) = g(s) + h(s)$ is the evaluation function.

A* maintains an *open list* of states (also called the frontier), from which the algorithm iteratively pops a state s with lowest $f(s)$ and then expands it. We assume that the open list is implemented as a priority queue ordered by f . To expand s , A* iterates over s 's successors and places them into the open list with their respective g , h , and f values. A key feature of A* is that it will stop as soon as it pops a goal from the open list, and any states that remain in the open list are discarded, not needing to be considered for constructing the optimal plan. For problems with large branching factors the number of such discarded states can get very large, which makes it expensive maintain the priority queue, and in extreme cases causes the algorithm to run out of memory. PEA* addresses this by using partial expansions. A partial expansion for s is defined as follows: let $SUCC$ denote the successor states of s , then the partial expansion only adds the greedy successor states

$$SUCC_{\leq 0} = \left\{ s_{\min} \in SUCC : f(s_{\min}) = \min_{s' \in SUCC} f(s') \right\}$$

and all non-greedy successors are captured by placing the parent state s back into the open list with an updated priority

$$F(s) = \min_{s' \in \text{SUCC} \setminus \text{SUCC}_{\leq 0}} f(s').$$

This $F(s)$ can be read as the smallest f -value of s 's non-greedy successors. Note that PEA* uses F to sort its priority queue, and we use $F(s) = f(s)$ if a state has not been partially expanded before. The key idea is that, if search determines that the states we have ignored become relevant, then s will be popped again as a proxy for its successors, and the partial re-expansion of s will place the newly greedy successors into the open list, and progress as though these states were always on the open list. Thus, the algorithm can maintain a smaller open list by ignoring unpromising states, at the cost of requiring additional re-expansion steps.

If iLAO* is the generalisation of A* to SSPs, then CG-iLAO* is the generalisation of PEA*, and CG-iLAO* shares the key behaviours of PEA* when applied to deterministic problems. It is awkward to compare CG-iLAO* and PEA* directly, because their settings are different: CG-iLAO* computes the states' costs-to-go in V and tracks the current greedy policy $\hat{\pi}_{\text{curr}}$, whereas PEA* is computing the lowest path costs g and implicitly tracking the best plan by recording each state's parent. Nevertheless, the order in which CG-iLAO* and PEA* handle states and actions is comparable. We illustrate this in fig. 4.6 and fig. 4.7. Fig. 4.6 is the example from Yoshizumi, Miura, and Ishida (2000) that demonstrates the expansion order of PEA*. Solid and dotted circles indicate that the state has or has not been added to the open list, respectively. The number at the bottom of each circle represents the corresponding state's priority on the open list, i.e., its $F(s)$, and when a value is crossed out this indicates that $F(s)$ was updated in this step. Fig. 4.7 shows how CG-iLAO* behaves on the same problem. Importantly, the numbers at the bottom of each node have a different semantic: they now represent $V(s)$, and the solid and dotted circles represent states that are either inside or outside the current partial SSP. We use the same heuristic for both algorithms, namely h with:

$$h(s_0) = 7, h(s_1) = 5, h(s_2) = 6, h(s_3) = 9, h(s_4) = 6, h(s_5) = 7, h(s_6) = 8, h(s_7) = 5, h(s_8) = 6, h(s_9) = 9.$$

At an intuitive level, we see that the expansions of CG-iLAO* behave similarly to PEA*'s first-time expansions, and CG-iLAO*'s fixing of violated constraints lines up with PEA*'s re-expansions, so that steps (a), (b), (c), and (d) match between the two figures. Within these steps, the candidate plans are the same, with the exception that in step (b) CG-iLAO* considers an extra action and state, since PEA* immediately recognises that s_I needs to be re-expanded, and CG-iLAO* needs to compute the policy first. To understand the relationship between CG-iLAO* and PEA*, we start by pointing out that iLAO* behaves similarly to A*: in each step of iLAO* in a deterministic problem, its current policy is a plan p in the partial SSP from s_I to $s \in \hat{G}$ such that $C(p) + h(s)$ is minimal; and A* considers the state in its open list with minimal $f(s) = g(s) + h(s)$. With consistent tie breaking, it turns out that in matching steps this s is the same, and $C(p) = g(s)$, so that $V(s_I) = f(s)$. CG-iLAO* and PEA* inherit this relationship, and it only remains to understand how CG-iLAO*'s fixing of violated constraints corresponds to PEA*'s re-expansions. This does not line up exactly, as we see in our example in step (b), because PEA* immediately places a state s back into the open list if $f(s) > f(s')$ for its parent s' , whereas CG-iLAO* does not take such shortcuts and computes the greedy policy. Nevertheless, the steps roughly correspond: CG-iLAO* recognises a violated constraint when a single action can be added to improve its candidate policy; and PEA*, re-expands s and brings in a new successor state s' with $F(s') = F(s)$, effectively improving the candidate plan by adding the missing action from s to s' . Thus, it seems clear that CG-iLAO* and PEA* have near-identical behaviour on deterministic problems.

We note that the definition of Yoshizumi, Miura, and Ishida (2000) is more general than what we have presented, because they allow $\text{SUCC}_{\leq C}$ for any cutoff $C \in \mathbb{R}_{\geq 0}$ and we only presented the special case $C = 0$. With the special case $\text{SUCC}_{\leq 0}$, PEA* is analogous to CG-iLAO* with the ALL-GREEDY-ACTIONS expansion (sec. 4.6.1). Larger values of C would translate to a CG-iLAO* expansion that adds actions that are within C of greedy. We investigated such expansions, but we did not find them theoretically insightful, and preliminary experiments were unpromising, so we have left them out of this paper.

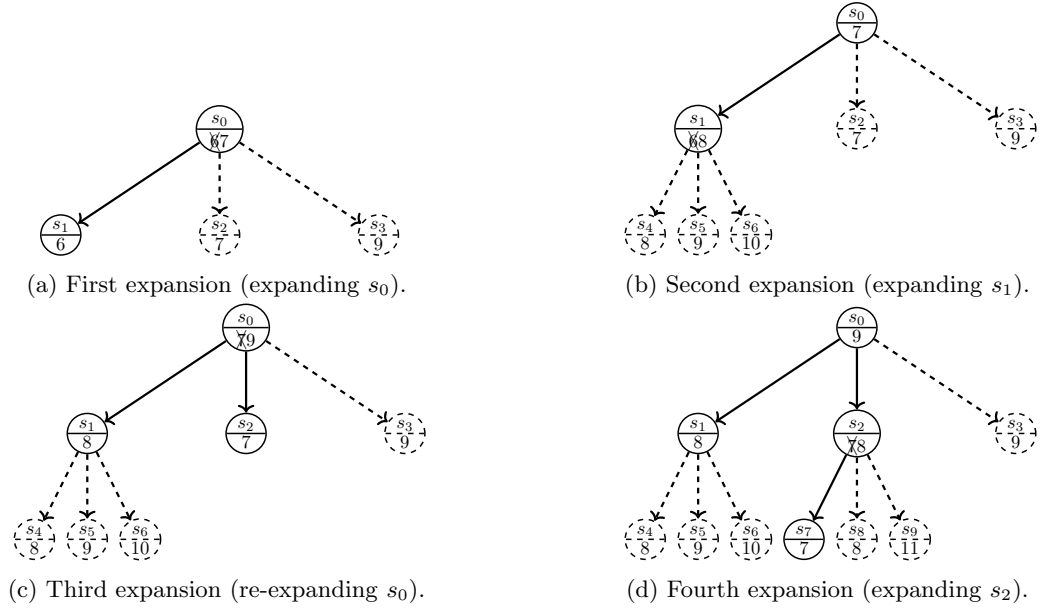


Figure 4.6: PEA*'s behaviour (comparing CG-iLAO* and PEA*).

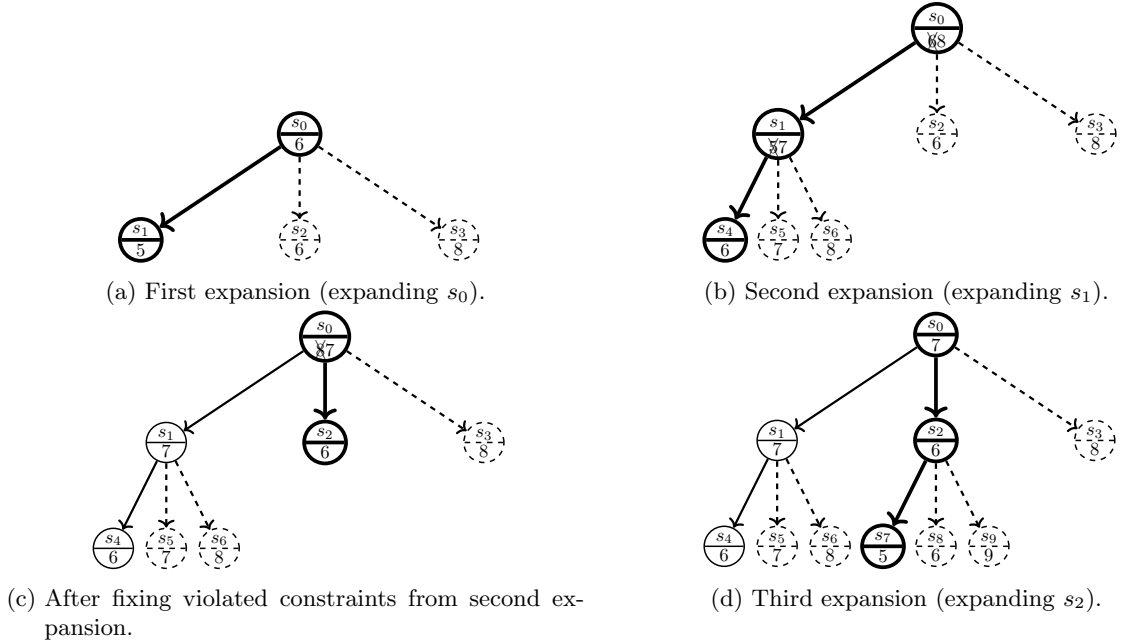


Figure 4.7: CG-iLAO*'s behaviour on a deterministic problem (comparing CG-iLAO* and PEA*). Thick lines indicate the greedy policy envelope.

4.9 Experiments

In this section, we experimentally determine the most efficient variants of CG-iLAO*, demonstrate how these variants compete with the state-of-the-art, and analyse these results in detail. We start by presenting the methodology of our experiments (sec. 4.9.1), describing the benchmark domains that we consider (sec. 4.9.2), and we explain the tables and plots that we use to present the results (sec. 4.9.3). Then, we compare the versions of CG-iLAO* to identify the best performers (sec. 4.9.4), and show that CG-iLAO*'s best performers are competitive with the state-of-the-art (sec. 4.9.5). We investigate CG-iLAO* further and show how many actions it is able to ignore (sec. 4.9.6), and how its mechanism for ignoring actions compares to action elimination (sec. 4.9.7).

4.9.1 Methodology

We compare different versions of CG-iLAO* with the state-of-the-art optimal heuristic-search algorithms iLAO* (Hansen and Zilberstein, 2001) and LRTDP (Bonet and Geffner, 2003b). We will give more details about the algorithms we use in the relevant sections. Heuristics have a major impact on the performance of heuristic-search algorithms, so we consider the following suite of admissible heuristics from the literature: LMCut (Helmert and Domshlak, 2009); ROC (Trevizan, Thiébaux, and Haslum, 2017); and PDB (Klößner et al., 2021; Klößner and Hoffmann, 2021). Following the original authors of ROC, on problems with dead ends we use ROC with $h^{\max}$ (Bonet and Geffner, 2000) as a dead-end detection mechanism. We only consider PDBs generated up to size 2, following the results of Klößner and Hoffmann (2021). We convert SSPs into dead-end free SSPs with the fixed-penalty transformation (Trevizan, Teichteil-Königsbuch, and Thiébaux, 2017) with a penalty of

- $D = 10^4$ for Elev, $\square$ TW, and Sched;
- $D = 10^7$ for PARC-R, PARC-N, and Sys;
- $D = 500$ for all other domains.

The domains with $D > 500$ require large penalties for different reasons. PARC-N and PARC-R have very large action costs. On problems such as $\square$ TW, the optimal policy π^* 's probability of reaching a goal (without using give-up actions) is very low, with the effect that the cost of π^* is within ϵ of 500, so D needs to be increased. On problems such as Elev, an optimal policy requires the agent to repeat a sequence of actions many times before it reaches a goal, so that the optimal policy's cost exceeds 500. As the parameter for ϵ -consistency, we use $\epsilon = 0.0001$, and we set CG-iLAO*'s parameter $\eta = \epsilon$.

For each triple of algorithm, heuristic, and problem, we run it 5 times with different random seeds. These triples coupled with a random seed are called **instances**. We run multiple instances per problem because, even if all other parameters are fixed, the random seed can cause a significant amount of variance: for algorithms that rely on random choices this is obvious (e.g., LRTDP), but even for seemingly deterministic algorithms, random seeds have a significant impact on behaviour in practice (e.g., on tie-breaking). To emphasise this point, we have observed a difference of over 1000 seconds on the same problem, algorithm, and heuristic, only changing the random seed. For each instance, the execution is cut off at 30 minutes of CPU time and 8GB of memory. The experiments were conducted in a cluster of Intel Xeon 3.2 GHz CPUs and each run used a single CPU core. The LP solver used for computing ROC was CPLEX version 20.1.

Our code, benchmarks, and results are available online (Schmalz and Trevizan, 2023). We now give descriptions of our benchmark domains.

4.9.2 Benchmark Domains

We use the benchmark problems from Klößner et al. (2021), which is a compilation of domains from the International Probabilistic Planning Competitions (IPPC) that were run in 2004 (Younes et al., 2005), 2006 (Bonet and Givan, 2006), and 2008 (Bryce and Buffet, 2008). These IPPCs encoded problems in Probabilistic PDDL (Younes et al., 2005); we do not consider later competitions because they use RDDDL (Sanner, 2010), which is incompatible with our heuristics. We consider three additional domains, which we also describe in this section. To reduce wasted compute time, we eliminate problems that are too difficult to solve. For a domain with problems enumerated from 1 to n , we consider the problems i to n too difficult if none of these were solved by LRTDP, iLAO*, or CG-iLAO* with any heuristics, given an extended deadline of 45 minutes. We now describe each domain and list each IPPC where the

domain appeared. We omit the domain “Boxworld” because all of its problems are too difficult, and we omit “Drive” and “Tireworld” because their problems are too easy: all algorithm configurations solved all problems in under 30 seconds (with a few exceptions).

Blocks World (BW) [IPPC 2004, 2006, 2008].

Deterministic Blocks World has featured in many of the (deterministic) International Planning Competitions (IPCs). A Blocks World problem consists of a table that has n unique blocks on it which can be stacked on top of each other, i.e., a block can be placed on top of any other, as long as there is no block on top of it already. The agent is presented with an initial configuration of blocks, and its task is to rearrange them into a specified configuration, e.g., block 1 should be on block 2, and block 2 should be on the table. To achieve this, the agent controls a “gripper” that can pick up one block at a time that has no other blocks on top of it, and then the gripper can put this block down on the table or on top of another block. The probabilistic version of Blocks World, which we consider, introduces to each of these actions a 0.25 probability that the block being handled slips out of the gripper and falls on the table. The probabilistic version also adds high-risk high-reward actions that enable the agent to handle a tower of two blocks, i.e., it can pick up a tower of two and then place it on the table or on top of a block, but it only succeeds with probability 0.1. In this domain, any action’s effects can be reversed, e.g., if the agent tries to place block 1 but drops it, it can repeat the pick-up action until it is holding block 1, returning it to its previous state. Consequently, the problems have no dead ends.

Core Security Pentesting (Coresec) (Steinmetz, Hoffmann, and Buffet, 2016; Klößner et al., 2021).

This domain represents a penetration testing (pentesting) scenario, where the agent aims to exploit vulnerabilities in a network to compromise specific targets in the fewest possible steps. Each exploit can be used once, succeeding or failing with some probability. Thus, Coresec problems violate reachability (assumption 1), because any exploit that is necessary to breach the network may fail, and since the exploit can not be used again, the agent has no way to achieve its goal. Pentesting is a real-world application of SSPs, where real-world networks are modelled and presented to the agent; the agent’s policies for breaching the network help administrators identify and address vulnerabilities. This particular domain is indirectly based on a test scenario from Core Security (<http://www.coresecurity.com/>). This domain has not featured in any IPPC.

Elevators (Elev) [IPPC 2006].

The agent is tasked with collecting all coins distributed within a building. The building can be visualised as a grid with y floors and x horizontal positions. Within a floor, the agent can freely walk between the horizontal positions. To move vertically between floors, the agent must use an elevator, which are available at particular horizontal positions. If the agent is at the same floor and horizontal position as an elevator, it can enter and move between floors as expected. The coins that the agent must collect are distributed throughout locations of the building, and the agent must try to navigate between them in the minimum number of steps. The probabilistic factor is that all locations except the ground floor have a “gate”, and whenever the agent walks past such a gate it has a 0.5 probability of falling to the left-most part of the ground floor. Thus, the agent should avoid walking past gates unnecessarily. This domain has no dead ends because the agent can simply try to move to the next coin repeatedly, regardless of how many times it gets sent back to the left-most part of the ground floor. Note that policies can get very expensive, particularly if there is a coin to which the closest elevator is far away horizontally, since it forces the agent to walk past many gates.

Exploding Blocks World (ExBW) [IPPC 2004, 2006, 2008].

This domain, like BW, is also based on the deterministic Blocks World from IPC. Instead of the block slipping out of the agent’s gripper, the probabilistic element is that each block is rigged with an explosive that can detonate once, instantly destroying the table or the block directly beneath it. When a block is placed, it detonates with probability 0.4 if placed on the table and 0.1 if placed on another block. Once the table or a block is destroyed, the agent can no longer interact with them, and if they were not in their goal position, the goal becomes unreachable. This means ExBW problems can have unavoidable dead ends: there may be a non-zero probability of reaching a state where a necessary block or the table have been destroyed. Under the fixed-penalty transformation, planners must avoid policies that incur

the give-up penalty with high probability. Earlier versions of this domain had a bug where a block can be placed on top of itself; this bug is fixed in all of the problems we consider.

Probabilistic PARC Printer (PARC) (Trevizan, Thiébaux, and Haslum, 2017).

The deterministic PARC Printer domain from the IPC models a modular printer consisting of various components, where each page scheduled for printing must pass through multiple components in a specific order. The agent’s task is to print a document of s pages in as few steps as possible, where each page can have different requirements which require different components in the printer. For example, the printer has coloured and black-and-white printing components, so pages in the document must be routed accordingly. In the probabilistic variant, c components have a 0.1 probability of jamming, rendering the component unusable and requiring the affected page to be reprinted. The domain has two variants:

- PARC-R (with repair): Jammed components can be repaired and reused with a high cost.
- PARC-N (without repair): Jammed components remain permanently unusable.

This domain does not come from Klößner et al. (2021), and has not featured in any IPPC.

Random (Rand) [IPPC 2006].

In this domain, problems are generated randomly with no associated semantics. Rand is exceptional among the domains we consider, because it can have significantly more applicable actions in each state. The other domains have a number applicable actions on the order of 10^1 , occasionally 10^2 ; whereas Rand has on the order of 10^4 (see tab. 4.7 for the number of states and actions considered in a particular Rand problem).

Schedule (Sched) [IPPC 2006, 2008].

This is a network scheduling scenario. There are m distinct packets being sent through the network, and n “classes” that can receive them. The agent’s task is to ensure that each class receives and serves one or more packets, with the network staying alive. In each timestep, each class may randomly receive an incoming packet, which sets the packet’s “time-to-live” to k timesteps. Then, the agent must choose whether the class should serve the packet, or hold onto it until the time-to-live expires, which causes the packet to be reclaimed, i.e., the current class no longer has access to the packet, and the packet gets randomly sent to one of the classes, as before. A complicating factor is that, whenever a packet gets reclaimed, the previous owner class has a small probability of locking up, which makes the network not alive, i.e., the agent fails its task. Different classes have different probabilities of locking up, so the agent should use the low-probability ones for reclaiming packets, and avoid reclaiming on the high-probability ones.

Search and Rescue (S&R) [IPPC 2008].

An autonomous helicopter must find a safe spot to land in order to save a survivor. There are n many locations, and it is not known a priori whether it is possible to land there or not. The agent must explore the locations – each explore action probabilistically determines whether it is possible to land at that location, or not. Once the agent finds a safe location to land, it can pick up and deliver the survivor to safety. The more actions the agent applies, the less likely the human survives. The agent’s task is to maximise the human’s chance of survival.

Sysadmin (Sys) [IPPC 2008].

The agent must manage a network of servers. In each timestep, a server has a probability of failing. If the server’s neighbouring servers are all operational, then this server’s probability of failing is low (probability 0.05), but if any of its neighbours have failed, then this server’s probability of failing is significantly higher (probability 0.25). The only decision available to the agent is to reboot one problematic server per timestep, fixing it with probability 0.9. In the version we consider, all servers are initially broken, and the agent must make all servers operational while minimising the number of reboots over expectation. Note that Sys has zero-cost actions to handle the bookkeeping of which servers fail, which violates our assumption that costs are strictly positive. However, this is not an issue because we manually verified that there are no zero-cost cycles and assumption 2 holds.

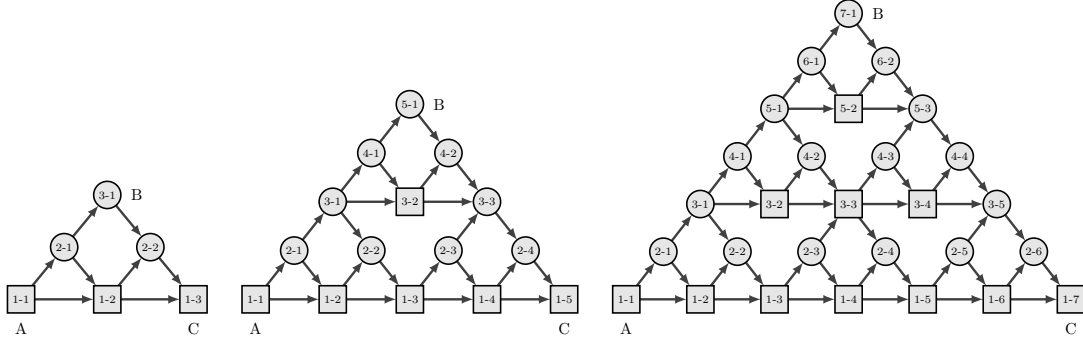


Figure 4.8: Triangle Tire World problems 1, 2, 3 (left to right).

Triangle Tire World (Δ TW) [IPPC 2008].

The agent is presented with a network of locations arranged in a triangular layout with corners A, B, C , as in fig. 4.8, and the agent’s task is to drive from corner A to C . Each time the agent moves, the car gets a flat tyre with probability 0.5. If the car gets a flat tyre the agent must replace the flat with a spare tyre if it has one; when the agent has no spare tyre, the car is stuck, i.e., it is in a dead end with no actions available. The agent can only load spare tyres in specific locations (circles in fig. 4.8), and can only keep one spare tyre at a time. This problem was introduced by [Little and Thiebaux \(2007\)](#) to be probabilistically interesting: if the probabilistic effects are ignored, the agent will take the direct path from A to C , which maximises the probability of encountering a dead end; whereas an optimal policy navigates from A to B to C , so that spare tyres are always available and it can reach the goal with probability 1. Thus, Δ TW has dead ends, but satisfies the reachability assumption (assumption 1) because a proper policy exists. Δ TW problems are specified by $n \in \mathbb{N}$. We show the layouts of Δ TW with $n = 1, 2, 3$ in fig. 4.8; the problem with $n = 1$ can be considered a base case, and the problems with $n > 1$ are constructed by connecting n copies of the $n = 1$ problem along each edge of the triangle. With this construction, the state-space grows exponentially with respect to n , which is inconvenient for running experiments: it can happen that problem n is solved easily by all algorithms, but all algorithms time-out for $n + 1$. To address this shortcoming, we introduced Δ TW with Head-start ([Schmalz and Trevizan, 2024b](#)), which gives a finer granularity of problem difficulty. This extension was not taken from [Klößner et al. \(2021\)](#), and has not featured in any IPPCs. A Δ TW with Head-start problem is defined by Δ TW(n, d), where $n \in \mathbb{N}$ denotes the problem size as before, and $d \in \mathbb{N}$ denotes that the agent is given a head-start along the edge AB , thus the agent has the initial location $x-y$ with $x = (d + 1)$ and $y = 1$ in fig. 4.8. For fixed n , d can take the values $0 \leq d \leq 2n$ where

- $d = 0$ defines the original Δ TW problem, with no head-start, where the agent starts at corner A ,
- $d = 2n$ is the easiest variant of the problem where the agent is given the biggest head-start and starts at corner B .

So, by starting at $d = 0$ we have the original Δ TW problem, and by increasing d we make the problem easier, giving us problems of intermediate difficulty between the standard Δ TW problems.

Rectangle Tire World ($\square$ TW) [IPPC 2008].

The agent must drive from the bottom-left corner of a rectangular network of roads to the top-right, avoiding dangerous areas as much as possible. This domain is inspired by Δ TW ([Little and Thiebaux, 2007](#)), but with two key changes. First, as the name suggests, the road network is rectangular, rather than triangular, which makes it easier to encode large road systems compactly. On this network, the agent can drive to orthogonally adjacent cities, as well as diagonally adjacent ones. Second, the agent can no longer get a flat tyre, instead there are dangerous locations. If the agent enters a dangerous location, it crashes with certainty, killing the agent, so it can not achieve any goal. Entire rows and columns of the rectangular grid are specified to be unsafe. Driving diagonally is “somewhat dangerous,” so anytime the agent performs this action it crashes with probability 0.2. The agent should minimise the number of drives between cities, while minimising the probability of crashing. Unsafe locations may be unavoidable in this domain, so $\square$ TW does not satisfy the reachability assumption (assumption 1). Moreover, randomly generated instances of this problem may be unsolvable in the sense that there is no

way for the agent to reach the goal with probability > 0 .

Zenotravel (Zeno) [IPPC 2004, 2006, 2008].

The agent must route aeroplanes to move people between cities. When grounded, aeroplanes board and debark passengers as expected. Aeroplanes can fly at a normal speed or “zoom” at a faster speed, allowing the vehicle to reach its destination faster at the cost of more fuel consumption. Once an aeroplane’s fuel is exhausted, it must refuel. The boarding, debarking, flying to a destination (at any speed), and refuelling actions all succeed only with a specified probability, and upon failure nothing happens. This means the domain has no dead ends since any action can be repeated until it succeeds.

4.9.3 Presentation of Results

In the following sections, we will be using similar tables and plots to summarise the results of our experiments:

- **Rank tables.** To obtain these tables, we start by creating a ranking of algorithms’ runtimes for each instance, where the fastest algorithm on an instance gets rank 1, the second-fastest gets 2, and so on. If multiple algorithms tie with identical runtimes, then these algorithms receive the average rank of the group, e.g., if algorithm a is strictly fastest, algorithms b, c, d are tied second, and algorithm e is strictly slowest, then a receives rank 1, b, c, d all receive $\frac{2+3+4}{3} = 3$, and e gets 5. If an algorithm times out on an instance, it receives the largest rank for that instance, tied among all algorithms that timed out. Our rank tables present each algorithm’s average (mean) rank over all relevant instances. Rankings are an intuitive way to determine which algorithms are fastest overall, and abstract away from the concrete times taken. Different rank tables *are not comparable*.
- **Coverage tables.** This is a standard way to summarise results in the planning community. For each domain, we present the number of instances that an algorithm and heuristic were able to solve before timing out. This gives a good indication of how well an algorithm scales, again without worrying about concrete timings. Different coverage tables *are comparable*.
- **Cumulative plots.** These plots show how many instances an algorithm and heuristic was able to solve with a budget over time, number of Q -values, or number of calls to the heuristic. In the cumulative plot over time, every point x, y on a curve denotes that the relevant algorithm and heuristic solved y -many instances within x seconds, and the other plots are similar w.r.t. their respective budgets. Note that these plots effectively show total coverage, as in the coverage table, but over slices of time (or Q -values, or heuristic calls). Different cumulative plots *are comparable*.
- **Partial SSP size percentage tables.** These tables show how large an algorithm and heuristic’s final partial SSP is, as a percentage of a baseline’s final partial SSP sizes. Thus, the $|\hat{S}| \%$ and $|\hat{A}| \%$ give the number of states $|\hat{S}|$ and actions $|\hat{A}| = \sum_{s \in \hat{S}} |\hat{A}(s)|$ in the relevant algorithm’s final partial SSP, as a percentage of the baseline’s counterparts. We use a percentage w.r.t. the baseline rather than the entire SSP because we want to emphasise the difference from the baseline, rather than how much smaller the partial SSPs are than the original SSP.⁶ Note that these tables can only feature iLAO* and CG-iLAO* type algorithms, because we have not defined partial SSPs for others. Different partial SSP size tables *are only comparable if the baseline is the same*. We consider a single baseline across all these tables, so they are indeed comparable.
- **Per-problem tables.** We present more detailed tables that give the coverage, average runtime, and average Q -values for each algorithm and heuristic per problem (rather than per domain) at <https://github.com/schmlz/cgilao/tree/main/results> and (Schmalz and Trevizan, 2023). These are only given externally, but we will occasionally refer to them in this section.

4.9.4 What Are The Best Versions of CG-iLAO*?

We investigate which settings of CG-iLAO* perform best, in order to eliminate unsuccessful variants and focus on only successful ones in the following sections. In particular, we are comparing CG-iLAO* with different expansion settings:

- CG-iLAO*_{single}: expand with a single greedy action (Single-Greedy-Action from sec. 4.6.1).

⁶Moreover, our SSPs are so large that it is impractical to compute the reachable states to calculate its size — recall that we use Probabilistic PDDL to represent our problems, and the state-space can be exponentially large w.r.t. the encoding.

ROC		PDB		LMCut	
CG-iLAO _{tied} *	2.12 ± 0.08	CG-iLAO _{tied} *	2.24 ± 0.08	CG-iLAO _{tied} *	2.25 ± 0.07
CG-iLAO _{single} *	2.17 ± 0.07	CG-iLAO _{single} *	2.39 ± 0.07	CG-iLAO _{single} *	2.30 ± 0.07
CG-iLAO _{trial} *	2.68 ± 0.09	CG-iLAO _{trial} *	2.78 ± 0.09	CG-iLAO _{trial} *	2.63 ± 0.08
CG-iLAO _{FF-AO} *	3.96 ± 0.08	CG-iLAO _{FF-AO} *	3.75 ± 0.08	CG-iLAO _{FF-AO} *	3.80 ± 0.07
CG-iLAO _{FF-MLO} *	4.08 ± 0.08	CG-iLAO _{FF-MLO} *	3.85 ± 0.08	CG-iLAO _{FF-MLO} *	4.02 ± 0.08

Table 4.1: Runtime ranking of CG-iLAO* variants within a specified heuristic (mean and 95% C.I. over all instances).

- CG-iLAO_{tied}*: expand with all tied-greedy actions (**Add-All-Greedy-Actions** from sec. 4.6.1).
- CG-iLAO_{trial}*: expand with rollout using trial (sec. 4.6.2). We use $t_{\max} = 100$, because preliminary results suggested this was a reasonable choice.
- CG-iLAO_{FF-AO}*: expand with rollout using FF on all-outcomes determinisation (sec. 4.6.2).
- CG-iLAO_{FF-MLO}*: expand with rollout using FF on most-likely-outcomes determinisation (sec. 4.6.2).

Tab. 4.1 is a rank table that presents the average ranks and 95% C.I. for the variants of CG-iLAO*. For all heuristics, the ordering of ranks is identical: from 1 to 5, we have CG-iLAO_{tied}*, CG-iLAO_{single}*, CG-iLAO_{trial}*, CG-iLAO_{FF-AO}*, CG-iLAO_{FF-MLO}*. The 95% C.I. shows that our experiments were unable to find a statistical difference between the average rank of the top two (CG-iLAO_{tied}* and CG-iLAO_{single}*), but there is a statistically significant difference between the average ranks of the top two and CG-iLAO_{trial}*, and between CG-iLAO_{trial}* and the variants with FF expansion. In the coverage table tab. 4.2, we see that CG-iLAO_{tied}*, CG-iLAO_{single}*, and CG-iLAO_{trial}* have similar total coverage, and the FF expansions are clustered together with significantly lower total coverage. Observe that with ROC, CG-iLAO_{single}* has a slightly higher total coverage than CG-iLAO_{tied}* by 7 instances, even though it has a lower average rank. This is not a contradiction, and happens because CG-iLAO_{tied}* tends to be faster on the other instances. Furthermore, the instances that CG-iLAO_{single}* solves and CG-iLAO_{tied}* fails to solve are not solved by the other algorithms either, so CG-iLAO_{tied}*'s tied-last rank remains small there. Overall, the coverage supports that CG-iLAO_{tied}* and CG-iLAO_{single}* are the fastest variants. Finally, we consider the cumulative plot over time in fig. 4.9. This clearly shows that CG-iLAO_{tied}*, CG-iLAO_{single}*, and CG-iLAO_{trial}* outperform the FF expansions with a significant margin over all heuristics. This performance gap becomes clear for easy problems that take around 50 seconds to solve, and is carried through all larger problems. Among the top three algorithms (CG-iLAO_{tied}*, CG-iLAO_{single}*, CG-iLAO_{trial}*), it is difficult to determine a clear winner, but we observe that CG-iLAO_{trial}*'s curve is underneath the curves of CG-iLAO_{tied}* and CG-iLAO_{single}* most of the time. Combining these observations, we declare that overall, CG-iLAO_{tied}* and CG-iLAO_{single}* are the tied-best variants of CG-iLAO*, CG-iLAO_{trial}* is a close second, and the FF expansions are last by a significant margin. We now compare the variants in more detail.

We have seen that CG-iLAO_{tied}* and CG-iLAO_{single}* are the best-performing variants of CG-iLAO*, with no significant difference between them, overall. However, we highlight that their behaviours are domain dependent, i.e., they have different strengths and weaknesses, and are not simply duplicates of each other. In particular, we focus on Rand with ROC, where CG-iLAO_{single}* has a higher coverage than CG-iLAO_{tied}* by 7 instances. Rand is an outlier among our domains, because it is the only one that is not inspired by real-world problems, and it has many more applicable actions per state than our other domains, on the order of 10^4 where the others have 10^1 or 10^2 (as we will see in tab. 4.7). The key difference is that CG-iLAO_{single}* has significantly fewer actions in its final partial SSP, by up to two orders of magnitude (we will see this later in sec. 4.9.6). Interestingly, this does not translate to savings in Q -value computations nor heuristic calls, but rather, the difference in performance comes from the overhead of managing the partial SSP's data-structures. Clearly, CG-iLAO_{tied}* is adding many tied-greedy actions to its partial SSP that are not necessary for finding an optimal policy, and CG-iLAO_{single}* avoids the issue in this case by adding only one greedy action in its expansion. CG-iLAO_{single}* also produces smaller partial SSPs on Rand with the other heuristics, but neither of the algorithms are able to solve the large problems with these heuristics, so the effect of differently sized partial SSPs is not noticeable. Over other domains, CG-iLAO_{single}* generally produces smaller partial SSPs than CG-iLAO_{tied}* (we investigate this further in sec. 4.9.6), but the size difference is not as extreme, and does not result in a significant performance difference. On the other hand, CG-iLAO_{tied}* has a higher coverage on S&R with LMCut and PDB. As we will see in sec. 4.9.5, these are problems where iLAO* performs well, i.e., adding all applicable

		BW	Coresec	Elev	ExBW	PARC-N	PARC-R	Rand	□TW	S&R	Sched	Sys	ΔTW	Zeno	total
ROC	# of instances	110	35	75	105	30	30	75	70	25	45	20	40	45	705
	CG-iLAO* _{tied}	105	25	75	105	30	25	36	60	20	45	20	35	40	621
	CG-iLAO* _{single}	105	25	75	105	30	25	43	60	20	45	20	35	40	628
	CG-iLAO* _{trial}	105	25	70	105	30	24	35	60	21	45	20	35	40	615
	CG-iLAO* _{FF-AO}	100	20	59	95	30	20	35	60	20	35	10	25	30	539
	CG-iLAO* _{FF-MLO}	85	20	61	95	30	20	40	60	20	35	15	26	26	533
PDB	CG-iLAO* _{tied}	90	30	75	90	0	0	30	60	24	30	20	38	40	527
	CG-iLAO* _{single}	90	30	75	90	0	0	30	60	20	30	20	37	40	522
	CG-iLAO* _{trial}	90	30	70	90	0	0	30	60	24	30	20	35	40	519
	CG-iLAO* _{FF-AO}	85	30	61	90	5	0	30	60	20	35	10	21	35	482
	CG-iLAO* _{FF-MLO}	85	30	61	90	5	0	30	60	20	35	15	22	30	483
	CG-iLAO* _{tied}	45	25	70	101	30	20	20	65	24	45	20	30	25	520
LMCut	CG-iLAO* _{single}	45	25	71	100	30	20	20	65	20	45	20	30	25	516
	CG-iLAO* _{trial}	45	25	70	100	30	20	20	65	22	45	20	30	25	517
	CG-iLAO* _{FF-AO}	45	20	63	95	25	15	25	65	20	35	10	20	20	458
	CG-iLAO* _{FF-MLO}	45	20	62	95	22	15	25	65	20	35	15	20	15	454

Table 4.2: Coverage for each CG-iLAO* variant and considered heuristic over the benchmark domains. The highest coverage for each problem is marked with boldface.

actions pays off. In these problems, the partial SSP sizes are similar, but CG-iLAO*_{tied} has fewer Q -values, presumably because it avoids the overhead of inserting missing tied-greedy actions. Outside these configurations, there is no significant difference between the two. Overall, this comparison shows that CG-iLAO*_{tied} and CG-iLAO*_{single}, while similar, have different strengths and weaknesses that make them more or less suitable to specific problems.

Now, we investigate CG-iLAO*_{trial}. Using the cumulative plots (fig. 4.9), it is difficult to distinguish CG-iLAO*_{trial} from CG-iLAO*_{tied} and CG-iLAO*_{single} in terms of time and Q -values, but we can clearly see that it is less performant in terms of heuristic calls from 0.53×10^6 calls for ROC and PDB, and for LMCut it is again difficult to distinguish. The similarity in Q -values makes it clear that CG-iLAO*_{trial}'s trials fail to add all useful actions in a way that saves Q -values, but it is also not adding a significant number of unneeded actions that would inflate the number of Q -values. The larger number of heuristic calls tells us that CG-iLAO*_{trial} considers more states than CG-iLAO*_{tied} and CG-iLAO*_{single}, which makes sense because its trials deliberately probe states beyond the policy envelope. This is arguably a weakness because CG-iLAO*_{trial} needs to perform more heuristic computations without savings in Q -values. However, with the less informative LMCut heuristic, this weakness disappears and CG-iLAO*_{trial} has a similar number of heuristic calls to the others. This is because CG-iLAO*_{tied} and CG-iLAO*_{single} need to eventually expand the same states to prove optimality. In this case, CG-iLAO*_{trial} is more competitive, and even has a slightly higher total coverage than CG-iLAO*_{single} by 1 instance. This suggests that the trials do not pay off with informative heuristics: it is more efficient to expand greedily; whereas with less informative heuristics, it can be beneficial to run a trial before deeming an action useful.

Overall, the FF expansions performed poorly over our benchmarks, which can be attributed to two factors: the FF expansion (often) adds unneeded states and actions, and calling FF for each expansion is expensive. In the cumulative plots in fig. 4.9, we can see that the FF expansions incur significantly more Q -value computations and heuristic calls, which confirms that more states and actions are being considered than necessary. Since Q -value computations and heuristic calls are the main computation expenses of CG-iLAO*, it is clear that the FF expansion variants can not keep up. This happens because FF's plans do not coincide with an optimal policy, which is to be expected, given that it completely ignores probabilistic effects with the determinisations. These issues are only exacerbated by the additional cost of running FF for each expansion, rather than a single Bellman backup. The only case where FF expansions pay off in our experiments is in PARC-N with PDB. This is because the PDB heuristic is very uninformative on the PARC-N domain, and FF expands states and actions that are sufficiently close to the optimal policy, so that CG-iLAO* can terminate.

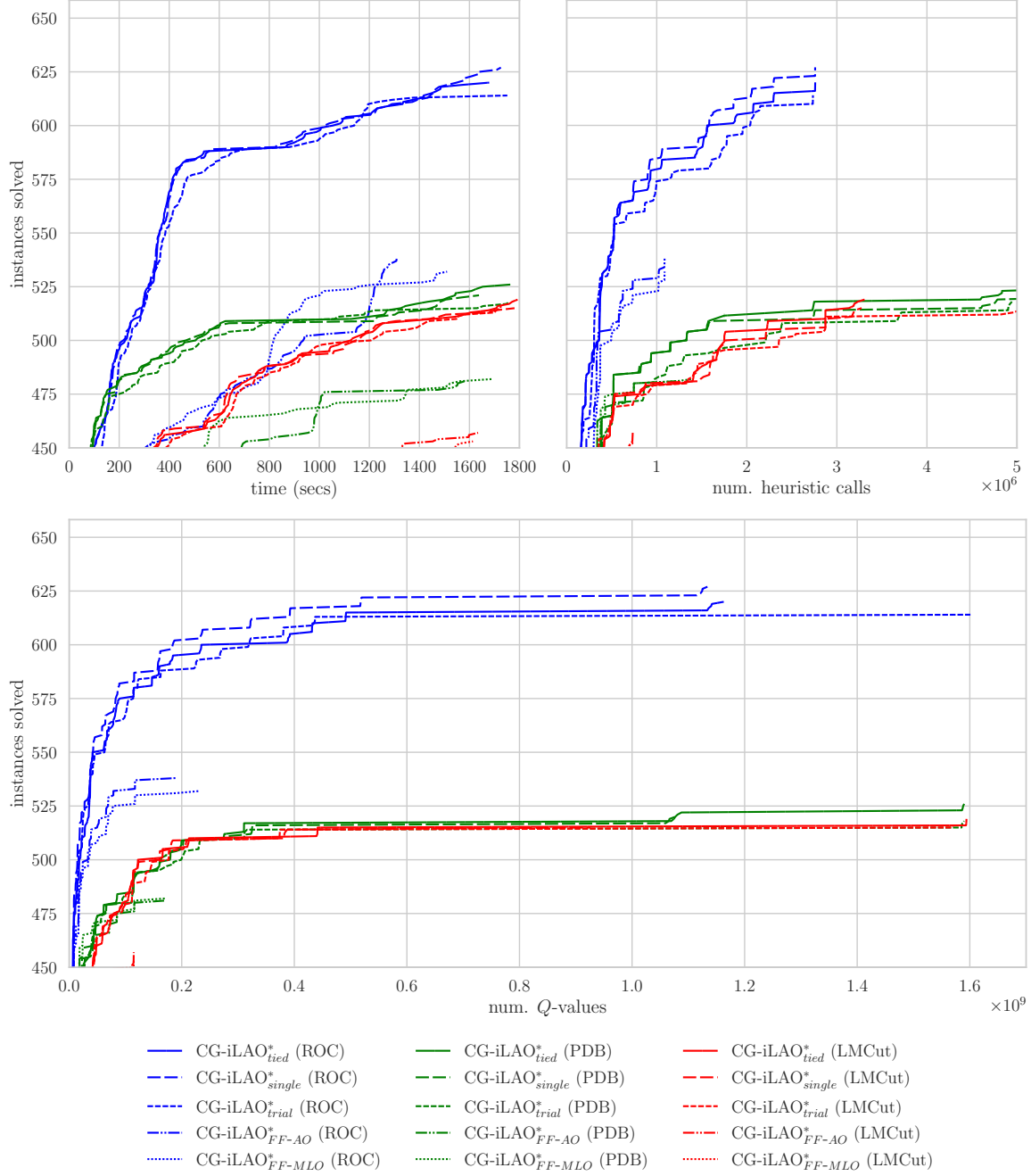


Figure 4.9: For each algorithm and heuristic, the cumulative plot of how many problems and seeds were solved w.r.t. time in seconds (top left), number of Q -values (top right), and number of calls to heuristic (bottom). To focus on where the algorithms are different we start the y -axis at 450 instances, and end the x -axis at 5×10^6 heuristic calls.

4.9.5 Comparing CG-iLAO* To The State-of-the-art

Now, we compare CG-iLAO* with two state-of-the-art optimal heuristic-search algorithms: iLAO* (Hansen and Zilberstein, 2001) and LRTDP (Bonet and Geffner, 2003b). We only consider CG-iLAO*_{tied} and CG-iLAO*_{single}, the best-performing variants of CG-iLAO* according to sec. 4.9.4. We consider two implementations of iLAO* in this experiment: iLAO* as implemented in the original paper (Schmalz and Trevizan, 2024b), and CG-iLAO*_{all}, which is CG-iLAO* with the complete Bellman expansion (see sec. 4.6.1). Although CG-iLAO*_{all} implements iLAO*, there are some subtle differences in implementation (discussed in sec. 4.2 and sec. 4.4) with large impacts on performance.

Using tab. 4.3, we can see that the ordering of the first three average ranks is consistent over all heuristics,

ROC		PDB		LMCut	
CG-iLAO* _{tied}	2.13 ± 0.08	CG-iLAO* _{tied}	2.27 ± 0.08	CG-iLAO* _{tied}	2.32 ± 0.07
CG-iLAO* _{single}	2.23 ± 0.08	CG-iLAO* _{single}	2.43 ± 0.08	CG-iLAO* _{single}	2.40 ± 0.08
CG-iLAO* _{all}	2.96 ± 0.08	CG-iLAO* _{all}	2.86 ± 0.07	CG-iLAO* _{all}	2.78 ± 0.07
iLAO*	3.84 ± 0.09	LRTDP	3.63 ± 0.10	iLAO*	3.68 ± 0.09
LRTDP	3.84 ± 0.11	iLAO*	3.82 ± 0.09	LRTDP	3.81 ± 0.09

Table 4.3: Runtime ranking of CG-iLAO* and state-of-the-art within a specified heuristic (mean and 95% C.I. over all instances).

from 1 to 3: CG-iLAO*_{tied}, CG-iLAO*_{single}, CG-iLAO*_{all}. The remaining orderings are not consistent across heuristics: LRTDP and iLAO* alternate. The 95% C.I. reveals that our experiments can not distinguish between CG-iLAO*_{tied} and CG-iLAO*_{single} (consistent with sec. 4.9.4), nor can we distinguish between LRTDP and iLAO*. For the other ordering of rankings there is no overlap between 95% C.I.s: CG-iLAO*_{tied} and CG-iLAO*_{single} are ranked better than CG-iLAO*_{all}, which is itself ranked better than iLAO* and LRTDP.

In terms of total coverage (tab. 4.4), CG-iLAO*_{single}(ROC) is the leader with 628 instances solved, then CG-iLAO*_{tied}(ROC) and CG-iLAO*_{all}(ROC) are close second with 621 instances solved. The next best is iLAO*(ROC) with 580 instances solved, 41 behind. Note that CG-iLAO*_{tied}(ROC) has a lower rank than CG-iLAO*_{all}(ROC), even though they have the same coverage. This is because for the instances that both algorithms solved, CG-iLAO*_{tied}(ROC) tends to be faster, which can be seen in the cumulative plot over time (fig. 4.10): the curve of CG-iLAO*_{all}(ROC) lies underneath the curve of CG-iLAO*_{tied}(ROC), i.e., CG-iLAO*_{tied}(ROC) can solve more instances in the same time. Thus, CG-iLAO*_{tied}(ROC) has better performance than CG-iLAO*_{all}(ROC), because it has the same coverage but a lower rank over the solved instances. By similar argument CG-iLAO*_{single}(ROC) also has better performance than CG-iLAO*_{all}(ROC), strengthened by the fact that it has slightly higher coverage. Thus, CG-iLAO*_{tied}(ROC) and CG-iLAO*_{single}(ROC) are the best-performing algorithms.

This trend is similar over the other heuristics: CG-iLAO*_{tied} and CG-iLAO*_{single} are ranked first, and have the highest coverage; in this case, with strictly larger coverage than the next-best CG-iLAO*_{all}, strengthening the arguments from before. The cumulative plot over time (fig. 4.10) confirms this because the curves for CG-iLAO*_{tied} and CG-iLAO*_{single} lie above the other algorithms' curves within the same heuristic. Thus, we declare CG-iLAO*_{tied} and CG-iLAO*_{single} as the best algorithms overall.

Now, we analyse the different performances in more detail. The fundamental operations that take up most of an algorithm's runtime are Q -values computations and calls to the heuristic. There are other operations that may take significant runtime, but they can be considered overhead, e.g., tracking the current partial SSP and ensuring the applicable actions are up-to-date. It is important to recognise that timings are dependent on many factors, e.g., how optimised the implementation is for the hardware in use, whereas the number of Q -values and heuristic calls is a more robust measure. In that sense, the following discussions are more insightful than discussing the fastest algorithm.

We start by looking at the number of Q -values. Focusing on ROC, we see in the cumulative plot over Q -values in fig. 4.10 that CG-iLAO*_{single}(ROC) and CG-iLAO*_{tied}(ROC) require the fewest Q -values to solve their instances. Between CG-iLAO*_{single}(ROC) and the third-best CG-iLAO*_{all}(ROC), the lead is clear almost immediately, and it maintains a large gap over all problems. From 0.04×10^9 to 0.64×10^9 Q -values, CG-iLAO*_{tied}(ROC) leads over CG-iLAO*_{all}(ROC) with a gap of around 5-15 instances to CG-iLAO*_{all}(ROC). From 0.64×10^9 to 1.13×10^9 Q -values CG-iLAO*_{all}(ROC) catches up to CG-iLAO*_{tied}(ROC), and then CG-iLAO*_{tied}(ROC) makes a jump of 4 instances; this is happening because two problems (Elev 04-13 and S&R 05) require significantly more Q -values to solve than the rest. In fact, this emphasises how many fewer Q -values CG-iLAO*_{tied}(ROC) requires, because it solves these large problems with 1.16×10^9 Q -values, and CG-iLAO*_{all}(ROC) requires 1.67×10^9 . The gap between the CG-iLAO* variants and other algorithms is even more significant. Thus, CG-iLAO*_{tied}(ROC) computes the fewest Q -values. For the other heuristics, we observe a similar trend, but more extreme in favour of CG-iLAO*_{tied} and CG-iLAO*_{single}. Thus, the top two CG-iLAO* variants are the algorithms that compute the fewest Q -values.

To investigate the number of heuristic calls, we use the cumulative plot over heuristic calls (fig. 4.10). We see that CG-iLAO*_{tied}, CG-iLAO*_{single}, and CG-iLAO*_{all} are decisively ahead of iLAO* and LRTDP

		BW	Coresec	Elev	ExBW	PARC-N	PARC-R	Rand	□TW	S&R	Sched	Sys	ΔTW	Zeno	total
ROC	# of instances	110	35	75	105	30	30	75	70	25	45	20	40	45	705
	LRTDP	55	25	75	105	30	20	33	55	25	45	20	35	20	543
	iLAO*	105	25	69	100	30	25	38	48	20	35	20	30	35	580
	CG-iLAO* _{all}	105	25	70	105	30	27	35	60	25	45	20	35	39	621
	CG-iLAO* _{tied}	105	25	75	105	30	25	36	60	20	45	20	35	40	621
	CG-iLAO* _{single}	105	25	75	105	30	25	43	60	20	45	20	35	40	628
PDB	LRTDP	84	30	75	90	0	0	30	60	25	35	20	35	30	514
	iLAO*	85	35	70	85	0	0	30	57	20	30	20	30	35	497
	CG-iLAO* _{all}	85	30	70	90	0	0	30	60	25	30	20	37	40	517
	CG-iLAO* _{tied}	90	30	75	90	0	0	30	60	24	30	20	38	40	527
	CG-iLAO* _{single}	90	30	75	90	0	0	30	60	20	30	20	37	40	522
LMCut	LRTDP	45	20	75	100	30	5	20	59	25	40	20	25	15	479
	iLAO*	45	25	70	95	30	20	20	55	20	35	20	25	25	485
	CG-iLAO* _{all}	45	25	70	100	30	20	20	65	25	40	20	30	25	515
	CG-iLAO* _{tied}	45	25	70	101	30	20	20	65	24	45	20	30	25	520
	CG-iLAO* _{single}	45	25	71	100	30	20	20	65	20	45	20	30	25	516

Table 4.4: Coverage for CG-iLAO* and state-of-the-art with each considered heuristic over the benchmark domains. The highest coverage for each problem is marked with boldface.

over all heuristics, in as few calls as 0.5×10^6 . Between these top three, there is no consistent trend over heuristics.

4.9.6 How Many Actions Does CG-iLAO* Ignore?

The key idea of CG-iLAO* is that it ignores inactive actions and only adds actions when they are needed, with the aim of producing a smaller partial SSP. In this section, we confirm experimentally that indeed CG-iLAO* has smaller partial SSPs than iLAO*, and quantify by how much, i.e., how many actions CG-iLAO* ignores.

For CG-iLAO*_{tied}, CG-iLAO*_{single}, and iLAO*, we present the number of states and actions in their final partial SSPs as a percentage of the number of states and actions in CG-iLAO*_{all}'s final partial SSP in tab. 4.5. The table gives for each domain, the percentages averaged over instances where the algorithm being considered and the baseline both terminate. Since we require the baseline to terminate, we use CG-iLAO*_{all} rather than iLAO*, because it solves more instances, letting us consider more instances. In tab. 4.5 we see that the number of states in the final partial SSPs $|\hat{S}|$ %, is around 100% for all algorithms and heuristics. This is what we expect, because CG-iLAO* ignores actions, and has no mechanism for ignoring more states than iLAO*. We acknowledge the notable outliers at Rand with PDB, where CG-iLAO*_{single} has less than 60% and iLAO* has over 2000% of CG-iLAO*_{all}'s states, but this is a unique exception which does not contribute much.

More importantly, the number of actions in the final partial SSPs $|\hat{A}|$ %, is always smaller than 101% for both CG-iLAO* variants, and below 50% in most problems. This confirms that indeed CG-iLAO* is able to leave out many actions that iLAO* considers. Exceptions to this are S&R and Sys, where the CG-iLAO* variants have over 90% of CG-iLAO*_{all}'s actions. Note that this is reflected by CG-iLAO*_{all} having higher coverage on S&R than the CG-iLAO* variants. It is not entirely clear why CG-iLAO* has such large partial SSPs on these problems, but we conjecture that it is because the heuristics are not informative enough, and with more informative heuristics CG-iLAO* would be able to ignore more actions. Between CG-iLAO*_{tied} and CG-iLAO*_{single}, we see that CG-iLAO*_{single} tends to have fewer actions in its final partial SSP. This is unsurprising, because CG-iLAO*_{tied} can add unneeded actions in its expansion, whereas CG-iLAO*_{single} only adds actions that are necessary at some point. Note, that this difference in partial SSP size is not reflected in the number of Q -values computed, as discussed in sec. 4.9.4, because CG-iLAO*_{single} incurs additional overhead in checking which actions should be added. The exception is again Rand where we have seen CG-iLAO*_{single}(ROC) obtains highest coverage, precisely because its final partial SSP is significantly smaller.

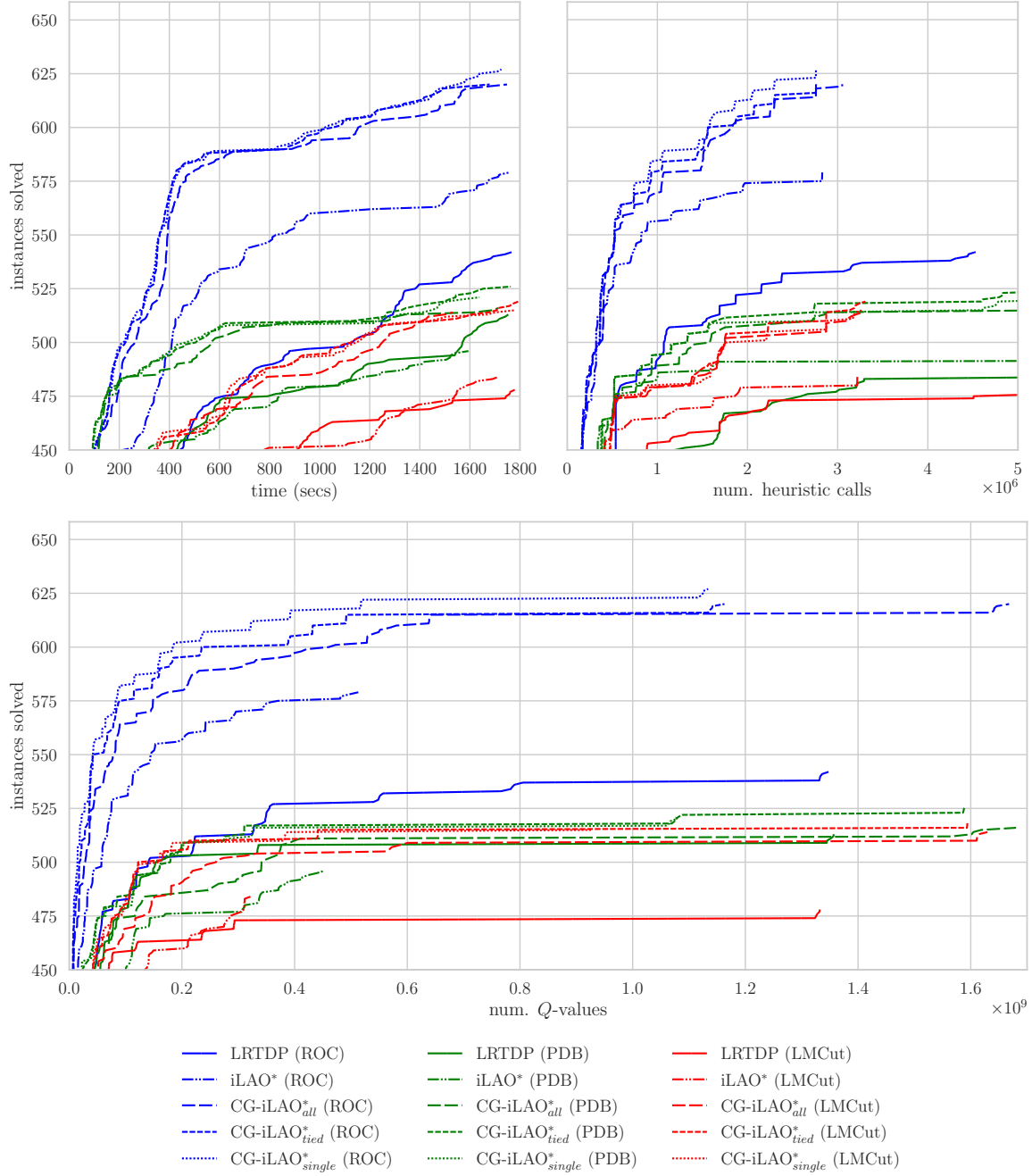


Figure 4.10: For each algorithm and heuristic, the cumulative plot of how many problems and seeds were solved w.r.t. time in seconds (top left), number of Q -values (top right), and number of calls to heuristic (bottom). To focus on where the algorithms are different we start the y -axis at 450 instances, and end the x -axis at 5×10^6 heuristic calls.

To make the final partial SSP sizes more concrete, we present the actual final partial SSP sizes for select problems in tab. 4.7. This gives a sense of scale of how many actions are being saved, bearing in mind that we picked smaller problems where all algorithms had close-to-complete coverage.

To summarise, the CG-iLAO* variants often have significantly smaller final partial SSPs than the iLAO* variants, confirming that our mechanism for ignoring actions has a significant impact.

To gain a better insight of how many actions are ignored by CG-iLAO*, we investigate how dense CG-iLAO*'s partial SSPs are, i.e., how many actions were added to CG-iLAO*'s partial SSP, out of the possible actions that could have been added. We define the density of state s as $|\hat{A}(s)|/|A(s)|$. To start, we restrict our attention to a single representative problem from each domain, solved by CG-iLAO*_{tied}

		CG-iLAO [*] _{tied}		CG-iLAO [*] _{single}		iLAO [*]			
		$ \hat{S} %$	$ \hat{A} %$	$ \hat{S} %$	$ \hat{A} %$	$ \hat{S} %$	$ \hat{A} %$	$ \hat{S} %$	$ \hat{A} %$
BW	ROC	91.8 ± 1.3	40.8 ± 1.2	98.7 ± 3.5	33.2 ± 2.8	86.8 ± 1.5	85.3 ± 1.5	86.8 ± 1.5	85.3 ± 1.5
	LMCut	97.5 ± 0.6	45.5 ± 1.4	97.6 ± 0.5	42.5 ± 2.0	88.3 ± 0.8	84.7 ± 1.2	88.3 ± 0.8	84.7 ± 1.2
	PDB	90.9 ± 1.4	39.7 ± 0.9	90.2 ± 3.0	30.9 ± 1.6	85.9 ± 1.4	84.1 ± 1.5	85.9 ± 1.4	84.1 ± 1.5
Coresec	ROC	100.2 ± 0.3	33.6 ± 1.9	100.2 ± 0.4	20.5 ± 1.1	85.7 ± 0.6	142.0 ± 1.8	85.7 ± 0.6	142.0 ± 1.8
	LMCut	100.0 ± 0.1	39.0 ± 1.9	99.9 ± 0.1	24.9 ± 1.1	95.8 ± 0.7	193.4 ± 2.2	95.8 ± 0.7	193.4 ± 2.2
	PDB	99.6 ± 0.5	29.0 ± 1.6	100.0 ± 0.4	22.2 ± 1.6	87.1 ± 0.5	113.9 ± 1.5	87.1 ± 0.5	113.9 ± 1.5
Elev	ROC	97.6 ± 1.1	60.5 ± 1.6	98.1 ± 1.3	53.3 ± 2.2	94.9 ± 1.1	94.9 ± 1.1	94.9 ± 1.1	94.9 ± 1.1
	LMCut	93.1 ± 3.0	45.0 ± 3.1	96.0 ± 2.1	40.4 ± 3.0	90.2 ± 1.4	90.3 ± 1.4	90.2 ± 1.4	90.3 ± 1.4
	PDB	97.0 ± 1.5	56.9 ± 1.7	98.2 ± 2.0	50.1 ± 2.2	92.3 ± 1.6	92.5 ± 1.6	92.3 ± 1.6	92.5 ± 1.6
ExBW	ROC	95.8 ± 1.4	44.8 ± 1.5	99.9 ± 3.2	39.6 ± 1.7	97.9 ± 0.7	103.5 ± 1.8	97.9 ± 0.7	103.5 ± 1.8
	LMCut	93.9 ± 2.0	39.2 ± 2.1	99.2 ± 3.3	37.1 ± 1.8	97.8 ± 0.9	102.2 ± 2.1	97.8 ± 0.9	102.2 ± 2.1
	PDB	95.7 ± 1.9	53.9 ± 3.4	98.7 ± 3.4	49.9 ± 3.7	97.6 ± 1.1	101.7 ± 2.0	97.6 ± 1.1	101.7 ± 2.0
PARC-N	ROC	96.6 ± 0.6	44.7 ± 0.9	97.0 ± 0.6	32.7 ± 0.9	87.5 ± 1.1	92.7 ± 1.1	87.5 ± 1.1	92.7 ± 1.1
	LMCut	93.5 ± 0.9	44.0 ± 1.2	94.5 ± 0.8	35.2 ± 1.3	75.3 ± 1.5	80.9 ± 1.8	75.3 ± 1.5	80.9 ± 1.8
PARC-R	ROC	99.6 ± 0.2	44.1 ± 1.0	99.6 ± 0.2	43.0 ± 1.0	86.9 ± 1.7	86.6 ± 1.6	86.9 ± 1.7	86.6 ± 1.6
	LMCut	101.1 ± 0.6	51.9 ± 1.0	101.9 ± 0.9	50.1 ± 1.2	95.7 ± 1.0	95.8 ± 0.9	95.7 ± 1.0	95.8 ± 0.9
Rand	ROC	89.8 ± 8.6	9.1 ± 1.8	93.7 ± 11.3	0.9 ± 0.3	96.3 ± 3.1	96.3 ± 3.1	96.3 ± 3.1	96.3 ± 3.1
	LMCut	100.0 ± 0.0	9.9 ± 3.9	100.0 ± 0.0	1.8 ± 0.7	100.0 ± 0.0	100.0 ± 0.0	100.0 ± 0.0	100.0 ± 0.0
	PDB	95.1 ± 5.3	17.0 ± 8.1	62.5 ± 16.7	1.0 ± 0.5	2369.7 ± 3093.0	3987.8 ± 5296.5	2369.7 ± 3093.0	3987.8 ± 5296.5
□TW	ROC	100.0 ± 0.2	31.5 ± 0.8	100.0 ± 0.2	33.1 ± 1.6	100.0 ± 0.0	126.0 ± 12.9	100.0 ± 0.0	126.0 ± 12.9
	LMCut	99.9 ± 0.2	30.9 ± 0.9	100.0 ± 0.1	32.8 ± 1.8	99.7 ± 0.3	125.8 ± 12.9	99.7 ± 0.3	125.8 ± 12.9
	PDB	97.9 ± 1.0	31.0 ± 1.5	98.8 ± 1.1	31.5 ± 1.8	100.0 ± 1.4	127.0 ± 11.3	100.0 ± 1.4	127.0 ± 11.3
S&R	ROC	100.0 ± 0.1	91.6 ± 0.8	100.2 ± 0.1	91.4 ± 0.7	101.0 ± 0.2	104.5 ± 0.4	101.0 ± 0.2	104.5 ± 0.4
	LMCut	100.1 ± 0.1	92.0 ± 0.8	100.0 ± 0.1	91.3 ± 0.7	100.9 ± 0.2	104.5 ± 0.4	100.9 ± 0.2	104.5 ± 0.4
	PDB	100.3 ± 0.1	92.2 ± 0.8	100.5 ± 0.2	91.4 ± 0.7	101.2 ± 0.2	104.2 ± 0.3	101.2 ± 0.2	104.2 ± 0.3
Sched	ROC	100.7 ± 0.3	78.1 ± 0.7	100.9 ± 0.5	50.7 ± 1.1	99.0 ± 0.8	100.0 ± 0.1	99.0 ± 0.8	100.0 ± 0.1
	LMCut	99.0 ± 0.5	73.8 ± 3.2	101.4 ± 0.8	50.2 ± 1.2	99.3 ± 0.6	100.2 ± 0.1	99.3 ± 0.6	100.2 ± 0.1
	PDB	100.0 ± 0.0	79.6 ± 0.5	100.0 ± 0.0	51.1 ± 1.3	100.0 ± 0.0	100.0 ± 0.0	100.0 ± 0.0	100.0 ± 0.0
Sys	ROC	100.0 ± 0.1	99.5 ± 0.3	100.0 ± 0.0	97.4 ± 0.3	100.0 ± 0.1	100.0 ± 0.0	100.0 ± 0.1	100.0 ± 0.0
	LMCut	100.0 ± 0.0	99.5 ± 0.4	100.0 ± 0.0	97.3 ± 0.4	100.1 ± 0.1	100.1 ± 0.0	100.1 ± 0.1	100.1 ± 0.0
	PDB	100.0 ± 0.0	100.0 ± 0.0	100.0 ± 0.0	98.4 ± 0.4	100.0 ± 0.0	100.0 ± 0.0	100.0 ± 0.0	100.0 ± 0.0
ΔTW	ROC	98.0 ± 0.8	64.4 ± 0.8	97.2 ± 1.0	63.8 ± 0.9	99.7 ± 0.4	99.8 ± 0.4	99.7 ± 0.4	99.8 ± 0.4
	LMCut	100.7 ± 0.6	68.6 ± 1.5	100.0 ± 0.9	67.0 ± 1.5	98.0 ± 0.9	98.3 ± 0.9	98.0 ± 0.9	98.3 ± 0.9
	PDB	98.2 ± 0.8	65.7 ± 0.7	98.0 ± 0.8	65.5 ± 0.7	99.6 ± 0.4	99.8 ± 0.5	99.6 ± 0.4	99.8 ± 0.5
Zeno	ROC	88.4 ± 0.5	37.1 ± 0.6	86.7 ± 0.3	30.1 ± 0.8	80.5 ± 1.1	77.2 ± 1.0	80.5 ± 1.1	77.2 ± 1.0
	LMCut	89.0 ± 1.2	34.3 ± 0.4	88.5 ± 1.3	31.6 ± 0.5	78.0 ± 1.4	74.2 ± 1.1	78.0 ± 1.4	74.2 ± 1.1
	PDB	89.9 ± 1.0	44.0 ± 1.3	88.0 ± 0.9	34.0 ± 1.1	82.1 ± 1.6	79.1 ± 1.4	82.1 ± 1.6	79.1 ± 1.4

Table 4.5: Size of each algorithm’s final partial SSP, as a percentage of CG-iLAO^{*}_{all}’s final partial SSP. These values are means over instances where the considered algorithm and CG-iLAO^{*}_{all} both terminated.

	CG-iLAO* _{tied}					CG-iLAO* _{single}					iLAO*					CG-iLAO* _{all}				
	$ \hat{S} $	$ \hat{A} $	$ \hat{A}^{max} $	$ \hat{S} $	$ \hat{A} $	$ \hat{A}^{max} $	$ \hat{S} $	$ \hat{A} $	$ \hat{A}^{max} $	$ \hat{S} $	$ \hat{A} $	$ \hat{A}^{max} $	$ \hat{S} $	$ \hat{A} $	$ \hat{A}^{max} $	$ \hat{S} $	$ \hat{A} $	$ \hat{A}^{max} $		
BW	ROC	38 197 ± 187	107 485 ± 619	264 000 ± 1337	38 041 ± 160	69 964 ± 384	263 097 ± 1068	35 601 ± 245	249 229 ± 1769	42 269 ± 332	310 921 ± 2560									
	PDB	38 604 ± 308	112 018 ± 782	266 356 ± 2200	38 141 ± 274	70 045 ± 740	263 724 ± 1758	35 514 ± 198	248 260 ± 1479	42 229 ± 398	310 213 ± 3083									
	08-11	ROC	12 961 ± 776	42 333 ± 2215	107 413 ± 6639	12 530 ± 402	22 106 ± 549	104 209 ± 3392	12 172 ± 797	102 326 ± 6804	14 114 ± 907	116 851 ± 7586								
	08-12	ROC	12 961 ± 776	42 333 ± 2215	107 413 ± 6639	12 530 ± 402	22 106 ± 549	104 209 ± 3392	12 172 ± 797	102 326 ± 6804	14 114 ± 907	116 851 ± 7586								
Elev	ROC	511 935 ± 243	2 236 293 ± 8866	3 063 040 ± 1474	512 158 ± 286	2 143 683 ± 7449	3 064 324 ± 1739	513 039 ± 171	3 069 532 ± 959	515 568 ± 988	3 084 272 ± 5788									
	LMC _{cut}	441 272 ± 717	1 652 518 ± 6622	2 649 696 ± 4449	441 162 ± 810	1 551 021 ± 8734	2 648 887 ± 4855	425 933 ± 3021	2 557 899 ± 18 032	452 195 ± 3016	2 714 150 ± 17 807									
	PDB	506 141 ± 399	2 134 977 ± 8328	3 028 903 ± 2315	506 320 ± 222	2 045 625 ± 6661	3 029 960 ± 1276	504 455 ± 808	3 018 725 ± 4709	510 224 ± 1157	3 052 950 ± 6776									
	ROC	8 ± 5	40 ± 31	707 ± 582	8 ± 5	12 ± 9	707 ± 582	8 ± 5	707 ± 582	8 ± 5	707 ± 582									
Rand	LMC _{cut}	3 ± 0	54 ± 49	264 ± 52	3 ± 0	4 ± 1	264 ± 52	3 ± 0	264 ± 52	3 ± 0	264 ± 52									
	PDB	4 ± 0	16 ± 2	265 ± 52	4 ± 1	5 ± 1	265 ± 52	4 ± 0	265 ± 52	4 ± 0	265 ± 52									
	ROC	4 ± 1	506 ± 33	3690 ± 954	5 ± 2	6 ± 5	4731 ± 2508	4 ± 1	3690 ± 954	4 ± 1	3690 ± 954									
	PDB	10 ± 2	468 ± 50	9712 ± 2326	10 ± 5	15 ± 9	9790 ± 4901	10 ± 2	9448 ± 2300	10 ± 2	9448 ± 2300									
S&R	ROC	25 ± 5	7634 ± 1621	46 300 ± 9419	25 ± 6	42 ± 11	45 070 ± 10 729	26 ± 4	48 143 ± 7394	26 ± 4	48 143 ± 7394									
	LMC _{cut}	7 ± 0	845 ± 3	11 992 ± 137	7 ± 0	10 ± 0	11 992 ± 137	7 ± 0	12 001 ± 42	7 ± 0	12 001 ± 42									
	PDB	128 ± 12	37 172 ± 4838	208 000 ± 20 635	98 ± 23	171 ± 41	162 899 ± 39 785	197 ± 29	325 885 ± 48 159	197 ± 29	325 905 ± 48 161									
	ROC	9910 ± 8	35 309 ± 32	39 577 ± 4	9921 ± 13	35 132 ± 30	39 583 ± 7	10 022 ± 15	39 633 ± 7	9904 ± 5	38 081 ± 2									
Sys	LMC _{cut}	9914 ± 8	35 152 ± 12	39 580 ± 4	9913 ± 4	35 088 ± 27	39 579 ± 2	10 024 ± 24	39 635 ± 12	9919 ± 14	38 088 ± 7									
	PDB	9890 ± 10	35 316 ± 34	39 573 ± 5	9894 ± 6	35 133 ± 20	39 575 ± 3	9993 ± 19	39 625 ± 9	9863 ± 2	38 102 ± 1									
	ROC	113 810 ± 17	527 263 ± 203	578 498 ± 8	113 820 ± 47	525 092 ± 167	578 502 ± 23	114 249 ± 76	578 717 ± 38	113 769 ± 30	560 917 ± 15									
	LMC _{cut}	113 817 ± 22	525 083 ± 43	578 501 ± 11	113 828 ± 13	524 661 ± 99	578 506 ± 7	114 234 ± 78	578 709 ± 39	113 799 ± 26	560 932 ± 13									
08-04	PDB	113 763 ± 32	527 146 ± 311	578 481 ± 16	113 766 ± 44	524 952 ± 158	578 483 ± 22	114 178 ± 78	578 689 ± 39	113 624 ± 14	560 916 ± 7									
	ROC	—	—	—	—	—	—	—	—	—	—									
	LMC _{cut}	1 251 642 ± 57	7 078 350 ± 557	7 682 167 ± 29	—	—	—	—	—	1 251 455 ± 60	7 485 144 ± 30									
	PDB	1 251 444 ± 65	7 109 673 ± 538	7 682 078 ± 33	—	—	—	—	—	1 250 771 ± 40	7 484 911 ± 20									
08-02	ROC	1541 ± 0	1725 ± 0	1729 ± 0	1541 ± 0	1697 ± 4	1729 ± 0	1539 ± 0	1727 ± 0	1541 ± 0	1729 ± 0									
	LMC _{cut}	1580 ± 0	1768 ± 0	1768 ± 0	1580 ± 0	1738 ± 2	1768 ± 0	1584 ± 0	1772 ± 0	1580 ± 0	1768 ± 0									
	PDB	1592 ± 0	1780 ± 0	1780 ± 0	1592 ± 0	1760 ± 3	1780 ± 0	1592 ± 0	1780 ± 0	1592 ± 0	1780 ± 0									
	ROC	4676 ± 0	5231 ± 0	5231 ± 0	4676 ± 0	5130 ± 7	5231 ± 0	4677 ± 0	5232 ± 0	4676 ± 0	5231 ± 0									
08-03	LMC _{cut}	4692 ± 0	5247 ± 0	5247 ± 0	4692 ± 0	5135 ± 6	5247 ± 0	4693 ± 1	5248 ± 1	4692 ± 0	5247 ± 0									
	PDB	4699 ± 1	5254 ± 1	5254 ± 1	4699 ± 1	5159 ± 8	5254 ± 1	4703 ± 2	5258 ± 2	4699 ± 1	5254 ± 1									
	ROC	33 016 ± 1	37 041 ± 1	37 041 ± 1	33 017 ± 1	35 730 ± 29	37 042 ± 0	33 038 ± 2	37 063 ± 2	33 016 ± 1	37 041 ± 1									
	LMC _{cut}	33 021 ± 4	37 046 ± 4	37 046 ± 4	33 023 ± 4	35 680 ± 19	37 048 ± 4	33 042 ± 2	37 067 ± 2	33 022 ± 3	37 047 ± 3									
08-01	PDB	33 074 ± 4	37 099 ± 4	37 099 ± 4	33 071 ± 6	36 030 ± 28	37 096 ± 6	33 096 ± 1	37 121 ± 1	33 074 ± 4	37 099 ± 4									

Table 4.6: The final partial SSP sizes of CG-iLAO* and iLAO*. The number of states in the final partial SSP $|\hat{S}|$, the number of actions in the final partial SSP $|\hat{A}| = \sum_{s \in \hat{S}} |\hat{A}(s)|$, and for CG-iLAO* the number of actions available to its partial states $|\hat{A}^{\max}| = \sum_{s \in \hat{S}} |A(s)|$. We do not count give-up actions from the fixed-penalty transformation. We restrict the results to the larger problems of each domain.

and $\text{CG-iLAO}^*_{\text{single}}$ with ROC. The representative problems were chosen to be the largest problem that was still solved in time (or some large problem that was solved in time, if there is no clear hierarchy). We give cumulative plots over density for these problems in fig. 4.11, where each point (x, y) represents that a proportion x of states in the final partial SSP has actions with density y or less, e.g., $(0.5, 0.3)$ tells us that 50% of the partial SSP’s states have a density of 0.3 or less. Note that the proportion x of states for $\text{CG-iLAO}^*_{\text{tied}}$ and $\text{CG-iLAO}^*_{\text{single}}$ are evaluated w.r.t. to their respective final partial SSP sizes, i.e., they may be referring to a different number of states. However, we have seen in sec. 4.9.6 that the number of states in their partial SSPs is similar, so the curves are roughly comparable. A plot that grows quickly at the start, with the extreme case $y = 1$ for $x \geq 0$ (shaped $\lceil$), indicates that all states have high density. Conversely, if the plot grows slowly and then jumps up for large x , with the extreme case $y = 0$ for $x < 1$ (shaped $\lfloor$), it means that most states have low density. Another way to understand this curve is that, if the area under the curve is large then most states are dense, and if the area under the curve is small then most states are not dense. Variants of iLAO^* have the extreme $\lceil$ -shaped curve because iLAO^* adds all actions, resulting in density 1 for all states.

Looking at fig. 4.11, we see that the density profiles vary dramatically across problems. For problems such as Coresec, Rand, and $\square\text{TW}$, both CG-iLAO^* variants have mostly low-density states, which is reflected by their small final partial SSPs in tab. 4.5. In contrast, Elev, S&R, and Sys have mostly high-density states, which is respectively reflected by relatively large final partial SSPs in tab. 4.5. This is particularly extreme in Sys, where the CG-iLAO^* variants have over 99% of the number of actions as $\text{CG-iLAO}^*_{\text{all}}$. $\text{CG-iLAO}^*_{\text{tied}}$ and $\text{CG-iLAO}^*_{\text{single}}$ have significantly different profiles for BW, Coresec, ExBW, PARC-N, Rand, and Sched; and nearly identical profiles for the other problems. This is generally reflected by the differences in partial SSP sizes for ROC in tab. 4.5. This confirms, again, that $\text{CG-iLAO}^*_{\text{single}}$ is generally able to ignore more actions than $\text{CG-iLAO}^*_{\text{tied}}$. Otherwise, we can only conclude that CG-iLAO^* equipped with ROC is able to ignore many actions on some problems, and very few on others.

Recall thm. 9, which tells us that $\text{CG-iLAO}^*_{\text{single}}$ with the perfect heuristic adds only the actions necessary to define its optimal policy, i.e., with the perfect heuristic its density plots would have $y = 1$ for $x \geq 0$. In that sense, the density of $\text{CG-iLAO}^*_{\text{single}}$ is heuristic-dependent, rather than problem-dependent. Consequently, fig. 4.11 can be interpreted as a measure of heuristic quality. This motivates that CG-iLAO^* fails to produce small partial SSPs on some problems due to uninformative heuristics, rather than any property of the problems themselves.

To get a higher-level picture we now present a summary of density over entire domains in fig. 4.12. Here, we generate the density curves for each algorithm, heuristic, and problem triple (averaged over instances in the problem), and then compute the area under these curves (AUC); we call this value the density AUC. An instance with mostly dense states and therefore a $\lceil$ -shaped curve will have AUC close to 1, and conversely an instance with few dense states and a $\lfloor$ -shaped curve will have AUC close to 0. We aggregate density AUC over domains with box-and-whisker plots to show the distribution over different problems within the domain. These results are generally consistent with the previous discussion and let us make similar conclusions: the density varies a lot over different domains, and generally $\text{CG-iLAO}^*_{\text{single}}$ tends to have final partial SSPs with lower density than $\text{CG-iLAO}^*_{\text{tied}}$. We note that in some domains the densities are very similar across problems within the domain, e.g., PARC-N, PARC-R, S&R; but in others the densities are varied, e.g., Elev, ExBW, Sched. This does not give any deep insight, only that some of our benchmarks have problems of similar structure within a domain, and others do not. One must be careful comparing the behaviour over different heuristics, because they are associated with different coverages and therefore have different datasets; nevertheless, it seems that the behaviour does not change significantly over different heuristics.

	CG-iLAO* _{tied}					CG-iLAO* _{single}					iLAO*					CG-iLAO* _{all}				
	$ \hat{S} $	$ \hat{A} $	$ \hat{A}^{max} $	$ \hat{S} $	$ \hat{A} $	$ \hat{A}^{max} $	$ \hat{S} $	$ \hat{A} $	$ \hat{A}^{max} $	$ \hat{S} $	$ \hat{A} $	$ \hat{A}^{max} $	$ \hat{S} $	$ \hat{A} $	$ \hat{A}^{max} $	$ \hat{S} $	$ \hat{A} $	$ \hat{A}^{max} $		
BW	ROC	38 197 ± 187	107 485 ± 619	264 000 ± 1337	38 041 ± 160	69 964 ± 384	263 097 ± 1068	35 601 ± 245	249 229 ± 1769	42 269 ± 332	310 921 ± 2560									
	PDB	38 604 ± 308	112 018 ± 782	266 356 ± 2200	38 141 ± 274	70 045 ± 740	263 724 ± 1758	35 514 ± 198	248 260 ± 1479	42 229 ± 398	310 213 ± 3083									
	08-11	ROC	12 961 ± 776	42 333 ± 2215	107 413 ± 6639	12 530 ± 402	22 106 ± 549	104 209 ± 3392	12 172 ± 797	102 326 ± 6804	14 114 ± 907	116 851 ± 7586								
	08-12	ROC	12 961 ± 776	42 333 ± 2215	107 413 ± 6639	12 530 ± 402	22 106 ± 549	104 209 ± 3392	12 172 ± 797	102 326 ± 6804	14 114 ± 907	116 851 ± 7586								
Elev	ROC	511 935 ± 243	2 236 293 ± 8866	3 063 040 ± 1474	512 158 ± 286	2 143 683 ± 7449	3 064 324 ± 1739	513 039 ± 171	3 069 532 ± 959	515 568 ± 988	3 084 272 ± 5788									
	LMC _{cut}	441 272 ± 717	1 652 518 ± 6622	2 649 696 ± 4449	441 162 ± 810	1 551 021 ± 8734	2 648 887 ± 4855	425 933 ± 3021	2 557 899 ± 18 032	452 195 ± 3016	2 714 150 ± 17 807									
	PDB	506 141 ± 399	2 134 977 ± 8328	3 028 903 ± 2315	506 320 ± 222	2 045 625 ± 6661	3 029 960 ± 1276	504 455 ± 808	3 018 725 ± 4709	510 224 ± 1157	3 052 950 ± 6776									
	ROC	8 ± 5	40 ± 31	707 ± 582	8 ± 5	12 ± 9	707 ± 582	8 ± 5	707 ± 582	8 ± 5	707 ± 582									
Rand	LMC _{cut}	3 ± 0	54 ± 49	264 ± 52	3 ± 0	4 ± 1	264 ± 52	3 ± 0	264 ± 52	3 ± 0	264 ± 52									
	PDB	4 ± 0	16 ± 2	265 ± 52	4 ± 1	5 ± 1	265 ± 52	4 ± 0	265 ± 52	4 ± 0	265 ± 52									
	ROC	4 ± 1	506 ± 33	3690 ± 954	5 ± 2	6 ± 5	4731 ± 2508	4 ± 1	3690 ± 954	4 ± 1	3690 ± 954									
	PDB	10 ± 2	468 ± 50	9712 ± 2326	10 ± 5	15 ± 9	9790 ± 4901	10 ± 2	9448 ± 2300	10 ± 2	9448 ± 2300									
S&R	ROC	25 ± 5	7634 ± 1621	46 300 ± 9419	25 ± 6	42 ± 11	45 070 ± 10 729	26 ± 4	48 143 ± 7394	26 ± 4	48 143 ± 7394									
	LMC _{cut}	7 ± 0	845 ± 3	11 992 ± 137	7 ± 0	10 ± 0	11 992 ± 137	7 ± 0	12 001 ± 42	7 ± 0	12 001 ± 42									
	PDB	128 ± 12	37 172 ± 4838	208 000 ± 20 635	98 ± 23	171 ± 41	162 899 ± 39 785	197 ± 29	325 885 ± 48 159	197 ± 29	325 905 ± 48 161									
	ROC	9910 ± 8	35 309 ± 32	39 577 ± 4	9921 ± 13	35 132 ± 30	39 583 ± 7	10 022 ± 15	39 633 ± 7	9904 ± 5	38 081 ± 2									
Sys	LMC _{cut}	9914 ± 8	35 152 ± 12	39 580 ± 4	9913 ± 4	35 088 ± 27	39 579 ± 2	10 024 ± 24	39 635 ± 12	9919 ± 14	38 088 ± 7									
	PDB	9890 ± 10	35 316 ± 34	39 573 ± 5	9894 ± 6	35 153 ± 20	39 575 ± 3	9993 ± 19	39 625 ± 9	9863 ± 2	38 102 ± 1									
	ROC	113 810 ± 17	527 263 ± 203	578 498 ± 8	113 820 ± 47	525 092 ± 167	578 502 ± 23	114 249 ± 76	578 717 ± 38	113 769 ± 30	560 917 ± 15									
	LMC _{cut}	113 817 ± 22	525 083 ± 43	578 501 ± 11	113 828 ± 13	524 661 ± 99	578 506 ± 7	114 234 ± 78	578 709 ± 39	113 799 ± 26	560 932 ± 13									
08-04	PDB	113 763 ± 32	527 146 ± 311	578 481 ± 16	113 766 ± 44	524 952 ± 158	578 483 ± 22	114 178 ± 78	578 689 ± 39	113 624 ± 14	560 916 ± 7									
	ROC	—	—	—	—	—	—	—	—	—	—									
	LMC _{cut}	1 251 642 ± 57	7 078 350 ± 557	7 682 167 ± 29	—	—	—	—	—	—	—									
	PDB	1 251 444 ± 65	7 109 673 ± 538	7 682 078 ± 33	—	—	—	—	—	—	—									
08-02	ROC	1541 ± 0	1725 ± 0	1729 ± 0	1541 ± 0	1697 ± 4	1729 ± 0	1539 ± 0	1727 ± 0	1541 ± 0	1729 ± 0									
	LMC _{cut}	1580 ± 0	1768 ± 0	1768 ± 0	1580 ± 0	1738 ± 2	1768 ± 0	1584 ± 0	1772 ± 0	1580 ± 0	1768 ± 0									
	PDB	1592 ± 0	1780 ± 0	1780 ± 0	1592 ± 0	1760 ± 3	1780 ± 0	1592 ± 0	1780 ± 0	1592 ± 0	1780 ± 0									
	ROC	4676 ± 0	5231 ± 0	5231 ± 0	4676 ± 0	5130 ± 7	5231 ± 0	4677 ± 0	5232 ± 0	4676 ± 0	5231 ± 0									
08-03	LMC _{cut}	4692 ± 0	5247 ± 0	5247 ± 0	4692 ± 0	5135 ± 6	5247 ± 0	4693 ± 1	5248 ± 1	4692 ± 0	5247 ± 0									
	PDB	4699 ± 1	5254 ± 1	5254 ± 1	4699 ± 1	5159 ± 8	5254 ± 1	4703 ± 2	5258 ± 2	4699 ± 1	5254 ± 1									
	ROC	33 016 ± 1	37 041 ± 1	37 041 ± 1	33 017 ± 1	35 730 ± 29	37 042 ± 0	33 038 ± 2	37 063 ± 2	33 016 ± 1	37 041 ± 1									
	LMC _{cut}	33 021 ± 4	37 046 ± 4	37 046 ± 4	33 023 ± 4	35 680 ± 19	37 048 ± 4	33 042 ± 2	37 067 ± 2	33 022 ± 3	37 047 ± 3									
08-01	PDB	33 074 ± 4	37 099 ± 4	37 099 ± 4	33 071 ± 6	36 030 ± 28	37 096 ± 6	33 096 ± 1	37 121 ± 1	33 074 ± 4	37 099 ± 4									

Table 4.7: The final partial SSP sizes of CG-iLAO* and iLAO*. The number of states in the final partial SSP $|\hat{S}|$, the number of actions in the final partial SSP $|\hat{A}| = \sum_{s \in \hat{S}} |\hat{A}(s)|$, and for CG-iLAO* the number of actions available to its partial states $|\hat{A}^{max}| = \sum_{s \in \hat{S}} |A(s)|$. We do not count give-up actions from the fixed-penalty transformation. We restrict the results to the larger problems of each domain.

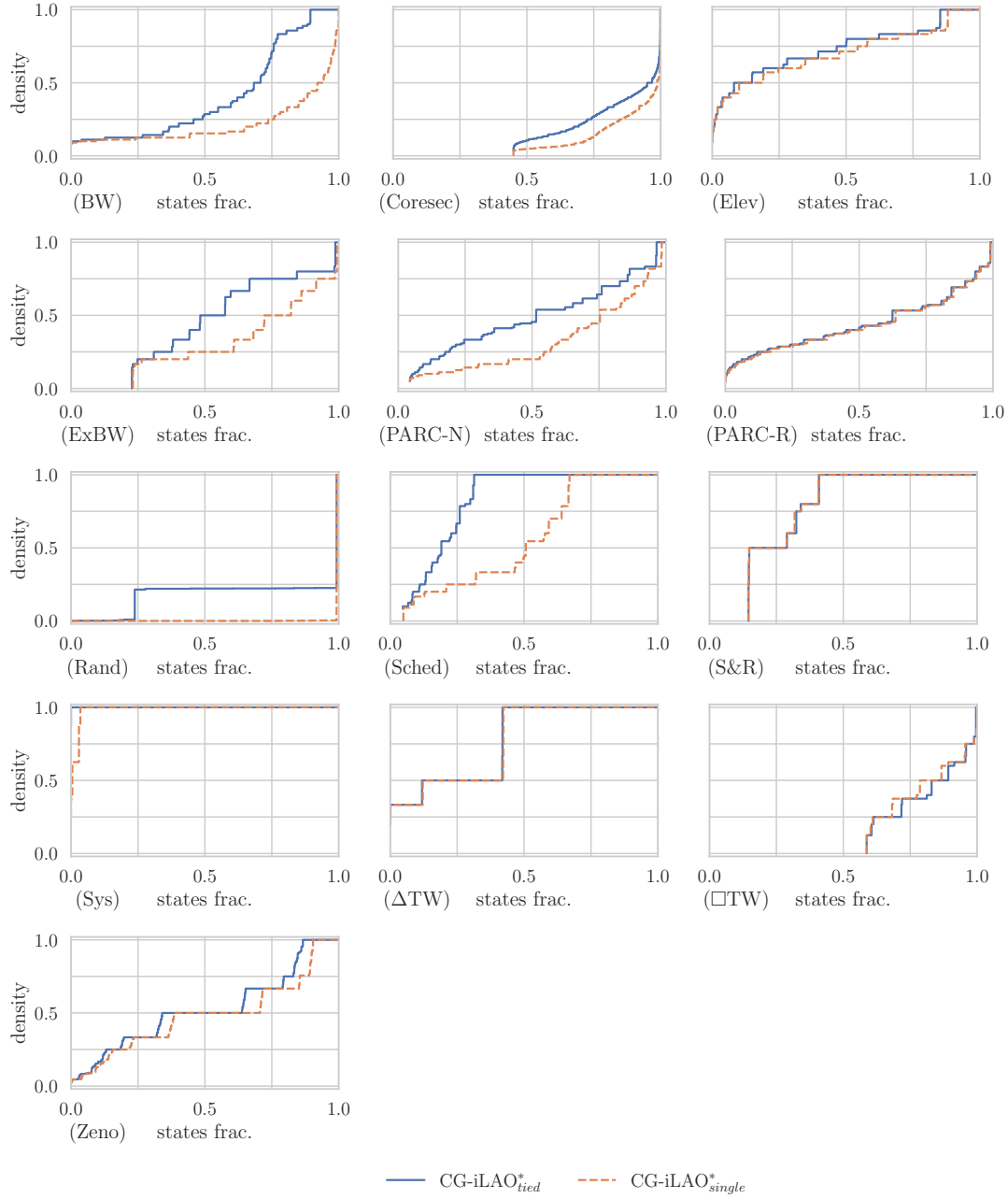


Figure 4.11: Cumulative plot of state density w.r.t. the fraction of states in the final partial SSP over 5 instances for one representative problem per domain.

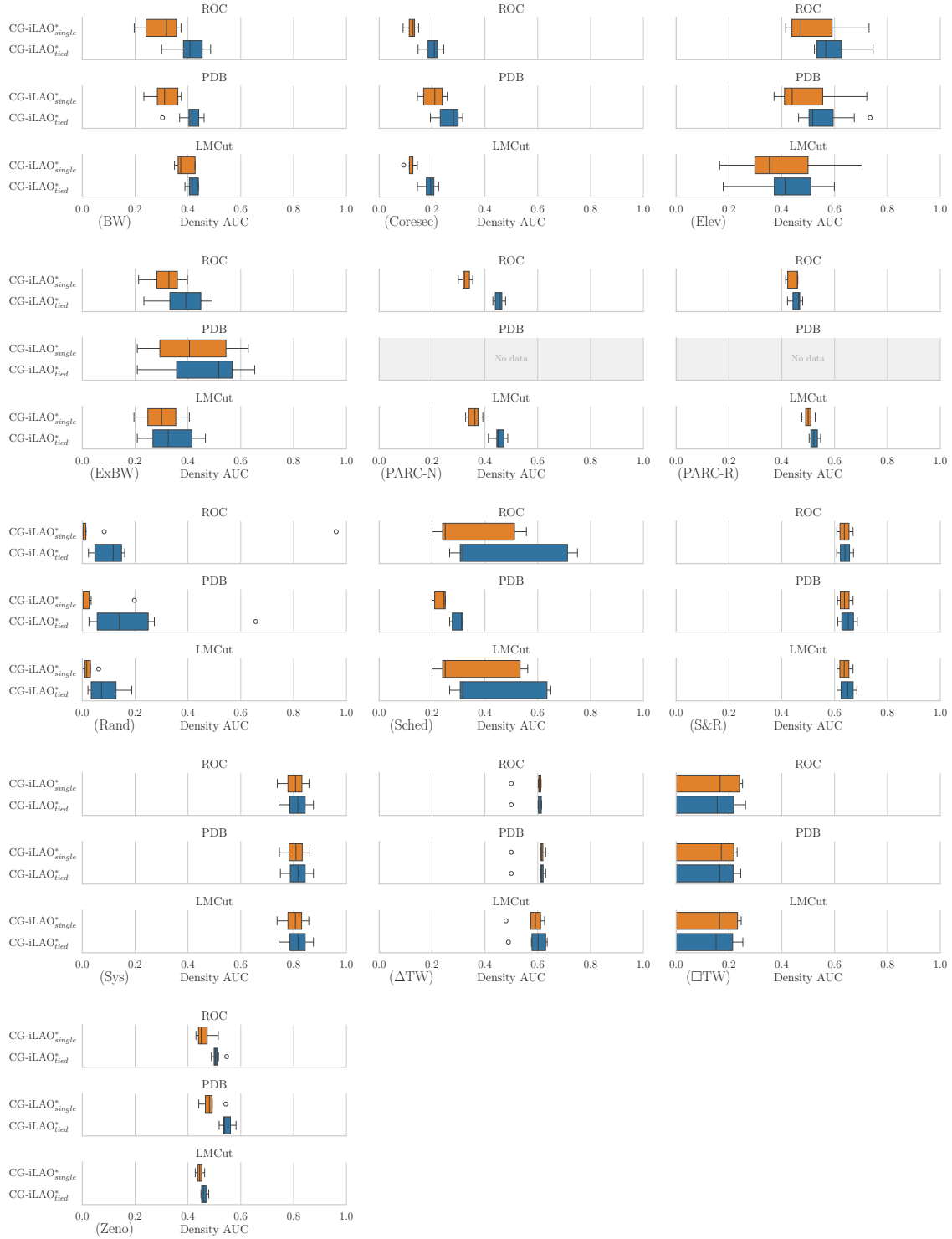


Figure 4.12: Box-and-whisker plots of area under density curve (density AUC) aggregated over problems within each domain.

4.9.7 How Does CG-iLAO* Compare To Action Elimination?

We have discussed that CG-iLAO*'s mechanism for ignoring actions has two key advantages over action elimination: it does not require an upper bound, and it starts with a minimal set of actions and only adds more when they are required. By adding actions as they are required, CG-iLAO* enjoys small partial SSPs from the start and keeps them relatively small; compared to action elimination, which starts with all actions and only removes them when they are proved suboptimal, which is towards the end of the algorithm's lifetime. These advantages result in CG-iLAO* being more effective than action elimination. This section confirms our claim experimentally, and furthermore shows that action elimination is able to remove only few actions that CG-iLAO* inserts into its partial SSPs, demonstrating that CG-iLAO* adds few provably suboptimal actions. In order to compare CG-iLAO*'s mechanism with action elimination we now describe how to augment our algorithms with action elimination.

Adding Action Elimination to iLAO*. In general, Bellman-backup-based algorithms can be easily modified to use action elimination with the following modifications:

1. Rename the lower bound V to V_{lb} , and additionally track an upper bound V_{ub} . We initialise V_{ub} with the trivial upper bound $V_{ub}(g) = 0$ for all goals $g \in \mathcal{G}$ and $V_{ub}(s) = d$ for all other states $s \in \mathcal{S} \setminus \mathcal{G}$, where $d = \infty$ or the penalty term if we are using the fixed-penalty transformation.
2. For each backup to V_{lb} , also apply a backup to V_{ub} (ignoring its residual).
3. After computing $Q_{lb}(s, a)$ and $Q_{ub}(s, a)$ for each $a \in \hat{\mathcal{A}}(s)$, permanently remove (eliminate) any action $\tilde{a} \in \hat{\mathcal{A}}$ from the partial SSP if it has $Q_{lb}(s, \tilde{a}) > \min_{a \in \mathcal{A}(s)} Q_{ub}(s, a)$.

If $V_{lb} \leq V^* \leq V_{ub}$, then these modifications preserve the correctness of the algorithm, because we are only removing suboptimal actions with action elimination (thm. 10). Indeed, if iLAO* is initialised with $V_{ub} \geq V^*$, then it preserves $V_{ub} \geq V^*$ as an invariant, by using similar arguments to show iLAO* has the invariant $V_{lb} \leq V^*$ when initialised with an admissible heuristic (Hansen and Zilberstein, 2001). Thus, this definition of iLAO* with action elimination behaves as we expect and remains optimal.

Adding Action Elimination to CG-iLAO*. We can apply the same modifications to CG-iLAO*, but with a subtle yet important change. Recall that CG-iLAO* does not guarantee $V_{lb} \leq V^*$ in general, and if we apply action elimination when $V_{lb} \not\leq V^*$ then optimal actions may be permanently removed, breaking the optimality of the algorithm. So, we only allow CG-iLAO* to apply action elimination when $\Gamma = \emptyset$, at which point $V_{lb} \leq V^*$ holds (see lem. 4), and action elimination can be performed safely.⁷ There is no analogous concern for V_{ub} , and it is always a valid upper bound because the absence of actions can only increase V_{ub} , since optimal actions may be missing.

Thus, we define CG-iLAO*_{tied-elim}, CG-iLAO*_{single-elim}, and CG-iLAO*_{all-elim} by adding action elimination to the relevant variant of CG-iLAO*. We make note of two technical details:

1. Initially, the only values of V_{ub} with $V_{ub}(s) < d$ are goal states, so Bellman backups can not decrease V_{ub} until goal states enter the partial SSP. Thus, in our implementation, we only start computing Q -values for V_{ub} after a goal state enters the partial SSP, avoiding needless computation of Q -values fixed at d .
2. We still use ϵ -consistency as the termination condition. With access to V_{ub} , other termination conditions are possible, e.g., $V_{ub}(s) - V_{lb}(s) \leq \epsilon \forall s \in \mathcal{S}$ (McMahan, Likhachev, and Gordon, 2005), which have slightly different guarantees, and can in some cases be satisfied sooner.

In these experiments, we consider the following algorithms:

- CG-iLAO*_{tied} and CG-iLAO*_{tied-elim}
- CG-iLAO*_{single} and CG-iLAO*_{single-elim}
- CG-iLAO*_{all} and CG-iLAO*_{all-elim}
- FTVI (Dai, Mausam, and Weld, 2009).

⁷Actually, lem. 4 gives us $V_{lb} \leq V^* + \epsilon N(s, V, \mathcal{S})$, i.e., there is an error term. This error term is insignificant in practice, but can theoretically affect optimality.

ROC		PDB		LMCut	
CG-iLAO* _{tied}	3.01 ± 0.12	CG-iLAO* _{tied}	3.01 ± 0.12	CG-iLAO* _{tied}	3.22 ± 0.11
CG-iLAO* _{single}	3.14 ± 0.13	CG-iLAO* _{single}	3.30 ± 0.13	CG-iLAO* _{single}	3.38 ± 0.12
CG-iLAO* _{tied-elim}	3.47 ± 0.12	CG-iLAO* _{tied-elim}	3.63 ± 0.11	CG-iLAO* _{single-elim}	3.42 ± 0.12
CG-iLAO* _{single-elim}	3.49 ± 0.13	CG-iLAO* _{single-elim}	3.86 ± 0.12	CG-iLAO* _{tied-elim}	3.79 ± 0.11
CG-iLAO* _{all}	4.33 ± 0.13	CG-iLAO* _{all}	4.17 ± 0.12	CG-iLAO* _{all}	4.08 ± 0.12
CG-iLAO* _{all-elim}	4.52 ± 0.13	CG-iLAO* _{all-elim}	4.46 ± 0.12	CG-iLAO* _{all-elim}	4.45 ± 0.11
FTVI	6.05 ± 0.13	FTVI	5.57 ± 0.13	FTVI	5.67 ± 0.13

Table 4.8: Runtime ranking of CG-iLAO* and action elimination methods within a specified heuristic (mean and 95% C.I. over all instances).

		BW	Coresec	Elev	ExBW	PARC-N	PARC-R	Rand	□TW	S&R	Sched	Sys	ΔTW	Zeno	total
ROC	# of instances	110	35	75	105	30	30	75	70	25	45	20	40	45	705
	FTVI	55	20	50	65	0	10	33	54	20	30	20	30	5	392
	CG-iLAO* _{all}	105	25	70	105	30	27	35	60	25	45	20	35	39	621
	CG-iLAO* _{all-elim}	105	25	70	105	30	25	35	60	25	45	20	35	35	615
	CG-iLAO* _{tied}	105	25	75	105	30	25	36	60	20	45	20	35	40	621
	CG-iLAO* _{tied-elim}	105	25	74	105	30	25	37	60	20	45	20	35	38	619
	CG-iLAO* _{single}	105	25	75	105	30	25	43	60	20	45	20	35	40	628
	CG-iLAO* _{single-elim}	105	25	72	105	30	25	43	60	20	45	20	35	38	623
PDB	FTVI	45	20	50	57	0	0	22	55	20	30	20	30	5	354
	CG-iLAO* _{all}	85	30	70	90	0	0	30	60	25	30	20	37	40	517
	CG-iLAO* _{all-elim}	85	30	70	90	0	0	30	60	25	30	20	35	40	515
	CG-iLAO* _{tied}	90	30	75	90	0	0	30	60	24	30	20	38	40	527
	CG-iLAO* _{tied-elim}	90	30	74	90	0	0	30	60	25	30	20	35	40	524
	CG-iLAO* _{single}	90	30	75	90	0	0	30	60	20	30	20	37	40	522
	CG-iLAO* _{single-elim}	90	30	72	90	0	0	30	60	20	30	20	35	40	517
	FTVI	45	20	50	70	0	1	18	60	20	30	20	25	5	364
LMCut	CG-iLAO* _{all}	45	25	70	100	30	20	20	65	25	40	20	30	25	515
	CG-iLAO* _{all-elim}	45	25	70	100	30	20	20	65	25	40	20	30	25	515
	CG-iLAO* _{tied}	45	25	70	101	30	20	20	65	24	45	20	30	25	520
	CG-iLAO* _{tied-elim}	45	25	70	100	30	20	20	65	21	40	20	30	25	511
	CG-iLAO* _{single}	45	25	71	100	30	20	20	65	20	45	20	30	25	516
	CG-iLAO* _{single-elim}	45	25	70	101	30	20	20	65	20	42	20	30	25	513
	FTVI	45	20	50	70	0	1	18	60	20	30	20	25	5	364
	CG-iLAO* _{all}	45	25	70	100	30	20	20	65	25	40	20	30	25	515

Table 4.9: Coverage for action elimination experiment with each considered heuristic over the benchmark domains. The highest coverage for each problem is marked with boldface.

To verify that CG-iLAO* is more effective than action elimination, we compare the performance of CG-iLAO*_{tied} and CG-iLAO*_{single} with their action-elimination extended counterparts, and additionally compare them to CG-iLAO*_{all} and its action-eliminated counterpart, which does not use the mechanism for ignoring inactive actions and implements iLAO*. We also include FTVI, as a baseline of algorithms that use action elimination. The ranks (tab. 4.8), coverage (tab. 4.9), and cumulative plots (fig. 4.13) paint a consistent picture:

- algorithms with action elimination are slower than their counterparts without action elimination,
- CG-iLAO*_{tied} and CG-iLAO*_{single} are faster than any of the algorithms with action elimination that we consider.

The issue with action elimination is that it must track and compute Bellman backups for the additional value function V_{ub} , incurring an additional overhead of Q -value computations, and this can only pay off if it eliminates sufficiently many actions to save more Q -value computations. By the cumulative plot over Q -values (fig. 4.13), we see that this is not the case.

We now investigate how many actions are removed by action elimination, compared to how many actions

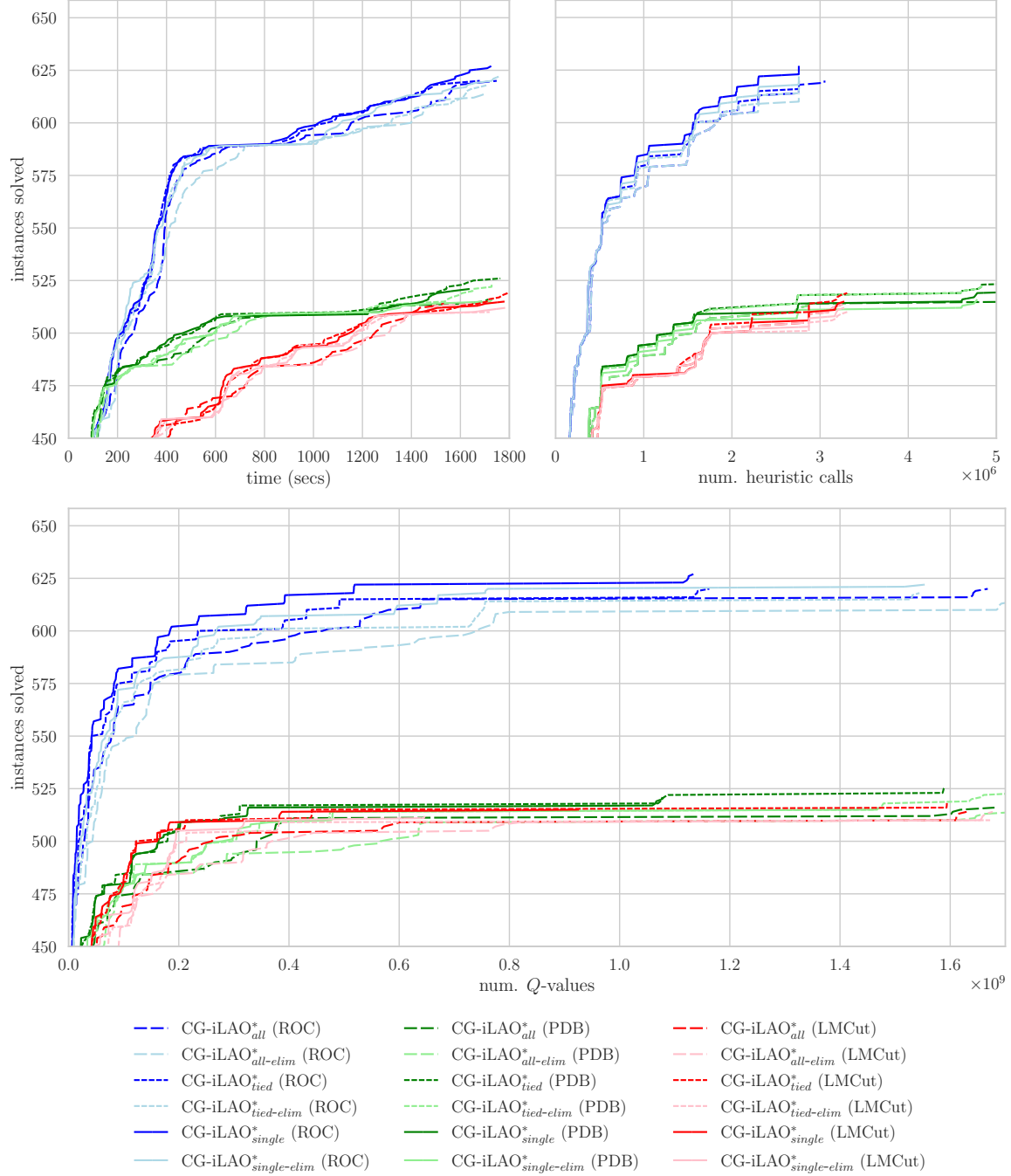


Figure 4.13: For each algorithm and heuristic, the cumulative plot of how many problems and seeds were solved w.r.t. time in seconds (top left), number of Q -values (top right), and number of calls to heuristic (bottom). To focus on where the algorithms are different we start the y -axis at 450 instances, and end the x -axis at 5×10^6 heuristic calls.

are ignored by CG-iLAO*. Tab. 4.10 shows the number of actions in the final partial SSPs of the CG-iLAO* variants. We do not include states in this table – action elimination has no way to reduce the number of states, so the percentage of states is equal between algorithms and their counterpart with action elimination (excluding some noise). In terms of actions, we see that CG-iLAO*_{all-elim} never has significantly fewer actions in its final partial SSP than CG-iLAO*_{tied} and CG-iLAO*_{single}, and often has significantly more actions. This tells us that CG-iLAO* is more effective at reducing partial SSP sizes than action elimination. Moreover, if we compare CG-iLAO*_{tied} and CG-iLAO*_{single} with their action elimination counterparts, we see that action elimination is able to shrink the partial SSP slightly, but never significantly, i.e., CG-iLAO* adds very few actions that can be removed by action elimination. This

		CG-iLAO* _{tied}	CG-iLAO* _{tied-elim}	CG-iLAO* _{single}	CG-iLAO* _{single-elim}	CG-iLAO* _{all-elim}
BW	ROC	40.8 ± 1.2	40.7 ± 1.3	33.2 ± 2.8	33.1 ± 2.8	98.8 ± 0.3
	LMCut	45.5 ± 1.4	44.6 ± 1.3	42.5 ± 2.0	41.5 ± 1.8	97.6 ± 0.3
	PDB	39.7 ± 0.9	39.3 ± 0.9	30.9 ± 1.6	30.6 ± 1.6	97.4 ± 0.5
Coresec	ROC	33.6 ± 1.9	33.5 ± 1.8	20.5 ± 1.1	20.4 ± 1.0	77.6 ± 4.0
	LMCut	39.0 ± 1.9	37.4 ± 1.5	24.9 ± 1.1	23.8 ± 0.9	82.6 ± 3.1
	PDB	29.0 ± 1.6	28.4 ± 1.5	22.2 ± 1.6	22.0 ± 1.5	73.0 ± 3.3
Elev	ROC	60.5 ± 1.6	60.1 ± 1.5	53.3 ± 2.2	53.0 ± 2.1	98.0 ± 0.6
	LMCut	45.0 ± 3.1	44.5 ± 3.0	40.4 ± 3.0	40.0 ± 2.8	97.0 ± 1.1
	PDB	56.9 ± 1.7	56.5 ± 1.6	50.1 ± 2.2	49.8 ± 2.2	97.5 ± 0.8
ExBW	ROC	44.8 ± 1.5	44.1 ± 1.5	39.6 ± 1.7	38.8 ± 1.7	85.6 ± 2.2
	LMCut	39.2 ± 2.1	38.6 ± 2.0	37.1 ± 1.8	36.5 ± 1.7	88.3 ± 2.4
	PDB	53.9 ± 3.4	53.3 ± 3.4	49.9 ± 3.7	49.4 ± 3.7	93.4 ± 1.5
PARC-N	ROC	44.7 ± 0.9	34.8 ± 1.0	32.7 ± 0.9	23.9 ± 0.9	44.8 ± 1.6
	LMCut	44.0 ± 1.2	37.1 ± 1.3	35.2 ± 1.3	29.0 ± 1.1	62.0 ± 1.7
PARC-R	ROC	44.1 ± 1.0	42.9 ± 0.7	43.0 ± 1.0	41.9 ± 0.8	92.7 ± 2.0
	LMCut	51.9 ± 1.0	50.8 ± 1.1	50.1 ± 1.2	49.1 ± 1.4	89.8 ± 1.8
Rand	ROC	9.1 ± 1.8	9.1 ± 1.8	0.9 ± 0.3	0.9 ± 0.3	100.0 ± 0.0
	LMCut	9.9 ± 3.9	9.9 ± 3.9	1.8 ± 0.7	1.8 ± 0.7	76.1 ± 18.6
	PDB	17.0 ± 8.1	17.0 ± 8.0	1.0 ± 0.5	1.0 ± 0.5	75.0 ± 14.8
□TW	ROC	31.5 ± 0.8	26.6 ± 1.1	33.1 ± 1.6	27.9 ± 1.7	66.2 ± 4.2
	LMCut	30.9 ± 0.9	26.4 ± 1.2	32.8 ± 1.8	27.9 ± 1.8	67.4 ± 4.2
	PDB	31.0 ± 1.5	26.3 ± 1.3	31.5 ± 1.8	26.7 ± 1.8	66.2 ± 3.9
S&R	ROC	91.6 ± 0.8	83.3 ± 2.3	91.4 ± 0.7	83.0 ± 2.3	91.1 ± 1.9
	LMCut	92.0 ± 0.8	83.5 ± 2.3	91.3 ± 0.7	82.9 ± 2.3	90.8 ± 2.1
	PDB	92.2 ± 0.8	85.1 ± 2.5	91.4 ± 0.7	82.7 ± 2.4	91.3 ± 2.0
Sched	ROC	78.1 ± 0.7	77.6 ± 0.9	50.7 ± 1.1	50.2 ± 1.1	81.4 ± 0.9
	LMCut	73.8 ± 3.2	73.2 ± 3.5	50.2 ± 1.2	49.8 ± 1.3	79.0 ± 0.5
	PDB	79.6 ± 0.5	79.6 ± 0.5	51.1 ± 1.3	51.1 ± 1.3	79.6 ± 0.5
Sys	ROC	99.5 ± 0.3	95.8 ± 0.8	97.4 ± 0.3	93.6 ± 0.8	95.9 ± 0.8
	LMCut	99.5 ± 0.4	95.4 ± 0.9	97.3 ± 0.4	93.3 ± 0.8	95.5 ± 0.9
	PDB	100.0 ± 0.0	95.6 ± 1.0	98.4 ± 0.4	93.8 ± 0.7	95.6 ± 1.0
ΔTW	ROC	64.4 ± 0.8	63.0 ± 0.9	63.8 ± 0.9	62.3 ± 0.9	86.3 ± 2.9
	LMCut	68.6 ± 1.5	67.8 ± 1.8	67.0 ± 1.5	66.7 ± 1.6	88.5 ± 3.9
	PDB	65.7 ± 0.7	64.2 ± 0.8	65.5 ± 0.7	64.0 ± 0.8	86.2 ± 3.2
Zeno	ROC	37.1 ± 0.6	37.0 ± 0.6	30.1 ± 0.8	30.1 ± 0.8	99.9 ± 0.1
	LMCut	34.3 ± 0.4	34.3 ± 0.4	31.6 ± 0.5	31.6 ± 0.5	99.9 ± 0.1
	PDB	44.0 ± 1.3	44.0 ± 1.3	34.0 ± 1.1	34.0 ± 1.1	99.8 ± 0.1

Table 4.10: Number of actions in each algorithm’s final partial SSP, as a percentage of CG-iLAO*_{all}’s final partial SSP.

is a strong argument in favour of CG-iLAO*: its mechanism for ignoring actions induces similarly sized final partial SSPs to action elimination, but it does not require upper bounds, and it avoids these actions from the start, rather than eliminating them towards the end of the algorithm’s lifetime.

4.10 Summary

This chapter introduced CG-iLAO*, an optimal heuristic-search algorithm for SSPs. Its novelty is that it can use its heuristic to ignore inactive actions, and only add actions when they are deemed necessary. This addresses the open issue that previous methods for solving SSPs consider all applicable actions, even when the heuristic makes it clear that some are not needed. To do this, we built on existing connections between operations research and planning to reframe iLAO* in terms of constraint and variable generation for LPs. Under this lens, we were able to refine the existing method’s separation oracle, which enables the algorithm to ignore inactive actions, and only add actions when they are deemed necessary. Bringing this back into a dynamic programming implementation yields CG-iLAO*.

CG-iLAO*’s novel ability to ignore inactive actions gives it different theoretical properties from existing methods, e.g., it does not guarantee that its value function remains admissible, and the algorithm is not monotonic. We discussed these properties and proved that CG-iLAO* is an optimal algorithm. We showed experimentally that CG-iLAO*’s mechanism for ignoring actions pays off, and it outperforms the state-of-the-art on our benchmarks.

CoGNe: Solving SSPs via Plans and Cycles

In this chapter, we introduce CoGNe, the first algorithm that finds optimal policies for SSPs via plans and cycles. The key insight is that proper policies for SSPs correspond to network flow, which follows from the OMLP (LP 2) for finding policies and its interpretation as network flow. Network flow theory lets us decompose a policy’s flow into flow across plans and cycles, inducing a new LP that finds policies in terms of plans and cycles from the SSP’s all-outcomes determinisation. Constructing an LP that enumerates all plans and cycles is impractical for large problems, so CoGNe applies variable generation. Importantly, the subproblem solved by CoGNe is to find sufficiently cheap plans and cycles on the SSP’s all-outcomes determinisation, i.e., CoGNe finds optimal policies for SSPs by solving a sequence of deterministic planning problems. These deterministic planning problems deviate from classical planning because they have state-dependent costs and negative costs, allowing negative cycles. CoGNe represents a novel approach for solving SSPs, and brings with it new open problems; solving these may enable CoGNe to solve problems that can not be solved by existing algorithms.

We give a breakdown of this chapter:

- Sec. 5.1 gives the necessary background for plans and cycles.
- Sec. 5.2 derives the key theorem that policies can be expressed in terms of plans and cycles.
- Sec. 5.3 describes the CoGNe algorithm.
- Sec. 5.4 evaluates CoGNe, both theoretically and empirically.
- Sec. 5.5 discusses previous determinisation-based algorithms for SSPs, and relates them to CoGNe.
- Sec. 5.6 summarises this chapter.

5.1 Background: Trajectories, Plans, and Cycles

Given the policy π , a trajectory from s is a sequence of states and actions that the agent may encounter when following π from s , formally:

Definition 29 (Trajectory). *A trajectory T from s is the sequence $\langle s_0, a_0, \dots, a_{n-1}, s_n \rangle$ where $s_0 = s$, $\pi(s_i) = a_i$, and $P(s_{i+1}|s_i, a_i) > 0$ for all $i \in \{0, \dots, n-1\}$. We also require that, if $\pi(s_i)$ is defined, then s_i must be followed by $\pi(s_i)$ in the sequence, i.e., we disallow trajectories that terminate in states s_n where π is defined. The set of all possible trajectories for a policy π from state s is denoted $\mathcal{T}_{\pi,s}$.*

If the agent can encounter a cycle following π from s , then $\mathcal{T}_{\pi,s}$ is infinite because $\mathcal{T}_{\pi,s}$ must contain a trajectory going through the cycle once, another trajectory going through the cycle twice, another three times, etc. Conversely, if the agent cannot encounter any cycles from s , then $\mathcal{T}_{\pi,s}$ is finite. We restrict our discussion of trajectories to proper policies, which ensures that each trajectory is finite (although it can be arbitrarily long in the presence of cycles), and that all trajectories end in a goal, i.e., $s_n \in \mathbf{G}$. With this restriction to finite trajectories, we can safely define for a trajectory $T = \langle s_0, a_0, \dots, s_n \rangle$ its

- probability of occurring $P(T|\pi) = \prod_{i=0}^{n-1} \pi(s_i, a_i)P(s_{i+1}|s_i, a_i)$, and

- cost $C(T) = \sum_{i=0}^{n-1} C(a_i)$.

With these definitions, we can recover the policy from its trajectories $\mathcal{T}_{\pi, s_I}$, and we can also compute qualities such as its cost $C(\pi) = \sum_{T \in \mathcal{T}_{\pi, s_I}} P(T)C(T)$.¹ This ability to encode policies with its trajectories foreshadows how we can use plans and cycles to encode policies.

We use the standard definitions of plans and cycles from the deterministic shortest path setting.

Definition 30 (Plans and Cycles). *For an SSP's determinisation $\langle \mathbb{S}, s_I, \mathbb{G}, \mathbb{A}_{det}, P_{det}, C_{det} \rangle$:*

- A *plan* is a state-action sequence $\langle s_0, a_0, \dots, s_{n-1}, a_{n-1}, s_n \rangle$ with $s_0 = s_I$, $s_n \in \mathbb{G}$, and the sequence must respect the determinisation's action applicability and effects, i.e., $a_i \in \mathbb{A}_{det}(s_i)$ and $P(s_{i+1}|s_i, a_i) = 1 \ \forall i \in \{0, \dots, n-1\}$.
- A *cycle* is a sequence $\langle s_0, a_0, \dots, s_{n-1}, a_{n-1}, s_n \rangle$ with $s_0 = s_n$, and it must similarly respect the determinisation's action applicability and effects.

Additionally, we require that states may not be repeated within plans or cycles, except the first and last states of a cycle, i.e., $i \neq j \implies s_i \neq s_j \ \forall i, j \in \{0, \dots, n-1\}$. The cost of a plan or cycle $w = \langle s_0, a_0, \dots, s_n \rangle$ is $C(w) = \sum_{i \in \{0, \dots, n-1\}} C_{det}(a_i)$. $\mathcal{P}$ and $\mathcal{C}$ refer to the set of all plans and cycles, respectively.

We abuse notation, and in the context of SSPs talk about a plan or cycle $w = \langle s^0, a^0, \dots, s^n \rangle$ using the SSP's actions, which corresponds to the plan or cycle $\langle s^0, a_{s^0 \rightarrow s^1}^0, \dots, s^n \rangle$ in the all-outcomes determinisation. These two representations are equivalent and we can convert between them since preceding and succeeding states are defined uniquely for each action in the sequence. This lets us interpret plans as trajectories, with probabilities of success $P(p|\pi) = \prod_{i=0}^{n-1} \pi(s_i, a_i)P(s_{i+1}|s_i, a_i)$ for $p = \langle s_0, a_0, \dots, s_n \rangle$, where the actions are from the SSP. Recall that trajectories allow state repetition and plans do not, so plans are a special case of trajectory. Cycles can not be interpreted as trajectories because we have restricted to trajectories that terminate in goal states. For an SSP $\mathbb{S}$, we write $\mathcal{P}$ and $\mathcal{C}$ to denote the set of all plans and cycles on $\mathbb{S}$'s all-outcomes determinisation.

For convenience, we will use the following notation to denote that *one sequence is a subsequence of another*: given two state-action sequences $\sigma = \langle s_0, a_0, \dots, s_n \rangle$ and $\sigma' = \langle s'_0, a'_0, \dots, s'_m \rangle$, we write $\sigma \subseteq \sigma'$ to denote that σ is a subsequence of σ' , i.e., $\exists i, j$ s.t. $0 \leq i < j \leq n$ and $\sigma = \langle s'_i, a'_i, \dots, s'_j \rangle$. We will apply this notation to check if sequences occur within plans or cycles.

5.2 Expressing Policies as Plans and Cycles

In this section we derive the central result of this chapter: *proper policies can be expressed as a combination of plans and cycles*. We derive this result by first defining an LP that is equivalent to the Occupation Measure LP but considers flow across the SSP's all-outcomes determinisation (sec. 5.2.1), and then we apply a network-flow theorem to decompose it into an LP that operates over plans and cycles (sec. 5.2.2), leading to the key theorem (thm. 11). Finally, we explain the semantics of the new plan and cycle representation (sec. 5.2.3).

5.2.1 Network flow on determinisation

We introduce the Determinised Occupation Measure LP (DOMLP), which is equivalent to the Occupation Measure LP (OMLP), but operates over the all-outcomes determinisation.

As a reminder, we present the OMLP in LP 5. This is the same LP as we presented in LP 2, but the macro actions $in(s)$ and $out(s)$ have been simplified out because this representation is more convenient in this chapter.

We mention again that the explicit sink constraint $\sum_{g \in \mathbb{G}} in(g) = 1$ is not necessary (thm. 5).

Now, we introduce the DOMLP in LP 6, which is equivalent to LP 5, but operates over the all-outcomes determinisation. The point is that actions and preservation of flow are expressed over the all-outcomes determinisation instead of the SSP, and then the missing probabilities are recovered via the regrouping

¹Similarly, with a careful treatment of improper policies' trajectories, a policy π 's probability of reaching the goal can be given by $P(\mathbb{G}|\pi, s_I) = \sum_{T \in \mathcal{T}_{\pi, s_I}} P(T)$.

Occupation Measure LP (OMLP)

$$\begin{aligned}
& \min_{\mathbf{x}} \sum_{s \in \mathbf{S}, a \in \mathbf{A}(s)} x_{s,a} \cdot C(a) & (\text{LP 5}) \\
& \text{s.t.} \quad \sum_{a \in \mathbf{A}(s)} x_{s,a} - \sum_{s' \in \mathbf{S}, a' \in \mathbf{A}(s')} x_{s',a'} \cdot P(s|s',a') = \llbracket s = s_I \rrbracket & \forall s \in \mathbf{S} \setminus \mathbf{G} \quad (\text{C10}) \\
& \quad x_{s,a} \geq 0 & \forall s \in \mathbf{S}, a \in \mathbf{A}(s) \quad (\text{C11})
\end{aligned}$$

constraints (C13). This will be useful later because network flow over deterministic graphs is better understood than network flow over probabilistic graphs.

LP 6 operates over the occupation measure $\mathbf{y}$. Recall that the all-outcomes determinisation splits each probabilistic action a into the deterministic actions $a_{s \rightarrow s'}$ for each $s' \in \text{succ}(s, a)$, so $y_{s,a,s'}$ denotes the flow passing through the determinised action $a_{s \rightarrow s'}$. C12 are the preservation-of-flow constraints which ensure that any flow entering a non-goal state s must equal to flow exiting s (except at s_I), where $y_{s,a} \stackrel{\text{def}}{=} \sum_{s' \in \text{succ}(s,a)} y_{s,a,s'}$. The out-flow and in-flow for LP 6 are $\text{out}'(s) \stackrel{\text{def}}{=} \sum_{a \in \mathbf{A}(s)} y_{s,a}$ and $\text{in}'(s) \stackrel{\text{def}}{=} \sum_{s' \in \mathbf{S}, a' \in \mathbf{A}(s')} y_{s',a',s}$, which lets us rewrite C12 as $\text{out}'(s) - \text{in}'(s) = \llbracket s = s_I \rrbracket$. Note that this is similar to the preservation-of-flow in LP 5, except that we are dealing with deterministic actions, which makes it easier to express the in-flow. Since we are considering the all-outcomes determinisation, and no longer encoding the probabilistic effects with flow distributions, we have to recover the probabilities with additional constraints. These additional constraints are the regrouping constraints (C13), similar to the regrouping constraints used in (Trevizan, Thiébaux, and Haslum, 2017). They simply ensure that, after applying a to s , the amount of flow passing through a particular effect s' is equal to the total flow passing through s and a multiplied by $P(s'|s, a)$, i.e., $y_{s,a,s'} = P(s'|s, a) \cdot y_{s,a}$. Regrouping constraints are not needed for probabilistic actions with a single effect, so we only define them for Υ where

$$\Upsilon \stackrel{\text{def}}{=} \{(s, a) : s \in \mathbf{S}, a \in \mathbf{A}(s) \text{ and } |\text{succ}(s, a)| > 1\}.$$

Analogously to LP 5, a solution $\mathbf{y}$ induces the policy $\pi_{\mathbf{y}}(s, a) = \frac{y_{s,a}}{\text{out}'(s)}$. To gain an intuitive understanding of why LP 6 is equivalent to LP 5, observe that $y_{s,a}$ is tightly related to $x_{s,a}$.

Determinised Occupation Measure LP (DOMLP)

$$\begin{aligned}
& \min_{\mathbf{y}} \sum_{s \in \mathbf{S}, a \in \mathbf{A}(s)} y_{s,a} \cdot C(a) & (\text{LP 6}) \\
& \text{s.t.} \quad \sum_{a \in \mathbf{A}(s)} y_{s,a} - \sum_{s' \in \mathbf{S}, a' \in \mathbf{A}(s')} y_{s',a',s} = \llbracket s = s_I \rrbracket & \forall s \in \mathbf{S} \setminus \mathbf{G} \quad (\text{C12}) \\
& \quad y_{s,a,s'} = P(s'|s, a) \cdot y_{s,a} & \forall (s, a) \in \Upsilon, s' \in \text{succ}(s, a) \quad (\text{C13}) \\
& \quad y_{s,a,s'} \geq 0 & \forall s \in \mathbf{S}, a \in \mathbf{A}(s) \quad (\text{C14})
\end{aligned}$$

We now show that LP 6 is equivalent to LP 5:

Lemma 6. *LP 5 and LP 6 are equivalent, in the sense that there is a simple bijection between feasible solutions that preserves the objective.*

Proof. We define the function φ that maps solutions of LP 5 to LP 6 thus:

$$\varphi(\mathbf{x}) = \mathbf{y} \text{ s.t. } y_{s,a,s'} = P(s'|s, a) \cdot x_{s,a} \quad \forall s \in \mathbf{S}, a \in \mathbf{A}(s), s' \in \text{succ}(s, a).$$

We show that, for feasible $\mathbf{x}$, $\varphi(\mathbf{x}) = \mathbf{y}$ is indeed a feasible solution for LP 6 with the same objective as $\mathbf{x}$ for LP 5. Observe that $y_{s,a} = x_{s,a}$, since

$$y_{s,a} = \sum_{s' \in \text{succ}(s,a)} y_{s,a,s'} = \sum_{s' \in \text{succ}(s,a)} (x_{s,a} \cdot P(s'|s, a)) = x_{s,a}.$$

Now, we show that each of LP 6's constraints are satisfied by $\mathbf{y}$.

Preservation-of-flow constraints (C12):

$$\llbracket s = s_I \rrbracket = \sum_{a \in A(s)} x_{s,a} - \sum_{s' \in S, a' \in A(s')} x_{s',a'} \cdot P(s|s',a') = \sum_{a \in A(s)} y_{s,a} - \sum_{s' \in S, a' \in A(s')} y_{s',a'} \cdot P(s|s',a').$$

Regrouping constraints (C13):

$$y_{s,a,s'} = P(s, |s, a) \cdot x_{s,a} = P(s'|s, a) \cdot y_{s,a}.$$

Non-negativity constraints (C14): trivial, since $\mathbf{x} \geq 0$.

Finally, we see that the objective is preserved:

$$\sum_{s \in S, a \in A(s)} y_{s,a} \cdot C(a) = \sum_{s \in S, a \in A(s)} x_{s,a} \cdot C(a).$$

The inverse function φ^{-1} is defined everywhere as

$$\varphi^{-1}(\mathbf{y}) = \mathbf{x} \text{ s.t. } x_{s,a} = y_{s,a} = \sum_{s' \in \text{succ}(s,a)} y_{s,a,s'} \quad \forall s \in S, a \in A(s).$$

To prove that, for feasible $\mathbf{y}$, $\varphi^{-1}(\mathbf{y}) = \mathbf{x}$ gives a feasible solution with equal objective, we start by observing that $y_{s,a,s'} = P(s'|s, a) \cdot x_{s,a}$ by the regrouping constraints (C13). Consequently, the proof for φ^{-1} is similar to the one for φ , and uses the same equalities. This proves that φ is an objective-preserving bijection between feasible solutions of LP 5 and LP 6, as required. $\square$

5.2.2 Decomposing into plans and cycles

We define the Plan and Cycle Flow LP in LP 7. This LP is equivalent to LP 6 (and respectively LP 5), but operates over the flow through plans and cycles, rather than individual actions.

LP 7 searches over plan and cycle flows $\mathbf{f}$, with a variable for each plan and cycle $w \in \mathcal{P} \cup \mathcal{C}$ where f_w denotes the amount of flow routed through w . Thus, $\mathbf{f}$ is a kind of occupation measure, but over plans and cycles, rather than individual state-action pairs. LP 7 has the convexity constraint (C15), which ensures that the plans carry a unit of flow, i.e., the probability mass, from s_I to the goals. This works because plans carry the whole flow entering s_I to the goals, and the cycles do not introduce any flow. Consequently, LP 7 does not require the preservation-of-flow constraints, since they are automatically satisfied by flow through plans and cycles. For a sequence $\sigma = \langle s_0, a_0, \dots \rangle$, we write $\Phi(\sigma)$ to denote the total flow through σ , i.e., $\Phi(\sigma) \stackrel{\text{def}}{=} \sum_{w \in \mathcal{P} \cup \mathcal{C}: \sigma \subseteq w} f_w$, and by abuse of notation we omit the angled brackets for sequences, e.g., $\Phi(s_0, a_0, s_1)$ is $\Phi(\langle s_0, a_0, s_1 \rangle)$. Then, C16 are the regrouping constraints and $\mathbf{f}$ induces the policy $\pi_{\mathbf{f}}(s, a) = \frac{\Phi(s,a)}{\Phi(s)}$. To see an intuitive connection to LP 6 and LP 5, note that $\Phi(s, a, s')$ is tightly related to $y_{s,a,s'}$; $\Phi(s, a)$ is related to $y_{s,a}$ and $x_{s,a}$; and $\Phi(s)$ is related to $out(s)$ and $out'(s)$.

Plan and Cycle Flow LP (P&C LP)	
$\min_{\mathbf{f}} \quad \sum_{w \in \mathcal{P} \cup \mathcal{C}} f_w \cdot C(w)$	(LP 7)
$\text{s.t.} \quad \sum_{p \in \mathcal{P}} f_p = 1$	(C15)
$\Phi(s, a, s') = P(s' s, a) \cdot \Phi(s, a) \quad \forall (s, a) \in \Upsilon, s' \in \text{succ}(s, a)$	(C16)
$f_w \geq 0$	(C17)

We now prove that LP 7 is indeed equivalent to LP 6.

Lemma 7. *There is a 1:n mapping from network-flow solutions $\mathbf{y}$ that satisfy C12, and plan-and-cycle-flow solutions $\mathbf{f}$ that satisfy C15. That is, each $\mathbf{f}$ can be mapped onto a unique solution $\mathbf{y}$, and $\mathbf{y}$ can be mapped onto a (potentially non-unique) solution $\mathbf{f}$. This mapping preserves the flow, which means that $y_{s,a,s'} = \Phi(s, a, s')$.*

Proof. This is a specialisation of the theorem by [Ahuja, Magnanti, and Orlin \(1993, pp. 79–81\)](#), which gives a 1:n mapping between arbitrary network flow and a flow across paths and cycles.

(Flow across paths and cycles $\rightarrow$ network flow.) Given $\mathbf{f}$, we obtain a unique $\mathbf{y}$ by setting $y_{s,a,s'} \leftarrow \Phi(s, a, s') = \sum_{w \in \mathcal{P} \cup \mathcal{C}: \langle s, a, s' \rangle \subseteq w} f_w$. This $\mathbf{y}$ must satisfy the preservation-of-flow constraints [C12](#) because for all $s \in \mathcal{S} \setminus \mathcal{G}$

$$\sum_{a \in A(s)} y_{s,a} - \sum_{s' \in \mathcal{S}, a' \in A(s')} y_{s',a',s} = \sum_{\substack{a \in A(s), \\ s' \in \text{succ}(s,a)}} \Phi(s, a, s') - \sum_{s' \in \mathcal{S}, a' \in A(s')} \Phi(s', a', s) = \llbracket s = s_I \rrbracket.$$

The second inequality can be shown by cases: (Case 1: $s \neq s_I$). Then

$$\sum_{\substack{a \in A(s), \\ s' \in \text{succ}(s,a)}} \Phi(s, a, s') - \sum_{s' \in \mathcal{S}, a' \in A(s')} \Phi(s', a', s) = \Phi(s) - \Phi(s) = 0.$$

(Case 2: $s = s_I$). The argument from the first case does not apply: $\sum_{s' \in \mathcal{S}, a' \in A(s')} \Phi(s', a', s) \neq \Phi(s)$ because the LHS term does not include the plans starting at s_I , it only considers the cycles that enter s_I from another state. Thus, we can split $\sum_{a \in A(s_I), s' \in \text{succ}(s_I, a)} \Phi(s_I, a, s')$ into flow across plans and cycles, while $\sum_{s' \in \mathcal{S}, a' \in A(s')} \Phi(s', a', s)$ is only flow across cycles, which gives us:

$$\begin{aligned} \sum_{\substack{a \in A(s_I), \\ s' \in \text{succ}(s_I, a)}} \Phi(s_I, a, s') - \sum_{s' \in \mathcal{S}, a' \in A(s')} \Phi(s', a', s_I) &= \sum_{\substack{a \in A(s_I), \\ s' \in \text{succ}(s_I, a), \\ p \in \mathcal{P}: \langle s_I, a, s' \rangle \in p}} f_p + \sum_{\substack{a \in A(s_I), \\ s' \in \text{succ}(s_I, a), \\ c \in \mathcal{C}: \langle s_I, a, s' \rangle \in c}} f_c - \sum_{\substack{s' \in \mathcal{S}, a' \in A(s'), \\ c \in \mathcal{C}: \langle s', a', s_I \rangle \in c}} f_c \\ &= \sum_{p \in \mathcal{P}: \langle s_I \rangle \in p} f_p + \sum_{c \in \mathcal{C}: \langle s_I \rangle \in p} f_c - \sum_{c \in \mathcal{C}: \langle s_I \rangle \in c} f_c \\ &= \sum_{p \in \mathcal{P}} f_p \\ &= 1 \end{aligned}$$

where the final equality follows from the convexity constraint ([C15](#)).

(Network flow $\rightarrow$ flow across plans and cycles.) [Ahuja, Magnanti, and Orlin \(1993, pp. 79–81\)](#) give an algorithm that converts $\mathbf{y}$ into $\mathbf{f}$. It considers the quantity $e(s)$ for each state, called the node imbalance, which represents how much more flow is coming into a state, than how much is exiting. If $e(s) > 0$, i.e., more flow is entering than exiting, then s is called an excess state; if $e(s) < 0$, i.e., more flow is exiting than entering, then s is a deficit state; and otherwise, if $e(s) = 0$, then s is balanced. As a prerequisite, they require that the network flow $\mathbf{y}$ satisfies the following two equations:

$$in'(s) - out'(s) = -e(s) \quad \forall s \in \mathcal{S} \quad (5.1)$$

$$\sum_{s \in \mathcal{S}} e(s) = 0. \quad (5.2)$$

Indeed, $-e(s) = \llbracket s = s_I \rrbracket$ for all states $s \in \mathcal{S} \setminus \mathcal{G}$ by [C12](#), and $\sum_{s \in \mathcal{G}} e(s) = 1$ by thm. 5, and thereby $\sum_{s \in \mathcal{S}} e(s) = 0$. Importantly, the only deficit state is s_I , and the only excess states are in $\mathcal{G}$. The algorithm works by picking a deficit state s_{deficit} , and routing its deficit through the graph. The routing forward stops when it reaches an excess state s_{excess} , in which case it returns the path from s_{deficit} to s_{excess} ; or it stops when it encounters a cycle, in which case it returns the cycle. Since s_I is the only deficit node, and the only excess states are in $\mathcal{G}$, the only extracted paths must go from s_I to $\mathcal{G}$, i.e., they are plans. Furthermore, $e(s_I) = -1$ and $\sum_{s \in \mathcal{G}} e(s) = 1$, so a precisely one unit of flow must be routed from s_I to the goals through the plans, thus $\mathbf{f}$ must satisfy [C15](#). We do not worry about the extracted cycles, because we have no constraints for them. $\square$

Lemma 8. *There is a 1:n mapping from solutions of [LP 6](#) to solutions of [LP 7](#) that preserve objective costs.*

Proof. By lem. 7, we know there is a 1:n mapping between $\mathbf{y}$ that satisfy [C12](#), and $\mathbf{f}$ that satisfy [C15](#), so that $y_{s,a,s'} = \Phi(s, a, s')$. Then, we see that, if one satisfies the regrouping constraints, then the other

one must also satisfy them, since $y_{s,a,s'} = \Phi(s, a, s')$ and $y_{s,a} = \Phi(s, a)$. The non-negativity constraints are similarly satisfied, and by a similar argument the objectives must be equal. $\square$

We have derived and shown that [LP 7](#) is equivalent to [LP 6](#) by exploiting results from network flow theory, but in fact, we can also obtain [LP 7](#) by applying Dantzig-Wolfe decomposition to [LP 6](#) ([Desrosiers and Lübbecke, 2005](#)). Using Dantzig-Wolfe decomposition on [LP 6](#), we categorise [C12](#) as “simple” constraints, and [C13](#) as “complicating” constraints. The simple constraints describe a convex polyhedron, which we re-express as a convex combination of extreme points and a non-negative combination of extreme rays using the Weyl-Minkowski theorem. For the polyhedron described by [C12](#), the extreme points correspond to plans and extreme rays correspond to cycles. Thus, we replace the simple constraints [C12](#) by the convex combination of extreme points and non-negative combination of extreme rays:

$$y_{s,a,s'} = \sum_{w \in \mathcal{P} \cup \mathcal{C}} [\langle s, a, s' \rangle \in w] \cdot f_w \quad \forall s \in \mathcal{S}, a \in \mathcal{A}(s), s' \in \text{succ}(s, a) \quad (\text{C18})$$

$$\sum_{p \in \mathcal{P}} f_p = 1 \quad (\text{C19})$$

$$f_w \geq 0 \quad \forall w \in \mathcal{P} \cup \mathcal{C} \quad (\text{C20})$$

with the new variables $\mathbf{f}$. However, the variables $\mathbf{y}$ are no longer needed, so they can be replaced with the right-hand side of [C18](#), and the tying constraints [C18](#) can subsequently be removed. This leaves us with the convexity constraint [C19](#), the positive constraints [C20](#), and the complicating constraints [C16](#) (with $\mathbf{y}$ replaced by equivalent $\mathbf{f}$), which precisely yields [LP 7](#).

By chaining together the equivalences we have shown, we can see that [LP 7](#) is equivalent to [LP 5](#) and is able to express the same policies as [LP 5](#), i.e., all proper policies. Since [LP 7](#) expresses its policies as combinations of plans and cycles, this leads to the key result:

Theorem 11. *Proper policies can be expressed as a combination of plans and cycles.*

5.2.3 Semantics of Plan and Cycle Decomposition

Solutions $\mathbf{x}$ for [LP 5](#) and $\mathbf{y}$ for [LP 6](#) have the intuitive semantic that $x_{s,a}$ and $y_{s,a,s'}$ respectively correspond to the expected number of times that $\langle s, a \rangle$ or $\langle s, a, s' \rangle$ is encountered when following their policies $\pi_{\mathbf{x}}$ and $\pi_{\mathbf{y}}$. This semantic gives us a natural understanding of the LPs, and importantly, enables us to express additional constraints over the SSP, e.g., an upper bound over the expected fuel usage can be expressed with $\sum_{s \in \mathcal{S}, a \in \mathcal{A}(s)} x_{s,a} \cdot \text{fuel-usage}(a) \leq 1000$. The plan and cycle representation does not in general have an equivalent semantic. This is because the plan and cycle representation is not unique, and introduces “plan mixing,” where two or more plans are used to encode the flow of other plans. We explain this phenomenon and show how a meaningful semantic can be recovered.

As an example of plan mixing, consider the SSP in [fig. 5.1](#), its plans $\mathcal{P}$ are:

- $p_{\uparrow\uparrow} = \langle s_0, a_0, s_{1\uparrow}, a_{1\uparrow}, s_2, a_2, s_{3\uparrow}, a_{3\uparrow}, s_g \rangle$
- $p_{\uparrow\downarrow} = \langle s_0, a_0, s_{1\uparrow}, a_{1\uparrow}, s_2, a_2, s_{3\downarrow}, a_{3\downarrow}, s_g \rangle$
- $p_{\downarrow\uparrow} = \langle s_0, a_0, s_{1\downarrow}, a_{1\downarrow}, s_2, a_2, s_{3\uparrow}, a_{3\uparrow}, s_g \rangle$
- $p_{\downarrow\downarrow} = \langle s_0, a_0, s_{1\downarrow}, a_{1\downarrow}, s_2, a_2, s_{3\downarrow}, a_{3\downarrow}, s_g \rangle$

where $a_i \in \mathcal{A}(s_i)$ for $i \in \{0, 1\uparrow, \dots, 3\downarrow\}$, noting that each $\mathcal{A}(s_i)$ contains a single action, so the action is unique.

The following solutions satisfy this SSP’s [LP 7](#):

1. $f_{p_{\uparrow\uparrow}} = f_{p_{\downarrow\downarrow}} = 0.5$ and $f_{p_{\uparrow\downarrow}} = f_{p_{\downarrow\uparrow}} = 0$
2. $f_{p_{\uparrow\downarrow}} = f_{p_{\downarrow\uparrow}} = 0.5$ and $f_{p_{\uparrow\uparrow}} = f_{p_{\downarrow\downarrow}} = 0$
3. $f_{p_{\uparrow\uparrow}} = f_{p_{\uparrow\downarrow}} = f_{p_{\downarrow\uparrow}} = f_{p_{\downarrow\downarrow}} = 0.25$
4. any other convex combination of solutions 1 and 2.

All of these solutions encode the same policy π with $\pi(s_i, a_i) = 1$ for all $i \in \{0, 1\uparrow, \dots, 3\downarrow\}$, which is the only closed policy for this SSP. Solution 3 is the most intuitive, with $P(p|\pi) = 0.25$ for each $p \in \mathcal{P}$,

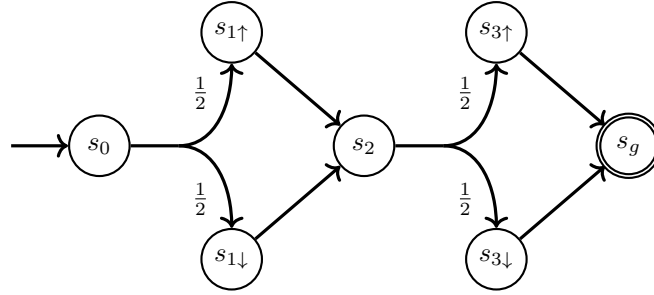


Figure 5.1: Consider the plans $p_{\uparrow\uparrow}$ and $p_{\downarrow\downarrow}$ passing through the top ($\uparrow$) and bottom ($\downarrow$) states respectively. The combination $f_{p_{\uparrow\uparrow}} = 0.5 = f_{p_{\downarrow\downarrow}}$ satisfies all the constraints of LP 7, but the active plans do not give all the ensuing policy’s trajectories. The plans are “mixing.”

representing that the probability of a plan p occurring when following π is 0.25. In contrast, solutions 1 and 2 mix two plans together to obtain all trajectories, without any clear semantics attached to the flow. Solutions 1 and 2 are the only basic feasible solutions for this LP, and solution 3 is a convex combination of the two. Consequently, algorithms that work over basic feasible solutions (notably the simplex method) are unable to recover the semantically intuitive solution in the presence of plan mixing. This is true in general.

In order to recover a relation between $\mathbf{f}$ and the trajectory probabilities, we introduce the notion of a canonical plan and cycle decomposition, which disallows plan mixing. We start by restricting our attention to *acyclic SSPs*.

Definition 31 (Acyclic SSP). *An SSP is acyclic if its all-outcomes determinisation contains no cycles, i.e., $\mathcal{C} = \emptyset$.*

An important feature of acyclic SSPs is that their policies can be decomposed into plans only, without any cycles. Note that the definition of acyclic SSPs can be weakened to allow cycles in parts of the state space such that optimal policies do not encounter them, but we do not pursue this. For acyclic SSPs, the definition of canonical plan and cycle decomposition is straightforward:

Definition 32. *For an acyclic SSP and policy π , a plan flow representation $\mathbf{f}$ is canonical if $f_p = P(p|\pi)$ for all $p \in \mathcal{P}$ (noting that no cycles are needed due to acyclicity).*

A canonical solution gives us the desired semantic $f_p = P(p|\pi)$ by construction. Canonical solutions represent the policy π by breaking it down into its finitely-many trajectories $\mathcal{T}_{\pi, s_I}$, which are plans, and assigns them their probability of occurring. They have the following properties:

- given an acyclic SSP $\mathbb{S}$ and a proper policy π , a canonical representation exists, and is unique;
- there is a bijection between canonical solutions and proper policies;
- canonical representations disallow plan mixing.

Thus, if we restrict LP 7 to canonical solutions, we can still find the optimal policy, but with the additional semantics.

For acyclic SSPs, we can ensure LP 7 only finds canonical plan-and-cycle flows with a simple modification: add the set of constraints

$$f_p \leq \prod_{\substack{s \in \mathbb{S}, a \in \mathbf{A}(s), \\ s' \in \text{succ}(s, a) \text{ s.t.} \\ \langle s, a, s' \rangle \subseteq p}} P(s'|s, a) \quad \forall p \in \mathcal{P}. \quad (\text{C21})$$

We call these the canonical constraints. Importantly, the right-hand side is cheap to compute for its plan. It is clear that the canonical constraints allow canonical solutions, but it is less obvious that they disallow non-canonical ones. It relies on the fact that non-canonical solutions with plan mixing must route additional flow through some plan to represent the flow from missing plans, and precisely this additional flow is disallowed by the canonical constraints.

We have only discussed acyclic SSPs, and we do not consider the cyclic case. The semantic becomes more complicated because a cycle may occur multiple times within a single trajectory, and a single cycle may

be associated with multiple plans. One possible semantic is

$$f_w = \sum_{T \in \mathcal{T}_{\pi, s_I}} P(p|\pi) \cdot \text{number of occurrences of } w \text{ in } T.$$

That is, f_w gives the expected number of times that w is encountered when following π . The technique we used to ensure a canonical representation for acyclic SSPs does not work here, and it is not clear how the semantic can be enforced efficiently; the cyclic case introduces many challenges, including the issue that the number of trajectories becomes infinite.

5.3 CoGNe

LP 7 finds an optimal policy as a combination of plans and cycles, but it has impractically many variables to solve directly. In this section, we present Column Generation over Network Flow (CoGNe), a framework for solving LP 7 efficiently. We start by applying variable generation in sec. 5.3.1, and then go into detail about the ensuing subproblems (pricing problems) in sec. 5.3.2

5.3.1 Variable Generation

CoGNe solves LP 7 but doing so directly is impossible in practice because the set of plans and cycles $\mathcal{P} \cup \mathcal{C}$ is impractically large. There are in the worst case exponentially-many plans and cycles w.r.t. the number of states and actions (as we will show in sec. 5.4.1). To make matters worse, the number of states can be exponentially larger than the input problem we consider, because they are specified in a compact encoding, namely PPDDL (Younes et al., 2005), i.e., LP 7 may have double-exponentially many variables w.r.t. the input. To address this issue, CoGNe applies variable generation to LP 7 (see sec. 3.5). Recall that variable generation is a technique from OR that lets us handle LPs with many variables by starting with a subset of variables, and only adding variables as they are needed.

We will explain briefly how it is applied to LP 7. For this purpose, it is convenient to write LP 7 as $\min_{\mathbf{f}} \mathbf{c} \cdot \mathbf{f}$ s.t. $\mathbf{A}\mathbf{f} = \mathbf{b}, \mathbf{f} \geq 0$ and its dual as $\max_{\boldsymbol{\mu}} \mathbf{b} \cdot \boldsymbol{\mu}$ s.t. $\mathbf{A}^T \boldsymbol{\mu} \leq \mathbf{c}$. We will also write $\mathbf{a}_w$ to refer to the column corresponding to the plan or cycle w in the constraint matrix $\mathbf{A}$.

LP 7 is the Master Problem (MP): it is the problem we want to solve, but we can not do so directly because it has too many variables. The variable generation algorithm works in the following steps:

1. Construct the reduced master problem (RMP), which is a copy of the master problem, but it is restricted to a subset of variables $\widehat{\mathcal{P}} \cup \widehat{\mathcal{C}} \subset \mathcal{P} \cup \mathcal{C}$.
2. Solve the RMP to obtain an optimal solution $\bar{\mathbf{f}}$, and the corresponding dual solution $\bar{\boldsymbol{\mu}}$. This is a feasible solution to the master problem, but may not be optimal. (We discuss the possibility that no feasible solution exists later.)
3. Construct the pricing problem. This is a new optimisation problem whose solution determines which variables should be added to the RMP. The reduced cost of a variable w is defined $rc(w, \boldsymbol{\mu}) = C(w) - \boldsymbol{\mu} \cdot \mathbf{a}_w$. This reduced cost $rc(w, \boldsymbol{\mu})$ is a bound on how much the objective can improve if w is added, so w is only worthwhile adding if $rc(w, \boldsymbol{\mu}) < 0$.
4. Solve the pricing problem, where a solution is a plan or cycle w with $rc(w, \boldsymbol{\mu}) < 0$.
 - If a solution w exists: add w to $\widehat{\mathcal{P}} \cup \widehat{\mathcal{C}}$ to construct the new RMP, and loop to step 2.
 - If no solution exists: the variable generation framework guarantees that $\bar{\mathbf{f}}$ is an optimal solution to the master problem, so we terminate and return $\bar{\mathbf{f}}$.

CoGNe's RMP for $\widehat{\mathcal{P}} \cup \widehat{\mathcal{C}}$ is given in LP 8. We use $\widehat{\Upsilon} \subseteq \Upsilon$ to represent state-action pairs that require regrouping constraints in the RMP, defined by

$$\widehat{\Upsilon} \stackrel{\text{def}}{=} \{(s, a) \in \mathbf{S} \times \mathbf{A} : \exists w \in \widehat{\mathcal{P}} \cup \widehat{\mathcal{C}} \text{ s.t. } \langle s, a \rangle \subseteq w\} \cap \Upsilon.$$

Technically, variable generation only produces new variables and assumes that all RMPs contain all constraints. However, constraints are only meaningful if there is at least one variable affected, and in practice, unneeded constraints are left out of RMPs. Thus, $\widehat{\Upsilon}$ specifies which constraints affect a variable in $\widehat{\mathcal{P}} \cup \widehat{\mathcal{C}}$, and must be included in the RMP; constraints in $\Upsilon \setminus \widehat{\Upsilon}$ are not needed and are left out. Leaving

RMP for Plan and Cycle Flow

$$\min_{\mathbf{f}} \sum_{w \in \hat{\mathcal{P}} \cup \hat{\mathcal{C}}} f_w \cdot C(w) \quad (\text{LP } 8)$$

$$\text{s.t.} \quad \sum_{p \in \hat{\mathcal{P}}} f_p = 1 \quad (\text{C22})$$

$$\Phi(s, a, s') = P(s'|s, a) \cdot \Phi(s, a) \quad \forall (s, a) \in \hat{\Upsilon}, s' \in \text{succ}(s, a) \quad (\text{C23})$$

$$f_w \geq 0 \quad \forall w \in \hat{\mathcal{P}} \cup \hat{\mathcal{C}} \quad (\text{C24})$$

out constraints for $\Upsilon \setminus \hat{\Upsilon}$ does not affect the primal, nor the dual solution. Our implementation uses LP 8, and only generates constraints when a relevant variable is added to $\hat{\mathcal{P}} \cup \hat{\mathcal{C}}$.

As we foreshadowed in step 2, it is possible that an RMP is infeasible, even though the master problem is itself feasible. This happens when the RMP is missing variables that are necessary for a feasible solution, e.g., if $\hat{\mathcal{P}} \cup \hat{\mathcal{C}}$ only contains a single plan with probabilistic effects, then the convexity and regrouping constraints can not be simultaneously satisfied. This is an issue, because a pricing problem can not be constructed if step 2 fails. We present and discuss four techniques to address this issue.

Big- M . We introduce a “big- M ” variable, which is effectively a dummy plan that satisfies all regrouping constraints but incurs a large cost $M \in \mathbb{R}_{>0}$. Thus, the RMP is always satisfiable by routing flow through the dummy plan, but its high cost encourages flow to be routed through real plans. Note that M has to be selected by the user in a way that ensures the optimal policy has a lower cost than M , otherwise CoGNe will output the dummy plan as a solution. This big- M approach happens naturally under the finite-penalty transformation if we add the “give-up” action from s_I as a plan to the initial RMP. A disadvantage of this approach is that the RMP solutions will be the meaningless dummy plan until a proper policy is found, which can take a long time. This means CoGNe is unable to produce meaningful intermediate solutions, and only produces a useful output once it finds its first proper policy. This is problematic in the online setting, but inconsequential in the optimal setting. We are considering the optimal setting in this work, so we will use this big- M approach for our implementation of CoGNe.

Farkas Pricing Problem. The Farkas pricing problem is similar to the pricing problem, but instead of finding columns that improve the RMP’s objective, we find columns that bring the RMP closer to being feasible. The key insight is that an unbounded LP has an infeasible dual LP; so, if we can make the RMP’s dual bounded (and feasible), it will make the RMP itself feasible (Bertsimas and Tsitsiklis, 1997). Farkas’ lemma tells us that either $A\mathbf{f} = \mathbf{b}, \mathbf{f} \geq 0$ is feasible or there is a vector $\boldsymbol{\mu}_{\text{ray}}$ with $A^T \boldsymbol{\mu}_{\text{ray}} \leq 0$ and $\boldsymbol{\mu}_{\text{ray}} \cdot \mathbf{b} > 0$ (Lübbecke, 2011). This vector $\boldsymbol{\mu}_{\text{ray}}$ is an unbounded ray in the RMP’s dual, which means that $\lambda \boldsymbol{\mu}_{\text{ray}}$ gives a feasible dual solution for any $\lambda \in \mathbb{R}_{\geq 0}$, and can produce an arbitrarily large objective as λ gets larger. We want to find a variable f_w for the primal RMP, so that it induces a constraint in the dual that blocks this ray, i.e., we are looking for f_w s.t. adding it to A and $\mathbf{c}$ has the effect $A^T \boldsymbol{\mu}_{\text{ray}} \not\leq \mathbf{c}$, making $\boldsymbol{\mu}_{\text{ray}}$ infeasible for the RMP’s dual. Thus, for $\boldsymbol{\mu}_{\text{ray}}$, we define the Farkas cost for a plan or cycle w as

$$fc(w, \boldsymbol{\mu}_{\text{ray}}) = -\boldsymbol{\mu}_{\text{ray}} \cdot \mathbf{a}_w$$

and the Farkas pricing problem is to find a w with $fc(w, \boldsymbol{\mu}_{\text{ray}}) < 0$. If w has $fc(w, \boldsymbol{\mu}_{\text{ray}}) < 0$, it should be added because it blocks the infeasible ray, and if no such w exists we have proved that the master problem is infeasible. The advantage of the Farkas pricing approach over the big- M approach is that it does not require a user-selected variable M . However, it still has the disadvantage of the big- M approach that we are unable to extract a meaningful solution until CoGNe finds a proper policy, and based on preliminary experiments it does not offer better performance, so we do not consider it further.

Allowing Leaks. We add dummy variables that can be used to satisfy regrouping constraints, but with a large penalty. One way to do this is to introduce a variable $\text{leak}_{s,a,s'}$ for each state-action pair’s effect, representing flow “leaking” at that effect. That is, the regrouping constraints C23 become

$$\Phi(s, a, s') + \text{leak}_{s,a,s'} = P(s'|s, a) \cdot \left(\Phi(s, a) + \sum_{s'' \in \text{succ}(s, a)} \text{leak}_{s,a,s''} \right) \quad \forall (s, a) \in \hat{\Upsilon}, s' \in \text{succ}(s, a)$$

and the objective becomes

$$\min_{\mathbf{f}, \mathbf{leak}} \sum_{w \in \hat{\mathcal{P}} \cup \hat{\mathcal{C}}} f_w \cdot C(w) + L \sum_{\substack{(s,a) \in \hat{\Upsilon}, \\ s' \in \text{succ}(s,a)}} \text{leak}_{s,a,s'}$$

where $L \in \mathbb{R}_{>0}$ is a large penalty term to discourage leaks. Clearly, if a solution to the new LP has $\mathbf{leak} = 0$ then that solution is also valid for the original LP, with the same objective. Once the RMP can describe a proper policy, and provided that L is large enough, then the new LP's optimal solution will indeed have $\mathbf{leak} = 0$, and thereby gives a correct solution to the original LP. An interesting feature of this approach is that it can, in contrast to the previous approaches, encode improper policies. This is beneficial in the anytime setting because we can interrogate CoGNe for a solution before it has found a proper policy, and it is able to return an improper one, which is more valuable than a dummy policy. However, it is important to recognise that this formulation does not have a nice semantic, and $\text{leak}_{s,a,s'}$ does not represent the probability that the policy must give up after applying a to s .² Consider the example in fig. 5.2. With the plan $p = \langle s_0, a_0, s_1, a_1, s_g \rangle$, a feasible solution requires

$$\frac{1}{2} \text{leak}_{s_0,a_0,s_2} = \frac{1}{2} f_p \quad \text{and} \quad \frac{1}{2} \text{leak}_{s_1,a_1,s_2} = \frac{1}{2} f_p.$$

This is a problem because the two leaks should not be equal: the amount of flow reaching $\text{leak}_{s_1,a_1,s_2}$ should be less than the flow reaching $\text{leak}_{s_0,a_0,s_2}$, since some flow has been leaked.

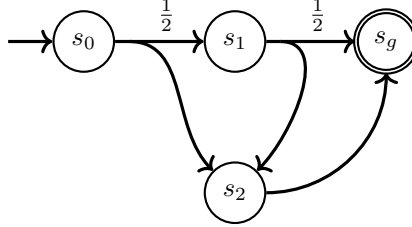


Figure 5.2: SSP where the leakage formulation does not have a nice semantic.

Consequently, since the leakage does not encode the actual probability of policy failure, it can happen that an RMP's optimal solution has a lower probability of success than another feasible solution. Even with this drawback, in our preliminary experiments, allowing leaks was the most promising approach we tested in the online setting. Again, this work focuses on the optimal setting, so we do not consider this formulation further.

Adding all Give-up Plans. The final technique we consider, is to add for each plan and cycle in the RMP, the corresponding set of plans and cycles with give-up actions. That is, when the plan $p = \langle s_0, a_0, \dots, s_n \rangle$ gets added to the RMP, we also add the plans

$$\langle s_0, a_{\text{give-up}}, s_g \rangle, \langle s_0, a_0, s_1, a_{\text{give-up}}, s_g \rangle, \dots$$

and similar for cycles. That way, the RMP can always construct a proper policy with the give-up actions, but the large give-up cost encourages the RMP to find policies that avoid give-up actions. In practice, this method is impractical because it requires a very large number of plans and cycles.

Equipped with one of these techniques for handling infeasible RMPs, the variable generation algorithm is guaranteed to produce an optimal policy. The remaining step to fully specify CoGNe is to show how to solve the pricing problem efficiently, which we do now.

5.3.2 The Pricing Problem

So far, we have discussed CoGNe's pricing problem in abstract; now, we make it concrete. CoGNe's pricing problem is a deterministic planning problem with a particular structure that can be exploited, and certain quirks that make it difficult to solve. First, we explain the structure of LP 7's pricing problems in detail, and then highlight the features that make it easier and harder to solve.

²Note that this absence of semantics is separate from the issue of plan mixing (sec. 5.2.3), i.e., even if no plan mixing occurs, the flow with leaks does not correspond to probabilities.

Suppose an RMP has just been solved, and the dual variables of the solution are μ . We refer to the dual variables that correspond to the convexity constraint and regrouping constraints over s, a, s' as μ_{conv} and $\mu_{s,a,s'}$ respectively. The reduced cost for w is $rc(w, \mu) = C(w) - \mu \cdot \mathbf{a}_w$ which expands to

$$rc(w, \mu) = C(w) - \llbracket w \in \mathcal{P} \rrbracket \cdot \mu_{\text{conv}} - \sum_{\substack{(s,a) \in \hat{\mathcal{T}}, \\ s' \in \text{succ}(s,a)}} (\llbracket \langle s, a, s' \rangle \in w \rrbracket - P(s'|s, a)) \mu_{s,a,s'}.$$

Notice that $C(w)$ and the summation can be decomposed over transitions, which lets us define reduced cost over transitions

$$rc(s, a, s', \mu) \stackrel{\text{def}}{=} C(a) - \sum_{s'' \in \text{succ}(s,a)} (\llbracket s'' = s' \rrbracket - P(s''|s, a)) \mu_{s,a,s''}.$$

A solution to the pricing problem is any plan or cycle with $rc(w, \mu) < 0$. While variable generation algorithms often search for columns that minimise the reduced cost, it is sufficient to find any column with negative reduced cost. Therefore,

CoGNe's pricing problem is a deterministic search problem over the input SSP's all-outcomes determination with the modified cost function $C'(s, a, s') \stackrel{\text{def}}{=} rc(s, a, s', \mu)$. A solution to the pricing problem is

- a plan p with $rc(p, \mu) = C'(p) - \mu_{\text{conv}} < 0$; or
- a cycle c with $rc(c, \mu) = C'(c) < 0$; or
- a guarantee that no such plans or cycles exist.

Complicating Features

CoGNe's pricing problem has two important features that differentiate it from standard deterministic planning problems, making it more difficult to solve and preventing us from using standard deterministic planning algorithms.

State-dependent costs. The cost of an action in the pricing problem is given by $C' : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$, i.e., the cost depends on the state it is applied in, as well as the state it leads into. Unfortunately, state-of-the-art domain-independent heuristics are incompatible with state-dependent costs because such heuristics require a compact encoding in which state-dependent costs can not be expressed. State-dependent costs can be encoded in PDDL, for example with conditional effects (Haslum et al., 2019); however, state-of-the-art domain-independent heuristics rely on a less expressive encoding such as STRIPS (Fikes and Nilsson, 1971) or SAS⁺ (Bäckström and Nebel, 1995), and the translation from PDDL with conditional effects into STRIPS or SAS⁺ has potentially exponential blowup. The large resulting problem encoding makes existing heuristics ineffective. To restate this: current domain-independent heuristics are not amenable to problems with state-dependent costs, and this is an important issue because effective domain-independent heuristics are the key component that makes planning efficient — without access to good heuristics the problem becomes much harder to solve. It is worth mentioning that state-dependent costs do not affect most algorithms, e.g., A* can handle state-dependent costs.

Negative action costs and negative cycles. The pricing problem's cost function C' may have negative costs, and importantly, the problem can have negative cycles, which are solutions to the pricing problem. Negative costs are a minor inconvenience, but the more significant issue is that our pricing problems contain negative cycles, and these are solutions that must be found. This prevents us from using the algorithms and heuristics for standard deterministic shortest path problems, which assume non-negative costs and therefore the absence of negative cycles. For example, if A* encounters a negative cycle it will get stuck there forever. It is possible to modify A* so that it detects negative cycles and can break out of the loop, but this does not address the fundamental issue that A* is designed to find plans and has no way to deliberately search for negative cycles. In other words, even if the methods for standard deterministic shortest path problems can be adapted to be compatible with negative action costs, they are not guaranteed to find all negative cycles and be complete for our pricing problems.

Simplifying Features

Now, we list some features of the pricing problem that make it easier to solve.

Sufficiently cheap (suboptimal) solutions. As we have already emphasised, any sufficiently cheap plan or cycle is a solution, and we do not need to search for an optimal one. This opens the door for suboptimal search algorithms. Solving pricing problems suboptimally is a well-known technique in the context of variable generation: it is called heuristic pricing and has been used extensively in the vehicle routing problem, e.g., (Laporte et al., 2000). Importantly, adding the plan or cycle with most negative reduced cost is not guaranteed to yield a solution in fewer steps than adding some plan or cycle with negative reduced cost, and the speedup from using suboptimal search usually pays off, especially for large problems. However, to guarantee that CoGNe converges to the optimum, we must ensure that we always find a sufficiently cheap plan or cycle if one exists, i.e. the algorithm we use must be complete.

Similarity between pricing problems. Another useful feature is that the states and transitions of all pricing problems is identical to the input SSP’s all-outcomes determinisation. The only part of the pricing problem that changes is the cost function C' . This suggests that the solution of one pricing problem may help to find a solution to the next one. It may be possible to exploit this in the form of “warm starts” or incremental search.

Special semantics for negative reduced cost solutions. Finally, we give some insight into the structure of C' , and what makes a negative reduced cost solution. A transition has $C'(s, a, s') = C(s, a)$ if all $s'' \in \text{succ}(s, a)$ have $\mu_{s,a,s''} \neq 0$. In particular, if a transition is not involved in any active regrouping constraints, then its cost corresponds to the standard SSP cost. Respectively, a transition’s cost only has $C'(s, a, s') \neq C(s, a)$ if it is involved in an active regrouping constraint. An intuitive case where this happens is when the candidate policy has an unaccounted probability effect at that (s, a) pair, preventing the policy from being proper. These observations lets us partition plans with negative reduced costs into two categories:

- i. Plans for closing or improving a policy: a plan p is in this category if it has some $(s, a, s') \in p$ with $rc(s, a, s', \mu) < 0$. If the RMP currently has no way to satisfy the regrouping constraint for (s, a, s') then this plan is added to make the policy (more) closed; if there is already a way to satisfy the regrouping constraint, then this plan offers a cheaper way to satisfy the regrouping constraint.
- ii. Plans for starting a new policy: such a plan p has $C'(s, a, s') = C(a)$ for all $(s, a, s') \in p$, and only has a negative reduced cost because $C'(p) < \mu_{\text{conv}}$. Note that $\mu_{\text{conv}} = \text{obj}(\mathbf{f})$ because the dual LP’s objective is $\max_{\mu} \mu_{\text{conv}}$ and we have strong duality. Thus, p ’s cost must be less than the current candidate policy’s expected cost; in other words, p will induce a cheaper policy than the current candidate under the assumption that π ’s probabilistic effects can be closed without significant cost.

Cycles with negative reduced cost can never start a new policy, and can only be valuable for closing or improving a policy, i.e., the first category can be extended to plans and cycles for closing or improving a policy. Note that a plan or cycle from the first category may address multiple regrouping constraints, not just one. An efficient pricing problem solver may be able to exploit this categorisation. For example, if our RMP induces a proper policy, we can search over the original SSP’s action costs C with existing deterministic planning methods, since C is not state-dependent and has strictly positive action costs. As another example, if we use the RMP formulation with “allowing leaks” then a plan from s' (and the appropriate prefix) will be useful to close the candidate policy wherever $\text{leak}_{s,a,s'} > 0$.

5.3.3 Solving the Pricing Problem

Now that we have explained the pricing problem, and its features that differentiate it from standard deterministic shortest path problems, we describe our algorithms. We describe a complete algorithm for the pricing problem, and a fast incomplete algorithm. We also mention other approaches that did not pay off in our preliminary experiments.

Complete Method: Modification of Bellman-Ford. The Bellman-Ford algorithm is the go-to algorithm for deterministic shortest path problems with negative costs. It is essentially VI for the deterministic setting, but instead of computing the cost-to-go from each state it computes for the cost to reach each state s from s_I using an optimal plan, which we call the cost-to-reach; Bellman-Ford tracks

the cost-to-reach with $g : \mathcal{S} \rightarrow \mathbb{R}_{\geq 0} \cup \{\infty\}$. Suppose we are working over an SSP’s all-outcomes determination; the cost-to-reach is initialised with $g(s) = 0$ for $s \in \mathcal{G}$ and $g(s) = \infty$ for $s \in \mathcal{S} \setminus \mathcal{G}$, then the algorithm iterates over each action $a_{s \rightarrow s'}$ and updates $g(s') \leftarrow \min\{g(s) + C(a_{s \rightarrow s'}), g(s')\}$. Each of these updates is called a “relaxation” of the transition. These “sweeps” of relaxations are repeated until g is a fixed point, which is guaranteed to happen in at most $|\mathcal{S}| - 1$ steps in the absence of negative cycles. This standard implementation of Bellman-Ford is completely impractical for our pricing problems because it fails to scale to our large search spaces, so we use a modified version that it is significantly more efficient for our pricing problems. We list the standard implementation’s weaknesses, and how we combat them in our modified version:

- **The number of states is too large.** Even enumerating all reachable states can be prohibitively expensive. Instead of sweeping across all state-action-state transitions, we maintain a queue of states whose outgoing transitions need to be updated in the next iteration. This queue is initialised with $A(s_I)$, and updated with $A(s)$ for any s whose $g(s)$ is updated. This turns the algorithm into a kind of Breadth-First Search with re-expansions. We keep track of the encountered states $\hat{\mathcal{S}}$, and we can terminate after $|\hat{\mathcal{S}}|$ iterations (as long as this value is current), instead of $|\mathcal{S}| - 1$. Another benefit of this approach is that we avoid wasting compute on relaxations for states that will not update g .
- **We need a sufficiently cheap solution, not an optimal one.** Bellman-Ford searches for an optimal plan or negative cycle, but we only need some plan or cycle with negative reduced cost. To address this, we simply stop the search as soon as we detect a negative reduced cost plan or cycle and return it. If such a plan or cycle is not found, then the modified algorithm behaves like the standard Bellman-Ford, and guarantees completeness.
- **Detecting negative cycles efficiently.** The standard way for detecting negative cycles is to keep track of each state’s “distance” from s_I , i.e., $\text{dist} : \mathcal{S} \rightarrow \mathbb{N}$ where $\text{dist}(s_I) = 0$ and $\text{dist}(s) = 1$ for states that can be reached from s_I with one transition, etc. If our problem has unit costs, then $\text{dist}(s) = g(s)$. The optimal plan can have no more than $|\mathcal{S}| - 1$ steps, so if we iterate $|\mathcal{S}|$ times and discover a state with $\text{dist}(s) > |\mathcal{S}| - 1$, then s must be part of a negative cycle. This extends to our modification: if we find s with $\text{dist}(s) > |\hat{\mathcal{S}}|$, then s must be involved in a negative cycle. There are more sophisticated methods for detecting negative cycles, e.g., (Cherkassky and Goldberg, 1999), but we found that the distance-based approach was most efficient in our preliminary experiments.
- **Reusing information between iterations.** As we have pointed out in sec. 5.3.2, the states and transitions of the pricing problem are fixed, and only the costs change between pricing problems, potentially in relatively few places. Therefore, we can use the previous values of g as a “warm-start” for the next pricing problem, or in other words, we can frame the iterative searches over g as an incremental search. We implemented the technique from Lifelong Planning A* (Koenig, Likhachev, and Furcy, 2004) to update g so that it can be reused by the next Bellman-Ford, without violating correctness. Unfortunately, the overhead of propagating changes to g did not pay off in our preliminary experiments, so our version of Bellman-Ford does not use incremental search. As future work, one can consider different techniques for warm starts, e.g., (Frigioni, Marchetti-Spaccamela, and Nanni, 2000) as used by (Seipp, von Allmen, and Helmert, 2020), or even CG-iLAO*’s mechanism for tracking potential constraint violations (see sec. 4.4).

Thus, our modification of Bellman-Ford exploits the simplifying feature that we only need a sufficiently cheap solution, but fails to exploit the similarity between problems and the special semantics for negative reduced cost solutions. Our modification remains **complete**: when no negative reduced cost plans or cycles are detected, then our modified version effectively becomes the standard Bellman-Ford (with the reachable state space computed on the fly), and we can terminate it after $|\hat{\mathcal{S}}|$ iterations, with the guarantee that no negative reduced cost plans or cycles exist.

Incomplete, Fast Method: Large Neighbourhood Search. Large Neighbourhood Search (LNS) is a standard technique from optimisation, where some incumbent solution is partially “destroyed” and then “repaired” multiple times, until the updated solution is sufficiently good. In our case, we start with a plan in the RMP, i.e., $p \in \hat{\mathcal{P}}$; we partially destroy $p = \langle s_0, a_0, \dots, s_n \rangle$ by removing some subsequence of the plan, resulting in two partial plans $p_{\text{head}} = \langle s_0, a_0, \dots, s_j \rangle$ and $p_{\text{tail}} = \langle s_k, a_k, \dots, s_n \rangle$ where $j < k$; these partial plans are repaired by running a depth-bounded search from s_j to some state in p_{head} or p_{tail} . There are three possible outcomes of the repair:

1. The repair constructs a plan p' that is cheaper than the starting p . If p' is a negative reduced cost plan, then p' is returned, otherwise p' is set as the new incumbent and the process continues.
2. The repair constructs a cycle c . This happens if the depth-bounded search finds a path to a state in p_{head} . If the cycle has a negative reduced cost it is returned, otherwise it is ignored and the process continues.
3. The repair fails to find a cheaper plan or cycle. Then the process continues.

The steps of destroy and repair are repeated multiple times, until a negative reduced cost solution is found, or the process times out. Note that the destruction has to be different from one iteration to the next, otherwise we may unsuccessfully try to repair the same partial plans repeatedly. We tested various destruction techniques, but found that randomly selecting a subsequence of bounded size performs best.

LNS exploits the fact that we do not require optimal solutions, only negative reduced cost ones. It also makes use of the other simplifying features of pricing problems, to some extent. LNS's initial plan is a negative reduced cost plan p from a previous pricing problem, and by destroying and repairing relatively small parts of p , it is searching for similar solutions, implicitly relying on the property that the new pricing problem should be similar to the old one. Furthermore, if p has regrouping constraints that can be satisfied with some plan or cycle, then LNS has a good chance of finding it, if it destroys the part of p where the regrouping constraints occur.

In practice LNS is quite effective, as we will show in the experiments (sec. 5.4.2). However, it is important to recognise that LNS is incomplete and may fail to find a solution even if one exists. Recall that CoGNe can only be terminated with a guarantee of optimality if we are certain that no negative reduced cost plans or cycles exist; so, if LNS fails to find a solution we must use a complete algorithm (such as our modified version of Bellman-Ford) to find solutions that LNS missed, or to prove that no such solutions exist.

Other Incomplete Methods. We considered many other techniques for finding some negative reduced cost solution quickly. However, none of them offered significant improvements in our preliminary experiments over the methods we have already described. Therefore, we only give a brief description of some notable candidates. **Envelope Search:** run the modified Bellman-Ford over a subset of states and actions that contain all the regrouping constraints, and then run a deterministic planning such as FF (Hoffmann and Nebel, 2001) from the fringes of this search space. The idea is that we separate the “envelope” that contains all regrouping constraints because this is where the state-dependent and negative cost actions appear. We solve the envelope exactly with our modified Bellman-Ford, and propagate the cost-dependent and negative costs to the fringe of the envelope. Then, the problems from the fringes are standard deterministic shortest path problems, which we can solve with standard algorithms such as FF. **Outside Length:** for each plan and cycle added to the RMP, find its “undo” and “skip” actions, and add them as cycles and plans, respectively. For the plan or cycle w with transition s, a, s' the state $s'' \in \text{succ}(s, a)$ is an unaccounted probabilistic effect if $s'' \neq s'$, and an undo action simply takes s'' back to s or one of its predecessors in w , so that the unintended effect is “undone.” Similarly, a skip action takes s'' to some successor of s' in w , thereby “skipping” part of w . Such actions are easy to detect and can be added to the RMP as the relevant plan for skip actions or cycle for undo actions. However, doing this naïvely adds a lot of unnecessary variables to the RMP, and it is not trivial to avoid duplicates.

5.4 Evaluating CoGNe

In this section, we evaluate the effectiveness of CoGNe as an algorithm for solving SSPs optimally. We start by doing a theoretical comparison between CoGNe and existing algorithms (sec. 5.4.1), where we contrast the number of variables and constraints needed to find and encode an optimal policy. Then, we run experiments to compare CoGNe to the state-of-the-art (sec. 5.4.2), showing that on existing benchmarks CoGNe does not perform well, and identify the reasons.

5.4.1 Theoretical Comparison

We compare the representation used by CoGNe to the representations used by existing methods. Recall that existing algorithms for SSPs consider one of two search spaces: the primal space with a value function $V : \mathbf{S} \rightarrow \mathbb{R}_{\geq 0}$ (e.g., VI, VLP); or dual space with occupation measures $x_{s,a}$ for each $s \in \mathbf{S}, a \in \mathbf{A}(s)$ (e.g.,

OMLP). The primal and dual spaces are intimately related, and as problem sizes increase, it is unlikely that one representation is significantly more advantageous than the other. In contrast, LP 7 and CoGNe’s search spaces are over plans and cycles. This search space is completely different: it may be significantly smaller, or it can be exponentially larger. This suggests that CoGNe is complementary to the existing methods, i.e., they are able to solve different problems efficiently.

We give an example where the plan and cycle representation is more efficient than primal or dual representations. Consider the family of path-branching SSPs $\text{PATHS}(b, d)$ — we show $\text{PATHS}(2, 3)$ and $\text{PATHS}(3, 3)$ in fig. 5.3, and other PATHS problems are constructed similarly, i.e., there is a chain $s_0, s_{1,j}, \dots, s_{d,j}, s_g$ for each $j \in \{1, \dots, b\}$, and they all share s_0 and s_g . Note that PATHS problems consist entirely of deterministic actions and have no probabilistic branching.

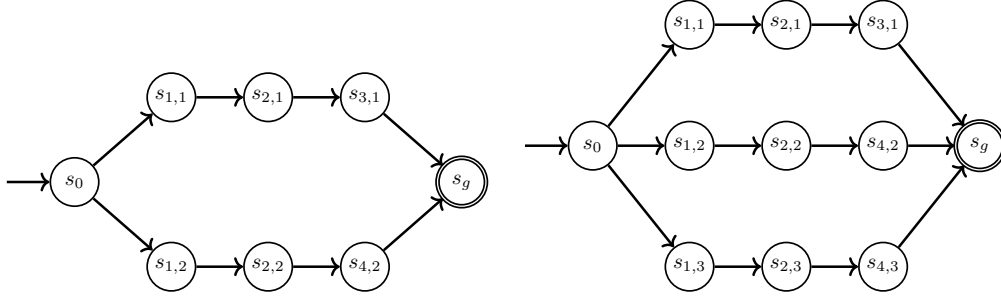


Figure 5.3: Path-branching SSPs $\text{PATHS}(b, d)$ with $b = 2, d = 3$ (left) and $b = 3, d = 3$ (right).

The problem $\text{PATHS}(b, d)$ has $bd + 2$ states and $bd + 1$ actions, and its optimal policy is any plan with length $d + 1$. We show in tab. 5.1 how many variables and constraints each representation requires to encode the optimal policy, and how many variables and constraints are required in the worst case, if search considers the whole search space.

	optimal policy		worst case	
	variables	constraints	variables	constraints
primal/dual	$O(d)$	$O(d)$	$O(bd)$	$O(bd)$
plan and cycle	1	0	b	0

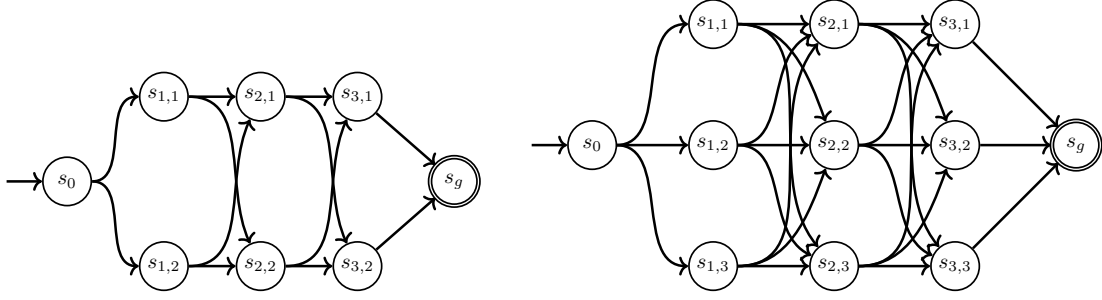
Table 5.1: Representation sizes for $\text{PATHS}(b, d)$.

The plan and cycle representation is significantly more compact than the primal and dual representations; in fact, it is arbitrarily more compact as b and d increase. Of course, each variable of the plan and cycle representation is a plan of length $d + 1$, but importantly, CoGNe is agnostic of this. The increased complexity of finding longer plans is delegated to the pricing problem, where we can leverage deterministic planning algorithms to find such plans efficiently.

In contrast, we now give an example where the plan and cycle representation is exponentially less compact. We introduce the family of probabilistic-branching SSPs $\text{PB}(b, d)$; fig. 5.4 shows $\text{PB}(2, 3)$ and $\text{PB}(3, 3)$. Similarly to $\text{PATHS}(b, d)$, $\text{PB}(b, d)$ has a $d \times b$ arrangement of states with a single action applicable in each $s \in \mathcal{S} \setminus \mathcal{G}$, but now, these have probabilistic effects that lead with uniform distribution to the states in the column immediately to the right. That is, for $s \in \mathcal{S} \setminus (\mathcal{G} \cup \{s_0\})$ the problem $\text{PB}(b, d)$ has $A(s_{i,j}) = \{a_{i,j}\}$ with $P(s_{i',j'} | s_{i,j}, a_{i,j}) = \frac{1}{b} \cdot \mathbb{I}[i' = i + 1]$.

The problem $\text{PB}(b, d)$ again has $bd + 2$ states and $bd + 1$ actions, but now its optimal and only closed policy is defined for all states, and applies the single applicable action in each non-goal state. The number of variables and constraints required for the different representations are in tab. 5.2. Note that the optimal policy in these problems requires the entire space to be encoded in all representations, so the table’s columns associated with the optimal policy and the worst case are collapsed. The critical result from this table is that indeed, the plan and cycle representation requires an exponential number of plans to encode the optimal policy, whereas the primal and dual representations only need polynomially many, i.e., the plan and cycle representation is exponentially less compact.

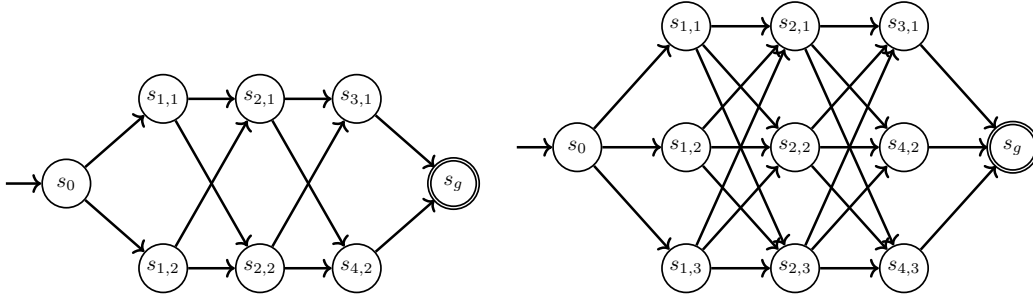
We present one final example, which shows that within a single problem the plan and cycle representation may be more compact in the best case, and exponentially less compact in the worst case. We introduce

Figure 5.4: Probability-branching SSPs $\mathbb{PB}(b, d)$ with $b = 2, d = 3$ (left) and $b = 3, d = 3$ (right).

	optimal policy / worst case	
	variables	constraints
primal/dual	$O(bd)$	$O(bd)$
plan and cycle	$O(b^d)$	$O(bd)$

Table 5.2: Representation sizes for $\mathbb{PB}(b, d)$.

the action-branching SSPs $\mathbb{AB}(b, d)$, where $\mathbb{AB}(2, 3)$ and $\mathbb{AB}(3, 3)$ are shown in fig. 5.5, and in general $\mathbb{AB}(b, d)$ is the all-outcomes determinisation of $\mathbb{PB}(b, d)$.

Figure 5.5: Action-branching SSPs $\mathbb{AB}(b, d)$ with $b = 2, d = 3$ (left) and $b = 3, d = 3$ (right).

$\mathbb{AB}(b, d)$ has $bd + 2$ states and $b^2d + 2b$ actions, and its optimal policy is any plan with length $d + 1$. Tab. 5.3 shows how many variables and constraints each representation requires. The plan and cycle representation is significantly more compact than the primal and dual representations for the optimal policy, but its search space is exponentially larger. This foreshadows the problem that, even if CoGNe can find an optimal policy much faster than existing algorithms, it may be exponentially slower to prove that its solution is optimal, as we discuss further in sec. 5.4.2.

	optimal policy		worst case	
	variables	constraints	variables	constraints
primal	$O(d)$	$O(d)$	$O(bd)$	$O(b^2d)$
dual	$O(d)$	$O(d)$	$O(b^2d)$	$O(bd)$
plan and cycle	1	0	$O(b^d)$	0

Table 5.3: Representation sizes for $\mathbb{AB}(b, d)$.

This makes it clear there are some types of SSPs that are handled more efficiently with the plan and cycle representation, and others where it is more efficient to use the primal or dual representations. In particular, our examples motivate that problems with low probabilistic branching have policies that are efficiently encoded with plans and cycles, whereas problems with high probabilistic branching do not.

5.4.2 Empirical Evaluation

In these experiments, we compare different versions of CoGNe with state-of-the-art optimal SSP algorithms. We consider two versions of CoGNe:

- CoGNe: the basic version of CoGNe with the modified version of Bellman-Ford,
- CoGNe-LNS: CoGNe augmented with LNS (see sec. 5.3.3 for details).

We compare these to iLAO* (Hansen and Zilberstein, 2001), LRTDP (Bonet and Geffner, 2003b), and CG-iLAO* (Schmalz and Trevizan, 2024b) with LM-Cut (Helmert and Domshlak, 2009) and ROC (Trevizan, Thiébaux, and Haslum, 2017) as their heuristics. On problems with dead ends, we use ROC with h-max as a dead-end detector, following Trevizan, Thiébaux, and Haslum (2017). We consider the domains from sec. 4.9.2, but leave out problems with unavoidable dead ends, i.e., problems that violate the reachability assumption (assumption 1), e.g., ExBW. This restriction is made for technical reasons, but CoGNe can be adapted to solve such problems with the fixed-cost penalty (defn. 9) or MCMP (Trevizan, Teichteil-Königsbuch, and Thiébaux, 2017). Each run was given a deadline of 30 minutes of CPU time and 8GB memory. These experiments were performed on AMD Ryzen 9 7950X. We only considered 5 instances per problem, because this gives sufficiently small variance for our discussions.

The results are given in tab. 5.4. We omit problems where CoGNe has no coverage. In the problems that remain, the state-of-the-art algorithms dominate CoGNe in terms of proving an optimal solution, and they are between one and three orders of magnitude faster.

We also report when CoGNe first finds an optimal policy. Recall that CoGNe finds an optimal policy as soon as all the necessary plans and cycles have been added to its RMP, but it can only prove that its solution is optimal once no negative reduced cost plans or cycles exist in the pricing problem. Thus, finding an optimal policy can be significantly faster than proving its optimality because CoGNe may be required to add a lot of plans and cycles before exhausting pricing solutions, and then it has to use a complete algorithm (we use the modified version of Bellman-Ford) to prove the non-existence of negative reduced cost plans and cycles, which is expensive. As an example of this, observe the large gap between CoGNe finding an optimal solution and proving optimality in the results for BW. We emphasise that CoGNe on its own has no way to determine that its candidate policy is optimal until it proves so; for the experiments, we precomputed each problem’s optimal policy and passed its expected cost to CoGNe, so that it could report when its candidate policy had attained the optimal expected cost. If we consider the times for finding an optimal policy, then CoGNe-LNS is slightly able to redeem itself: it has similar runtime on BW to the state-of-the-art algorithms equipped with LMCut, and even manages to find some solutions where LRTDP (LMCut) fails. This hints at the possibility that CoGNe may be an effective anytime algorithm. However, on the other domains, even CoGNe-LNS’s time to find a solution is dominated by the state-of-the-art methods. Overall, it is clear that CoGNe is not competitive over our benchmarks. We now discuss why CoGNe fails.

CoGNe’s poor performance can be attributed to two simple reasons: it is considering too many columns, and finding columns takes too long. In more detail:

Too many columns. Consider for CoGNe’s proved solution entries $[C/D]$, the number of plans/cycles in CoGNe’s final RMP; and compare this to the state-of-the-art’s number of states required to find the optimal solution $\langle E \rangle$. CoGNe’s entries are obtained by extracting the variables from its final RMP, and the state-of-the-art’s entries are computed by the number of calls to the heuristic. In the best case, CoGNe has fewer than 20% of the state-of-the-art’s terms; however it is more common that the number of terms is comparable, and when equipped with ROC the state-of-the-art often considers fewer terms than CoGNe. This large number of columns makes it impossible for CoGNe to compete, because finding a negative reduced cost plan or cycle is necessarily more expensive than finding a single state to expand, as we discuss further in the next paragraph. *Why does CoGNe require so many columns?* The first part of this, is that there are problems where the plan and cycle representation is inefficient (see sec. 5.4.1). Consider the π^* columns in tab. 5.4, which indicate the best-case encoding sizes. On BW, the sum of plans and cycles is approximately equal to the number of states required, and on the other problems the sum of plans and cycles is around 50% of the number of states required. This suggests that, indeed, the plan and cycle representation, and therefore CoGNe, is not well-suited to these domains. This is an inherent feature of CoGNe, and does not indicate an algorithmic issue, but simply that the problems are not amenable. The second reason CoGNe requires so many columns is that it is effectively doing blind search over the space of plans and cycles, and it has no guidance for the quality of plans and cycles. Recall that a solution to CoGNe’s pricing problem is some plan or cycle with negative reduced cost, and we have no mechanism to guide which plans and cycles will bring more “value” to the RMP. CoGNe does not take into consideration what other columns will be required for a particular plan or cycle. This increases the number of plans and cycles CoGNe has to consider before finding the optimal solution, and

	CoGNe			$ \mathcal{P}^* $	CoGNe-LNS		CG-iLAO*		iLAO*		LRTDP		$ S^* $
	found solution	proved solution	found solution		found solution	proved solution	LMCut	ROC	LMCut	ROC	LMCut	ROC	
6-15934	$\frac{5}{(44.8 \pm 11.6)}$ [72 $\pm$ 8/138 $\pm$ 45]	$\frac{5}{(62.3 \pm 10.0)}$ [133 $\pm$ 5/145 $\pm$ 43]	$\frac{5}{(1.6 \pm 0.8)}$ [38 $\pm$ 14/35 $\pm$ 11]	[2/8]	$\frac{5}{(21.3 \pm 2.2)}$ [123 $\pm$ 12/47 $\pm$ 3]		$\frac{5}{(1.3 \pm 0.1)}$ [1705 $\pm$ 21]	$\frac{5}{(0.1 \pm 0.0)}$ (86 $\pm$ 8)	$\frac{5}{(1.5 \pm 0.0)}$ (1640 $\pm$ 19)	$\frac{5}{(0.1 \pm 0.0)}$ (93 $\pm$ 9)	$\frac{5}{(6.9 \pm 0.3)}$ (8138 $\pm$ 173)	$\frac{5}{(0.1 \pm 0.0)}$ (495 $\pm$ 114)	(13)
6-20752	$\frac{5}{(408.9 \pm 126.5)}$ [507 $\pm$ 140/583 $\pm$ 197]	$\frac{5}{(482.3 \pm 134.5)}$ [691 $\pm$ 157/593 $\pm$ 187]	$\frac{5}{(2.0 \pm 1.0)}$ [65 $\pm$ 27/49 $\pm$ 14]	[2/8]	$\frac{5}{(86.9 \pm 5.5)}$ [412 $\pm$ 42/94 $\pm$ 9]		$\frac{5}{(2.2 \pm 0.1)}$ [2769 $\pm$ 55]	$\frac{5}{(0.1 \pm 0.0)}$ (482 $\pm$ 45)	$\frac{5}{(2.3 \pm 0.0)}$ (2523 $\pm$ 54)	$\frac{5}{(0.1 \pm 0.0)}$ (447 $\pm$ 22)	$\frac{5}{(7.4 \pm 0.2)}$ (8683 $\pm$ 43)	$\frac{5}{(0.2 \pm 0.0)}$ (1538 $\pm$ 265)	(16)
6-4006	$\frac{5}{(39.6 \pm 21.1)}$ [125 $\pm$ 47/123 $\pm$ 60]	$\frac{5}{(70.0 \pm 19.2)}$ [240 $\pm$ 34/130 $\pm$ 60]	$\frac{5}{(0.9 \pm 0.6)}$ [47 $\pm$ 15/35 $\pm$ 7]	[2/6]	$\frac{5}{(33.5 \pm 2.7)}$ [210 $\pm$ 23/48 $\pm$ 8]		$\frac{5}{(0.7 \pm 0.0)}$ [964 $\pm$ 17]	$\frac{5}{(0.1 \pm 0.0)}$ (66 $\pm$ 2)	$\frac{5}{(0.8 \pm 0.0)}$ (926 $\pm$ 18)	$\frac{5}{(0.1 \pm 0.0)}$ (67 $\pm$ 5)	$\frac{5}{(4.0 \pm 0.2)}$ (5712 $\pm$ 183)	$\frac{5}{(0.1 \pm 0.0)}$ (138 $\pm$ 58)	(12)
8-23171			1 (1771.9) [136/121]				$\frac{5}{(349.0 \pm 41.5)}$ (116560 $\pm$ 10527)	$\frac{5}{(0.5 \pm 0.1)}$ (1854 $\pm$ 305)	$\frac{5}{(339.2 \pm 29.8)}$ (105285 $\pm$ 8905)	$\frac{5}{(0.6 \pm 0.1)}$ (1832 $\pm$ 287)		$\frac{5}{(4.8 \pm 0.3)}$ (22587 $\pm$ 1354)	(22)
8-24967			$\frac{5}{(193.5 \pm 70.9)}$ [348 $\pm$ 96/181 $\pm$ 38]				$\frac{5}{(92.1 \pm 21.7)}$ (38658 $\pm$ 11129)	$\frac{5}{(0.4 \pm 0.0)}$ (1415 $\pm$ 17)	$\frac{5}{(91.9 \pm 24.7)}$ (34800 $\pm$ 10341)	$\frac{5}{(0.4 \pm 0.0)}$ (1247 $\pm$ 21)		$\frac{5}{(4.0 \pm 0.3)}$ (18518 $\pm$ 1489)	(19)
8-25241			1 (92.3) [13/20]				$\frac{5}{(354.0 \pm 59.4)}$ (114252 $\pm$ 23781)	$\frac{5}{(0.3 \pm 0.1)}$ (947 $\pm$ 502)	$\frac{5}{(355.5 \pm 65.0)}$ (105768 $\pm$ 21583)	$\frac{5}{(0.3 \pm 0.1)}$ (745 $\pm$ 479)		$\frac{5}{(4.5 \pm 2.4)}$ (20335 $\pm$ 10977)	(19)
Sys	p01	$\frac{5}{(6.1 \pm 0.8)}$ [41 $\pm$ 2/353 $\pm$ 48]	$\frac{5}{(6.2 \pm 0.7)}$ [42 $\pm$ 2/357 $\pm$ 46]		$\frac{5}{(6.5 \pm 0.8)}$ [42 $\pm$ 8/322 $\pm$ 54]	$\frac{5}{(6.6 \pm 0.8)}$ [42 $\pm$ 8/324 $\pm$ 54]	$\frac{5}{(0.0 \pm 0.0)}$ (442 $\pm$ 0)	$\frac{5}{(0.1 \pm 0.0)}$ (442 $\pm$ 0)	$\frac{5}{(0.0 \pm 0.0)}$ (443 $\pm$ 0)	$\frac{5}{(0.1 \pm 0.0)}$ (443 $\pm$ 0)	$\frac{5}{(0.1 \pm 0.0)}$ (444 $\pm$ 0)	$\frac{5}{(0.1 \pm 0.0)}$ (444 $\pm$ 0)	(173)
	p02	$\frac{5}{(229.2 \pm 71.9)}$ [81 $\pm$ 27/3013 $\pm$ 734]	$\frac{5}{(232.6 \pm 70.5)}$ [81 $\pm$ 27/3044 $\pm$ 722]		$\frac{5}{(206.5 \pm 65.7)}$ [79 $\pm$ 18/2635 $\pm$ 686]	$\frac{5}{(209.4 \pm 67.9)}$ [79 $\pm$ 18/2659 $\pm$ 703]	$\frac{5}{(0.1 \pm 0.0)}$ (1601 $\pm$ 0)	$\frac{5}{(0.2 \pm 0.0)}$ (1564 $\pm$ 0)	$\frac{5}{(0.3 \pm 0.1)}$ (1603 $\pm$ 0)	$\frac{5}{(0.4 \pm 0.0)}$ (1565 $\pm$ 0)	$\frac{5}{(0.5 \pm 0.0)}$ (1603 $\pm$ 3)	$\frac{5}{(0.6 \pm 0.0)}$ (1568 $\pm$ 2)	
ΔTW	p01	$\frac{5}{(0.0 \pm 0.0)}$ [19 $\pm$ 1/0 $\pm$ 0]	$\frac{5}{(0.0 \pm 0.0)}$ [19 $\pm$ 1/0 $\pm$ 0]		$\frac{5}{(0.1 \pm 0.0)}$ [26 $\pm$ 2/0 $\pm$ 0]	$\frac{5}{(0.1 \pm 0.0)}$ [26 $\pm$ 2/0 $\pm$ 0]	$\frac{5}{(0.0 \pm 0.0)}$ (34 $\pm$ 1)	$\frac{5}{(0.0 \pm 0.0)}$ (33 $\pm$ 0)	$\frac{5}{(0.0 \pm 0.0)}$ (44 $\pm$ 1)	$\frac{5}{(0.0 \pm 0.0)}$ (44 $\pm$ 1)	$\frac{5}{(0.0 \pm 0.0)}$ (44 $\pm$ 2)	$\frac{5}{(0.0 \pm 0.0)}$ (44 $\pm$ 1)	(21)
	p02	$\frac{5}{(21.1 \pm 3.3)}$ [1200 $\pm$ 159/0 $\pm$ 0]	$\frac{5}{(21.1 \pm 3.3)}$ [1200 $\pm$ 159/0 $\pm$ 0]		$\frac{5}{(18.7 \pm 6.1)}$ [1056 $\pm$ 253/0 $\pm$ 0]	$\frac{5}{(18.8 \pm 6.1)}$ [1057 $\pm$ 253/0 $\pm$ 0]	$\frac{5}{(0.0 \pm 0.0)}$ (405 $\pm$ 14)	$\frac{5}{(0.0 \pm 0.0)}$ (320 $\pm$ 2)	$\frac{5}{(0.0 \pm 0.0)}$ (441 $\pm$ 9)	$\frac{5}{(0.0 \pm 0.0)}$ (380 $\pm$ 9)	$\frac{5}{(0.0 \pm 0.0)}$ (993 $\pm$ 82)	$\frac{5}{(0.1 \pm 0.0)}$ (625 $\pm$ 44)	

Table 5.4: Coverage, average runtime (secs), and average solution sizes. For each average we present the mean and its 95% C.I. over the converged instances. For CoGNe we give $A(B)[C/D]$ at the moment the optimal solution is found, and then proved; for state-of-the-art algorithms, we give $A(B)\langle E \rangle$ when the solution is proved. A is the coverage (out of 5), B is the average time (secs), C/D are the average number of plans and cycles used to encode the solution, and E is the number of states considered. We also give $|\mathcal{P}^*|$ and $|\mathcal{C}^*|$ as $[C/D]$, denoting the minimum number of plans (C) and cycles (D) required to represent π^* ; and $|S^{\pi^*}|$ as $\langle E \rangle$, denoting the number of states required to encode π^* (i.e., the number of states in π^* 's policy envelope).

moreover, significantly increases the number of plans and cycles required to prove the optimal solution. Recall that in heuristic search algorithms, states with large heuristic values can be ignored and their successors do not need to be considered, because these can not improve the current solution. It would benefit CoGNe if it had a similar heuristic function for plans and cycles, such that w is only added to the RMP if $rc(w) + H(w) < 0$. Currently, our $H(w) = 0$, i.e., CoGNe is using the analogue of the zero heuristic. It may be possible to construct domain-independent heuristics over plans and cycles to address this issue, as we discuss further in the future work section (sec. 9.3.6).

Finding columns takes too long. CoGNe in its current form considers between 0.5 and 6 columns for each state considered by CG-iLAO* with ROC. CG-iLAO* takes a mean of 10^{-4} seconds per state, whereas both versions of CoGNe take 10^{-1} seconds per column.³ Thus, if we do not speed up CoGNe’s pricing problems, CoGNe would need to consider three orders of magnitude fewer columns than CG-iLAO* considers states, which we have seen is impossible for many domains. Consequently, the pricing problem needs to become faster in order for CoGNe to be competitive. Recall that our algorithms for solving pricing problems do not exploit all its simplifying properties (sec. 5.3.2); we believe that an algorithm that exploits all these properties is possible, and that should speed up the pricing problem solver. We discuss this further in the future work section (sec. 9.3.7).

5.5 Related Work: Solving SSPs via Determinisation

In this section, we describe some algorithms that construct policies for SSPs via plans on determinisations and predate CoGNe. We explain that CoGNe is novel because it can guarantee optimality and it constructs policies out of plans and cycles explicitly.

The most straight-forward determinisation-based planner is FF-Replan (Yoon, Fern, and Givan, 2007). Given an SSP, it runs FF (Hoffmann and Nebel, 2001) from s_I to G on the SSP’s determinisation to obtain the plan $\langle s^0, a_{s^0 \rightarrow s^1}, \dots, s_n \rangle$, which FF-Replan uses to define its policy π with $\pi(s^i) \leftarrow a^i$ for each $i \in \{0, \dots, n-1\}$. This policy may not be closed, so whenever the agent encounters a state where $\pi(s)$ is undefined, FF-Replan is called in an online way to “replan” and update its policy. FF-Replan updates its policy at state s in the same way that it constructed its initial policy: it runs FF on the SSP’s determinisation from the state s to G in order to obtain a plan that it uses to update π . This algorithm is effective and won the International Probabilistic Planning Competition (IPPC) in 2004 (Younes et al., 2005). Robust-FF (Teichteil-Königsbuch, Kuter, and Infantes, 2010) extends FF-Replan: it runs simulations of its policy from s_I to find states where $\pi(s)$ is undefined to avoid online computations. As an additional performance improvement, Robust-FF closes a state s by running FF from s to a disjunction of states s where $\pi(s)$ is defined or s is a goal. This allows FF to produce significantly shorter plans in some cases, thus speeding up the overall search. Robust-FF won IPPC 2008 (Bryce and Buffet, 2008).

These approaches were effective in the IPPC, but it is important to recognise that they have no guarantees of optimality. They have no mechanism to minimise expected cost, and moreover, they may not produce a proper policy even if proper policies exist. To demonstrate this, Little and Thiebaux (2007) introduced the Triangle Tireworld domain (ΔTW , see sec. 4.9.2) where determinisation-based algorithms produce improper policies. To be precise, the shortest plan in a ΔTW ’s all-outcomes determinisation produces a policy with the smallest probability of success, and only the all-outcome determinisation’s longest plan can induce a proper policy, with the consequence that determinisation-based algorithms produce policies that have a low probability of success.

More work has been done on determinisation-based planning, e.g., FF-Hindsight which constructs and solves multiple determinisations to approximate the SSP’s value function (Yoon et al., 2008), and the ReTrASE algorithm which combines determinisation-based search with other techniques to mitigate the drawbacks (Kolobov, Mausam, and Weld, 2009). Neither of these can guarantee that they produce optimal policies, and as far we are aware, no determinisation-based planner before CoGNe can guarantee optimal policies.

CoGNe separates itself from these approaches because it does have optimality guarantees. Moreover, it is worth noting that CoGNe is the only algorithm that explicitly encodes its policies as combinations

³CG-iLAO*’s metric of seconds per state is problematic because the complexity of an SSP does not grow linearly with the number of states, it depends on the structure of the problem. Nevertheless, this flawed comparison gives us an idea of how fast the pricing problems need to be solved in order for CoGNe to compete.

of plans and cycles; the other approaches use plans to update their policy and do not consider cycles explicitly — cycles are formed implicitly in their policies.

It is also worth mentioning that there has been recent work in constructing admissible heuristics for SSPs from its determinisation’s plans (Peterson et al., 2022). This technique works by constructing a set of determinisations whose costs-to-go (obtained from computing optimal plans) can be combined into an admissible heuristic for the SSP. The key idea is that determinisations can be constructed in a coordinated way to account for different probabilistic outcomes. For example, given an SSP $\mathbb{S}$ with a probabilistic action a that has $P(s_1|\cdot, a) = 0.7$ and $P(s_2|\cdot, a) = 0.3$ we can construct a pair of determinisations:

- Determinisation 1: $\mathbb{S}$ ’s all-outcomes determinisation with the additional constraint that the first application of a results in s_1 ,
- Determinisation 2: $\mathbb{S}$ ’s all-outcomes determinisation but this one requires that the first application of a results in s_2 .

Thus, the two determinisations account for all possible outcomes a ’s first application. Consequently, if V_1 and V_2 are the costs-to-go for the respective determinisations, then $0.7V_1(s) + 0.3V_2(s)$ produces a heuristic for the SSP at state s that is admissible and more informative than the cost-to-go from the all-outcomes determinisation on its own. Intuitively, this method approximates an SSP’s optimal value function with a finite set of trajectories. For acyclic SSPs it is possible to enumerate all trajectories as plans and produce the optimal value function, but in the presence of cycles, we would require infinitely-many trajectories to get an exact solution. In contrast, CoGNe finds an SSP’s optimal policy exactly using a finite representation of policies with plans and cycles.

5.6 Summary

In this chapter, we showed that an SSP’s policies can be decomposed into plans and cycles over the SSP’s all-outcomes determinisation. Consequently, optimal policies for an SSP can be constructed from the SSP’s all-outcomes determinisation’s plans and cycles. Using this idea, we defined CoGNe, which applies variable generation to find an SSP’s optimal policy via plans and cycles. Importantly, CoGNe’s pricing problem is a deterministic search, which means we can optimally solve SSPs by solving a sequence of deterministic subproblems. Although earlier algorithms exist for constructing policies from the SSP’s all-outcomes determinisation, such as FF-Replan (Yoon, Fern, and Givan, 2007) and Robust-FF (Teichteil-Königsbuch, Kuter, and Infantes, 2010), CoGNe is the first to do so with guarantees of optimality.

Part II

Constrained Stochastic Shortest Path Problems

Background

6.1 The Problem and its Solutions

Constrained Stochastic Shortest Path problems (CSSPs) generalise SSPs by introducing secondary costs for actions and imposing constraints over these secondary costs. Thus, CSSPs represent problems where an agent must move between states using actions with probabilistic effects that are known a priori while ensuring that the expected secondary costs that it incurs are below a specified threshold. CSSPs have been used to model real-world problems, e.g., routing an aeroplane's flight path, subject to unpredictable weather and timing constraints (Geißer et al., 2020). They have also been used for hospital admission scheduling, infrastructure maintenance, and other real-world problems (Altman, 1995).

Formally, CSSPs are defined as follows:

Definition 33 (Constrained Stochastic Shortest Path Problem (CSSP) (Altman, 1995; Trevizan et al., 2016)). *A CSSP is defined by the tuple $\mathbb{C} = \langle S, s_0, G, A, P, \mathbf{C}, \mathbf{u} \rangle$ with $n \in \mathbb{N}_{>0}$ secondary constraints, where*

- $S, s_I, G, A,$ and P are defined identically to SSPs (see defn. 1);
- $\mathbf{C}$ is a cost vector with the primary cost $C_0 : A \rightarrow \mathbb{R}_{>0}$ and secondary costs $C_i : A \rightarrow \mathbb{R}_{\geq 0}$ for $i \in \{1, \dots, n\}$ (we will explain later why only the primary cost needs to be strictly positive);
- $\mathbf{u} \in \mathbb{R}_{\geq 0}^n$ is a vector of upper bounds, which may not be exceeded by the corresponding secondary costs over expectation.

CSSPs, similarly to SSPs, are solved by policies. Again, we consider deterministic policies $\pi : S \rightarrow A$ that map states to actions deterministically (defn. 2), and stochastic actions $\pi : S \rightarrow \text{distr}(A)$ that map actions to a probability distribution over actions, from which the agent must pick randomly (defn. 3). These policies are identical to the SSP part, so we reuse the following definitions from chap. 3

- successors $\text{succ}(s, a) = \{s' \in S : P(s'|s, a) > 0\}$ (defn. 4),
- policy envelopes $S^{\pi, s}$ (defn. 5),
- closed and open policies (defn. 6),
- proper and improper policies (defn. 7).

The first difference is that policy costs must now be defined over the cost vector

Definition 34 (Policy Cost). *Given a closed policy π for the CSSP $\mathbb{C}$ with $\mathbf{C} : A \rightarrow \mathbb{R}_{\geq 0}^{n+1}$, the policy cost $C_i(\pi)$ for $i \in \{0, \dots, n\}$ denotes the expected cost incurred over the cost function C_i , by the agent following π from s_I . $C_i(\pi)$ can be computed by $C_i(\pi) = \sum_{s \in S, a \in A(s)} x_{s,a} \cdot C_i(a)$ where $\mathbf{x}$ is an occupation measure that describes π with $\pi_{\mathbf{x}} = \pi$.*

We make two assumptions for CSSPs, which extend the assumptions from SSPs (assumption 1 and assumption 2).

Assumption 4 (CSSP Reachability). *There exists at least one policy π that is proper w.r.t. s_I and feasible.*

Assumption 5. *All improper policies have an infinite primary expected cost C_0 .*

Note that we have defined CSSPs with strictly positive primary costs to accommodate assumption 5, with the same reasoning that we make SSP costs strictly positive to accommodate assumption 2 (see sec. 3.1). It is sufficient that the primary cost is strictly positive, and the secondary costs do not require the same restrictions. In fact, it is quite common that actions have zero as their secondary costs, e.g., consider a domain where the agent must get to work with a taxi or by walking, while subject to constraints over money spent (this will be a running example later, see ex. 1); the walking action does not incur any secondary costs over money spent.

To handle CSSP that violate reachability (assumption 1), we extend the fixed-penalty transformation (defn. 9):

Definition 35 (Fixed-penalty Transformation (Kolobov, Mausam, and Weld, 2012; Trevizan et al., 2016)). *Given a CSSP $\mathbb{C} = \langle S, s_0, G, A, P, C, \mathbf{u} \rangle$ and a penalty term $\mathbf{d} \in \mathbb{R}_{\geq 0}^{n+1}$, its fixed-penalty transformation is the new CSSP $\mathbb{C}' = \langle S, s_I, G, A', P', C', \mathbf{u} \rangle$ with:*

- $S, s_I, G, \mathbf{u}$ are identical;
- $A'(s) = A(s) \cup \{\text{give-up}\}$;
- $C'(a) = \begin{cases} C(a) & \text{if } a \in A(s) \\ \mathbf{d} & \text{if } a = \text{give-up}; \end{cases}$
- $P'(s'|s, a) = \begin{cases} 1 & \text{if } s' = g \text{ and } a = \text{give-up} \\ 0 & \text{if } s' \neq g \text{ and } a = \text{give-up} \\ P(s'|s, a) & \text{else} \end{cases}$

where $g \in G$ is some representative goal state.

Such transformed CSSPs $\mathbb{C}'$ always satisfy reachability (assumption 1), because the agent can always reach the goal with probability 1 by applying the give-up action. For sufficiently large $\mathbf{d}$, the optimal policy of $\mathbb{C}'$ is what we intuitively want: it maximises the probability of reaching goals while satisfying secondary-cost constraints, and then minimises primary cost.

Note that $\mathbf{d}$ may require different penalty terms for each cost function. Each cost function may be in different units (dollars, litres of fuel, hours of battery left, etc.), with different consequences for violating the constraint, so a single value is often inappropriate.

Finally, we formally state what feasible and optimal policies for a CSSP are:

Definition 36 (Feasible Policy). *A policy π is feasible for the CSSP $\mathbb{C} = \langle S, s_0, G, A, P, C, \mathbf{u} \rangle$ if all secondary-cost constraints are satisfied by π :*

$$C_i(\pi) \leq u_i \quad \forall i \in \{1, \dots, n\}.$$

Definition 37 (Optimal Policy). *A policy π is optimal for CSSP $\mathbb{C}$ if it is proper, feasible, and a minimiser of C_0 over all proper, feasible policies.*

An important difference between SSPs and CSSPs, is that CSSPs may not have an optimal deterministic policy (Altman, 1995; Trevizan et al., 2016), which we discuss further in the next section.

6.2 Deterministic and Stochastic Policies

In this section, we go into detail about the properties of deterministic and stochastic policies in the context of CSSPs.

First, we show how to relate the two types of policies. Policies, both deterministic and stochastic, can be combined into a new stochastic policy in the following way:

Definition 38 (Combination of Policies). *Consider policies $\pi_0, \dots, \pi_k$ and a probability distribution $\mu_0, \dots, \mu_k$ over the policies, i.e., each π_i has a corresponding $\mu_i \in [0, 1]$ and $\sum_{i=0}^k \mu_i = 1$. This induces the new stochastic policy π defined by*

$$\pi(s) = \sum_{i=0}^k \mu_i \pi_i(s, a) \quad \forall s \in \bigcup_{i=0}^k S^{\pi_i}.$$

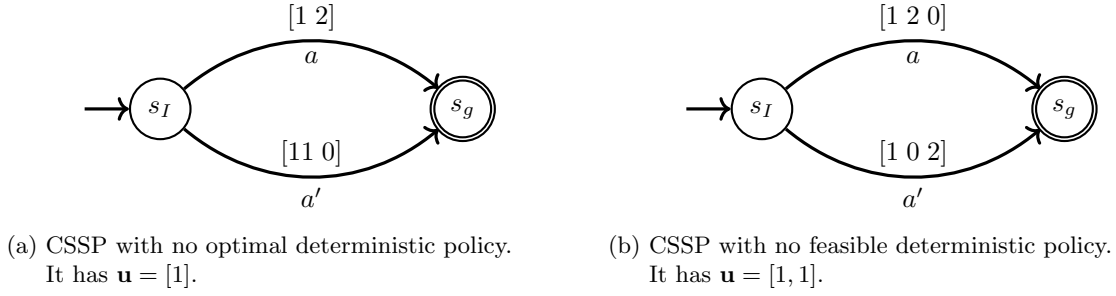


Figure 6.1: Two CSSPs with no optimal deterministic policies.

To express π in terms of its constituent policies, we write $\pi = \mu_0\pi_0 + \dots + \mu_k\pi_k$. This definition assumes that each π_i is defined for all $s \in \bigcup_{i=0}^k S^{\pi_i}$.

The probability distribution over constituent policies can also be interpreted as a convex combination over constituent policies. It turns out that any stochastic policy can be decomposed into constituent deterministic policies:

Theorem 12 (Stochastic Policies are Convex Combinations of Deterministic Policies (Geißer et al., 2020)). *For stochastic policy π , there exists a set of deterministic policies $\pi_0, \dots, \pi_k$, called π 's constituent deterministic policies and some positive numbers $\mu_0, \dots, \mu_k \in \mathbb{R}_{>0}$ that satisfy $\mu_0 + \dots + \mu_k = 1$, which decompose π into $\pi = \mu_0\pi_0 + \dots + \mu_k\pi_k$.*

Thus, stochastic policies can be interpreted as convex combinations or probabilistic mixtures of deterministic policies, where the agent randomly selects which constituent deterministic policy to use.

As we mentioned before, CSSPs may not have an optimal deterministic policy (Altman, 1995; Trevizan et al., 2016). So, if a CSSP is solvable and has a set of optimal solutions, then they may all be strictly stochastic, with no deterministic ones among them. To motivate this phenomenon, consider the CSSP in fig. 6.1a with $\mathbf{u} = [1]$. Its optimal policy is $\pi^* = \{a \mapsto 0.5, a' \mapsto 0.5\}$ with $C_0(\pi^*) = 6$ and $C_1(\pi^*) = 1 \leq u_1$. Clearly, using a with any higher probability would make the policy infeasible, and using a' with higher probability would increase the primary cost. Moreover, a CSSP may not even have any feasible deterministic solutions, as we see in the CSSP in fig. 6.1b with $\mathbf{u} = [1 \ 1]$. In this example, the only feasible (and therefore optimal) policy is $\pi^* = \{a \mapsto 0.5, a' \mapsto 0.5\}$ with $C_1(\pi^*) = 1 \leq u_1$ and $C_2(\pi^*) = 1 \leq u_2$, and there is no way of changing its probabilities without violating one of the secondary-cost constraints. In particular, there is no feasible deterministic policy.

So, a CSSP may have no optimal deterministic policies, and may not even have any feasible deterministic policies. Nevertheless, there is often interest in finding, among the feasible deterministic policies, the minimiser of primary cost:

Definition 39 (Deterministic-optimal Policy). *A deterministic policy π is deterministic-optimal for a CSSP $\mathbb{C}$, if it is deterministic, proper, feasible, and a minimiser of C_0 over all deterministic, proper, feasible policies.*

Recall that we write π^* to denote an optimal policy; analogously we write π_{det}^* to denote a deterministic-optimal policy. We introduce this terminology to distinguish between optimal deterministic policies, which are optimal policies that happen to be deterministic; and deterministic-optimal policies that are not necessarily optimal for the CSSP, but optimal among the CSSP's deterministic policies. Relating these two concepts: an optimal deterministic policy is immediately a deterministic-optimal policy, but not vice versa. A deterministic-optimal policy may not be an optimal deterministic policy, and moreover, the existence of a deterministic-optimal policy does not imply the existence of an optimal deterministic policy.

In terms of complexity, finding an optimal policy for a CSSP can be done in polynomial time w.r.t. the CSSP size; we show in the next section that CSSPs can be solved by polynomially large LPs, which demonstrates this point. In contrast, finding a deterministic-optimal policy is NP-complete (Feinberg, 2000).

Nevertheless, even though deterministic-optimal policies have fewer guarantees and are more difficult to find, there are practical scenarios in which deterministic policies are preferred or required:

- in multi-agent systems, an agent can predict another agent’s actions if they are deterministic, and potentially significant communication overhead is required if agents use stochastic policies (Dolgov and Durfee, 2005);
- making probabilistic decisions in medical contexts can have ethical issues (Rojers et al., 2013);
- aviation regulations may not permit stochastic policies (Geißer et al., 2020);
- and more generally, stochastic policies can introduce issues with accountability and explainability (Hong and Williams, 2023).

To highlight the differences between deterministic and stochastic policies for CSSPs and their properties, we present the following example:

Example 1 (Getting to Work CSSP). *The agent needs to make its way from home to work. It can run, use a taxi, or walk to the train station where it can try to take the train. The train is cancelled with 50% probability. Each action has a cost vector $[t \ p \ e]$ in terms of time (t), price (p), and personal effort (e). The agent’s task is to get to work in minimal time s.t. price ≤ 15 and effort ≤ 10 over expectation. The CSSP is shown in fig. 6.2.*

The CSSP in ex. 1, has the optimal policy π^* with $\pi^*(s_I, \text{run}) = 0.5$ and $\pi^*(s_I, \text{taxi}) = 0.5$ where

- $C_0(\pi^*) = 1$ (time),
- $C_1(\pi^*) = 15 \leq 15$ (price),
- $C_2(\pi^*) = 10 \leq 10$ (effort).

It also has the deterministic-optimal policy $\pi_{\text{det}}^* = \{s_I \mapsto \text{walk}, s_1 \mapsto \text{train}, s_2 \mapsto \text{walk}\}$ which costs

- $C_0(\pi_{\text{det}}^*) = 3$ (time),
- $C_1(\pi_{\text{det}}^*) = 10 \leq 15$ (price),
- $C_2(\pi_{\text{det}}^*) = 4 \leq 10$ (effort).

We emphasise here that π^* satisfies the constraints *over expectation*. Both actions of π^* individually violate the secondary-cost constraints, and it is only by alternating between the two that the secondary-cost constraints become satisfied. This makes sense for our problem if the agent needs to get to work every day, so the policy is repeated, and the average price stays within the agent’s budget. It does not make sense if the agent only has 15 monetary units in its wallet, and can not exceed that budget in any single day. In certain situations, practitioners dislike that the policy’s constituent deterministic policies can violate constraints. Now, consider the deterministic-optimal policy π_{det}^* . This policy addresses the issue of stochastic policies because its only constituent deterministic policy is itself, and it is feasible. However, we point out that among π_{det}^* ’s executions, there is the trajectory $\langle s_I, \text{walk}, s_1, \text{train}, g \rangle$ which incurs a cost of $20 \geq u_1$ in terms of price when the train is not cancelled (with a 50% probability). This means that, although the constituent deterministic policies all satisfy the secondary-cost constraints, it is not the case that all executions of a policy, deterministic or stochastic, satisfy the constraints.

Thus, if the user requires (or prefers) that the secondary-cost constraints are satisfied over all executions of the policy, then CSSPs are the wrong model, and restricting to deterministic policies does not help. CSSP’s secondary-cost constraints over expectation are useful for modelling scenarios that are repeated many times and we have constraints over the long term, not for each individual run. We emphasise that this is not an issue with CSSPs, but stochastic optimisation in general; a similar behaviour occurs with SSPs: $V^*(s_I)$ is an SSP’s expected optimal cost, but any single execution for π^* may observe a cost smaller or larger than $V^*(s_I)$.

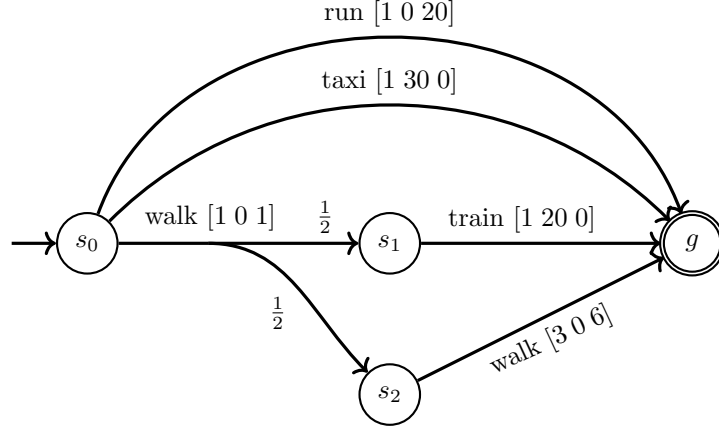


Figure 6.2: The CSSP associated with “Getting to Work” (ex. 1).

6.3 Algorithms

In this section, we present the Constrained Occupation Measure LP, which underlies all techniques for solving CSSPs that we consider. Then, we describe i²-dual and LAny, which are existing state-of-the-art algorithms for solving CSSPs optimally and deterministically, respectively. These will be important to present our contributions in later chapters.

6.3.1 Constrained Occupation Measure LP

The COMLP (Trevizan et al., 2016), presented in LP 9, extends the OMLP for SSPs (LP 2). As with the OMLP, the COMLP optimises over the occupation measures $\mathbf{x}$, where $x_{s,a}$ represents the expected number of times that the agent encounters state s and then applies action a . It can also be interpreted as a network flow problem, where a unit of flow is injected into s_I and must be routed to the goals G . The COMLP (LP 9) adapts and extends the OMLP (LP 2) into the CSSP setting by replacing the scalar cost C in the objective with C_0 , and appending the secondary-cost constraints C29.

Constrained Occupation Measure LP (COMLP)

$$\begin{aligned}
 & \min_{\mathbf{x}} \sum_{s \in S, a \in A(s)} x_{s,a} \cdot C_0(a) & (\text{LP } 9) \\
 & \text{s.t. } out(s) - in(s) = \mathbb{I}[s = s_I] & \forall s \in S \setminus G \text{ (C25)} \\
 & out(s) = \sum_{a \in A(s)} x_{s,a} & \forall s \in S \text{ (C26)} \\
 & in(s) = \sum_{s' \in S, a' \in A(s')} x_{s',a'} \cdot P(s|s', a') & \forall s \in S \text{ (C27)} \\
 & x_{s,a} \geq 0 & \forall s \in S \text{ (C28)} \\
 & \sum_{s \in S, a \in A(s)} x_{s,a} \cdot C_i(a) \leq u_i & \forall i \in \{1, \dots, n\} \text{ (C29)}
 \end{aligned}$$

A solution $\mathbf{x}$ defines the policy $\pi_{\mathbf{x}}(s) = \frac{x_{s,a}}{out(s)}$ for $s \in S \setminus G, a \in A(s)$. As before, an optimal solution $\mathbf{x}^*$ induces an optimal policy $\pi_{\mathbf{x}^*}$. Recall that the OMLP for SSPs had the convenient property that its basic feasible solutions describe deterministic policies. This is no longer the case for the COMLP, because the secondary-cost constraints C29 may now require $x_{s,a} > 0$ for multiple $a \in A(s)$. In fact, we know that this is required for the problems that have no optimal deterministic policy. Thus, $\pi_{\mathbf{x}}$ may be strictly stochastic.

6.3.2 I-dual and i²-dual

I-dual (Trevizan et al., 2016) and its successor i²-dual (Trevizan, Thiébaux, and Haslum, 2017) are state-of-the-art heuristic-search algorithms for solving CSSPs optimally (i.e., they may produce stochastic policies). We will use these algorithms as a baseline in chap. 7, and as a building block in chap. 8. We first explain i-dual, and then build on top of it to explain i²-dual.

I-dual. I-dual works by solving a sequence of partial CSSPs with the COMLP, incrementally growing them until the partial CSSP’s solution produces an optimal solution for the original CSSP. Recall from sec. 4.2 that iLAO* (Hansen and Zilberstein, 2001) works similarly, but for SSPs: it solves a sequence of partial SSPs with Bellman backups, growing them until its partial SSP’s solution gives an optimal solution for the original SSP. In that sense, i-dual generalises iLAO*: i-dual replaces iLAO*’s partial SSPs with partial CSSPs, and solves them with the COMLP in place of Bellman backups. Another way to interpret i-dual, is simply as constraint and variable generation over the COMLP. This is to be expected, given the connection between iLAO* and constraint and variable generation (sec. 4.3), but the constraint and variable generation is even more explicit because i-dual is solving each CSSP with the COMLP.

We extend the definitions of SSPs with terminal costs (defn. 24) and partial SSPs (defn. 25) from iLAO* to CSSPs:

Definition 40 (CSSP with Terminal Costs). *A CSSP with terminal costs is given by the tuple $\mathbb{C}_T = \langle \mathcal{S}, s_I, \mathcal{G}, \mathcal{A}, P, \mathbf{C}, \mathbf{u}, \mathbf{T} \rangle$. All components that appear in the definition of CSSPs are defined identically. If $\mathbb{C}_T$ has n secondary costs and $\mathbf{C} : \mathcal{S} \rightarrow \mathbb{R}_{\geq 0}^{n+1}$, then the new term $\mathbf{T} : \mathcal{G} \rightarrow \mathbb{R}_{\geq 0}^{n+1}$ is the terminal-cost function that applies the vector cost $\mathbf{T}(g)$ once, whenever the agent reaches the goal $g \in \mathcal{G}$.*

Definition 41 (Partial CSSP). *Consider a CSSP $\mathbb{C} = \langle \mathcal{S}, s_0, \mathcal{G}, \mathcal{A}, P, \mathbf{C}, \mathbf{u} \rangle$ with n secondary costs, and a heuristic vector $\mathbf{H} : \mathcal{S} \rightarrow \mathbb{R}_{\geq 0}^{n+1}$. A partial CSSP for $\mathbb{C}$ with $\mathbf{H}$ is a CSSP with terminal costs $\hat{\mathbb{C}} = \langle \hat{\mathcal{S}}, s_I, \hat{\mathcal{G}}, \hat{\mathcal{A}}, P, \mathbf{C}, \mathbf{H}, \mathbf{u} \rangle$ where*

- $s_I, P, \mathbf{C}$, and $\mathbf{u}$ are identical to $\mathbb{C}$,
- $\hat{\mathcal{S}}$ and $\hat{\mathcal{A}}$ are subsets of their counterparts in $\mathbb{C}$, i.e., $\hat{\mathcal{S}} \subseteq \mathcal{S}$ and $\hat{\mathcal{A}}(s) \subseteq \mathcal{A}(s) \forall s \in \hat{\mathcal{S}}$,
- $\hat{\mathcal{G}}$ is a set of artificial goals that satisfies $\hat{\mathcal{G}} \subseteq \hat{\mathcal{S}}, \mathcal{G} \cap \hat{\mathcal{S}} \subseteq \hat{\mathcal{G}}$, and
- $\mathbf{H}$ is the terminal cost.

A partial CSSP with $\mathbf{H}$ is solved by LP 10, which leaves the flow constraints from LP 9 untouched, but adds the heuristic values to the objective and secondary-cost constraints C34.

COMLP for Partial CSSP $\hat{\mathbb{C}}$ with $\mathbf{H}$

$$\begin{aligned}
 & \min_{\mathbf{x}} \quad \sum_{s \in \hat{\mathcal{S}}, a \in \hat{\mathcal{A}}(s)} x_{s,a} \cdot C_0(a) + \sum_{g \in \hat{\mathcal{G}}} in(g) \cdot H_0(g) & (\text{LP 10}) \\
 & \text{s.t.} \quad out(s) - in(s) = \llbracket s = s_I \rrbracket & \forall s \in \hat{\mathcal{S}} \setminus \hat{\mathcal{G}} \quad (\text{C30}) \\
 & \quad \quad out(s) = \sum_{a \in \hat{\mathcal{A}}(s)} x_{s,a} & \forall s \in \hat{\mathcal{S}} \quad (\text{C31}) \\
 & \quad \quad in(s) = \sum_{s' \in \hat{\mathcal{S}}, a' \in \hat{\mathcal{A}}(s')} x_{s',a'} \cdot P(s|s', a') & \forall s \in \hat{\mathcal{S}} \quad (\text{C32}) \\
 & \quad \quad x_{s,a} \geq 0 & \forall s \in \hat{\mathcal{S}} \quad (\text{C33}) \\
 & \quad \quad \sum_{s \in \hat{\mathcal{S}}, a \in \hat{\mathcal{A}}(s)} x_{s,a} \cdot C_i(a) + \sum_{g \in \hat{\mathcal{G}}} in(g) \cdot H_i(g) \leq u_i & \forall i \in \{1, \dots, n\} \quad (\text{C34})
 \end{aligned}$$

Now, we can describe i-dual with the high-level pseudocode in alg. 4. For a candidate policy π , its *fringe states* are states in its envelope $s \in \mathcal{S}^\pi$ that are artificial goals in the partial CSSP, i.e., $s \in \hat{\mathcal{G}} \setminus \mathcal{G}$. Thus, i-dual computes an optimal (potentially stochastic) policy by iterating the two key steps (analogously to iLAO* in sec. 4.2):

1. **Optimise:** solve the partial CSSP to get a candidate policy π .

2. **Close:** expand π 's fringe states.

The optimisation step makes sure that the candidate policy is optimal for the partial CSSP, and the close step makes sure that π is eventually closed for the original CSSP $\mathbb{C}$. The two steps together ensure that i-dual finds an optimal policy for $\mathbb{C}$.

Algorithm 4: i-dual

```

1 Function find-optimal-policy(CSSP  $\mathbb{C}$ , heuristic  $\mathbf{H}$ )
2   initialise partial CSSP  $\hat{\mathbb{C}}$  with fringe  $s_I$ 
3   initialise empty candidate policy  $\pi$ 
4   while policy envelope  $S^\pi$  contains fringes do
5      $\hat{\mathbb{C}} \leftarrow$  expand fringes
6      $\mathbf{x} \leftarrow$  solve COMLP for  $\hat{\mathbb{C}}$  with  $\mathbf{H}$ 
7     if infeasible then
8       return infeasible
9      $\pi \leftarrow \pi_{\mathbf{x}}$ 
10  return  $\pi$ 

```

I²-dual. This is an improvement to i-dual, which replaces the external heuristic $\mathbf{H}$ with an embedded version of *C-POM* (Trevizan, Thiébaux, and Haslum, 2017). The advantage of embedding C-POM is that the COMLP's candidate policy can provide information to the heuristic, strengthening it, while simultaneously the heuristic guides search for the candidate policy, creating a powerful flow of information between heuristic and solver.

For a CSSP $\mathbb{C}$, the C-POM heuristic constructs a set of smaller CSSPs $\mathbb{C}^v$ for various v . These smaller CSSPs are constructed with atomic projection and encoded in the probabilistic SAS⁺ formalism, which is explained in (Trevizan, Thiébaux, and Haslum, 2017); for our purposes, it is sufficient to understand:

- we have a separate $\mathbb{C}^v$ for each $v \in \mathcal{V}$ ($\mathcal{V}$ are the SAS⁺ variables),
- each $\mathbb{C}^v$ is itself a CSSP (an atomic projection for the SAS⁺ variable $v \in \mathcal{V}$),
- each $\mathbb{C}^v$ has the states D_v (D_v is variable v 's SAS⁺ domain),
- these $\mathbb{C}^v$ CSSPs are much smaller than $\mathbb{C}$ and can be solved directly with the COMLP, while retaining some structural information about the original CSSP.

In particular, a proper policy π for $\mathbb{C}$ must induce a proper policy on $\mathbb{C}^v$. Consequently, we can obtain an admissible heuristic by solving the COMLP for each $\mathbb{C}^v$, and taking the maximum over their objectives. To improve on this, Trevizan, Thiébaux, and Haslum (2017) combine all these $\mathbb{C}^v$ COMLPs into a single LP, and connect the separate $\mathbb{C}^v$ s by introducing the *tying constraints*

$$\sum_{d \in D_{v_i}} x_{d,a}^{v_i} = \sum_{d \in D_{v_j}} x_{d,a}^{v_j} \quad \forall v_i, v_j \in \mathcal{V}, \forall a \in A,$$

which force the expected usage of actions to be identical across $\mathbb{C}^v$ s. $\mathbb{C}$'s optimal policy still induces a feasible solution for the new combined LP, so the combined LP with tied atomic projections still produces an admissible heuristic. Instead of computing C-POM for a particular state $s \in S$, we can compute it for a probability distribution over states, where we specify $p_0^v(d)$ for each $v \in \mathcal{V}$ and $d \in D_v$ with the relevant probability mass. Thus, $p_0^v(d)$ denotes the probability mass that is injected into state d in the CSSP $\mathbb{C}^v$ and $\sum_{d \in D_v} p_0^v(d) = 1$ for all $v \in \mathcal{V}$. Then, for $\mathbf{p}_0$, C-POM is computed by LP 11, where $x_{d,a}^v$ denotes the occupation measure over the state d and action a for $\mathbb{C}^v$. C35–C39 are simply the constraints for solving each $\mathbb{C}^v$, and C40 are the tying constraints. Note that the term $\bar{v} \in \mathcal{V}$ that appears in the objective and secondary-cost constraints can be selected arbitrarily from $\mathcal{V}$, and the choice does not matter thanks to the tying constraints.

I²-dual's LP for solving its partial CSSPs combines the COMLP with C-POM into a single LP, routing all the flow into the partial CSSP's artificial goals through the candidate policy into C-POM's $\mathbf{p}_0$. We call this the i²-dual LP for the given partial CSSP. Importantly, this LP combines the secondary-cost

C-POM for $\mathbf{p}_0$

$$\begin{aligned}
& \min \sum_{d \in \mathcal{D}_{\overline{v}}, a \in \mathcal{A}} x_{d,a}^{\overline{v}} \cdot C_0(a) & (\text{LP } 11) \\
& \text{s.t. } out^v(d) - in^v(d) = p_0^v(d) & \forall v \in \mathcal{V}, \forall d \in \mathcal{D}_v \quad (\text{C35}) \\
& out^v(d) = \sum_{a \in \mathcal{A}} x_{d,a}^v & \forall v \in \mathcal{V}, \forall d \in \mathcal{D}_v \quad (\text{C36}) \\
& in^v(d) = \sum_{d' \in \mathcal{D}_v, a' \in \mathcal{A}} x_{d',a'}^v \cdot P(d|d', a') & \forall v \in \mathcal{V}, \forall d \in \mathcal{D}_v \quad (\text{C37}) \\
& x_{d,a}^v \geq 0 & \forall v \in \mathcal{V}, \forall d \in \mathcal{D}_v \quad (\text{C38}) \\
& \sum_{d \in \mathcal{D}_{\overline{v}}, a \in \mathcal{A}} x_{d,a}^{\overline{v}} \cdot C_i(a) \leq u_i & \forall i \in \{1, \dots, n\} \quad (\text{C39}) \\
& \sum_{d \in \mathcal{D}_{v_i}} x_{d,a}^{v_i} = \sum_{d \in \mathcal{D}_{v_j}} x_{d,a}^{v_j} & \forall v_i, v_j \in \mathcal{V}, \forall a \in \mathcal{A} \quad (\text{C40})
\end{aligned}$$

constraints C34 from the COMLP with the secondary-cost constraints C39 from C-POM into the merged constraints

$$\sum_{s \in \widehat{\mathcal{S}}, a \in \widehat{\mathcal{A}}(s)} x_{s,a} \cdot C_i(a) + \sum_{d \in \mathcal{D}_{\overline{v}}, a \in \mathcal{A}} x_{d,a}^{\overline{v}} \cdot C_i(a) \leq u_i \quad \forall i \in \{1, \dots, n\}.$$

This merging of constraints is key because it connects the two LPs, otherwise they would be effectively separate. Now, C-POM can take into account the secondary costs incurred by the candidate policy, strengthening the heuristic, while the COMLP component can use the secondary costs estimated by C-POM to prune infeasible policies that would not be recognised as infeasible without that information.

Finally, i^2 -dual is given by alg. 5. It is similar to i -dual (alg. 4), with the modification that no heuristic $\mathbf{H}$ is provided, and the LP for solving the partial CSSP with $\mathbf{H}$ in alg. 4 line 6 is replaced by solving the relevant i^2 -dual LP in alg. 5 line 6. As mentioned before, the unique strength of i^2 -dual is that it combines the search component (the COMLP) with a heuristic (C-POM), so that they interact and “help each other.”

Algorithm 5: i^2 -dual

```

1 Function find-optimal-policy(CSSP  $\mathbb{C}$ )
2   initialise partial CSSP  $\widehat{\mathcal{C}}$  with fringe  $s_I$ 
3   initialise empty candidate policy  $\pi$ 
4   while policy envelope  $\mathcal{S}^\pi$  contains fringes do
5      $\widehat{\mathcal{C}} \leftarrow$  expand fringes
6      $\mathbf{x} \leftarrow$  solve  $i^2$ -dual LP for  $\widehat{\mathcal{C}}$ 
7     if infeasible then
8       return infeasible
9      $\pi \leftarrow \pi_{\mathbf{x}}$ 
10  return  $\pi$ 

```

6.3.3 LAny

We explain the Lagrangian Anytime algorithm for CSSPs (LAny) (Hong and Williams, 2023).¹ This was the first heuristic-search algorithm for finding deterministic-optimal solutions for CSSPs. LAny is useful to explain because we will use similar techniques in chap. 7, and it will serve as an important baseline in chap. 8. The algorithm works in two stages: it first solves the CSSP’s *Lagrangian dual*, and in stage 2, if necessary, it closes the gap between the Lagrangian dual solution and a deterministic-optimal policy by enumerating deterministic policies in a methodical way until it can prove its solution is deterministic-optimal.

What is the Lagrangian dual? To explain the Lagrangian dual in the context of deterministic-optimal policies, we consider the following problem:

$$\min_{\pi \in \Pi_{\text{det}}} C_0(\pi) \text{ s.t. } g_i(\pi) \leq 0 \text{ for } i \in \{1, \dots, n\}$$

where Π_{det} is the set of deterministic policies and $g_i(\pi) = C_i(\pi) - u_i$. For a particular CSSP and a vector $\lambda \in \mathbb{R}_{\geq 0}^n$, called the Lagrange multiplier, we obtain:

- Lagrangian function: $L(\lambda, \pi) = C_0(\pi) + \lambda \cdot g(\pi)$, and
- Lagrangian relaxation: $L(\lambda) = \min_{\pi \in \Pi_{\text{det}}} L(\lambda, \pi)$.

The Lagrangian relaxation removes the CSSP’s secondary-cost constraints and puts them inside the objective function so that any violated constraints are penalised, and satisfied constraints are rewarded. That is, violated constraints have $g_i(\pi) > 0$, and satisfied constraints have $g_i(\pi) \leq 0$, making the objective contribution $\lambda_i \cdot g_i(\pi)$ either non-negative or non-positive, respectively. Since the secondary-cost constraints have been removed, a Lagrangian relaxation describes an unconstrained SSP that can be solved with any SSP algorithm, which will be exploited later. Importantly, all Lagrangian relaxations produce a lower bound on the original CSSP, i.e., $L(\lambda) \leq C_0(\pi_{\text{det}}^*)$ for all $\lambda \in \mathbb{R}_{\geq 0}^n$. The Lagrangian dual finds the largest of these lower bounds:

$$L^* = L(\lambda^*) = \max_{\lambda \in \mathbb{R}_{\geq 0}^n} L(\lambda).$$

LPs enjoy the property that L^* is equal to the original problem’s optimal objective cost z^* (strong duality). However, finding deterministic-optimal policies requires a more general technique (*Mixed Integer Programs*), which can only guarantee that $L^* \leq z^*$ (weak duality). The Lagrangian dual’s lower bound will be useful, but the important quantity that LAny requires from the Lagrangian dual is λ^* and its associated $\mathbb{S}(\lambda^*)$, because this unconstrained SSP approximates the original CSSP.

Stage 1: Solving the Lagrangian dual. The first stage of LAny finds and solves $L(\lambda^*)$ for the input CSSP $\mathbb{C}$. Finding λ^* is a piecewise linear concave optimisation problem, i.e., if we plot $L(\lambda)$ over all values of λ , we get a piecewise linear concave surface (Hong and Williams, 2023). Intuitively, this is because for a fixed policy π and varying λ the function $L(\lambda, \pi)$ describes a plane, since changing λ linearly impacts the penalties and rewards of π ’s g_i functions. If we pick the optimal policy at each λ (which must be deterministic because $L(\lambda)$ describes an unconstrained SSP), then the surface is still linear over the regions where one single policy is optimal, but overall the surface becomes concave because we select the optimal policy that minimises $L(\lambda, \pi)$ at each point. An important consequence of L being a piecewise linear concave optimisation problem is that it can be solved with standard subgradient descent methods, as long as we have access to an oracle which takes as input λ and outputs $L(\lambda)$ and the subgradient at λ .² Note that $L(\lambda)$ has “sharp kinks” in the form of edges or vertices where multiple policies coincide with $L(\lambda, \pi) = L(\lambda, \pi')$, i.e., multiple policies are tied-optimal for that λ . The function $L(\lambda)$ can not be differentiated at these points, which is why we need subgradients rather than gradients.

Oracle for L . In our context, an oracle may be given any $\lambda \in \mathbb{R}_{\geq 0}^n$ by the outer optimisation method, and the oracle must return $L(\lambda)$ and a subgradient at λ . To describe an efficient implementation, we start by formalising that each λ induces an unconstrained SSP:

¹It is called the Anytime Algorithm for CSSPs by the original authors, but we use LAny to emphasise the use of the Lagrangian.

²Technically, subgradient descent is for convex problems, and for concave problems we would have supergradients instead of subgradients, and an optimisation method would be ascending, rather than descending. However, it is standard in the literature to use the terminology from the convex case.

Definition 42 (Lagrangian SSP). Given $\mathbb{C} = \langle S, s_0, G, A, P, \mathbf{C}, \mathbf{u} \rangle$ and $\boldsymbol{\lambda} \in \mathbb{R}_{\geq 0}^n$, its Lagrangian SSP $\mathbb{S}(\boldsymbol{\lambda})$ is a terminal-cost SSP (defn. 24) defined by the tuple $\langle S, s_I, G, A, P, C_{\boldsymbol{\lambda}}, T \rangle$ where

- S, s_I, G, A, P are inherited from $\mathbb{C}$, i.e., $\mathbb{S}(\boldsymbol{\lambda})$ is an SSP relaxation of $\mathbb{C}$,
- $C_{\boldsymbol{\lambda}} : A \rightarrow \mathbb{R}_{\geq 0}$ is the scalar cost function given by $C_{\boldsymbol{\lambda}}(a) \stackrel{\text{def}}{=} [1 \ \boldsymbol{\lambda}] \cdot \mathbf{C}(a) \ \forall a \in A$,
- $T : G \rightarrow \mathbb{R}_{\geq 0}$ is the terminal cost function $T(g) = -\sum_{i=1}^n \lambda_i u_i \ \forall g \in G$. The right-hand side is a constant term independent of g , noting that $\boldsymbol{\lambda}$ and $\mathbf{u}$ are fixed for $\mathbb{S}(\boldsymbol{\lambda})$.

We write a proper policy's cost as $C_{\boldsymbol{\lambda}}(\pi) = [1 \ \boldsymbol{\lambda}] \cdot \mathbf{C}(\pi) - \sum_{i=1}^n \lambda_i u_i$.

An optimal policy $\pi_{\boldsymbol{\lambda}}^*$ for $\mathbb{S}(\boldsymbol{\lambda})$ precisely satisfies $C_{\boldsymbol{\lambda}}(\pi_{\boldsymbol{\lambda}}^*) = L(\boldsymbol{\lambda})$. Moreover, $\mathbb{S}(\boldsymbol{\lambda})$'s optimal policy $\pi_{\boldsymbol{\lambda}}^*$ also gives a subgradient of $L(\boldsymbol{\lambda})$: the vector $[g_1(\pi_{\boldsymbol{\lambda}}^*) \ \dots \ g_n(\pi_{\boldsymbol{\lambda}}^*)]$ is a subgradient at $\boldsymbol{\lambda}$. Thus, an oracle for the Lagrangian dual can be implemented by solving $\mathbb{S}(\boldsymbol{\lambda})$, and evaluating its cost functions. [Hong and Williams \(2023\)](#) implement this by solving each $\mathbb{S}(\boldsymbol{\lambda})$ with a modified version of iLAO* ([Hansen and Zilberstein, 2001](#)). This implementation requires a vector heuristic $\mathbf{H} : S \rightarrow \mathbb{R}_{\geq 0}^{n+1}$, and in order to be admissible, $\mathbf{H}$ must satisfy $[1 \ \boldsymbol{\lambda}] \cdot \mathbf{H}(s) \leq V_{\boldsymbol{\lambda}}^*(s)$ for all $s \in S$, where $V_{\boldsymbol{\lambda}}^*$ is the optimal value function for $\mathbb{S}(\boldsymbol{\lambda})$. This precludes heuristics such as C-POM or C-ROC ([Trevizan, Thiébaux, and Haslum, 2017](#)), which take into account the interaction between secondary-cost constraints.

Piecewise linear concave optimisation. Equipped with the oracle that computes $L(\boldsymbol{\lambda})$ and a subgradient for each $\boldsymbol{\lambda}$, one can apply the standard subgradient descent method this optimisation problem ([Shor, 1985](#)). However, the standard subgradient descent method is generally slow to converge, and instead, [Hong and Williams \(2023\)](#) use a coordinate search that performs an exact line search over each coordinate of $\boldsymbol{\lambda}$, which has been shown to be efficient for piecewise linear concave models with few constraints.

Coordinate search maximises $L(\boldsymbol{\lambda})$ by focusing on one coordinate at a time. It sequentially solves for each i the problem $\max_{\lambda_i} L(\boldsymbol{\lambda})$, where all λ_j for $j \neq i$ are fixed from the previous step. Importantly, $\max_{\lambda_i} L(\boldsymbol{\lambda})$ is itself a piecewise linear concave maximisation problem, but now in one dimension. This can be solved efficiently with an exact line search.

We visualise the surface of $L(\boldsymbol{\lambda})$ for two different CSSPs in fig. 6.4. Fig. 6.4a shows $L(\boldsymbol{\lambda})$ associated with the Getting to Work CSSP (ex. 1). Recall that this CSSP has three proper deterministic policies:

- $\pi_{\text{run}} = \{s_I \mapsto \text{run}\}$,
- $\pi_{\text{taxi}} = \{s_I \mapsto \text{taxi}\}$,
- $\pi_{\text{train}} = \{s_I \mapsto \text{walk}, s_1 \mapsto \text{train}, s_2 \mapsto \text{train}\}$.

Each linear surface of the plot corresponds to one of these deterministic policies; we have labelled the linear surfaces associated with π_{run} and π_{train} with “run” and “train” respectively, and π_{taxi} is obscured. A linear surface indicates that the corresponding policy is optimal for the $\mathbb{S}(\boldsymbol{\lambda})$ at the relevant values of $\boldsymbol{\lambda}$; edges and vertices indicate that multiple deterministic policies are optimal at the $\mathbb{S}(\boldsymbol{\lambda})$. In particular, note that at $\boldsymbol{\lambda}^* = 0$ we have an edge between π_{run} and π_{taxi} , and π_{train} does not intersect there; this indicates that the optimal (in this case stochastic) policy consists of π_{run} and π_{taxi} , which we know to be true. This example is not interesting in terms of coordinate search because $\boldsymbol{\lambda}^* = 0$ is found in the first step, so we consider the following example.

Example 2 (Interesting Coordinate Search CSSP). Consider the CSSP in fig. 6.3, with $u_1 = u_2 = 15$. Note that it only has deterministic actions. Its optimal policy π^* is stochastic with

- $\pi^*(s_I, a_2) = 1$
- $\pi^*(s_1, a_4) = \frac{1}{4} \quad \pi^*(s_1, a_5) = \frac{3}{4}$

and it has $C_0(\pi^*) = 4$ and $C_1(\pi^*) = C_2(\pi^*) = 15$.

Fig. 6.4b shows $L(\boldsymbol{\lambda})$ for the CSSP from ex. 2. The red lines indicate “the path of $\boldsymbol{\lambda}$ ” with coordinate search, i.e., it starts at $\boldsymbol{\lambda} = 0$, then moves to $\boldsymbol{\lambda} = [0.025 \ 0]$, then $\boldsymbol{\lambda} = [0.025 \ 0.025]$, and so on, eventually converging to $\boldsymbol{\lambda}^* = [0.2 \ 0.2]$. Importantly, recognise that only a single coordinate changes in each step because the coordinate search only optimises one coordinate at a time.

It is noteworthy that the coordinate search is not complete for piecewise linear concave problems, and may not find an optimal solution ([Shi et al., 2017](#)). We illustrate this with the following example.

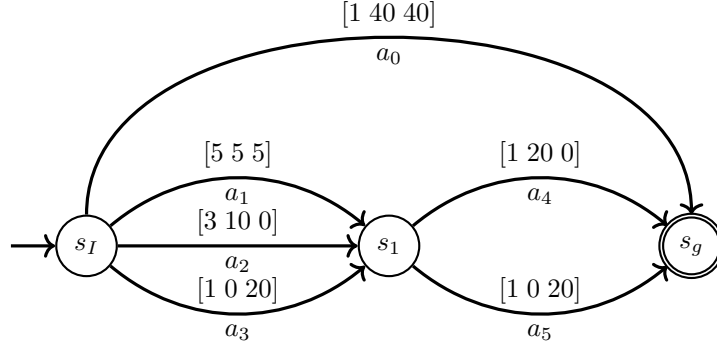
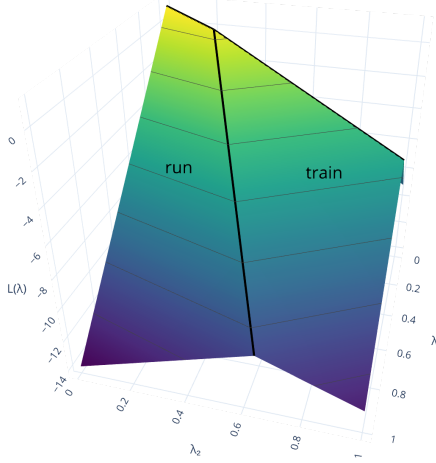
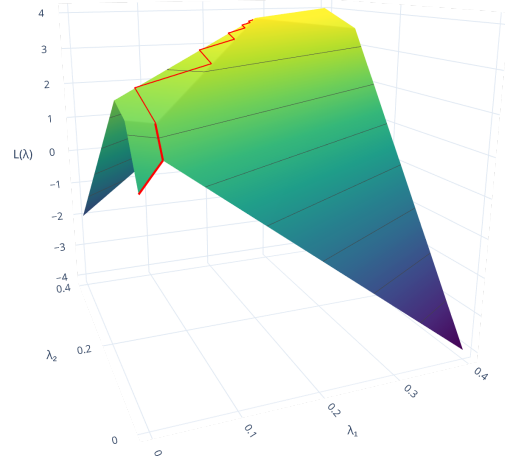


Figure 6.3: A CSSP that is interesting for coordinate search (ex. 2). It has $\mathbf{u} = [15 \ 15]$.



(a) Getting to Work CSSP (ex. 1).



(b) Interesting Coordinate Search CSSP (ex. 2).

Figure 6.4: $L(\boldsymbol{\lambda})$ for two different CSSPs.

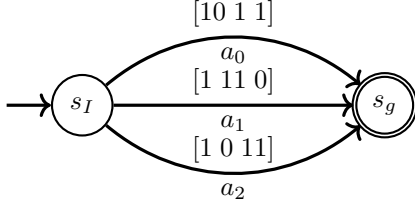
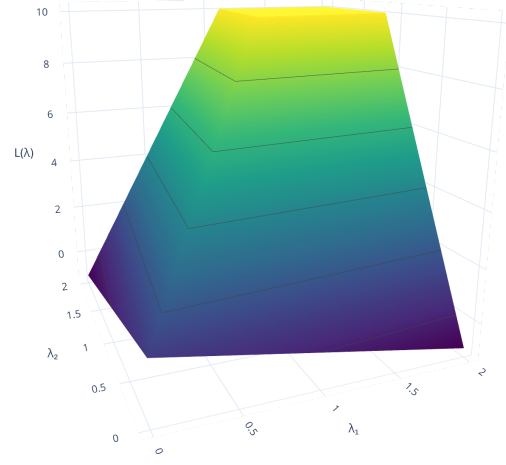
Example 3 (Coordinate Search is Incomplete CSSP). Consider the CSSP in fig. 6.5a with two secondary-cost constraints $u_1 = u_2 = 1$, i.e., a policy π is feasible if $C_1(\pi) \leq 1$ and $C_2(\pi) \leq 1$. The CSSP has three actions that lead deterministically from s_I to the goal with costs

- $\mathbf{C}(a_0) = [10 \ 1 \ 1]$,
- $\mathbf{C}(a_1) = [1 \ 11 \ 0]$, and
- $\mathbf{C}(a_2) = [1 \ 0 \ 11]$,

inducing the three deterministic policies π_0, π_1, π_2 with $\pi_i(s_I) = a_i$ for $i \in \{0, 1, 2\}$. Taking into account the terminal costs, the policy costs are

- $C_{\boldsymbol{\lambda}}(\pi_0) = 10 + \lambda_1(1 - u_1) + \lambda_2(1 - u_2) = 10$,
- $C_{\boldsymbol{\lambda}}(\pi_1) = 1 + \lambda_1(11 - u_1) + \lambda_2(0 - u_2) = 1 + 10\lambda_1 - \lambda_2$, and
- $C_{\boldsymbol{\lambda}}(\pi_2) = 1 + \lambda_1(0 - u_1) + \lambda_2(11 - u_2) = 1 - \lambda_1 + 10\lambda_2$.

$L(\boldsymbol{\lambda})$ selects the cheapest one of these, i.e., $L(\boldsymbol{\lambda}) = \min\{10, 1 + 10\lambda_1 - \lambda_2, 1 - \lambda_1 + 10\lambda_2\}$. This surface is shown in fig. 6.5b; note there are three linear surfaces corresponding to the three deterministic policies (the flat one is π_0 , and the sloped ones are π_1 and π_2). If we consider $\boldsymbol{\lambda} = 0$ as a starting point then $L([0 \ 0]) = 1$, and we can see that there is no way to get a larger $L(\boldsymbol{\lambda})$ by moving in a single coordinate.

(a) Pathological CSSP. It has $\mathbf{u} = [1 \ 1]$.(b) $L(\boldsymbol{\lambda})$ for the CSSP.Figure 6.5: A pathological CSSP where coordinate search fails to find $\boldsymbol{\lambda}^*$ and its $L(\boldsymbol{\lambda})$.

Algebraically,

$$L([x \ 0]) = \min\{10, 1 + 10x, 1 - x\} = 1 - x < 1$$

and symmetrically

$$L([0 \ x]) = \min\{10, 1 - x, 1 + 10x\} = 1 - x < 1$$

for all $x \in \mathbb{R}_{>0}$. Thus, coordinate search gets stuck at $\boldsymbol{\lambda} = [0 \ 0]$ with $L(\boldsymbol{\lambda}) = 1$ and fails to progress to an optimal solution such as $\boldsymbol{\lambda}^* = [2 \ 2]$ with $L(\boldsymbol{\lambda}^*) = 10$. Coordinate search's incompleteness is not important in the context of LAny because its second stage will ensure optimality regardless, but this will be important in chap. 7.

Tracking incumbent solutions and allowing early termination. During the search for $\boldsymbol{\lambda}^*$, the algorithm will solve many instances of $\mathbb{S}(\boldsymbol{\lambda})$ and produce the policy $\pi_{\boldsymbol{\lambda}}^*$. It can happen that one of these $\pi_{\boldsymbol{\lambda}}^*$ is feasible for the original CSSP. In this case, $\pi_{\boldsymbol{\lambda}}^*$ may become the new incumbent solution $\pi_{\text{incumbent}}$; $\pi_{\text{incumbent}} \leftarrow \pi_{\boldsymbol{\lambda}}^*$ if no previous incumbent solution exists or $\pi_{\boldsymbol{\lambda}}^*$ is cheaper than the previous incumbent w.r.t. C_0 . Recall that each $L(\boldsymbol{\lambda})$ is a lower bound on $C_0(\pi_{\text{det}}^*)$, where π_{det}^* is a deterministic-optimal policy for the original CSSP. Thus, LAny tracks the largest $L(\boldsymbol{\lambda})$ it has encountered so far as a lower bound lb , and it tracks an upper bound ub via $C_0(\pi_{\text{incumbent}})$. If $lb = C_0(\pi_{\text{incumbent}})$, then this proves that $\pi_{\text{incumbent}}$ is a deterministic-optimal policy for the CSSP, and the algorithm can return it, terminating early.

Stage 2: Closing the gap. Stage 1 solves the Lagrangian dual, terminating with $\boldsymbol{\lambda}^*$ and potentially an incumbent feasible solution $\pi_{\text{incumbent}}$.³ It is possible that the incumbent solution $\pi_{\text{incumbent}}$ is deterministic-optimal, or close to it, but it can also happen that stage 1 did not discover any feasible policies, and $\pi_{\text{incumbent}}$ is undefined. In such cases we write $C_0(\pi_{\text{incumbent}}) = \infty$. The task of stage 2 is to close the gap between the Lagrangian dual solution $L(\boldsymbol{\lambda}^*)$ and a solution for the primal problem, i.e., finding a deterministic-optimal policy. The algorithm is described in alg. 6. It works by enumerating the deterministic policies for $L(\boldsymbol{\lambda}^*)$ in ascending order of cost $C_{\boldsymbol{\lambda}^*}(\pi)$, breaking ties between policies with identical costs arbitrarily. For the i th cheapest policy π^i , its $L(\boldsymbol{\lambda}^*, \pi^i)$ gives a lower bound on the j th cheapest policies for $j \leq i$, and this lower bound is tracked with lb in alg. 6. Thus, $lb \geq C_0(\pi_{\text{incumbent}})$ is proof that $\pi_{\text{incumbent}}$ is a deterministic-optimal policy, and the algorithm can terminate. In the worst case, the algorithm has to enumerate every deterministic policy for $\mathbb{S}(\boldsymbol{\lambda}^*)$. This set is finite, so the algorithm is guaranteed to terminate eventually, and then output a deterministic-optimal policy. Importantly, this

³As we mentioned, the coordinate search is incomplete, so stage 1 may return a suboptimal solution instead of $\boldsymbol{\lambda}^*$, but this is not important for this discussion.

Algorithm 6: Stage 2 of LAny

```

1 Function closing-gap( $\mathbb{C}, \lambda^* \in \mathbb{R}_{\geq 0}^n, \pi_{\text{incumbent}}$ )
2    $lb \leftarrow L(\lambda^*)$ 
3   for  $i \in \{0, 1, \dots, |\Pi_{\text{det}}|\}$ 
4      $\pi^i \leftarrow i\text{th cheapest policy of } \mathbb{S}(\lambda^*)$ 
5      $lb \leftarrow L(\lambda^*, \pi^i)$ 
6     if  $\pi^i$  is feasible for  $\mathbb{C}$  and  $C_0(\pi) < C_0(\pi_{\text{incumbent}})$  then
7        $\pi_{\text{incumbent}} \leftarrow \pi^i$ 
8   until  $lb \geq C_0(\pi_{\text{incumbent}})$ 
9   return  $\pi_{\text{incumbent}}$ 

```

mechanism for closing the gap searches over the policy space. The policy space can be exponentially larger than the state space, which puts this LAny at a disadvantage compared to state-based search algorithms if LAny's stage 2 has to enumerate a lot of policies. Consequently, LAny is only practical when the Lagrangian dual gives a good approximation of the CSSP, and few policies have to be enumerated by stage 2.

Closing the gap approximately, without guarantees. To speed up the procedure for closing the gap, [Hong and Williams \(2023\)](#) propose an approximate scheme that prunes policies from consideration if they can not improve the incumbent by a sufficient amount. Pruning policies from consideration can enable LAny to find close-to deterministic-optimal solutions quicker, but stage 2 loses its termination condition, and the algorithm has to be stopped with a time-out. We will not consider the approximate scheme throughout this thesis because our focus is optimal algorithms. Furthermore, in preliminary experiments, the approximate scheme did not offer significant performance gains in our implementation.

CARL: Solving CSSPs with Scalarisation

This chapter presents CARL, a novel algorithm for finding optimal (potentially stochastic) policies for CSSPs. CARL uses a primal formulation, which means that it optimises a value function that can be used to construct optimal policies, similarly to VI and the VLP (LP 1). It works by solving the VLP extended to CSSPs; but instead of solving the LP directly, CARL solves it by running a subgradient method that solves unconstrained SSPs as subproblems. These subproblem SSPs are constructed by projecting the CSSP’s cost vectors onto a scalar cost function, i.e., scalarisation. In other words, CARL solves the Lagrangian dual of the SSP and produces an optimal (maximising) scalarisation λ^* , using similar methods to LAny’s stage 1 (see sec. 6.3.3). Then, CARL computes all optimal deterministic policies associated with the scalarised SSP of λ^* and combines them into an optimal (potentially stochastic) policy for the original CSSP.

We give a breakdown of this chapter:

- Sec. 7.1 motivates the use of a primal formulation for CSSPs.
- Sec. 7.2 extends the VLP to the Constrained VLP.
- Sec. 7.3 describes the CARL algorithm.
- Sec. 7.4 gives an intuition for why CARL works and proves CARL’s correctness.
- Sec. 7.5 discusses how LAny and Multi-Objective planning relate to LAny.
- Sec. 7.6 gives the methodology and results of our experiments.
- Sec. 7.7 summarises the chapter.

Parts of this chapter were published in ECAI 2025 (Schmalz and Trevizan, 2025).

7.1 Motivation

Recall the two standard ways to solve unconstrained SSPs:

- **Primal formulation:** we optimise over the value function to find V^* whose greedy policy is optimal, e.g., VI, iLAO*, LRTDP, CG-iLAO*.
- **Dual formulation:** we optimise over occupation measures to find the cheapest way to route flow from s_I to the goals, inducing an optimal policy, e.g., OMLP (LP 2).

Algorithms over the primal representation tend to be faster, and in particular the state-of-the-art algorithms such as CG-iLAO* rely on the primal representation. In contrast, the only previously existing heuristic-search algorithms for solving CSSPs optimally are i-dual (Trevizan et al., 2016) and i²-dual (Trevizan, Thiébaux, and Haslum, 2017), which use the dual formulation. This chapter addresses this gap, and introduces CARL as the first heuristic-search algorithm that finds optimal (potentially stochastic) policies for CSSPs using the primal space.

CARL solves the VLP (LP 1)’s extension to CSSPs by breaking it down into an optimisation problem that can be solved with a subgradient method, and unconstrained SSPs as subproblems. We can solve the sequence of unconstrained SSPs efficiently, using the techniques developed in sec. 4.4. CARL implicitly uses the well-established technique of Lagrangian duality and solving it via optimisation, which has been used successfully for constrained deterministic shortest path problems (Handler and Zang, 1980), Constrained POMDPs (Lee et al., 2018), and for deterministic-optimal policies for CSSPs (see LAny in sec. 6.3.3).

7.2 Constrained Value Function LP

Underlying CARL is the Constrained Value Function LP (CVLP) in LP 12. Recall that unconstrained SSPs can be solved by two LPs:

- VLP (LP 1): a primal formulation with variables V representing the value function;
- OMLP (LP 2): the dual of the VLP, which is respectively a dual formulation with occupation measures.

The COMLP (LP 9) extends the OMLP to CSSPs, and is the standard LP used to solve CSSPs; its dual CVLP has not received much attention. The CVLP is primal (because it is the dual of the dual) and optimises over the value function V , so we will write V_s and $V(s)$ interchangeably. Thanks to strong duality, the CVLP solves the same problem as the COMLP, i.e., it finds an optimal policy for CSSPs. We summarise the relationships between these LPs in tab. 7.1.

	primal	dual
SSP	VLP (LP 1)	OMLP (LP 2)
CSSP	CVLP (LP 12)	COMLP (LP 9)

Table 7.1: Primal and dual LPs for solving SSPs and CSSPs.

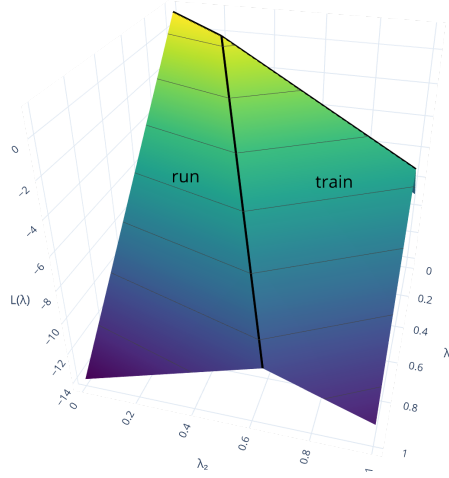
Observe that if we construct the CVLP for an unconstrained SSP then the LP’s λ_i and u_i terms disappear, leaving the VLP which encodes the SSP’s Bellman equations (3.1). For a CSSP, the CVLP has the additional variables λ which correspond to Lagrange multipliers under the interpretation of Lagrangian duality. These λ can also be interpreted as a scalarisation term which flattens the CSSP’s cost vector into a scalar cost function, yielding an unconstrained SSP. Thus, for a fixed $\lambda \in \mathbb{R}_{\geq 0}^n$, the CVLP describes the Bellman equations for an unconstrained SSP, namely the Lagrangian SSP $\mathbb{S}(\lambda)$ (defn. 42) with

- constant terminal costs $-\sum_{i=1}^n \lambda_i u_i$ (from the CVLP’s objective),
- action costs $C_\lambda(a) = [1 \ \lambda] \cdot \mathbf{C}(a)$ (from C41).

So, for a fixed λ the CVLP’s V induces an optimal deterministic policy for $\mathbb{S}(\lambda)$. Now, let us make λ free. Then, the CVLP’s optimal solution V^*, λ^* is a scalarisation λ^* that induces the most expensive Lagrangian SSP $\mathbb{S}(\lambda^*)$, and V^* is the value function of this $\mathbb{S}(\lambda^*)$. We know that V^*, λ^* must induce an optimal policy for the CSSP due to strong duality; however, it is not immediately clear how to extract an optimal (potentially stochastic) policy from these values. We can obtain a policy from V^* by constructing its greedy policy but this is deterministic, and existing methods for obtaining stochastic policies rely on the COMLP’s occupation measure. It turns out that if we compute all optimal policies for $\mathbb{S}(\lambda^*)$, which are all deterministic, then we can construct π^* as a convex combination of $\mathbb{S}(\lambda^*)$ ’s optimal deterministic policies. We give more details and derive this fact with complementary slackness in sec. 7.3.3.

Again, we know that the CVLP must produce an optimal policy thanks to strong duality, but this is not intuitive. To motivate how the CVLP produces an optimal policy, consider the Getting to Work CSSP (ex. 1). The CSSP has deterministic policies π_{run} , π_{taxi} , and π_{train} ; and its optimal policy π^* is stochastic with $\pi^*(s_I, \text{run}) = \frac{1}{2}$ and $\pi^*(s_I, \text{taxi}) = \frac{1}{2}$, i.e., π^* is the convex combination $\pi^* = 0.5\pi_{\text{run}} + 0.5\pi_{\text{taxi}}$. In fig. 7.1 we plot $L(\lambda)$ as λ varies, where $L(\lambda)$ is the cost $C_\lambda(\pi_\lambda^*)$ associated with $\mathbb{S}(\lambda)$ ’s optimal policy π_λ^* , as explained in sec. 6.3.3. Each linear surface in this plot corresponds to a deterministic policy; we have labelled the surfaces corresponding to π_{run} and π_{train} , and π_{taxi} is obscured. The CVLP is solved by finding λ that maximises $L(\lambda)$, and then finding the optimal value function associated with its $\mathbb{S}(\lambda)$. This problem’s optimal solution is $\lambda^* = 0$ attaining $L(\lambda^*) = 1$, which we can verify visually in the plot. Importantly, at λ^* we have the intersection of the linear surfaces of π_{run} and π_{taxi} ; and the surface of π_{train}

Constrained Value Function LP (CVLP) $\max_{V, \lambda} \quad V_{s_I} - \sum_{i=1}^n \lambda_i u_i \quad (\text{LP } 12)$ $\text{s.t.} \quad V_s \leq C_0(a) + \sum_{i=1}^n \lambda_i C_i(a) + \sum_{s' \in \mathcal{S}} P(s' s, a) V_{s'} \quad \forall s \in \mathcal{S} \setminus \mathcal{G}, a \in \mathcal{A}(s) \quad (\text{C41})$ $V_g = 0 \quad \forall g \in \mathcal{G} \quad (\text{C42})$ $\lambda_i \geq 0 \quad \forall i \in \{1, \dots, n\} \quad (\text{C43})$	
--	--

Figure 7.1: $L(\lambda)$ for the Getting to Work CSSP (ex. 1).

does not intersect. This indicates that the optimal policy is a convex combination of π_{run} and π_{taxi} , which is indeed true. Algebraically, if we know that $\lambda^* = 0$, then we get $V^* = \{s_I \mapsto 1, s_1 \mapsto 1, s_2 \mapsto 3, g \mapsto 0\}$ with the Q -values

- $Q^*(s_I, \text{run}) = 1$
- $Q^*(s_I, \text{taxi}) = 1$
- $Q^*(s_I, \text{walk}) = 3$

which means that π_{run} and π_{taxi} are the optimal deterministic policies for $\mathcal{S}(\lambda^*)$. This is again consistent with the fact that π^* must be a convex combination of π_{run} and π_{taxi} , and motivates that we can obtain π^* 's constituent policies from V^* and λ^* .

7.3 CARL

This section explains our Scalarisation Algorithm over the Lagrangian Dual (CARL) and how it finds optimal policies for CSSPs via scalarisation. We start with a high-level overview.

Essentially, CARL is solving the CVLP. However, doing so directly is impractical due to the number of variables and constraints as state and action spaces get large. Therefore, CARL separates the optimisation over V and λ into an outer problem with subproblems:

- $\max_{\lambda} L(\lambda)$ (outer problem with variable λ)
- $L(\lambda) = \max_V V_{s_I} - \sum_{i=1}^n \lambda_i u_i$ (subproblem with fixed λ and variable V).

Algorithm 7: Solve CSSP with Scalarisation

```

1 Function solve(CSSP  $\mathbb{C}$ , heuristic  $\mathbf{H} : \mathcal{S} \rightarrow \mathbb{R}_{\geq 0}^{n+1}$ )
  //  $\lambda^* \in \mathbb{R}_{\geq 0}^n, \mathbf{V} : \mathcal{S} \rightarrow \mathbb{R}_{\geq 0}^{n+1}, V_{\nabla}^* : \mathcal{S} \rightarrow \mathbb{R}_{\geq 0}$ 
2    $\lambda^*, \mathbf{V} \leftarrow \text{find-}\lambda^*(\mathbb{C}, \mathbf{H})$ 
3    $V_{\nabla}^* \leftarrow \text{find-ties-for-}\mathbb{S}(\lambda^*, \mathbf{V}, \mathbb{C}, \mathbf{H})$ 
4    $\pi^* \leftarrow \text{get-stochastic-policy}(\lambda^*, V_{\nabla}^*, \mathbb{C})$ 
5   return  $\pi^*$ 
6 Function find- $\lambda^*$ (CSSP  $\mathbb{C}$ ,  $\mathbf{H} : \mathcal{S} \rightarrow \mathbb{R}_{\geq 0}^{n+1}$ )
  //  $\lambda \in \mathbb{R}_{\geq 0}^n, \mathbf{V} : \mathcal{S} \rightarrow \mathbb{R}_{\geq 0}^{n+1}, \mathbf{g} \in \mathbb{R}^n$ 
7    $\lambda \leftarrow 0, \mathbf{V} \leftarrow \mathbf{H}$ 
8   repeat
9      $\mathbf{V} \leftarrow \text{solve-}\mathbb{S}(\lambda, \mathbf{V}, \mathbb{C})$ 
10    subgradient  $\mathbf{g} \leftarrow [V_1(s_I) - u_1 \dots V_n(s_I) - u_n]$ 
11     $\lambda \leftarrow \text{move from } \lambda \text{ with } \mathbf{g}$  (see sec. 7.3.1)
12  until  $\lambda$  has converged
13  return  $\lambda, \mathbf{V}$ 
14 Function solve- $\mathbb{S}$ ( $\lambda, \mathbf{V}, \mathbb{C}$ )
15    $\mathbf{V} \leftarrow \text{solve } \mathbb{S}(\lambda)$  (see defn. 42) with SSP algorithm modified to work on vector  $\mathbf{V}$  (see sec. 7.3.1)
16   return  $\mathbf{V}$ 
17 Function find-ties-for- $\mathbb{S}$ ( $\lambda, \mathbb{C}, \mathbf{H}$ )
18    $\mathbf{V}_{\nabla}^* \leftarrow \text{solve-}\mathbb{S}(\lambda, \mathbf{V}, \mathbb{C})$  with modification that satisfies strong  $\epsilon$ -consistency (defn. 46) as described in
    sec. 7.3.2
19   return  $V_{\nabla}^*$  s.t.  $V_{\nabla}^*(s) = [1 \ \lambda^*] \cdot \mathbf{V}_{\nabla}^*(s) \ \forall s \in \mathcal{S}$ 
20 Function get-stochastic-policy( $\lambda, V_{\nabla}^*, \mathbb{C}$ )
21    $\mathbf{x}^* \leftarrow \text{solution to SOL 1 with } \lambda, V_{\nabla}^*, \mathbb{C}$ 
22   return  $\pi_{\mathbf{x}^*}$ 

```

$L(\lambda)$ is a Lagrangian relaxation, and its value is the optimal policy cost for the associated $\mathbb{S}(\lambda)$. As λ varies, the function $L(\lambda)$ produces a piecewise linear concave surface (Hong and Williams, 2023). Thus, $\max_{\lambda} L(\lambda)$ can be solved with a subgradient method, provided that we have an oracle that can give $L(\lambda)$ and a subgradient for each λ . It turns out that solving $\mathbb{S}(\lambda)$ produces the information we need, so we can implement an oracle by using SSP algorithms to solve $\mathbb{S}(\lambda)$ s. Although our presentation is different, this part of CARL is effectively solving the COMLP’s Lagrangian dual, which is also the first step of LAny (Hong and Williams, 2023), as described in sec. 6.3.3. We use some of their techniques, but we also describe a new and improved way to solve the $\mathbb{S}(\lambda)$ subproblems, which is a novel contribution and can be used to improve LAny. Once λ^* is found, CARL completely deviates from LAny because it must extract an optimal policy for the CSSP, instead of a deterministic-optimal one. This requires CARL to find a V^* that describes all optimal deterministic policies for $\mathbb{S}(\lambda^*)$, deviating from existing SSP heuristic-search algorithms, which only look for a single optimal deterministic policy. We show how to do this efficiently with *strong ϵ -consistency*. Finally, CARL has to combine the tied-greedy policies for V^* , i.e., the optimal policies for $\mathbb{S}(\lambda^*)$, into an optimal policy for the CSSP via complementary slackness. CARL’s high-level pseudocode is shown in alg. 7.

In the remainder of this section, we give details of CARL’s three key steps:

1. find λ^* (*find- λ^** and *solve- $\mathbb{S}$*) (sec. 7.3.1)
2. find V^* given λ^* (*find-ties-for- $\mathbb{S}$*) (sec. 7.3.2)
3. extract optimal (potentially stochastic) policy (*get-stochastic-policy*) (sec. 7.3.3).

7.3.1 Finding a Maximal Scalarisation

In this section, we describe how CARL finds λ^* with *find- λ^** . CARL follows LAny and applies the same subgradient method over the outer problem $\max_{\lambda} L(\lambda)$, and solves $\mathbb{S}(\lambda)$ s as subproblems to compute $L(\lambda)$ and a subgradient for each λ . However, CARL solves its $\mathbb{S}(\lambda)$ s more efficiently than LAny by using CG-iLAO* instead of a combination of iLAO* and VI.

Outer optimisation over λ . Assume for now that we have an oracle that returns $L(\lambda)$ and a subgradient $\mathbf{g}$ for each λ via solving $\mathbb{S}(\lambda)$ s. Recall that the outer optimisation problem $\max_{\lambda} L(\lambda)$ is piecewise linear concave. We follow LAny and use an exact line search within coordinate search, which has been shown to be efficient for piecewise linear concave models with few constraints (see sec. 6.3.3 for details). However, as we discussed in sec. 6.3.3, coordinate search is incomplete for our problems because they are non-smooth. This is an issue because CARL can not find an optimal policy without λ^* . Fortunately, such failures do not happen often, as we see in our experiments (sec. 7.6). Moreover, the failure of coordinate search is detectable. Due to strong duality, we know that $L(\lambda^*) = C_0(\pi^*)$ for any optimal policy π^* ; thus, if coordinate search produces the policy $\pi^\dagger$ from $V^\dagger, \lambda^\dagger$ such that $L(\lambda^\dagger) < C_0(\pi^\dagger)$, then we know that $\lambda^\dagger \neq \lambda^*$ and coordinate search failed. If CARL detects that coordinate search failed, it falls back on the standard subgradient method (with projection to ensure $\lambda \geq 0$) (Shor, 1985), which is slower but is complete and will eventually converge to λ^* .

Oracle via $\mathbb{S}(\lambda)$ Subproblem. Recall that $\mathbb{S}(\lambda)$ (defn. 42) is an unconstrained SSP, and is therefore solved optimally by deterministic policies. $\mathbb{S}(\lambda)$'s optimal deterministic policy π_λ^* gives $L(\lambda)$ and a subgradient $\mathbf{g} \in \mathbb{R}^n$ in the following way:

- $L(\lambda) = C_\lambda(\pi_\lambda^*)$ and
- $\mathbf{g} = [C_1(\pi_\lambda^*) - u_1 \ \cdots \ C_n(\pi_\lambda^*) - u_n]$ (Hong and Williams, 2023).

Thus, we can implement an oracle for λ by finding π_λ^* and then evaluating it w.r.t. C_λ and each secondary-cost function. To find π_λ^* , we can use any optimal SSP algorithm with any admissible heuristic.

Efficient Heuristic Search over $\mathbb{S}(\lambda)$ s. Instead of using a scalar value function $V : \mathcal{S} \rightarrow \mathbb{R}_{\geq 0}$, we use a vector value function $\mathbf{V} : \mathcal{S} \rightarrow \mathbb{R}_{\geq 0}^{n+1}$, with entries for each cost function. This modification still solves the $\mathbb{S}(\lambda)$ SSP by projecting the vector value function onto $V_\lambda(s) = [1 \ \lambda] \cdot \mathbf{V}(s)$, but we now apply Bellman backups on $\mathbf{V}$ directly. We write $\mathbf{Q}(s, a) = \mathbf{C}(a) + \sum_{s' \in \text{succ}(s, a)} P(s'|s, a) \mathbf{V}(s')$, then the λ modified Bellman backups are given as follows:

Definition 43 (λ Bellman backup). *Given vector value functions $\mathbf{V}, \mathbf{V}' : \mathcal{S} \rightarrow \mathbb{R}_{\geq 0}^{n+1}$ and a scalarisation $\lambda \in \mathbb{R}_{\geq 0}^n$, a vector Bellman backup applied to state s populates $\mathbf{V}'$ with*

$$\mathbf{V}'(s) \leftarrow \mathbf{Q}(s, a_{\min}) \quad \text{for} \quad a_{\min} \in \underset{a \in \mathbf{A}(s)}{\operatorname{argmin}} [1 \ \lambda] \cdot \mathbf{Q}(s, a).$$

This allows us to solve $\mathbb{S}(\lambda)$ and evaluate its policy for each cost function $C_i(\pi_\lambda^*)$ at the same time. Note that we have to redefine the Bellman residual for these backups to ensure that each V_i converges and indeed evaluates its cost function.

To ensure optimality when using heuristic search, we require an admissible heuristic to solve $\mathbb{S}(\lambda)$, i.e., $H \leq V_\lambda^*$; in addition, we also require that the heuristic is a vector in $\mathbb{R}_{\geq 0}^{n+1}$ so that it can be used with $\mathbf{V}$. To address this, we introduce λ heuristics.

Definition 44 (λ heuristic). *For $\mathbb{C}$ and $\mathbb{S}(\lambda)$, a λ heuristic computes an admissible scalar SSP heuristic $H : \mathcal{S} \rightarrow \mathbb{R}_{\geq 0}$ with $H \leq V_\lambda^*$, and then decomposes it into the vector $\mathbf{H} : \mathcal{S} \rightarrow \mathbb{R}_{\geq 0}^{n+1}$ such that $[1 \ \lambda] \cdot \mathbf{H}(s) = H(s) \leq V_\lambda^*(s) \ \forall s \in \mathcal{S}$.*

To vectorise the scalar heuristic, we identify the actions that are considered by H for $\mathbb{S}(\lambda)$, and then use these actions' vector costs. For example, if the heuristic for a state s is $H(s) = C_\lambda(a_0) + C_\lambda(a_1)$, then we can reconstruct $\mathbf{H}(s)$ with $\mathbf{C}(a_0) + \mathbf{C}(a_1)$. For most heuristics, we can use their internal data structures to extract the selected actions, e.g., for LMCut (Helmert and Domshlak, 2009) we consider the representative action from each cut and its weight, and for ROC (Trevizan, Thiébaux, and Haslum, 2017) we consider actions' operator counts. Another way to obtain admissible vector heuristics is with ideal-point (IP) heuristics (Geißer et al., 2022):

Definition 45 (Ideal-Point (IP) heuristic). *For $\mathbb{C}$ and $\mathbb{S}(\lambda)$, an IP heuristic is $\mathbf{H} : \mathcal{S} \rightarrow \mathbb{R}_{\geq 0}^{n+1}$ given by $\mathbf{H} = \langle H_0, \dots, H_n \rangle$ where each H_i is admissible w.r.t. their cost function i . That is, if we relax $\mathbb{C}$ in to an SSP and set its scalar cost function $C = C_i$, then H_i must be admissible for this new SSP.*

The advantage of λ heuristics is that they are computed for the specific $\mathbb{S}(\lambda)$ and are therefore more informative, but with the trade-off that they have to be recomputed for each $\mathbb{S}(\lambda)$, whereas an IP heuristic is admissible for all $\mathbb{S}(\lambda)$ s.

Warm starting $\mathbb{S}(\lambda)$ s. Notice that all $\mathbb{S}(\lambda)$ s are the same problem only differing in their cost functions. This makes $\mathbf{V}$ from a previous $\mathbb{S}(\lambda_{\text{old}})$ a good candidate to start solving a new $\mathbb{S}(\lambda_{\text{new}})$, a technique known as warm start in Operations Research. However, we have to be careful in reusing $\mathbf{V}$. Suppose $\mathbf{V}$ is a solution to $\mathbb{S}(\lambda_{\text{old}})$ and we have

- $V_{\lambda_{\text{old}}}$ with $V_{\lambda_{\text{old}}}(s) = [1 \ \lambda_{\text{old}}] \cdot \mathbf{V}(s) \ \forall s \in \mathcal{S}$ and
- $V_{\lambda_{\text{new}}}$ with $V_{\lambda_{\text{new}}}(s) = [1 \ \lambda_{\text{new}}] \cdot \mathbf{V}(s) \ \forall s \in \mathcal{S}$.

Even if $V_{\lambda_{\text{old}}}$ is admissible for $\mathbb{S}(\lambda_{\text{old}})$, there is no guarantee that $V_{\lambda_{\text{new}}}$ is admissible for $\mathbb{S}(\lambda_{\text{new}})$, which may break the optimality of heuristic search algorithms. LAny makes sure that $V_{\lambda_{\text{new}}}$ is admissible by running VI, but this becomes inefficient as the number of states considered increases. Instead, we use CG-iLAO* and its built-in mechanism for efficiently handling potentially inadmissible states. Recall from sec. 4.4 that CG-iLAO* observes states whose values have increased or decreased to determine a set of state-action pairs that potentially correspond to constraint violations, and these are tracked in the set Γ . Thus, to handle the change from $V_{\lambda_{\text{old}}}$ to $V_{\lambda_{\text{new}}}$, CG-iLAO* tracks the states where $[1 \ \lambda_{\text{new}}] \cdot \mathbf{V}(s)$ decreases or increases w.r.t. $[1 \ \lambda_{\text{old}}] \cdot \mathbf{V}(s)$, and records the corresponding state-action pairs in Γ . Then, we run CG-iLAO* as normal, and it will fix all constraint violations in Γ and produce an optimal policy for $\mathbb{S}(\lambda_{\text{new}})$. Reusing $\mathbf{V}$ in this way speeds up CARL significantly because ensuring admissibility does not occur much overhead and the optimal policy for one $\mathbb{S}(\lambda)$ is often good for the next one; in the best case, if the change in C_λ does not affect the optimal policy, then $\mathbf{V}$ immediately solves the new $\mathbb{S}(\lambda')$.

7.3.2 Finding an Optimal Value Function

Once we have λ^* , we need to compute $V^* : \mathcal{S} \rightarrow \mathbb{R}_{\geq 0}$ that describes all optimal deterministic policies for $\mathbb{S}(\lambda^*)$. Let $\Pi(V)$ denote the set of tied-greedy policies for V , that is, the greedy policies of V obtained by breaking ties in all possible ways. Then, if V^* is the unique solution to the Bellman equations, $\Pi(V^*)$ gives precisely all optimal deterministic policies for the SSP. For us, V^* is produced by CG-iLAO*, which only guarantees ϵ -consistency. This means V^* need only have a small error within one policy's envelope, and any other policies' envelopes can have arbitrarily larger value functions, i.e., V^* need only encode a single greedy policy.

In order to obtain a value function that encodes all (approximately) optimal policies, without solving the Bellman equations exactly, we introduce strong ϵ -consistency. This condition requires a value function to be ϵ -consistent for all of its greedy policies. Formally,

Definition 46 (Strong ϵ -consistency). *A value function V is strongly ϵ -consistent for an SSP $\mathbb{S}$ if*

$$\text{RES}(s) \leq \epsilon \quad \forall s \in \bigcup_{\pi \in \Pi(V)} \mathcal{S}^\pi.$$

When a value function is strongly ϵ -consistent, we write it as $V_\forall^*$. Note that strong ϵ -consistency is a new extension of ϵ -consistency that is necessary for CARL, but ϵ -consistency is sufficient for SSP heuristic-search algorithms that only need to find some optimal policy, not all of them.

To illustrate why strong ϵ -consistency is necessary, we present the following example:

Example 4. *Consider the SSP in fig. 7.2, with the value function V specified in the bottom of each node. The SSP has two optimal policies:*

- $\pi_0^* = \{s_I \mapsto a'_0\}$
- $\pi_1^* = \{s_I \mapsto a_0, s_1 \mapsto a_1, s_3 \mapsto a_3\}$.

V has two greedy policies: the optimal π_0^ and a suboptimal policy*

- $\pi' = \{s_I \mapsto a_0, s_1 \mapsto a'_1, s_2 \mapsto a_2\}$.

V is ϵ -consistent w.r.t. the greedy policy π_0^ , and V is admissible; so, V could be produced by a FIND-AND-REVISE algorithm or CG-iLAO*. Importantly, V is not ϵ -consistent w.r.t. the other greedy policy π' because $\text{RES}(s_2) = |1 - 5| > \epsilon$ (assume small ϵ such as 0.0001), and therefore V is not strongly ϵ -consistent. This has the effect that V fails to capture the second optimal policy π_1^* because π_1^* is not a greedy policy of V , so we can not recover all optimal policies from V .*

Finding $V_\forall^*$. Existing SSP algorithms are designed to find ϵ -consistent solutions, so we must modify them to find strongly ϵ -consistent solutions. For FIND-AND-REVISE algorithms (defn. 21):

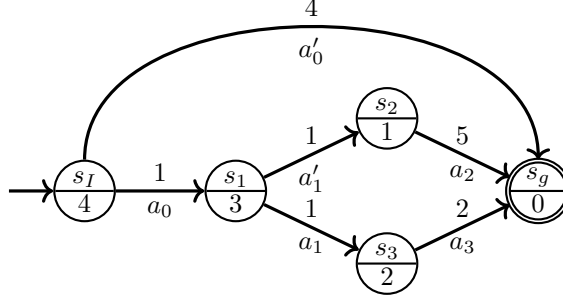


Figure 7.2: An SSP with a value function that is ϵ -consistent but not strongly ϵ -consistent, and fails to describe all optimal policies.

- replace ϵ -consistency with strong ϵ -consistency as the termination condition;
- FIND a state s in one of the greedy policy envelopes $s \in \bigcup_{\pi \in \Pi(V)} S^\pi$ s.t. $\text{RES}(s) > \epsilon$;
- REVISE stays the same.

To adapt CG-iLAO* for finding strongly ϵ -consistent solutions, we make the following modifications:

- Instead of tracking the candidate policy $\hat{\pi}_{\text{curr}} : S \rightarrow A$, we track the union of all tied-greedy policies (up to ϵ) with $\text{tied} : S \rightarrow \mathcal{P}(A)$ where

$$\text{tied}(s) = \{a \in A(s) : Q(s, a) \leq \min_{a' \in A(s)} Q(s, a') + \epsilon\}.$$

- CG-iLAO*'s policy envelope $\mathcal{E}$ over all tied-greedy policies is found by running the depth-first search from s_I over tied .
- In FIX-CONSTRS, instead of tracking only states with $Q(s, a) \leq V(s) - \epsilon$ in Γ , we additionally need to track any actions with $Q(s, a) \leq V(s) + \epsilon$ (the $-$ changed to $+$). This is necessary to ensure that tied is up to date.
- Also, we replace the termination condition $\hat{\pi}_{\text{old}} = \hat{\pi}_{\text{curr}}$, now requiring that no new actions have been added to any $\text{tied}(s)$. We only care about new actions and do not track removals, because they only shrink the envelope and do not affect V .

With these modifications, CG-iLAO* outputs a strongly ϵ -consistent value function V_∇^* . This can be shown with straightforward modifications of the proofs in sec. 4.4.

Recall that the version of CG-iLAO* that we use for CARL works over the vector value function $\mathbf{V} : S \rightarrow \mathbb{R}^{n+1}$ with the λ Bellman backups (see sec. 7.3.1). Consequently, CG-iLAO* must produce a vector value function $\mathbf{V}_\nabla^*$ that induces V_∇^* with $V_\nabla^*(s) = [1 \ \lambda^*] \cdot \mathbf{V}_\nabla^*(s)$. The modifications for strong ϵ -consistency can be applied directly to the vector version of CG-iLAO*, and all the arguments are identical.

7.3.3 Extracting Stochastic Policies

Suppose we have an optimal solution V_∇^* and λ^* , where $\Pi(V_\nabla^*)$ describes all optimal deterministic policies for $S(\lambda^*)$. By strong duality, this solution tells us the primary cost of the CSSP's optimal policy. Furthermore, it turns out that an optimal policy π^* can be constructed as a convex combination of policies in $\Pi(V_\nabla^*)$, which means that the actions used by π^* must be actions used by some $\pi \in \Pi(V_\nabla^*)$. We will show how to use this information to construct an optimal policy for the CSSP from V_∇^* and λ^* .

For this section, it will be useful to define the concept of a policy's support.

Definition 47 (Policy Support at a State). *The support of a policy π at state s is the set of actions that may be applied with non-zero probability*

$$\text{supp}(\pi, s) = \{a \in A(s) : \pi(s, a) > 0\}.$$

Definition 48 (Policy Support). *The support of a policy π is the set of state-action pairs that π may encounter during an execution from s_I*

$$\text{supp}(\pi) = \{(s, a) : s \in S^\pi, a \in \text{supp}(\pi, s)\}.$$

Note that for a policy $\pi_{\mathbf{x}}$ obtained from solving the COMLP, its $\text{supp}(\pi_{\mathbf{x}})$ corresponds to the state-action pairs (s, a) with $x_{s,a} > 0$.

For now, suppose that we have a stochastic policy π . If we want to compute $C_0(\pi)$, we can use the COMLP with the following modifications:

- restrict its variables to the support of π , i.e., the variables $x_{s,a}$ are only defined for $s, a \in \text{supp}(\pi)$,
- add the constraints $\frac{x_{s,a}}{\text{out}(s)} = \pi(s, a)$ for all $s, a \in \text{supp}(\pi)$,
- remove the objective function.

This defines a System of Linear Equations and Inequalities (SOL) whose solution $\mathbf{x}$ describes the occupation measures for π , i.e., $\pi = \pi_{\mathbf{x}}$. Consequently, we can get π 's primary cost with $C_0(\pi) = \sum_{s \in \mathcal{S}, a \in \mathcal{A}} x_{s,a} C_0(a)$. Note that this SOL is efficient to solve because it is not making any decisions, and it is just evaluating π^* 's flow.

Our situation is different because we are looking for an optimal policy π^* and we do not have it yet. However, we do have the following:

- $C_0(\pi^*) = V_{\nabla}^*(s_I) - \sum_{i=1}^n \lambda_i^* u_i$ (by strong duality),
- $\text{supp}(\pi^*) \subseteq \bigcup_{\pi \in \Pi(V_{\nabla}^*)} \text{supp}(\pi)$ (explained later).

Thus, we can take the SOL for evaluating π^* and

- replace $\text{supp}(\pi^*)$ with the approximation $\text{supp}(V_{\nabla}^*) \stackrel{\text{def}}{=} \bigcup_{\pi \in \Pi(V_{\nabla}^*)} \text{supp}(\pi)$; and respectively $\text{supp}(\pi^*, s)$ with $\text{supp}(V_{\nabla}^*, s) \stackrel{\text{def}}{=} \bigcup_{\pi \in \Pi(V_{\nabla}^*)} \text{supp}(\pi, s)$ and $\mathcal{S}^{\pi^*}$ with $\mathcal{S}^{V_{\nabla}^*} \stackrel{\text{def}}{=} \bigcup_{\pi \in \Pi(V_{\nabla}^*)} \mathcal{S}^{\pi}$,
- remove the constraints $\frac{x_{s,a}}{\text{out}(s)} = \pi^*(s, a)$ because we do not have π^* ,
- add the new constraint $\sum_{s \in \mathcal{S}, a \in \mathcal{A}(s)} x_{s,a} C_0(a) = C_0(\pi^*) = V_{\nabla}^*(s_I) - \sum_{i=1}^n \lambda_i^* u_i$.

This gives us **SOL 1** without **C50** and **C51** which we explain later. A solution $\mathbf{x}$ to **SOL 1** (with or without **C50** and **C51**) induces the policy $\pi_{\mathbf{x}}$ which must be proper and feasible, since it is a solution to the COMLP, and it has $C_0(\pi_{\mathbf{x}}) = C_0(\pi^*)$, i.e., $\pi_{\mathbf{x}}$ must be an optimal policy for the CSSP. Intuitively, this SOL is considering all deterministic policies that can be constructed within $\text{supp}(V_{\nabla}^*)$, and determining how to combine these into an optimal policy for the CSSP.

given V_{∇}^* and λ^* find $\mathbf{x}$	(SOL 1)
s.t. $\sum_{s,a \in \text{supp}(V_{\nabla}^*)} x_{s,a} C_0(a) = V_{\nabla}^*(s_I) - \sum_{i=1}^n \lambda_i^* u_i$	(C44)
$\text{out}(s) - \text{in}(s) = \llbracket s = s_I \rrbracket$	$\forall s \in \mathcal{S}^{V_{\nabla}^*} \setminus \mathcal{G}$ (C45)
$\text{out}(s) = \sum_{a \in \text{supp}(V_{\nabla}^*, s)} x_{s,a}$	$\forall s \in \mathcal{S}^{V_{\nabla}^*}$ (C46)
$\text{in}(s) = \sum_{s', a' \in \text{supp}(V_{\nabla}^*)} x_{s', a'} P(s s', a')$	$\forall s \in \mathcal{S}^{V_{\nabla}^*}$ (C47)
$x_{s,a} = 0$	$\forall s \in \mathcal{S}, a \in \mathcal{A}(s) \text{ s.t. } V_{\nabla}^*(s) < Q_{\nabla}^*(s, a)$ (C48)
$x_{s,a} \geq 0$	$\forall s \in \mathcal{S}, a \in \mathcal{A}(s) \text{ s.t. } V_{\nabla}^*(s) = Q_{\nabla}^*(s, a)$ (C49)
$\sum_{s,a \in \text{supp}(V_{\nabla}^*)} x_{s,a} C_i(a) \leq u_i$	$\forall i \in \{1, \dots, n\} \text{ s.t. } \lambda_i^* = 0$ (C50)
$\sum_{s,a \in \text{supp}(V_{\nabla}^*)} x_{s,a} C_i(a) = u_i$	$\forall i \in \{1, \dots, n\} \text{ s.t. } \lambda_i^* > 0$ (C51)

Formally, **SOL 1** comes from complementary slackness, a technique that lets us transform an optimal solution of an LP into an optimal solution for its dual (Bertsimas and Tsitsiklis, 1997). If V_{∇}^*, λ^* is

an optimal basic feasible solution for [LP 1](#) that is non-degenerate, i.e., all its basic variables have non-zero values assigned, then we can recover a solution for its dual [LP 2](#) with complementary slackness by constructing a system of linear equations according to the following rules, and then solving it:

$$\begin{aligned}
V_{\forall}^*(s) > 0 &\implies \text{C25 for } s \text{ is tight} \\
&\text{i.e. } out(s) - in(s) = \llbracket s = s_I \rrbracket \\
\lambda_i > 0 &\implies \text{C29 for } i \text{ is tight} \\
&\text{i.e. } \sum_{s \in \mathbb{S}, a \in \mathbb{A}(s)} x_{s,a} C_i(a) = u_i \\
\text{C41 for } s, a \text{ is loose} &\implies x_{s,a} = 0 \\
&\text{i.e. } V_{\forall}^*(s) < Q_{\forall}^*(s, a)
\end{aligned}$$

These are [SOL 1](#)'s constraints [C45](#), [C48](#), and [C51](#). Unfortunately, our $V_{\forall}^*, \lambda^*$ is often degenerate, so this system of linear equations is underspecified, and we must include additional constraints from the original problem. In particular, we have to reintroduce the secondary-cost constraints [C29](#) where $\lambda_i = 0$, which is what we do in [C50](#). We also include [C44](#) to ensure the resulting policy is indeed optimal.

We have been claiming that $\text{supp}(\pi^*) \subseteq \text{supp}(V_{\forall}^*)$ and that [SOL 1](#) is constructing π^* from the optimal deterministic policies of $\mathbb{S}(\lambda^*)$. This follows from the last complementary slackness rule: occupation measures $x_{s,a}$ are only allowed to be non-zero if $V_{\forall}^*(s) = Q_{\forall}^*(s, a)$, i.e., if a is a tied-greedy action and thereby $\pi(s) = a$ for some $\pi \in \Pi(V_{\forall}^*)$.

7.4 Correctness of CARL

In this section we show that CARL always finds an optimal solution. First, $\text{find-}\lambda^*(\mathbb{C}, \mathbf{H})$ ([alg. 7](#)) is guaranteed to find λ^* because $\max_{\lambda} L(\lambda)$ is a piecewise linear concave problem and solving $\mathbb{S}(\lambda)$ s yields subgradients ([Hong and Williams, 2023](#)). Therefore, using these subgradients with coordinate search falling back on the subgradient descent, which is a complete method, is guaranteed to converge to λ^* . It remains to show that CARL finds V^* for $\mathbb{S}(\lambda^*)$ such that the solution encodes an optimal policy.

Lemma 9. *If π^* is an optimal policy for the CSSP, then its constituent deterministic policies must be optimal for $\mathbb{S}(\lambda^*)$.*

Proof. We first show that π^* is optimal for $\mathbb{S}(\lambda^*)$, and then conclude that its constituent deterministic policies must also be optimal for $\mathbb{S}(\lambda^*)$. Policy π^* 's cost in $\mathbb{S}(\lambda^*)$ can be rewritten as $C_{\lambda^*}(\pi^*) = C_0(\pi^*) + \sum_{i=1}^n \lambda_i^*(C_i(\pi^*) - u_i)$. But π^* is feasible, so $C_i(\pi^*) - u_i \leq 0$ for all $i \in \{1, \dots, n\}$, and therefore $C_{\lambda^*}(\pi^*) \leq C_0(\pi^*)$. On the other hand, we know that $C_{\lambda^*}(\pi) \geq C_0(\pi^*)$ for any policy π , due to the semantic of λ^* within strong duality. Thus, $C_{\lambda^*}(\pi^*) = C_0(\pi^*)$, which is the minimal cost for $\mathbb{S}(\lambda^*)$, and π^* is optimal for $\mathbb{S}(\lambda^*)$. We know that π^* can be decomposed into the convex combination of deterministic policies $\mu_0\pi_0 + \dots + \mu_k\pi_k$, and $C_{\lambda^*}(\pi_i) = C_{\lambda^*}(\pi^*)$ for each $i \in \{0, \dots, k\}$, because otherwise π^* can not be optimal for $\mathbb{S}(\lambda^*)$. $\square$

This is a key result: for any policy π^* that is optimal for the CSSP, the optimal deterministic policies of $\mathbb{S}(\lambda^*)$ must be a superset of π^* 's constituent policies, i.e., it must be possible to construct π^* as some combination of $\mathbb{S}(\lambda^*)$'s optimal deterministic policies.

Theorem 13. *CARL's $\text{find-ties-for-}\mathbb{S}(\lambda^*, \mathbb{C}, \mathbf{H})$ finds $\mathbf{V}_{\forall}^*$ so that [SOL 1](#) with λ^* and $V_{\forall}^*$ constructed by $V_{\forall}^*(s) = \lambda^* \cdot \mathbf{V}_{\forall}^*(s)$ produces an optimal stochastic policy.*

Proof. Suppose $V_{\forall}^*$ encodes all the optimal deterministic policies for $\mathbb{S}(\lambda^*)$. If an optimal policy π^* exists for the CSSP, then its constituent deterministic policies must be optimal for $\mathbb{S}(\lambda^*)$ by [lem. 9](#); therefore its constituent policies are a subset of $\Pi(V_{\forall}^*)$. It follows that [SOL 1](#) contains the whole support of such policy π^* . Since $C_0(\pi^*) = \mathbf{V}_{\forall}^*(s_I) - \sum_{i=1}^n \lambda_i^* u_i$ due to strong duality, it must be the case that [SOL 1](#) induces an optimal policy, thanks to its connection to [LP 2](#). We have argued that the version of CG-iLAO* presented in [sec. 7.3.2](#) finds all policies for which V can be ϵ -consistent, which gives approximately optimal policies for small ϵ , and precisely the optimal policies as $\epsilon \rightarrow 0$. $\square$

Thus, assuming that CARL found λ^* exactly and ϵ is sufficiently small, CARL is proven to find an optimal policy.

Error terms. In practice, CARL only finds an approximation of λ^* , and hand-picked values of ϵ may not be “small enough” for ϵ -consistency, so it is important to discuss the effects of error terms, and how to handle them. For the sake of simplicity, we have omitted that *find- λ^** requires an error term in practice, let’s call it $\eta \in \mathbb{R}_{>0}$. The coordinate search for λ^* stops when one pass over all coordinates can not improve $L(\lambda)$ by more than η . If the failure of coordinate search is detected (see sec. 7.3.1), then the gradient descent method stops when its step size (which is monotonically decreasing over the algorithm’s iterations) is smaller than η . In both cases, this yields an approximate solution $\lambda' \approx \lambda^*$, and the error disappears as $\eta \rightarrow 0$. Similarly, as discussed in sec. 3.3 and sec. 4.4, our SSP solvers produce $V \approx V^*$ for their SSPs, and this error disappears as $\epsilon \rightarrow 0$. Thus, both ϵ and η can affect CARL’s solution, and we explain how this can be mitigated. Assume for now that ϵ is sufficiently small. If CARL finds $\lambda' \approx \lambda^*$ and some policy π , we have

$$L(\lambda') \leq C_0(\pi^*) \leq C_0(\pi)$$

where π^* is an optimal policy for the CSSP. This gives us the optimality gap $C_0(\pi) - L(\lambda')$, which upper bounds how far π ’s primary cost deviates from π^* ’s. If the gap is too large, we can reduce η . Now, we return our attention to ϵ . The value of ϵ affects the gradient search oracle’s value of $L(\lambda)$ and subgradient g at each λ , which in turn can significantly affect the found policy’s quality and breaks the optimality gap for η . Recall that this is standard for algorithms using ϵ -consistency, and they only guarantee an optimal policy as $\epsilon \rightarrow 0$. For additional guarantees, there are “outer loop” methods that run the algorithm and select ϵ automatically in order to guarantee ϵ -optimality (Hansen and Abdoulahi, 2015; Hansen, 2017). CARL is in a similar position and guarantees an optimal policy as $\epsilon \rightarrow 0$, but requires an outer loop method to select ϵ automatically for stronger guarantees. The mentioned methods for ensuring ϵ -optimality immediately apply to CARL’s ϵ -consistent subproblems, and should apply to the strongly ϵ -consistent subproblems with minimal changes, which lets us recover a correct optimality gap. Thus, it is possible to modify CARL to produce solutions with stronger guarantees. In practice, values of $\epsilon = \eta = 0.0001$ are usually “good enough.”

7.5 Related Work: LAny and Multi-Objective Planning

In this section, we explain how LAny and Multi-Objective planning are related to CARL.

It is important to acknowledge that CARL is inspired by LAny (Hong and Williams, 2023) and shares similarities, but also has significant differences that make it a novel contribution. CARL and LAny are similar in the sense that they both find policies for CSSPs via the Lagrangian dual and start by finding λ^* . The main difference is that CARL looks for optimal (potentially stochastic) policies whereas LAny looks for deterministic-optimal policies, which need not be optimal. Moreover, our method for finding λ^* is based on LAny’s but we make substantial algorithmic improvements. They solve $S(\lambda)$ s with iLAO* and reuse V as a warm start (see sec. 7.3.1); however, they need to run VI on iLAO*’s expanded states to make V admissible for the next step, which is inefficient as the partial SSPs grow, defeating the purpose of heuristic search. CARL uses CG-iLAO*, allowing it to focus only on the changes to λ as opposed to VI’s blind search. As a result, CARL finds λ^* up to $4.7\times$ faster than their method with a mean speedup of $2\times$.

Once λ^* has been found, the similarities between CARL and LAny end. Strong duality does not hold for deterministic policies, thus λ^* does not directly produce a deterministic-optimal policy for the original CSSP. To overcome this, LAny uses an expensive second stage that enumerates deterministic policies until the optimal one is found. CARL does not require this stage because strong duality holds for stochastic policies. Instead, we must ensure that V^* encodes all optimal policies for $S(\lambda^*)$, which can be done cheaply with dynamic programming, and then we use SOL 1 to extract the optimal policy for the CSSP. Both these steps have no analogue in their algorithm. Thus, CARL is guaranteed to find an optimal policy, stochastic or not, for CSSPs with relatively little computational effort after finding λ^* , whereas LAny can only find a deterministic-optimal policy, which can cost more than the optimal stochastic policy, and requires an expensive second stage to do so. Additionally, Hong and Williams (2023) did not consider domain-independent heuristics, and we do so by using λ heuristics and IP heuristics.

We now explain how Multi-Objective (MO) planning relates to CSSPs and CARL. MO-SSPs (Forejt, Kwiatkowska, and Parker, 2012; Chen, Trevizan, and Thiébaux, 2023) are SSPs with vector cost functions, similarly to CSSPs. The difference is that MO-SSPs treat all costs as primary costs, i.e., they do not distinguish between primary and secondary costs and do not have secondary-cost constraints. This leads to different solution criteria, and MO-SSPs are solved by finding all undominated trade-offs between

cost functions; restricting to proper policies, a policy π dominates the policy π' if $C_i(\pi) \leq C_i(\pi') \forall i \in \{0, \dots, n\}$ and $C_i(\pi) < C_i(\pi')$ for some $i \in \{0, \dots, n\}$, and respectively a policy π is undominated if no policy exists that dominates π . Thus, an MO-SSP's solution is the set of all undominated proper policies. This is a more general problem than CSSPs. In particular, if a CSSP is solved as an MO-SSP, i.e., all its cost functions are treated as primary costs and its secondary-cost constraints are ignored, then at least one of the CSSP's optimal policies must be among the MO-SSP's undominated policies. Consequently, one can use algorithms for MO-SSPs to solve CSSPs, e.g., [Forejt, Kwiatkowska, and Parker \(2012\)](#) and [Chen, Trevizan, and Thiébaux \(2023\)](#). However, the overhead of finding all undominated policies makes MO-SSP algorithms inefficient for CSSPs, which are solved by a single policy. As an analogue, the MO simplex algorithm can be used to solve MO LPs as well as single-objective LPs ([Luc, 2016](#)), but it is more efficient to solve LPs with a standard simplex implementation.

We also point out that scalarisation is a well-established technique in the MO literature ([Ehrgott, 2006](#)), and has been applied to MO-SSPs ([Forejt, Kwiatkowska, and Parker, 2012](#)). However, the MO use of scalarisation is quite different from the way CARL uses scalarisation because MO is trying to find all undominated trade-offs with different scalarisations, whereas CARL is looking for a single scalarisation that produces a single optimal policy. To reiterate, CARL uses scalarisation in the context of Lagrangian optimisation where scalarisations λ are also called Lagrangian multipliers, which is a single-objective technique. Thus, CARL progressively finds a better scalarisation vector, whereas [Forejt, Kwiatkowska, and Parker \(2012\)](#) accumulate a set of non-dominated scalarisation vectors.

7.6 Empirical Evaluation

We compare our novel algorithm CARL with i-dual ([Trevizan et al., 2016](#)) and i²-dual ([Trevizan, Thiébaux, and Haslum, 2017](#)), the state-of-the-art algorithms for finding optimal stochastic policies for CSSPs.

7.6.1 Methodology

In this section, we experimentally compare CARL with the state-of-the-art algorithms i-dual ([Trevizan et al., 2016](#)) and i²-dual ([Trevizan, Thiébaux, and Haslum, 2017](#)). We describe the considered benchmark domains in [sec. 7.6.2](#). Each domain consists of multiple problems, and for each problem, we consider multiple instances. In our test problems, we allow CSSPs to have dead ends, but transform them into CSSPs without dead ends with the fixed-penalty transformation ([defn. 35](#)). All code is implemented in C++, and we use CPLEX Version 22.1.1 to solve LPs and SOLs. We ran each planner and heuristic on 30 instances of each problem on a cluster with Intel Xeon 3.2 GHz CPUs limited to one CPU core with 30 mins and 4GB RAM. We now give details about the planners used.

CARL. For CARL, we consider the heuristics LMCut ([Helmert and Domshlak, 2009](#)) and ROC ([Trevizan, Thiébaux, and Haslum, 2017](#)) as λ heuristics ([defn. 44](#)) and also as IP heuristics ([defn. 45](#)). Additionally, we consider C-ROC ([Trevizan, Thiébaux, and Haslum, 2017](#)), noting that CARL(C-ROC) does not guarantee that it produces optimal solutions because C-ROC takes the interaction between secondary-cost constraints into account and is therefore inadmissible for CARL's $S(\lambda)$ subproblems. That is, we consider the heuristics λ -LMcut, IP-LMcut, λ -ROC, IP-ROC, and C-ROC. For problems with dead ends before the fixed-penalty transformation, we augment the ROC variants with h-max as a dead-end detector ([Trevizan, Thiébaux, and Haslum, 2017](#)).

I-dual. For i-dual, we follow the results of [Trevizan, Thiébaux, and Haslum \(2017\)](#) and use the heuristics IP-LMcut and C-ROC ([Trevizan, Thiébaux, and Haslum, 2017](#)). Importantly, C-ROC is admissible for i-dual, and i-dual(C-ROC) is guaranteed to produce optimal policies. As before, we use C-ROC with h-max as a dead-end detector ([Trevizan, Thiébaux, and Haslum, 2017](#)) on problems that have dead ends before the fixed-penalty transformation.

I²-dual. I²-dual uses C-POM ([Trevizan, Thiébaux, and Haslum, 2017](#)), which is built into the algorithm, again with h-max as a dead-end detector on problems that have dead ends before the fixed-penalty transformation.

We consider the following benchmark domains:

7.6.2 Benchmark Domains

Constrained Search and Rescue (C-S&R) (Trevizan et al., 2016).

This problem is based on the Search and Rescue problem for SSPs (see page 48). An unknown number of survivors is scattered on an $n \times n$ grid ($n \in \mathbb{N}$), and the agent’s task is to navigate a drone to bring exactly one of the survivors to a safe location as quickly as possible. One survivor’s position is known and has distance $d \in \mathbb{N}$ from the drone’s initial position. Some other locations, selected according to a density $r \in [0, 1]$, have a predetermined probability of having another survivor when explored. Thus, the agent must choose between rescuing the survivor at the known location, and exploring closer locations that may have another survivor. Additionally, the agent can move faster or slower, which uses more or less fuel respectively. C-S&R problems have the single secondary-cost constraint that fuel consumption should not exceed a given threshold. The threshold is selected automatically per problem as a function of the fuel usage of a policy in the SSP relaxation.

Constrained Elevators (C-Elev) (Trevizan et al., 2016).

In a 10-floor building with $e \in \mathbb{N}$ elevators, $w \in \mathbb{N}$ people have pressed the button to call an elevator and are waiting, and $h \in \mathbb{N}$ hidden people have not pressed the button yet. At each step, each hidden passenger presses the button with probability 0.75 at which point they become unhidden and stay unhidden. The task is to route the e elevators to deliver all people to their destinations in the fewest timesteps. Each person has a constraint over the time spent waiting, and the time spent travelling in an elevator, resulting in $2(w + h)$ secondary-cost constraints. The thresholds for waiting and travel times are $1.5n$ for all passengers except one VIP whose waiting time and travel time are constrained by $0.75n$ and $n + 1$ respectively. Note that the number of floors can also be parameterised with $n \in \mathbb{N}$, but we only consider $n = 10$.

Constrained Exploding Blocks World (C-ExBW) (Trevizan et al., 2016).

This problem is based on the Exploding Blocks World for SSPs (see page 47). The agent is presented with $N \in \mathbb{N}$ blocks, which must be rearranged into a specified arrangement with the minimal number of moves. This is complicated by the blocks being rigged with an explosive that can explode, destroying the block or table immediately underneath it. In the SSP variant, destroyed blocks and tables are permanently destroyed and can never be used again. In contrast, this variant has an action to repair the table with a large primary cost, but destroyed blocks remain unrepairable. There is a single secondary-cost constraint, limiting the expected number of destroyed blocks to $c \in \mathbb{R}_{\geq 0}$, which is selected manually to ensure the problem is feasible. We consider interesting arrangements of blocks from IPPC 2008 which we specify by their id.

Constrained PARC Printer (C-PARC) (Trevizan, Thiébaux, and Haslum, 2017).

This domain extends the PARC Printer domain for SSPs without repair (see page 48). A modular printer must print four pages with various requirements. The printer has $c \in \mathbb{N}$ unreliable components in which a page can get jammed with probability 0.1, simultaneously ruining the page and making the component unusable. C-PARC has two secondary-cost constraints:

- $f \in [0, 1]$ constrains how many components are allowed to fail and
- $u \in \mathbb{N}$ limits how often a particular component is allowed to be used ($u = \infty$ deactivates this constraint).

Note that the number of sheets to be printed can be parameterised with $s \in \mathbb{N}$, but we only consider $s = 4$.

Constrained Triangle Tireworld (C- Δ TW) (Schmalz and Trevizan, 2024c).

This domain extends the Triangle Tireworld with Headstarts for SSPs (see page 49). As before, the agent must drive across a triangular network of cities, where $n \in \mathbb{N}$ specifies the size of the network, and $d \in \mathbb{N}$ gives the starting distance from the goal. Between any two cities, the car gets a flat tyre with probability 0.5; with a spare tyre the flat can be replaced and the journey can continue; if no spare is loaded, the car is stuck and no actions are available. The car only fits one spare tyre at a time, and spare tyres can only be acquired in select cities. What is new in this CSSP variant is that tyres can be purchased in

these cities for one of $c \in \mathbb{N}$ currencies, and the problem has c secondary-cost constraints over the c -many currencies. Thus, the agent must choose which currency to spend so that it does not spend too much of any single one.

For C-S&R and C-Elev, a problem instance is a randomly generated problem for the specified parameters. For the other domains, the parameters specify a unique problem and an instance couples this problem with a random seed that is passed to the algorithm.

7.6.3 Results

The results of our experiments are presented in tab. 7.2. We highlight the fastest algorithm-heuristic pairs, where a pair is considered the fastest for a particular problem if it has the maximum coverage and then its mean time plus 95% C.I. is less than the next best algorithm's mean time minus 95% C.I. Recall that CARL(C-ROC) does not guarantee that its solutions are optimal due to C-ROC's inadmissibility for CARL's $\mathbb{S}(\lambda)$ subproblems; nevertheless, in our results, any time that CARL(C-ROC) produced a solution, it was in fact optimal. We now identify the best performing algorithm per domain.

C-S&R, C-ExBW, and C- Δ TW. CARL dominates in these domains and no variant of i-dual or i^2 -dual is highlighted as fastest in the table. In fact, all variants of CARL are faster than the fastest variant of i-dual and i^2 -dual, with the exception of C-ExBW (01, 5, 0.1) and (03, 6, 0.91) where i-dual(IP-LMcut) is faster than CARL with the ROC heuristics, which can be attributed to the difference in heuristic. Moreover, CARL can obtain substantially larger coverage with the extreme case of C-ExBW (06, 8, 0.3), where CARL obtained full coverage with all heuristics while all other planners had zero coverage. Comparing only entries with full coverage in these three domains, CARL with the admissible heuristics has $10\times$, $19\times$, and $42\times$ speedup on average w.r.t. i-dual (C-ROC), i-dual (IP-LMcut) and i^2 -dual, respectively. Between CARL with different heuristics, there is no clear winner overall. Looking at individual domains and restricting our discussion to admissible heuristics:

- C-S&R: the λ heuristics perform better than the IP heuristics, but there is no clear winner between λ -ROC and λ -LMcut;
- C-ExBW: the LMcut variants perform better than ROC, and in particular IP-LMcut tends to perform best;
- C- Δ TW: λ -ROC tends to be the fastest, then the LMcut variants are tied second, and IP-ROC tends to be the slowest.

This demonstrates that in some cases the difference between λ heuristics and IP heuristics is the most important factor, in other cases the difference between ROC and LMcut is more important, and other times both play a role. CARL with C-ROC (which is inadmissible for $\mathbb{S}(\lambda)$ s) has very similar behaviour to IP-ROC in these domains.

C-Elev. With one elevator i-dual dominates, and with two elevators CARL dominates. With one elevator, CARL fails to solve some instances for two reasons:

1. CARL's coordinate search fails and falls back on the subgradient method, as outlined in sec. 7.3.1, which is significantly slower than coordinate search;
2. $\mathbb{S}(\lambda)$ s are too difficult to solve in some cases, which suggests the heuristic does not give adequate guidance.

With two elevators, problem (1) does not occur, and any instances where CARL fails to find a solution is because the coordinate search's subproblems take too long to solve. This contrasts i-dual and i^2 -dual, which are not affected by problem structure. However, these scale worse with the larger search spaces with two elevators.

C-PARC. Among optimal algorithm-heuristic pairs (this excludes CARL with C-POM), i^2 -dual dominates when the constraint over failure is tighter ($f \leq 0.2$) and CARL dominates when the constraint is looser ($f \geq 0.4$). We observe that CARL becomes competitive with i^2 -dual on the tightly constrained problems when it is equipped with C-ROC, which takes secondary-cost constraints into account but is therefore inadmissible for CARL's $\mathbb{S}(\lambda)$ s. This suggests that CARL with the admissible heuristics is struggling on the tightly constrained problems because the heuristics do not take the interaction of constraints into account, and consequently give poor guidance. I^2 -dual has the opposite weakness: its heuristic is very informative for highly constrained problems and it prunes a lot of the search space but,

	CARL												i-dual		i ² -dual		
	λ-ROC				C-ROC †				λ-LMcut				IP-LMcut			C-ROC	
	IP-ROC		C-ROC †		λ-LMcut		IP-LMcut		IP-LMcut								
C-S&R (p, d, r)	(4, 3, 0.75)	30	5.6 ± 1.4	30	11.5 ± 2.3	30	12.1 ± 2.4	30	4.3 ± 1.0	30	9.2 ± 1.8	30	104.1 ± 51.0	30	45.8 ± 21.1	30	69.2 ± 56.4
	(4, 4, 0.75)	30	41.9 ± 5.3	30	75.5 ± 7.6	30	79.5 ± 8.3	30	34.1 ± 4.2	30	65.3 ± 7.6	5	1345.7 ± 534.1	9	979.8 ± 296.6	8	495.5 ± 417.8
	(5, 3, 0.50)	30	4.9 ± 0.9	30	9.0 ± 1.6	30	9.3 ± 1.7	30	4.8 ± 1.1	30	10.0 ± 2.4	30	25.1 ± 8.5	30	13.1 ± 4.6	30	11.1 ± 4.0
	(5, 3, 0.75)	30	12.3 ± 2.5	30	26.2 ± 5.8	30	27.9 ± 6.0	30	13.3 ± 2.9	30	33.1 ± 9.0	30	253.8 ± 110.6	30	113.3 ± 46.0	30	168.0 ± 105.9
	(5, 4, 0.50)	30	30.8 ± 6.8	30	52.3 ± 9.5	30	54.7 ± 9.8	30	36.2 ± 7.9	30	70.5 ± 12.9	17	695.0 ± 268.5	24	550.6 ± 166.1	27	637.8 ± 213.1
C-Elv (w, h)	(5, 4, 0.75)	30	126.6 ± 45.2	30	270.0 ± 80.7	30	282.9 ± 83.9	30	142.5 ± 49.3	30	353.9 ± 105.7	6	930.9 ± 369.3	6	499.2 ± 226.4	8	349.1 ± 288.1
	(1, 1, 1)	30	1.6 ± 0.6	30	2.0 ± 0.4	30	2.1 ± 0.3	30	1.2 ± 0.3	30	1.4 ± 0.3	30	1.5 ± 0.1	30	0.8 ± 0.1	30	6.2 ± 0.6
	(1, 1, 2)	29	50.9 ± 26.8	29	42.7 ± 12.6	29	42.2 ± 11.9	29	35.0 ± 12.7	29	44.4 ± 14.1	30	25.3 ± 1.0	30	21.1 ± 2.6	30	280.0 ± 46.5
	(1, 2, 1)	25	14.2 ± 5.6	25	17.7 ± 3.7	25	17.7 ± 3.7	25	13.1 ± 4.2	25	15.8 ± 4.3	30	13.9 ± 0.9	30	9.8 ± 1.4	30	174.2 ± 38.4
	(1, 2, 2)	22	313.3 ± 150.2	22	399.8 ± 99.9	22	407.9 ± 102.0	22	336.9 ± 115.8	22	460.1 ± 118.5	30	340.1 ± 35.7	30	392.5 ± 72.1	1	989.2
C-ExBW (id, N, c)	(2, 1, 1)	30	104.5 ± 23.2	30	131.7 ± 26.2	30	126.9 ± 25.1	30	121.8 ± 26.7	30	149.7 ± 31.5	23	670.6 ± 173.7	26	576.1 ± 181.4	1	1799.7
	(2, 1, 2)	20	1321.5 ± 161.1	4	1281.0 ± 114.2	5	1309.1 ± 230.8	3	1344.2 ± 341.5	–	–	1	1799.6	–	–	–	–
	(2, 2, 1)	28	569.2 ± 131.7	28	830.8 ± 160.8	28	796.4 ± 158.7	23	890.5 ± 164.2	18	1157.4 ± 180.4	5	1608.9 ± 229.1	5	1444.4 ± 352.4	–	–
	(01, 5, 0.1)	30	8.6 ± 0.1	30	8.5 ± 0.1	30	8.5 ± 0.1	30	1.6 ± 0.0	30	1.2 ± 0.0	30	97.2 ± 1.0	30	7.9 ± 0.1	30	626.6 ± 18.8
	(02, 5, 0.07)	30	40.4 ± 0.3	30	36.0 ± 0.2	30	35.3 ± 0.3	30	18.7 ± 0.1	30	15.2 ± 0.1	30	–	30	732.9 ± 30.6	–	–
C-PARC (f, u)	(03, 6, 0.91)	30	173.5 ± 5.1	30	138.0 ± 1.4	30	135.7 ± 1.2	30	7.5 ± 0.1	30	3.3 ± 0.0	–	–	30	10.9 ± 0.2	–	–
	(04, 6, 0.16)	30	161.2 ± 1.4	30	142.9 ± 1.0	30	142.1 ± 1.7	30	63.7 ± 0.5	30	45.6 ± 0.3	–	–	27	1462.2 ± 79.6	–	–
	(05, 7, 0.01)	30	30.7 ± 0.2	30	28.1 ± 0.2	30	27.2 ± 0.3	30	25.8 ± 0.2	30	12.8 ± 0.1	30	82.7 ± 1.5	30	38.5 ± 0.6	30	451.0 ± 14.5
	(06, 8, 0.3)	30	300.4 ± 10.9	30	276.0 ± 3.1	30	276.7 ± 4.1	30	91.6 ± 0.6	30	48.2 ± 0.4	–	–	–	–	–	–
	(07, 8, 0.5)	30	0.3 ± 0.0	30	0.5 ± 0.0	30	0.5 ± 0.0	30	0.2 ± 0.0	30	0.2 ± 0.0	30	2.2 ± 0.1	30	0.9 ± 0.1	30	12.1 ± 0.4
C-ΔTW (ind, c)	(08, 8, 0.63)	30	35.1 ± 0.5	30	40.5 ± 0.8	30	41.1 ± 1.0	30	34.8 ± 0.4	30	35.5 ± 0.7	6	1799.2 ± 0.1	13	1691.9 ± 45.5	–	–
	(09, 8, 0.4)	30	177.9 ± 1.4	30	374.6 ± 3.3	30	374.2 ± 5.2	30	14.9 ± 0.1	30	24.5 ± 0.1	–	–	30	202.3 ± 7.5	–	–
	(0, 0, 1)	30	311.2 ± 13.6	30	183.7 ± 6.9	30	1.5 ± 1.4	30	270.9 ± 8.3	30	252.5 ± 8.7	30	126.2 ± 9.1	30	103.1 ± 2.4	30	42.8 ± 3.1
	(0, 0, ∞)	30	100.7 ± 0.8	30	52.1 ± 0.9	30	0.1 ± 0.0	30	141.3 ± 1.0	30	64.7 ± 0.5	30	47.1 ± 2.5	30	42.1 ± 1.8	30	29.5 ± 1.8
	(0, 2, 1)	30	600.7 ± 3.6	30	234.8 ± 3.1	13	184.2 ± 3.5	30	863.4 ± 9.7	30	337.1 ± 3.8	–	–	–	–	30	65.1 ± 11.1
C-ΔTW (ind, c)	(0, 2, ∞)	30	121.7 ± 1.9	30	91.0 ± 1.3	30	56.9 ± 0.6	30	185.7 ± 1.2	30	79.2 ± 0.6	30	1537.7 ± 72.8	–	–	30	41.7 ± 4.9
	(0, 4, 1)	30	882.8 ± 31.7	30	247.5 ± 3.8	30	241.8 ± 3.2	30	564.1 ± 12.8	30	308.2 ± 4.0	–	–	–	–	3	801.5 ± 536.2
	(0, 4, ∞)	30	130.9 ± 2.5	30	98.0 ± 1.8	30	94.0 ± 1.7	30	188.1 ± 1.3	30	82.0 ± 1.2	–	–	–	–	3	665.9 ± 308.3
	(0, 6, 1)	30	318.4 ± 10.3	30	186.0 ± 3.6	30	148.3 ± 2.3	30	255.4 ± 6.9	30	40.9 ± 0.3	26	1508.9 ± 68.9	–	–	–	–
	(0, 6, ∞)	30	25.5 ± 0.5	30	40.0 ± 0.9	30	40.4 ± 0.9	30	39.6 ± 0.3	30	40.9 ± 0.3	–	–	–	–	–	–
C-ΔTW (ind, c)	(0, 8, 1)	30	345.5 ± 7.4	30	185.3 ± 2.4	30	148.2 ± 2.2	30	267.1 ± 8.0	30	148.1 ± 2.2	–	–	–	–	–	–
	(0, 8, ∞)	30	25.7 ± 0.6	30	40.4 ± 1.1	30	40.8 ± 1.0	30	39.8 ± 0.3	30	41.3 ± 0.4	29	1375.1 ± 89.9	–	–	–	–
	(1, 0, 1)	30	208.1 ± 6.8	30	184.6 ± 2.2	30	150.3 ± 3.1	30	226.0 ± 5.4	30	148.6 ± 1.6	–	–	–	–	–	–
	(1, 0, ∞)	30	25.6 ± 0.6	30	40.5 ± 1.1	30	41.5 ± 1.1	30	39.6 ± 0.3	30	41.1 ± 0.4	30	1298.1 ± 55.3	–	–	–	–
	(4, 4, 2)	30	1.6 ± 0.1	30	2.7 ± 0.1	30	3.2 ± 0.1	30	1.8 ± 0.1	30	1.8 ± 0.1	30	4.5 ± 0.1	30	24.2 ± 0.7	30	24.3 ± 0.8
C-ΔTW (ind, c)	(4, 4, 4)	30	2.4 ± 0.0	30	4.6 ± 0.1	30	5.4 ± 0.1	30	3.2 ± 0.0	30	3.2 ± 0.0	30	8.2 ± 0.5	30	43.0 ± 1.0	30	36.1 ± 2.0
	(4, 4, 6)	30	3.1 ± 0.0	30	6.6 ± 0.1	30	7.4 ± 0.1	30	4.6 ± 0.1	30	4.7 ± 0.1	30	10.5 ± 0.4	30	56.0 ± 2.4	30	42.5 ± 1.5
	(4, 5, 2)	30	3.9 ± 0.2	30	6.3 ± 0.3	30	7.4 ± 0.3	30	4.2 ± 0.3	30	4.2 ± 0.2	30	18.7 ± 0.8	30	119.1 ± 5.5	30	115.3 ± 6.3
	(4, 5, 4)	30	5.9 ± 0.1	30	11.3 ± 0.3	30	13.4 ± 0.6	30	7.5 ± 0.1	30	7.6 ± 0.2	30	32.4 ± 1.0	30	239.5 ± 9.0	30	157.0 ± 5.3
	(4, 5, 6)	30	7.3 ± 0.3	30	15.9 ± 0.4	30	18.6 ± 0.5	30	10.3 ± 0.4	30	11.1 ± 0.5	30	41.7 ± 1.0	30	310.6 ± 14.7	30	194.5 ± 7.1
C-ΔTW (ind, c)	(4, 6, 2)	30	9.3 ± 0.5	30	14.9 ± 0.6	30	17.8 ± 0.8	30	10.1 ± 0.6	30	9.8 ± 0.6	30	90.9 ± 4.7	30	840.2 ± 47.1	30	937.7 ± 45.7
	(4, 6, 4)	30	13.9 ± 0.3	30	27.1 ± 0.8	30	31.3 ± 0.9	30	17.8 ± 0.3	30	18.5 ± 0.3	30	187.0 ± 5.9	13	1656.6 ± 76.8	30	1281.6 ± 64.0
	(4, 6, 6)	30	18.2 ± 0.4	30	37.7 ± 0.9	30	44.7 ± 0.9	30	24.9 ± 0.7	30	26.5 ± 0.6	30	225.1 ± 10.0	4	1424.3 ± 88.9	30	1238.9 ± 81.6

Table 7.2: For the benchmark problems, we show each planner and heuristic's coverage (out of 30) and over the converged runs the mean runtime (secs) with 95% C.I. The fastest planner and heuristic per problem are in bold. (†) CARL with C-POM does not have optimality guarantees due to inadmissibility.

as the constraints become looser, its heuristic becomes less effective. On problems with the loosest constraint $f \geq 0.6$ and $u = 1$, CARL completely dominates by achieving full coverage while the baselines obtain zero coverage.

What makes CARL so effective on our benchmarks? The part of CARL that takes the most time is solving $\mathbb{S}(\lambda)$ s and, over the solved instances, CARL only solves a mean and maximum of 7 and 30 $\mathbb{S}(\lambda)$ s in its search, respectively. We identify two domain features that make coordinate search efficient and let CARL solve so few subproblems: single secondary-cost constraint and sparse λ . In the former, coordinate search only has to solve a single coordinate and it is guaranteed to converge to λ^* . C-S&R, C-ExBW, and C-PARC problems with $u = \infty$ have a single constraint. It is convenient if $\lambda^* = 0$ because $\lambda = 0$ is the initial guess and coordinate search then only has to verify that its candidate is optimal. C- Δ TW has $\lambda^* = 0$ for almost all its problems. More generally, coordinate search tends to work well when λ^* is sparse because it is inexpensive to determine these, but this trend is not guaranteed and depends on the search space structure. Over the solved instances, λ^* is quite sparse having a mean and maximum of 0.62 and 2 non-zero entries. Note that if $C_i(\pi^*) < u_i$, i.e., a secondary-cost constraint is satisfied with some slack, then $\lambda_i^* = 0$ by complementary slackness, which suggests that many problems will have a sparse λ^* . In contrast, we have seen that particular problems are structured such that coordinate search fails to find λ^* , e.g., some C-Elev instances with a single elevator. Fortunately, these seem to appear infrequently. We emphasise that these benchmarks were taken from the existing literature on CSSPs, and were not hand-picked to accommodate CARL. This may suggest that problems are more often amenable to CARL than not.

How well does CARL scale with constraints? Coordinate search performs well on our problems (except on the C-Elev problems where it fails to converge). However, since it needs to optimise each coordinate separately, and solve the corresponding $\mathbb{S}(\lambda)$ s, it is clear that it will stop performing well as the number of secondary-cost constraints increases. To investigate this, we ran CARL, i-dual, and i^2 -dual with the same settings as before, on a suite of C- Δ TW problems (page 114), where the number of “currencies” and thereby the number of secondary-cost constraints increases. The results are shown in fig. 7.3. We omit i-dual(IP-LMcut) and i^2 -dual because they failed to scale and only solved the problems with 6 or fewer constraints; we also omit CARL with IP heuristics, because they performed worse than the λ heuristics. This plot shows that CARL performs better than i-dual with few constraints, but eventually i-dual overtakes it as the number of constraints increases. For CARL(λ -LMcut) the overtake happens around 40 constraints, and for CARL(λ -ROC) it happens around 65 constraints. This supports that coordinate search is ill-suited to problems with many constraints; fortunately for CARL, it is still suitable for up to 65 constraints, which is significantly larger than any of the domains we have encountered.

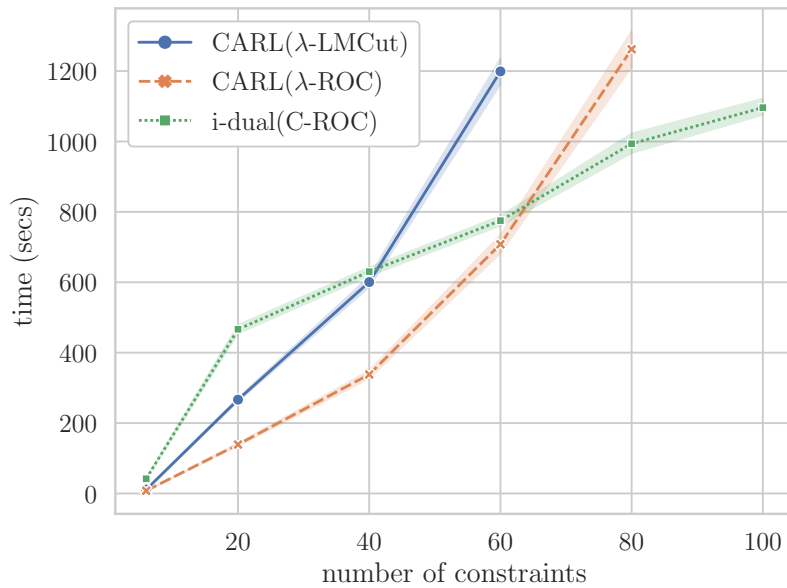


Figure 7.3: The mean and 95% C.I. of each planner’s time-to-solve as the number of constraints in the C- Δ TW problems increases. We only include times where the planner achieved full coverage.

7.7 Summary

This chapter introduced the CARL algorithm. It solves the CVLP (LP 12), which is a primal formulation that finds an optimal policy for CSSPs. However, instead of solving it directly, CARL breaks the LP down into an optimisation problem that is solved with a subgradient method, which in turn produces a sequence of unconstrained SSPs with scalarisation. We showed how the sequence of unconstrained SSPs can be solved efficiently, exploiting the techniques introduced in chap. 4. Then, we explained how, once CARL has obtained a solution to the CVLP, the resulting set of deterministic policies can be combined into an optimal (potentially stochastic) policy for the original CSSP. We presented various ways that CARL can be interpreted and understood intuitively, proved correctness, and demonstrated empirically that it outperforms the state-of-the-art.

I²-dual-det: Finding Deterministic Policies for CSSPs

This chapter is about finding deterministic-optimal policies for CSSPs. The foundational technique for finding deterministic-optimal policies for CSSPs is with a Mixed Integer Program (MIP) that was originally developed for Constrained MDPs (Dolgov and Durfee, 2005). A technical gap appears in the generalisation to CSSPs (see sec. 8.2). We identify this technical gap and present various techniques to address it. With the technical gap closed, we introduce the novel heuristic-search algorithm i²-dual-det for finding deterministic-optimal policies. Recall that i²-dual (sec. 6.3.2) finds optimal policies for CSSPs by iteratively solving COMLPs (LP 9); i²-dual-det uses the same framework, but uses a MIP for finding deterministic-optimal policies in place of the COMLP, resulting in an algorithm for finding deterministic-optimal policies. As another contribution of this chapter, we introduce a categorisation of how interesting CSSPs’ constraints are, separating CSSPs’ constraints into three groups: trivial, linearisable, and interesting, with the idea that interesting constraints make CSSPs more difficult, whereas trivial ones do not. These definitions are useful in comparing i²-dual-det with the state-of-the-art approaches.

We give a breakdown of this chapter:

- Sec. 8.1 describes the MIP for finding deterministic-optimal policies.
- Sec. 8.2 explains the MIP formulation’s technical gap for CSSPs, and how to address it.
- Sec. 8.3 describes the i²-dual-det algorithm.
- Sec. 8.4 introduces the categorisation of CSSP’s constraints.
- Sec. 8.5 evaluates i²-dual-det, and compares it to LAny with a theoretical comparison and empirical experiments.
- Sec. 8.6 summarises the chapter.

Parts of this chapter have been published in ECAI 2024 (Schmalz and Trevizan, 2024c).

8.1 Background: Finding Deterministic Policies with MIPs

LPs are not sufficient to find deterministic-optimal policies, and we need to use Mixed Integer Programs (MIPs) (Conforti, Cornuéjols, and Zambelli, 2014). The variables, constraints, and format of MIPs are similar to LPs, with the key difference that LPs require all variables to be continuous, whereas MIPs allow variables to be restricted to a discrete domain, e.g., $x \in \mathbb{Z}$ or $b \in \{0, 1\}$. This makes MIPs more expressive, and also more difficult to solve in general: while LPs are P-complete, MIPs are NP-complete (Conforti, Cornuéjols, and Zambelli, 2014), reflecting that finding optimal (potentially stochastic) policies for CSSPs is P-complete and finding deterministic-optimal ones is NP-complete (Feinberg, 2000).

A deterministic-optimal policy can be found with the Constrained Occupation Measure MIP (COM-MIP) (Dolgov and Durfee, 2005; Hong and Williams, 2023), as presented in MIP 12. The occupation

measure variables $\mathbf{x}$, the objective, and the constraints C52–C56 are identical to the COMLP (LP 9) for optimal (potentially stochastic) policies. As before, $x_{s,a}$ represents the expected number of times that the agent encounters state s , and then applies action a . To restrict solutions to deterministic policies, the COMMIP introduces the binary indicator variables $\Delta_{s,a}$ for each (s,a) , with the semantic $\Delta_{s,a} = \llbracket x_{s,a} > 0 \rrbracket$, i.e., $\Delta_{s,a}$ is 1 iff there is positive flow through the associated occupation measure $x_{s,a}$. Then, constraint C57 allows at most one action to be applied in each state, forcing the solution to induce a deterministic policy. The indicator variables' semantic is encoded by the constraints C58. Note that $M \in \mathbb{R}_{>0}$ is a large value, and C58 is a standard way to encode indicator variables, called the big- M approach. The COMMIP was initially presented by Dolgov and Durfee (2005) for Constrained Markov Decision Processes (CMDPs), and our COMMIP generalises it to CSSPs; in the setting of CMDPs it is straightforward to automatically derive a valid value of M , but their approach does not work for CSSPs, as we elaborate in the next section.

Constrained Occupation Measure MIP (COMMIP)

$$\begin{aligned}
 & \min_{\mathbf{x}, \Delta} \sum_{s \in S, a \in A(s)} x_{s,a} \cdot C_0(a) & (\text{MIP 12}) \\
 & out(s) - in(s) = \llbracket s = s_I \rrbracket & \forall s \in S \setminus G \quad (\text{C52}) \\
 & out(s) = \sum_{a \in A(s)} x_{s,a} & \forall s \in S \quad (\text{C53}) \\
 & in(s) = \sum_{s' \in S, a' \in A(s')} x_{s',a'} \cdot P(s|s',a') & \forall s \in S \quad (\text{C54}) \\
 & x_{s,a} \geq 0 & \forall s \in S \quad (\text{C55}) \\
 & \sum_{s \in S, a \in A(s)} x_{s,a} \cdot C_i(a) \leq u_i & \forall i \in \{1, \dots, n\} \quad (\text{C56}) \\
 & \sum_{a \in A(s)} \Delta_{s,a} \leq 1 & \forall s \in S \quad (\text{C57}) \\
 & x_{s,a} \leq M \cdot \Delta_{s,a} & \forall s \in S, a \in A(s) \quad (\text{C58}) \\
 & \Delta_{s,a} \in \{0, 1\} & \forall s \in S, a \in A(s) \quad (\text{C59})
 \end{aligned}$$

8.2 Technical Gap in MIPs: Selecting M

We have said that M in the COMMIP (MIP 12) is some large value $M \in \mathbb{R}_{>0}$, without giving any details on how to select it. It is important that M is selected appropriately, and it is not too small nor too large:

- If M is too small: the whole MIP may become infeasible, or more subtly, the MIP may remain feasible but the deterministic-optimal solution is made infeasible so that the MIP produces some feasible solution that is not deterministic-optimal.
- If M is excessively large: it can impact performance, introduce numerical instability, and may allow integrality constraints to be violated (called trickle flow), yielding non-deterministic policies.

This presents a dangerous situation, where the user may receive a deterministic-optimal solution, a suboptimal deterministic solution, a stochastic solution, or no solution at all, depending on the values of M . Selecting a valid M is often non-trivial, because it is difficult to account for the flow circulating through cycles in an SSP.

As mentioned before, the COMMIP was originally presented by Dolgov and Durfee (2005) for Constrained Markov Decision Processes (CMDPs), rather than CSSPs. CMDPs are very similar to CSSPs (inheriting the relationship between MDPs and SSPs), and the COMMIP can be applied to either formulation. However, the differences become significant for finding M . CMDPs typically have action rewards rather than action costs, and consequently try to maximise reward rather than minimising cost. More importantly: CMDPs have no goal states, and instead have a discount factor $\gamma \in [0, 1)$, with the semantic that an immediate reward is more valuable than a reward t steps in the future, which is penalised by the multiplicative term γ^t . A CMDP's optimal policy should maximise the expected discounted reward.

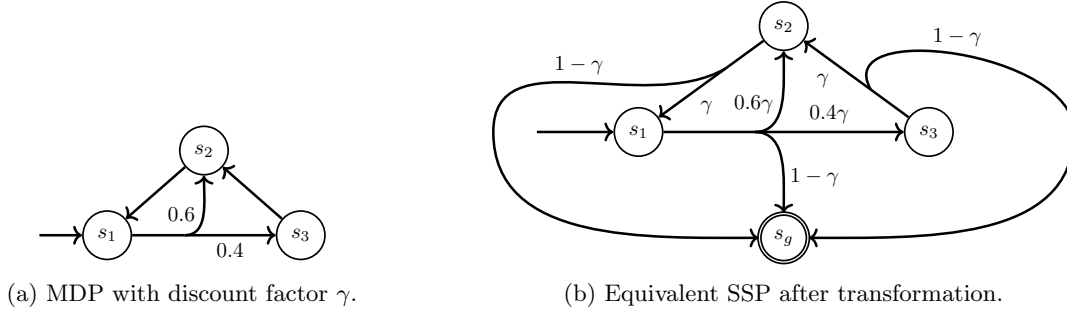


Figure 8.1: Example of transformation from MDP into equivalent SSP. Costs are omitted.

CMDPs are a special case of CSSPs, in the sense that CMDPs can be transformed into an equivalent CSSP (Bertsekas, 2019). We motivate this in fig. 8.1 by transforming the MDP in fig. 8.1a into the equivalent SSP in fig. 8.1b. In this example, we ignore costs and rewards, and focus on transition probabilities. We assume the MDP's initial state is s_1 with probability 1. In this transformation, the discount factor is represented by the artificial goal state s_g that is reached by all actions with a probability of $1 - \gamma$.

Dolgov and Durfee (2005) provide an automatic way to derive a valid M for their COMMIP in the context of CMDPs. They use an LP, which translates to LP 13 in the CSSP setting. This LP is identical to the OMLP for unconstrained SSPs, but the objective is changed to find the policy with maximum cumulative flow. It is clear that this LP's solution must yield an upper bound on the maximum $x_{s,a}$ for the OMLP, and consequently also gives an upper bound for the COMLP. However, it does not work for CSSPs because the LP may be unbounded.

LP for finding M for CSSPs (may be unbounded)	
$\max_{\mathbf{x}} \sum_{s \in S, a \in A(s)} x_{s,a}$	(LP 13)
$\text{s.t. } out(s) - in(s) = \mathbb{I}[s = s_I]$	$\forall s \in S \setminus G$ (C60)
$out(s) = \sum_{a \in A(s)} x_{s,a}$	$\forall s \in S$ (C61)
$in(s) = \sum_{s' \in S, a' \in A(s')} x_{s',a'} \cdot P(s s', a')$	$\forall s \in S$ (C62)
$x_{s,a} \geq 0$	$\forall s \in S$ (C63)

Consider the CSSP in fig. 8.2. The solution $x_{s_I, a_g} = 1$ and $x_{s_I, a_1} = x_{s_2, a_2} = y$ satisfies the LP for any $y \in \mathbb{R}_{\geq 0}$, which can be made arbitrarily large to maximise the objective, i.e., the LP is unbounded. Moreover, the solution $x_{s_I, a_g} = 1$ and $x_{s_3, a_3} = x_{s_4, a_4} = y$ satisfies the LP for any $y \in \mathbb{R}_{\geq 0}$, which again makes the LP unbounded. Thus, CSSPs with closed improper policies, even if these improper policies are unreachable from s_I , will have unbounded LP 13. The LP is guaranteed to be bounded for CMDPs. Recall that the transformation from CMDPs into CSSPs every action has a $1 - \gamma$ probability of reaching the artificial goal state, which means that every closed policy must also be proper, so all their closed policies are proper. Consequently, CMDPs can not have any improper policies, and LP 13 must always be bounded. Thus, we can not apply the results for CMDPs to CSSPs in this particular instance of finding M .

8.2.1 Finding M Automatically for CSSPs

In this section, we show how to find valid values of M without relying on domain knowledge. We start with a bound on M that applies to all CSSPs. This bound is impractically large, so we demonstrate more practical bounds for special cases and use this theory to develop an algorithm to find M automatically. Finally, we show how to avoid the problem of selecting M with special constraints.

The following terms will be useful for this section. For the CSSP $\mathbb{C}$, we define:

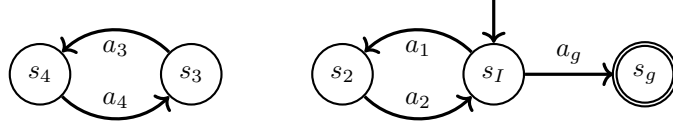


Figure 8.2: A CSSP where an improper policy exists and [Dolgov and Durfee \(2005\)](#)'s LP for finding M is unbounded. We omit costs because they do not affect the argument.

- $p_{\min} \stackrel{\text{def}}{=} \min\{P(s'|s, a) : s, s' \in \mathcal{S}, a \in \mathcal{A}, P(s'|s, a) > 0\}$ (smallest non-zero transition probability)
- $\bar{g} \stackrel{\text{def}}{=} \max_{a \in \mathcal{A}} C_0(a)$ (largest primary cost over actions)
- $\underline{g} \stackrel{\text{def}}{=} \min_{a \in \mathcal{A}} C_0(a)$ (smallest primary cost over actions).

Recall that we defined CSSPs with strictly positive primary costs $C_0(a) > 0 \forall a \in \mathcal{A}$ (defn. 33), so $\underline{g} > 0$, which will be important for our proofs. For $\mathbf{x}$ representing feasible solutions for $\mathbb{C}$'s COMMIP, we additionally define the term

- $x_{\max} \stackrel{\text{def}}{=} \max_{s \in \mathcal{S}, a \in \mathcal{A}} x_{s,a}$ (maximum occupation measure through solution $\mathbf{x}$).

With this terminology, a choice of M is sufficiently large if $M \geq x_{\max}^*$ for $\mathbf{x}^*$ associated with some deterministic-optimal policy π_{det}^* .

We start by giving a domain-independent formula for deriving an upper bound on $x_{\max}$ over all deterministic policies:

Theorem 14. *For any $\mathbb{C}$ and deterministic proper policy encoded by $\mathbf{x}$, $p_{\min}^{-|\mathcal{S} \setminus \mathcal{G}|}$ is an upper bound for $x_{\max}$. Moreover, $\mathbb{C}$ and $\mathbf{x}$ exist such that the bound is tight.*

Proof. To show this is an upper bound consider $\mathbf{x}$ that induces the deterministic proper policy π ; and for contradiction, suppose for some $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$ we have $x_{s,a} = x_{\max} = (1 + \epsilon) \cdot p_{\min}^{-|\mathcal{S} \setminus \mathcal{G}|}$ where $\epsilon \in \mathbb{R}_{>0}$. To derive a contradiction, we show that the amount of flow that reaches goals from s, a exceeds 1. Since π is proper, there exists an acyclic trajectory $\langle s_1, \dots, s_k \rangle$ with $s_1 = s$ and $s_k \in \mathcal{G}$. Observe that for $i \in \{1, \dots, k-1\}$ we have $\text{in}(s_{i+1}) \geq \text{out}(s_i) \cdot P(s_{i+1}|s_i, \pi(s_i))$ and $\text{out}(s_i) \geq \text{in}(s_i)$. We collapse these inequalities into $\text{in}(s_k) \geq \text{out}(s_1) \cdot \prod_{i=1}^{k-1} P(s_{i+1}|s_i, \pi(s_i)) \geq \text{out}(s_1) \cdot p_{\min}^{k-1}$. Recall that $s_1 = s$ and $x_{s,a} = x_{\max}$, so $\text{out}(s_1) \cdot p_{\min}^{k-1} = x_{\max} \cdot p_{\min}^{k-1} = (1 + \epsilon) \cdot p_{\min}^{k-1-|\mathcal{S} \setminus \mathcal{G}|}$. But, $k-1 \leq |\mathcal{S} \setminus \mathcal{G}|$ since an acyclic trajectory can pass through at most $|\mathcal{S} \setminus \mathcal{G}|$ states before encountering a goal. Therefore $(1 + \epsilon) \cdot p_{\min}^{k-1-|\mathcal{S} \setminus \mathcal{G}|} \geq 1 + \epsilon > 1$ which violates $\sum_{g \in \mathcal{G}} \text{in}(g) = 1$ (thm. 5), yielding the desired contradiction.

Now, we show the bound is tight by giving $\mathbb{C}$ and $\mathbf{x}$ where $x_{\max} = p_{\min}^{-|\mathcal{S} \setminus \mathcal{G}|}$. We give a CSSP with a single cost function, i.e., an SSP. The backwards-cycling chain SSP has $\mathcal{S} = \{s_0, \dots, s_k\}$; $s_0 = s_I$; $\mathcal{G} = \{s_k\}$; $\mathcal{A}(s_i) = \{a_i\}$ for $0 \leq i < k$; $P(s_{i+1}|s_i, a_i) = p_{\min}$ and $P(s_0|s_i, a_i) = 1 - p_{\min}$; and C is not important. We demonstrate this SSP with $k = 4$ in fig. 8.3. This SSP has a single solution $\mathbf{x}$ which satisfies $x_{s_0, a_0} = p_{\min}^{-|\mathcal{S} \setminus \mathcal{G}|}$ for any $k \in \mathbb{N}_{>0}$, which can be shown by straightforward induction proof. $\square$

A similar result was obtained independently by [Baier et al. \(2017\)](#) in the context of model-checking.

Thm. 14 gives us a way to obtain M for arbitrary CSSPs, but this term scales exponentially with the number of states, which becomes impractical for the reasons we discussed before. We now show methods to derive practical bounds, starting with some bounds on $x_{\max}$ that work in special cases. Note that the following bounds hold for stochastic and deterministic policies, but our focus is on deterministic policies.

1. If $\mathbb{C}$ is acyclic, then $x_{\max} = 1$.
2. Consider the case where each action incurs some secondary cost, i.e., $\sum_{i=1}^n C_i(a) > 0 \forall a \in \mathcal{A}$, and there are constraints associated with each of these costs. Then, we can construct the LP $\max_{\mathbf{x}} \sum_{s \in \mathcal{S}, a \in \mathcal{A}(s)} x_{s,a}$ s.t. C52–C56 that includes the secondary-cost constraints. This LP is bounded: if there were some $x_{s,a}$ that tended to infinity, then there must be i for which $\sum_{s \in \mathcal{S}, a \in \mathcal{A}} x_{s,a} C_i(a)$ tends to infinity and violates its constraint. The solution will yield an upper bound on $x_{\max}$.

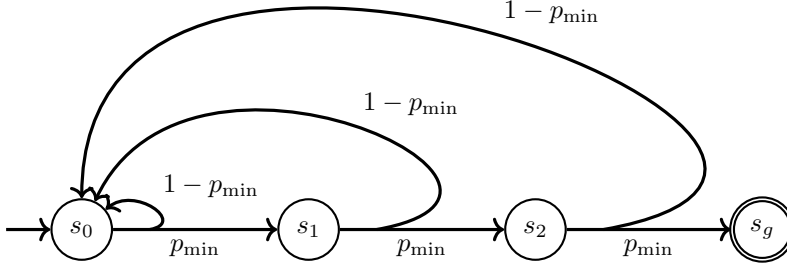


Figure 8.3: A backwards cycling chain SSP with four states.

Algorithm 8: Find M automatically

```

1 Function solve(MIP  $\mathcal{L}$ ,  $M_0 \in \mathbb{R}_{>0}$ ,  $\kappa \in \mathbb{R}_{>1}$ )
2   for  $i \in \{0, 1, \dots\}$ 
3      $\mathbf{x} \leftarrow$  optimise  $\mathcal{L}$  with  $M = \kappa^i \cdot M_0$ 
4     until  $\mathbf{x}$  is feasible
5     if  $\text{obj}(\mathbf{x}) \cdot \underline{g}^{-1} > M$  then
6        $\mathbf{x} \leftarrow$  optimise  $\mathcal{L}$  with  $M = \text{obj}(\mathbf{x}) \cdot \underline{g}^{-1}$ 
7   return  $\mathbf{x}$ 

```

3. If we have an upper bound U for the primary cost, we observe that COMMIP has the same optimal solution under the constraint

$$\sum_{s \in \mathcal{S}, a \in \mathcal{A}(s)} x_{s,a} \cdot C_0(a) \leq U \quad (\text{C64})$$

For this new MIP, the LP $\max_{\mathbf{x}} \sum_{s \in \mathcal{S}, a \in \mathcal{A}(s)} x_{s,a}$ s.t. C52–C55, C64 will give an upper bound on $x_{\max}$, and the LP must be bounded because primary action costs are strictly positive and C64 forces the LP’s primary cost to be bounded. Subsequently, this is a valid choice of M for COMMIP, with the important caveat that some solutions whose objectives exceed U may be infeasible under this M . In this sense, a smaller value of M can have a positive impact on the solving time by pruning suboptimal solutions.

4. Following on from the previous point, we observe that $U \geq \sum_{s \in \mathcal{S}, a \in \mathcal{A}(s)} x_{s,a}^* \cdot C_0(a) \geq \sum_{s \in \mathcal{S}, a \in \mathcal{A}(s)} x_{s,a}^* \cdot \underline{g} = \underline{g} \sum_{s \in \mathcal{S}, a \in \mathcal{A}(s)} x_{s,a}^*$, where $\mathbf{x}^*$ is the optimal solution. This lets us relate U , the upper bound on cost, to the flow of $\mathbf{x}^*$ thus: $U \cdot \underline{g}^{-1} \geq \sum_{s \in \mathcal{S}, a \in \mathcal{A}(s)} x_{s,a}^* \geq x_{\max}^*$. Note that action costs must be strictly positive for this bound to be finite.

We now exploit these insights to develop alg. 8, which solves the MIP and automatically finds a valid M . The idea is that we initialise M with some value $M_0 \in \mathbb{R}_{>0}$ that may be too small, and then we increase M exponentially until we find a feasible solution for the MIP. The rate with which M grows is controlled by the user-selected parameter $\kappa \in \mathbb{R}_{>1}$. When a feasible solution $\mathbf{x}$ is found, its objective $\text{obj}(\mathbf{x})$ is an upper bound U on the primary cost of the original MIP with an arbitrary M , so we can leverage special case 4 and produce a valid M with $M \leftarrow U \cdot \underline{g}^{-1}$; then we re-solve the MIP one last time to produce an optimal policy. Importantly, this algorithm only works if the input CSSP $\mathbb{C}$ has a feasible solution; if $\mathbb{C}$ is infeasible, alg. 8 will never terminate.

8.2.2 Avoiding M for CSSPs

We conclude this section by presenting a technique that avoids M altogether. Recall that the COMMIP searches for deterministic policies by modifying the COMLP (LP 9) with the additional constraints

C57–C59 encoding

$$x_{s,a} > 0 \text{ for at most one } a \in A(s)$$

with the big- M formulation. There is an alternative approach to encode this semantic with Special Ordered Sets of type 1 (SOS1) (Beale and Tomlin, 1969), a special type of constraint where an ordered set of continuous variables $\{x_0, \dots, x_k\}$ is specified, and the constraint requires that at most one x_i may have a non-zero value, i.e., one or zero x_i variables in the set are allowed to be non-zero. We emphasise that the variables $x_0, \dots, x_k$ are continuous, so a SOS1 constraint is **not** equivalent to $\sum_{i=0}^k x_i \leq 1$. Now, we can replace the big- M constraints C57–C59 with the corresponding SOS1 constraints

$$\{x_{s,a} : a \in A(s)\} \text{ for } s \in S.$$

Note that the order of states can be chosen arbitrarily within the ordered set.

SOS1 constraints are a standard feature in LP and MIP solving software, e.g., Gurobi and CPLEX support them. Solvers enforce that SOS1 constraints are satisfied in one of two ways:

1. branch and bound;
2. automatic big- M reformulation.

The branch-and-bound approach legitimately avoids M completely and works similarly to the branch-and-bound method of MIP solvers (Conforti, Cornuéjols, and Zambelli, 2014), i.e., it solves sequence of LP relaxations, branching into multiple new LP problems whenever a SOS1 constraint is violated. Thus, even though an LP with SOS1 constraints is not technically a MIP, solving it has very similar behaviour. With the second technique, solvers simply reformulate the SOS1 constraints into the big- M constraints we have already seen, deriving M automatically from the MIP. In this case, we have avoided M in the sense that we do not need to select it and the solver computes it automatically, but it is still in the formulation. Unfortunately, these solvers are closed source so it is not clear how they derive M , and whether their derivation gives any insight into upper bounds for CSSPs. In our implementation, we did not find significant performance difference between the two approaches, and we let the solver select whichever method it prefers automatically.

For the remainder of this chapter, the COMMIP will refer to MIP 12 with the big- M formulation, or with SOS1 constraints interchangeably.

8.3 I^2 -dual-det

Recall that solving the COMLP directly is impractical for large problems, resulting in the development of i-dual and i^2 -dual which perform a kind of constraint and variable generation over the COMLP. The COMMIP scales even worse to large problems because MIPs are (generally) more difficult to solve than LPs. Thus, an obvious next step is to redefine i^2 -dual over the COMMIP. We do this now, yielding the new algorithm i^2 -dual-det. Then, we explain why this algorithm is correct, and show some techniques for making it more efficient. Finally, we briefly discuss how i^2 -dual-det relates to an existing algorithm.

I^2 -dual-det. Recall the pseudocode for i^2 -dual (alg. 5), and the i^2 -dual LP which combines the COMLP (LP 9) with the LP for computing C-POM (LP 11). I^2 -dual-det extends the i^2 -dual LP by constraining it to deterministic policies (either with the big- M formulation or SOS1 constraints), which we call the i^2 -dual-det MIP. Then, i^2 -dual-det is described by i^2 -dual’s pseudocode alg. 5, but instead of solving the i^2 -dual LP in line 6, it solves the respective i^2 -dual-det MIP. Thus, i^2 -dual-det iteratively solves partial CSSPs by finding their deterministic-optimal policy, and grows the partial CSSP for the next iteration by expanding artificial goals that are reachable with the candidate policy, until the partial CSSP’s deterministic-optimal policy is also deterministic-optimal for the original CSSP.

Importantly, the i^2 -dual-det MIP only adds constraints that affect the COMLP (to restrict to deterministic policies), and does not affect the C-POM LP. Consequently, the C-POM component still computes admissible heuristics for stochastic policies; since deterministic policies are no cheaper than stochastic ones, we know that C-POM must be admissible in the new setting of deterministic policies. Then, i^2 -dual-det inherits its guarantee of optimality from i-dual and i^2 -dual.

Theorem 15. *Given a CSSP $\mathbb{C}$ for which a deterministic-optimal policy exists, i^2 -dual-det terminates and returns a deterministic-optimal policy for $\mathbb{C}$.*

Proof. This follows directly from the proof of i-dual (Trevizan et al., 2016). We start by showing that i^2 -dual-det produces a closed feasible policy. Observe that i^2 -dual-det, which follows the framework of i-dual, only terminates once its candidate policy π has no fringes in its envelope S^π (alg. 4 line 4), i.e., $S^\pi \cap \widehat{G} \subseteq G$. The COMMIP produces proper policies for its CSSP (thm. 4); in this case, we are dealing with a partial CSSP but the produced policy π has no fringes in its envelope, so π can only encounter states from the original CSSP and must therefore be closed w.r.t. the original CSSP. Furthermore, since the solution contains no artificial goals and $\sum_{s \in S, a \in A(s)} x_{s,a} \cdot C_i(a) \leq u_i \forall i \in \{1, \dots, n\}$ by C56, it must be the case that π satisfies the original CSSP’s secondary-cost constraints. Thus, π is a closed feasible policy. Next, we argue that each partial CSSP’s COMMIP has $\text{obj}(\mathbf{x}) \leq C_0(\pi^*)$. This follows because we inherit the C-POM heuristic’s admissibility from i^2 -dual (Trevizan, Thiébaux, and Haslum, 2017). Thus, we have a closed policy $\pi = \pi_{\mathbf{x}}$ with $C_0(\pi) = \text{obj}(\mathbf{x}) \leq C_0(\pi^*)$, and it is feasible w.r.t. the secondary-cost constraints, so indeed i^2 -dual-det’s π must be optimal. $\square$

Note that, if the CSSP has no deterministic-optimal policy, i.e., all its deterministic policies are infeasible, then i^2 -dual-det will terminate and assert that no solution exists as soon as one of its partial CSSPs can not be solved with the COMMIP.

Now, we discuss some techniques for improving i^2 -dual-det’s performance. One important implementation detail for i^2 -dual-det is that solving the MIP for the current $\widehat{C}$ should use the solution from the previous iteration as a warm-start. This significantly speeds up search because modern MIP solvers can leverage such warm-starts to find a new solution faster, e.g., they can, in some cases, transform the old solution into a feasible solution for the new problem. Note that similar warm-starts were used in the LPs for i-dual (Trevizan et al., 2016) and i^2 -dual (Trevizan, Thiébaux, and Haslum, 2017). Another technique for speeding up i^2 -dual-det is to solve relaxations of the intermediate partial CSSPs. An optimal solution for $\widehat{C}$ is not necessary to expand useful fringes, and an approximate solution obtained from a relaxation can suffice. Of course, in the final steps, we must solve $\widehat{C}$ optimally to ensure i^2 -dual-det’s optimality, but considerable time can be saved by approximating the intermediate partial CSSPs. Note that this is analogous to the improvement from LAO* to iLAO* (Hansen and Zilberstein, 2001). We present two approximation techniques that use LP relaxations of the MIPs, and a third method that is compatible with the first two and is based on optimality gaps.

1. Solve LP relaxations of each MIP and when a closed policy is found, solve the MIPs. In other words, run i^2 -dual first, then use its search tree and solution in i^2 -dual-det. This was done by Hong and Williams (2023) for i-dual (Trevizan et al., 2016).
2. Start by considering LP relaxations. Solve relaxations of each MIP and when a closed policy π is found determine at which states π is stochastic, then add determinising constraints only at these states. Repeat these steps until the policy is deterministic. This is effectively constraint generation.
3. MIP solvers keep track of the optimality gap, which is the gap between its lower and upper bounds. Usually, solvers run until the gap is close to zero in order to solve the problem optimally (up to a small error). In contrast, we relax our MIPs by allowing the solver to stop early with a larger optimality gap. Once i^2 -dual-det finds a feasible solution with no fringe states but a large optimality gap, it has found a closed feasible policy for the CSSP. At this stage, we impose that the optimality gap must be close to zero, forcing future solutions to be optimal, and continue running i^2 -dual-det with the feasible solution we found as a warm start.

I^2 -dual-det can be implemented with item 1, item 2, or neither; and in parallel, any of these variants may use item 3. Note that item 3 lets i^2 -dual-det behave like an anytime algorithm, since it can return closed feasible policies before termination, and the MIP’s optimality gap directly gives an optimality gap for the policy. For maximum performance, the key is to find a balance between finding a solution quickly, obtaining a sufficiently informative solution so that useful fringes are expanded, and maximising how effectively the solver can reuse information from the previous iteration with warm starts.

To conclude this section, we note that a similar generalisation of i^2 -dual was independently developed by Steinmetz et al. (2024) to find deterministic policies for SSPs constrained by policy properties. Their problem is more general than CSSPs, so their version of i^2 -dual can also be used to find deterministic policies for CSSPs. In this sense, our method i^2 -dual-det can be considered a special case of their algorithm. However, they only consider the big- M formulation for enforcing deterministic policies with a manually selected M , and do not consider the performance improvements discussed in this chapter.

This means that our theory and techniques from sec. 8.2 and the improvements in this chapter are novel contributions. Rather than devaluing I^2 -dual-det, the existence of their more general algorithm makes the contributions of this chapter more valuable because our results can be applied in their general setting.

8.4 How Interesting Are a Problem's Constraints?

In the context of deterministic-optimal policies, secondary-cost constraints generally make the problem more difficult; recall that finding optimal deterministic (and therefore deterministic-optimal) policies for unconstrained SSPs is a P-complete problem, whereas finding deterministic-optimal policies for CSSPs is NP-complete (Feinberg, 2000). On the hand, consider a CSSP with secondary-cost constraints that can not be violated by proper policy. These constraints do not restrict the space of feasible policies, and can be ignored, i.e., this CSSP can be solved as an SSP. Clearly, there are some constraints that are interesting and make the problem harder, and there are other constraints that are uninteresting and can be ignored.

We use the concept of probabilistically interesting problems (Little and Thiebaux, 2007) as an analogue. SSPs are probabilistically uninteresting if a good policy can be obtained by ignoring the SSP's probabilities, and solving it as a deterministic problem. For example, Blocks World (page 47) is probabilistically uninteresting: the problems can be solved by following a plan over the all-outcomes determinisation (defn. 22), and whenever the agent reaches an unexpected state it can replan over the all-outcomes determinisation. This is what the determinisation-based algorithms FF-Replan (Yoon, Fern, and Givan, 2007) and Robust-FF (Teichteil-Königsbuch, Kuter, and Infantes, 2010) rely on. In contrast, the Triangle Tireworld (page 49) is probabilistically interesting, because an agent that ignores probabilities will get stuck in dead-end states with a flat tyre. We apply similar arguments to the secondary-cost constraints of CSSPs: the constraints are uninteresting if they can be ignored or avoided, and interesting if they must be considered carefully to produce a solution.

For our categorisation, we make use of the concept of Lagrangian SSPs $\mathbb{S}(\boldsymbol{\lambda})$ (defn. 42). Recall that for a CSSP $\mathbb{C}$, its $\mathbb{S}(\boldsymbol{\lambda})$ is an SSP relaxation where the constraints have been pushed into the objective, and the violation (satisfaction) of the i^{th} secondary-cost constraints is penalised (rewarded) by λ_i . We categorise how interesting a CSSP's constraints are into three groups:

1. Trivial: $\mathbb{C}$'s constraints are trivial if they can be completely ignored, and an optimal policy for the SSP relaxation is feasible for $\mathbb{C}$, i.e., the optimal policy for $\mathbb{S}(\boldsymbol{\lambda})$ with $\boldsymbol{\lambda} = 0$ is feasible for $\mathbb{C}$. Note that the SSP relaxation's optimal policy π^* is immediately optimal for $\mathbb{C}$, if π^* is feasible for $\mathbb{C}$.
2. Linearisable: $\mathbb{C}$'s constraints are linearisable if there exists some Lagrangian multiplier $\boldsymbol{\lambda} \in \mathbb{R}_{\geq 0}^n$, such that its $\mathbb{S}(\boldsymbol{\lambda})$ has an optimal policy $\pi_{\boldsymbol{\lambda}}^*$ which is feasible for $\mathbb{C}$. We do **not** require that all of $\mathbb{S}(\boldsymbol{\lambda})$'s optimal policies are feasible for $\mathbb{C}$, only that at least one is feasible. Among $\mathbb{S}(\boldsymbol{\lambda})$'s optimal policies $\pi_{\boldsymbol{\lambda}}^*$ that are feasible for $\mathbb{C}$, we do **not** require that $\pi_{\boldsymbol{\lambda}}^*$ is optimal for $\mathbb{C}$. Note that trivial constraints are linearisable with $\boldsymbol{\lambda} = 0$.
3. Interesting: $\mathbb{C}$'s constraints are interesting if they are not linearisable.

We argue that interesting constraints are indeed more interesting than linearisable ones. Intuitively, constraints are more interesting if they interact with the primary cost and each other, i.e., satisfying a constraint either increases the primary cost, or makes it more difficult to satisfy another constraint. Respectively, constraints are less interesting if they can be ignored. In linearisable problems, the constraints are “aligned” in the sense that we can satisfy them by finding the correct $\boldsymbol{\lambda}$ and then solving $\mathbb{S}(\boldsymbol{\lambda})$. If the problem is not linearisable, then the constraints are interacting among themselves in a more complicated way, and there is no way to simplify them out of the problem.

To give an intuition for the different categories, we give an example of CSSPs with linearisable and interesting constraints. We do not give an example for trivial constraints because these are simple to imagine, e.g., consider a CSSP where all secondary costs are 0 or the upper bounds $\mathbf{u}$ are so large that they can never be exceeded by accumulating secondary costs; in these cases the secondary-cost constraints can be ignored.

Example 5 (Linearisable CSSP). *Consider the CSSP in fig. 8.4a. It has the deterministic policies $\pi_i = \{s_0 \mapsto a_i\}$ for $i \in \{0, 1, 2\}$. It is constrained by $u_1 = u_2 = 2$, so the policy π_0 is feasible, π_1 is deterministic-optimal, and π_2 is not feasible. The CSSP is linearisable because a $\mathbb{S}(\boldsymbol{\lambda})$ exists such that its optimal policy is feasible, for example, with $\boldsymbol{\lambda} = [0 \ 1]$ the policy π_0 is optimal for $\mathbb{S}(\boldsymbol{\lambda})$ with*

(a) Linearisable CSSP with $\mathbf{u} = [2 \ 2]$ (ex. 5).(b) Interesting CSSP with $\mathbf{u} = [10 \ 10]$ (ex. 6).

Figure 8.4: Example CSSPs with linearisable and interesting constraints. All their actions are deterministic.

$C_{\lambda}(\pi_0) = 3 + 0(1 - u_1) + 1(0 - u_2) = 1$. Note that π_0 is indeed feasible, but not deterministic-optimal. We plot the cost functions C_0 and C_2 for each policy in fig. 8.5a, and this shows visually that π_0 (a feasible policy) minimises in the direction of $\lambda = [0 \ 1]$, confirming that the CSSP is linearisable.

Example 6 (Interesting CSSP). Consider the CSSP in fig. 8.4b. This CSSP has $u_1 = u_2 = 10$, and the only deterministic policy that is feasible is $\pi_{det}^* = \{s_0 \mapsto a_1\}$. Importantly, there is no λ such that π_{det}^* is an optimal policy for $\mathbb{S}(\lambda)$, making this CSSP's constraints interesting. To see this, observe that as $\lambda = [\lambda_1 \ \lambda_2]$ varies, an optimal policy π_{λ}^* for $\mathbb{S}(\lambda)$ has

$$\begin{aligned} C_{\lambda}(\pi_{\lambda}^*) &= \min\{1 + (0 - u_1)\lambda_1 + (11 - u_2)\lambda_2, \\ &\quad 2 + (9 - u_1)\lambda_1 + (9 - u_2)\lambda_2, \\ &\quad 1 + (11 - u_1)\lambda_1 + (0 - u_2)\lambda_2\} \\ &= \min\{1 - 10\lambda_1 + \lambda_2, \\ &\quad 2 - \lambda_1 - \lambda_2, \\ &\quad 1 + \lambda_1 - 10\lambda_2\} \end{aligned}$$

where $C_{\lambda}(\pi_{det}^*)$ is the middle term $2 - \lambda_1 - \lambda_2$, and this term never attains the minimum for $\lambda_1, \lambda_2 \geq 0$. In order to do so, it would have to simultaneously satisfy

$$\begin{aligned} 1 &\leq -9\lambda_1 + 2\lambda_2 \\ 1 &\leq 2\lambda_1 - 9\lambda_2 \end{aligned}$$

which is impossible for $\lambda_1, \lambda_2 \geq 0$. We plot the cost functions C_0 and C_2 for each policy in fig. 8.5b, and this confirms visually that there is no $\lambda = [0 \ x]$ with $x \in \mathbb{R}_{\geq 0}$ so that the only feasible policy π_{det}^* (represented by a star) minimises in the direction of λ . Note that the plot only gives information for λ with $\lambda_1 = 0$ because this is a 2D plot with C_1 missing, but it motivates why there is no suitable $\lambda \in \mathbb{R}_{\geq 0}^2$ in general for this CSSP.

To conclude the discussion of constraint interestingness for deterministic-optimal policies we show that CSSPs with a single secondary-cost constraint are always linearisable.

Theorem 16. If a CSSP $\mathbb{C}$ has one secondary-cost constraint, then its constraint is linearisable.

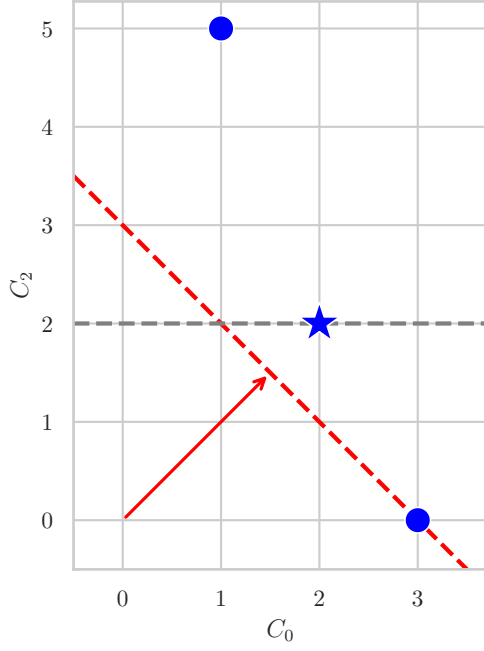
Proof. Consider a CSSP $\mathbb{C}$ with an arbitrary feasible policy π , and an arbitrary infeasible policy π_{inf} which satisfy

$$C_1(\pi) = u_1 - \alpha \text{ for } \alpha \in \mathbb{R}_{\geq 0} \text{ and } C_1(\pi_{inf}) = u_1 + \beta \text{ for } \beta \in \mathbb{R}_{> 0}.$$

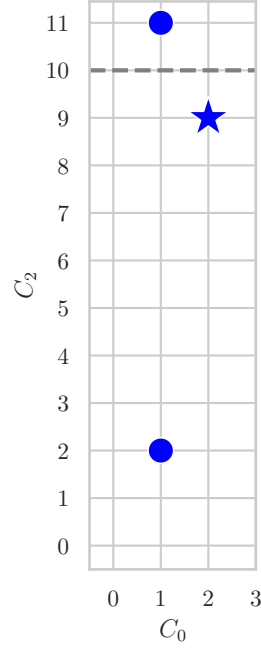
We will show that there is some $\lambda = [\lambda_1]$ such that π is a cheaper policy for $\mathbb{S}(\lambda)$ than π_{inf} . On $\mathbb{S}(\lambda)$, the policies' costs are

$$\begin{aligned} C_{\lambda}(\pi) &= C_0(\pi) + \lambda_1(C_1(\pi) - u_1) = C_0(\pi) - \lambda_1\alpha \\ C_{\lambda}(\pi_{inf}) &= C_0(\pi_{inf}) + \lambda_1(C_1(\pi_{inf}) - u_1) = C_0(\pi_{inf}) + \lambda_1\beta. \end{aligned}$$

Then, $C_{\lambda}(\pi) < C_{\lambda}(\pi_{inf})$ iff $\lambda_1 > \frac{C_0(\pi) - C_0(\pi_{inf})}{\alpha + \beta}$. Thus, for any feasible and infeasible policies there is always some $\mathbb{S}(\lambda)$ where the feasible policy is cheaper, so in particular there must be a $\mathbb{S}(\lambda)$ whose cheapest policy is feasible for $\mathbb{C}$. $\square$



(a) For linearisable CSSP (ex. 5).



(b) For interesting CSSP (ex. 6).

Figure 8.5: Costs C_0 and C_2 of the deterministic policies for linearisable and interesting CSSPs (ex. 5 and ex. 6). Blue circles (stars) represent the costs of deterministic (deterministic-optimal) policies, and the dashed grey line denotes u_2 for the secondary-cost constraint over C_2 . In (a) the red arrow represents the optimisation direction given by $\lambda = [0 \ 1]$ whose minimising policy is CSSP-feasible, and the dashed red line represents the associated optimality contour.

In other words, it is always possible to find a sufficiently large λ_1 so that infeasible policies are penalised so much that feasible ones are always cheaper. Note that there are pathological examples where λ_1 needs to be very large, which can cause numerical issues in practice.

Relation to Stochastic Policies. We have defined trivial, linearisable, and interesting constraints in the context of deterministic policies for CSSPs. Although these definitions can be applied to stochastic policies, the concept of linearisability does not make sense. Our definition requires that some $\mathbb{S}(\lambda)$ has an optimal policy π_λ^* that is feasible for the original CSSP $\mathbb{C}$; but with stochastic policies it does not make sense to require a single deterministic policy to be feasible. We could generalise linearisability to stochastic policies such that $\mathbb{C}$'s constraints are linearisable if there is some $\mathbb{S}(\lambda)$ whose optimal deterministic policies can be combined into a feasible policy; however, recall from chap. 7 that, given the Lagrangian dual λ^* , an optimal stochastic policy is constructed via a convex combination of the optimal deterministic policies for $\mathbb{S}(\lambda^*)$; thus, any solvable CSSP would be linearisable. So, the interestingness of constraints requires a different treatment in the context of stochastic policies.

8.5 Evaluation of i^2 -dual-det

8.5.1 Theoretical Comparison to LAny

Optimal vs Anytime. We define an algorithm to be optimal if it is guaranteed to eventually terminate with a proven optimal solution (up to ϵ); and anytime if it can produce feasible incumbent solutions and improves on them over time. i^2 -dual-det and all its variants are optimal. By default, i^2 -dual-det is not anytime, but variants that allow large optimality gaps for intermediate partial CSSPs, as per item 3, are anytime. LAny is both optimal and anytime.

Use of Heuristics. Both LAny and i^2 -dual-det use heuristic functions to guide their search, but in fundamentally different ways. Recall that LAny uses heuristic search to solve its $\mathbb{S}(\lambda)$ subproblems, in particular it uses a modified version of iLAO* that operates on the vector value function $\mathbf{V}$ with λ

Bellman backups (defn. 43) and requires some heuristic vector $\mathbf{H}$ that is admissible for the unconstrained SSP $\mathbb{S}(\lambda)$ with $[1 \ \lambda] \cdot \mathbf{H}(s) \leq V_\lambda^*(s)$ for all $s \in \mathcal{S}$. In chap. 7 we presented two ways to obtain such an admissible heuristic $\mathbf{H}$, namely, with λ heuristics (defn. 44) and Ideal-Point (IP) heuristics (defn. 45). Such heuristics have two key properties: (1) they are SSP heuristics and can not take into account the secondary-cost constraints, (2) they are based on standard heuristics that are computed externally and separately from the search algorithm. i^2 -dual-det differs in both of these points. Recall that i^2 -dual-det uses the built-in C-POM heuristic, which does take into account the secondary-cost constraints, and in fact feeds its partial CSSP's candidate policy costs into the C-POM heuristic to improve the heuristic's ability to prune infeasible solutions. Thus, i^2 -dual-det generally has much better-informed heuristics than LAny.

Infeasible Problems. We have implicitly assumed that CSSPs always have some feasible proper deterministic policy, adapting the reachability assumption for stochastic policies (assumption 4) into the deterministic setting; we briefly relax this assumption and consider infeasible CSSPs, i.e., CSSPs that have no feasible proper deterministic policy. Assuming that the underlying SSP still satisfies reachability (assumption 1), the CSSP can become infeasible if the secondary-cost constraints are too aggressive with $\mathbf{u}$ terms that are too small. Both LAny and i^2 -dual-det can handle infeasible CSSPs, and will eventually terminate, asserting that the CSSP is indeed infeasible. LAny must solve the Lagrangian dual (stage 1), and then enumerate every deterministic policy for $\mathbb{S}(\lambda^*)$ in its stage 2. Once all policies have been enumerated, and none of them are feasible, LAny can assert that the CSSP is infeasible. LAny has no mechanism for stopping before all policies have been enumerated. This is extremely inefficient, and can only be used for small problems. In contrast, i^2 -dual-det detects that the CSSP is infeasible as soon as any of its partial CSSPs are infeasible.¹ In the worst case, this occurs after i^2 -dual-det expands every state and its partial CSSP is the original CSSP, which can still be exponentially fewer steps than LAny's enumeration of all policies. In practice, thanks to the guidance and pruning power of its integrated C-POM, i^2 -dual-det proves infeasible CSSPs much sooner. To motivate this, we tested the two algorithms on an infeasible C-ExBW problem with 3 blocks (page 114). i^2 -dual-det proved that the problem is infeasible within a second, and LAny was stopped after enumerating 3000 policies over 30 mins, without having proved infeasibility.

Behaviour on Different Categories of Constraints. LAny's behaviour is highly dependent on the CSSP's constraints.

- **Trivial constraints.** LAny solves $\mathbb{S}(\lambda)$ for $\lambda = 0$ as the very first step and checks whether the ensuing policy is feasible for the CSSP. With trivial constraints, an optimal deterministic policy π_0 for $\mathbb{S}(0)$ must indeed be feasible for the CSSP, and it is therefore immediately a deterministic-optimal policy, so LAny can terminate and return it. Thus, LAny handles trivial constraints very efficiently and only needs to solve a single SSP.
- **Linearisable constraints.** In this category, LAny can have a range of different behaviours. During its Lagrangian search (stage 1), LAny may find a deterministic-optimal policy, it may find a suboptimal feasible policy, or it may not find a feasible policy at all. If it finds a deterministic-optimal policy then it can only prove optimality during stage 1 if strong duality holds; in all other cases, LAny is forced to close the gap (stage 2). The time spent in LAny stage 2 depends on how well $\mathbb{S}(\lambda^*)$ approximates the CSSP, and how good the initial incumbent solution is. As we will see in sec. 8.5.2, linearisable constraints are generally amenable to LAny; LAny often finds a good incumbent solution and does not require much work in stage 2 to produce an optimal one.
- **Interesting constraints.** LAny can not find any feasible solutions for the CSSP while solving the Lagrangian dual (stage 1), because none of its $\mathbb{S}(\lambda)$ subproblems will have optimal policies that are feasible for the CSSP. Consequently, LAny must close the gap (stage 2) without an initial incumbent value, which is generally inefficient.

In contrast, i^2 -dual-det is less dependent on CSSP constraints. It does not have any ways to exploit trivial or linearisable constraints, but it is also not disadvantaged by them. However, i^2 -dual-det does have an advantage for problems with difficult-to-satisfy constraints in the sense that its MIP that combines the COMMIP and C-POM is able to prune policies that violate constraints quite effectively, reducing the search space.

¹Recall that i^2 -dual-det with alg. 8 for finding a valid M does not terminate for infeasible CSSPs. The claim about i^2 -dual-det holds for all other implementations of i^2 -dual-det.

8.5.2 Empirical Evaluation

In this section, we experimentally compare our new algorithm i^2 -dual-det with LAny (Hong and Williams, 2023), the current state-of-the-art for finding optimal deterministic policies for CSSPs; and COM-MIP (Dolgov and Durfee, 2005), as a baseline. All algorithms are implemented in C++ and the code is available at (Schmalz and Trevizan, 2024a). For solving LPs and MIPs we use CPLEX Version 22.1.1. To decide which planner settings to use, we ran preliminary experiments testing various setting combinations over a subset of the benchmarks with three instances per problem, with 45 problem instances in total. We now list planner-specific settings:

I^2 -dual-det. We consider two variants of i^2 -dual-det:

1. i^2 -dual-det- M uses alg. 8 with $M_0 = 1, \kappa = 10$, and only makes intermediate partial CSSPs easier to solve with the optimality gap approach (item 3), with a value of 1.0; and
2. i^2 -dual-det-SOS1 uses SOS1 constraints, solves the LP relaxation of intermediate partial CSSPs (item 1), and then solves intermediate MIPs with an optimality gap requirement of 0.2 (item 3).

These were selected as best performers from preliminary experiments where we tested i^2 -dual-det with combinations of

- ways to deal with big- M (sec. 8.2): either with SOS1 constraints or with alg. 8 and its various settings;
- different relaxations of the intermediate partial CSSPs (sec. 8.3): solve LP relaxations until a closed policy is found (item 1), constraint generation for integrality constraints (item 2), and solving MIPs with a large optimality gap (item 3).

I^2 -dual-det with simultaneously the first and last relaxations (item 1 and item 3) solved 39-45 instances, and otherwise solved 23. The method for handling big- M did not make a significant difference in these tests, suggesting that the most important factor for i^2 -dual-det’s performance is the method for relaxing intermediate partial CSSPs. Note that i^2 -dual-det always uses the C-POM heuristic, which is embedded (Trevizan, Thiébaux, and Haslum, 2017), and additionally uses h-max as a dead-end detector on problems with dead ends before applying the fixed-penalty transformation.

LAny. We consider LAny (Hong and Williams, 2023) equipped with two heuristics:

1. LMCut (Helmert and Domshlak, 2009) and
2. POM with h-max as a dead-end detector (Trevizan, Thiébaux, and Haslum, 2017).

Both are used as Ideal-Point (IP) Heuristics (defn. 45). The algorithm’s error tolerances are set to 0.0001. LAny relies on an upper bound on each λ_i , which can be selected manually, but we use the automatic method used by the algorithm’s authors in their experiments. To select the remaining settings, we performed preliminary experiments with combinations of the following parameters, which were suggested by the authors of LAny:

1. the number of iterations in stage 1 is either limited to n iterations, where n is the number of secondary-cost constraints, or it is unrestricted;
2. we tested the exact version of the algorithm, as described in sec. 6.3.3, as well as the two approximate versions with “candidate pruning” (Hong and Williams, 2023).

The number of iterations in stage 1 did not have an effect on coverage. The approximate variants solved fewer instances fully and performed slightly worse in the anytime setting. So, in the full experiments, we only consider the exact version of LAny with LMCut and an unrestricted number of iterations in stage 1. We also tested LAny with ROC, but found that LAny(LMCut) solved 31-39 instances and LAny(ROC) solved 23-31 problems, so we do not consider ROC. Note that we consider LAny without the improvements from chap. 7 that speed up the search for λ^* ; including these changes would not affect results significantly because LAny’s bottleneck is its stage 2 where it enumerates policies, and its stage 1 for finding λ^* is already fast for the problems considered. Furthermore, we only consider IP heuristics. It would be interesting to consider λ heuristics, but based on the results from CARL (sec. 7.6), it would not change the conclusions of our results.

COMMIP. We implemented this as [MIP 12](#) with SOS1 constraints.

For benchmarks, we use the domains from sec. 7.6.2. As before, we apply the fixed-cost penalty formulation for dead ends (defn. 35) to ensure that all problems satisfy reachability (assumption 4). We give a brief reminder of each domain and give additional information that is relevant for this chapter.

Constrained Search and Rescue (C-S&R) (page 114). Multiple survivors are scattered on an $n \times n$ grid ($n \in \mathbb{N}$); the task is to save a survivor as quickly as possible while ensuring the drone does not run out of fuel. C-S&R problems have the single secondary-cost constraint that fuel consumption should not exceed a given threshold. Since they have a single constraint, **C-S&R problems are linearisable** (thm. 16). In the following experiments, we also consider problems where these thresholds are halved, denoted $h = T$ when halved and $h = F$ when original.

Elevators (C-Elev) (page 114). The agent must route $e \in \mathbb{N}$ elevators through a 10-floor building so that all passengers arrive at their destinations. There are $w \in \mathbb{N}$ actively waiting passengers and $h \in \mathbb{N}$ hidden passengers who have not yet pressed the call button. Each problem has a bound on all passengers' waiting and travel times, i.e., there are $2(w + h)$ constraints secondary cost functions and constraints. **Over 90% of the problem instances we consider with $e = 2$ have trivial constraints. For $e = 1$, there is a mix of trivial, linearisable, and interesting problems.**

Constrained Exploding Blocks World (C-ExBW) (page 114). $N \in \mathbb{N}$ blocks that must be stacked on top of each other in a particular arrangement, where each block is rigged with an explosive, which with a certain probability explodes and destroys the table or block that it is placed on. The only secondary-cost constraint is that the number of destroyed blocks is bounded by threshold $c \in \mathbb{R}_{\geq 0}$, which is selected manually to ensure the problem is feasible. Since this domain has a single secondary-cost constraint, **all its problems are linearisable** (thm. 16).

Constrained PARC Printer (C-PARC) (page 114). The agent must print four pages using a modular printer with various components. The printer has $c \in \mathbb{N}$ unreliable components in which a page can get jammed with probability 0.1. Once jammed, the component can not be used again and the relevant page must be reprinted. Using certain components incurs a secondary cost whose expected cost must be bounded by $u \in \mathbb{N} \cup \{\infty\}$, and the probability of failure must be bounded by $f \in [0, 1]$. In the problems we consider, they are **trivial when $f = 1$ and $u = \infty$, otherwise linearisable or interesting**.

Constrained Triangle Tireworld (C- Δ TW) (page 114). The agent must drive a car through a triangular-shaped network of roads and cities, and each time the agent drives between cities it has a 50% chance of getting a flat tyre. If the agent has a spare, it can fix the flat, but spare tyres can only be acquired in certain cities. In such cities, the agent can use one of $c \in \mathbb{N}$ currencies to purchase the spare tyre. There are c constraints, one for each currency. **C- Δ TW is linearisable but with high probability behaves like it has interesting constraints.** This is because, for any λ such that $\mathbb{S}(\lambda)$ has an optimal policy π_λ^* that is feasible for the CSSP, there are also many other optimal policies for $\mathbb{S}(\lambda)$ that are not feasible for the CSSP. Consequently, solving $\mathbb{S}(\lambda)$ is likely to yield a policy that is infeasible for the CSSP, even though a feasible policy exists; so, these problems will have the same properties as interesting problems with high probability.

For a given problem, a problem instance on C-Elev and C-S&R refers to a random instantiation of the problem, and on C-ExBW, C-PARC, and C- Δ TW refers to the fixed problem paired with a random seed that is passed to the planner. We ran each planner on 30 instances of each problem on a cluster of Intel Xeon 3.2 GHz CPUs. For each instance, the planner was limited to 30 minutes CPU time, 4GB memory, and one CPU core. We now present and discuss the results.

What is the best optimal planner? We present the coverage and runtimes of planners per problem in tab. 8.1. COMMIP works well for small problems, but fails as the problems grow large, so we focus on comparing i^2 -dual-det and LAny. LAny performs best on trivial problems, e.g., C-Elev with $e = 2$ and the trivial C-PARC problems. i^2 -dual-det does best on interesting problems, e.g., interesting C-PARC and C-Elev with $e = 1$ problems. It is worth noting that LAny may not find λ^* on C-Elev with $e = 1$ due to the incompleteness of coordinate search (sec. 6.3.3), analogous to CARL (sec. 7.6); these

MIP 1				LAny		i^2 -dual-det			
				LMCut	POM	M		SOS1	
C-S&R (n,h)	(4, F)	–		26 142.33 $\pm$ 53.93	25 220.22 $\pm$ 75.72	15 753.23 $\pm$ 228.96	25 605.83 $\pm$ 235.51		
	(4, T)	–		26 179.02 $\pm$ 99.71	27 256.52 $\pm$ 119.13	1 494.30	1 308.98		
	(5, F)	–		30 229.20 $\pm$ 71.15	30 609.57 $\pm$ 109.72	5 264.87 $\pm$ 276.37	–		
	(5, T)	–		27 249.75 $\pm$ 73.84	27 781.13 $\pm$ 128.25	2 564.38 $\pm$ 633.43	–		
C-Elev (e,w,h)	(1, 2, 2)	30 9.90 $\pm$ 3.54		10 13.57 $\pm$ 3.21	10 5.63 $\pm$ 1.65	30 247.64 $\pm$ 84.71	30 68.76 $\pm$ 42.79		
	(2, 1, 1)	30 5.16 $\pm$ 0.19		30 9.74 $\pm$ 2.26	30 18.89 $\pm$ 3.17	24 667.18 $\pm$ 199.89	30 118.77 $\pm$ 34.55		
	(2, 1, 2)	30 275.91 $\pm$ 103.93		29 125.35 $\pm$ 17.91	29 156.64 $\pm$ 18.63	–	–		
	(2, 2, 1)	30 196.21 $\pm$ 118.34		28 64.48 $\pm$ 12.83	28 79.44 $\pm$ 10.77	1 1260.28	8 527.71 $\pm$ 192.10		
C-ExBW (id,N,c)	(04, 5, 0.17)	–		30 838.48 $\pm$ 42.33	30 1335.18 $\pm$ 46.07	30 77.92 $\pm$ 20.23	30 16.58 $\pm$ 0.78		
	(04, 5, 0.2)	–		30 0.20 $\pm$ 0.00	30 3.22 $\pm$ 0.01	30 10.61 $\pm$ 1.13	30 1.62 $\pm$ 0.04		
	(05, 7, 0.1)	–		30 10.95 $\pm$ 0.06	30 21.00 $\pm$ 0.42	25 717.19 $\pm$ 139.25	30 19.74 $\pm$ 0.21		
	(07, 8, 0.48)	–		30 1.77 $\pm$ 0.11	30 66.83 $\pm$ 2.88	27 559.59 $\pm$ 144.51	30 35.33 $\pm$ 2.59		
C-PARC (c,f,n)	(1, 0.0, 1)	–		–	–	30 32.90 $\pm$ 5.87	30 16.95 $\pm$ 0.15		
	(1, 0.0, ∞)	–		30 53.74 $\pm$ 0.67	–	30 17.83 $\pm$ 0.12	30 16.68 $\pm$ 0.12		
	(1, 0.1, 1)	–		–	–	30 69.91 $\pm$ 24.25	25 121.63 $\pm$ 123.07		
	(1, 1.0, ∞)	–		30 20.74 $\pm$ 0.36	30 490.62 $\pm$ 8.15	–	–		
	(2, 0.0, 1)	–		–	–	30 32.83 $\pm$ 6.16	30 16.71 $\pm$ 0.13		
	(2, 0.0, ∞)	–		30 151.60 $\pm$ 1.73	–	30 17.28 $\pm$ 0.07	30 16.09 $\pm$ 0.17		
C- Δ TW (n,d,c)	(2, 1.0, ∞)	–		30 44.73 $\pm$ 0.52	30 595.88 $\pm$ 18.83	–	–		
	(4, 4, 2)	30 371.88 $\pm$ 58.04		30 349.01 $\pm$ 1.40	30 359.34 $\pm$ 1.30	30 110.65 $\pm$ 16.28	30 29.61 $\pm$ 2.13		
	(4, 4, 4)	30 583.43 $\pm$ 63.55		–	–	29 330.38 $\pm$ 101.19	30 44.88 $\pm$ 2.28		
	(4, 4, 6)	18 949.41 $\pm$ 91.60		–	–	29 513.92 $\pm$ 79.73	30 56.99 $\pm$ 2.14		
	(4, 5, 2)	14 1621.75 $\pm$ 60.58		–	–	30 385.90 $\pm$ 26.71	30 141.81 $\pm$ 14.66		
	(4, 5, 4)	–		–	–	27 850.25 $\pm$ 108.59	30 205.69 $\pm$ 11.66		
	(4, 5, 6)	–		–	–	25 1021.15 $\pm$ 88.20	30 246.11 $\pm$ 15.59		
	(4, 6, 2)	–		–	–	–	30 1064.31 $\pm$ 102.18		
	(4, 6, 4)	–		–	–	–	25 1376.67 $\pm$ 96.93		
	(4, 6, 6)	–		–	–	–	28 1281.45 $\pm$ 81.74		

Table 8.1: Results for optimal planners. For each problem, the coverage (out of 30) and the mean runtime (secs) with 95% C.I. over the instances where the planner converged in time.

cases put LAny at a further disadvantage. i^2 -dual-det also does best on C- Δ TW problems. Recall that C- Δ TW is linearisable, but for any linearisation λ the $\mathbb{S}(\lambda)$ has many optimal solutions that are not CSSP-feasible. This causes LAny to struggle because it is unlikely to find a CSSP-feasible policy in stage 1, and is forced to enter stage 2 and enumerate many policies before finding a CSSP-feasible one. On C-ExBW $id = 4, N = 5$, we see that i^2 -dual-det is faster on $c = 0.17$, and LAny is faster on $c = 0.2$, i.e., i^2 -dual-det performs better on problems with more challenging constraints. Incidentally, all our C-ExBW problems have strong duality except C-ExBW $id = 4, N = 5$, which intuitively has an effect on LAny, since weak duality requires the enumeration of more policies in stage 2. Strong and weak duality is not directly related to how interesting a problem is, but it does correlate with how well $\mathbb{S}(\lambda^*)$ approximates the CSSP.

Comparing LAny using LMCut and POM we have that: LMCut is 1.5-3 $\times$ faster on C-S&R and 1.5-38 $\times$ on C-ExBW; POM has less coverage on C-PARC and C- Δ TW; and both have comparable results on C-Elev. LAny with POM still has better coverage than i^2 -dual-det-SOS1 on C-S&R, is faster on the trivial C-Elev problems, but is slower than i^2 -dual-det-SOS1 on all C-ExBW problems. Since POM is closer to C-POM, the heuristic used by i^2 -dual-det, this suggests that LAny is outperforming i^2 -dual-det on C-ExBW largely thanks to LMCut. For C-S&R, this is only a small factor. Comparing i^2 -dual-det-SOS1 and i^2 -dual-det- M , the former dominates almost everywhere, except on large C-S&R problems and some C-PARC problems.

In summary, when considering optimal solutions, LAny is well suited to uninteresting CSSPs, i.e., trivial and linearisable CSSPs and generally underperforms on interesting problems and problems that are linearisable but with high probability the optimal solution for any $\mathbb{S}(\lambda)$ is infeasible, such as C- Δ TW. In contrast, i^2 -dual-det is not negatively affected by the complexity of the constraints and outperforms other planners in interesting CSSPs. Nonetheless, i^2 -dual-det is unable to take advantage of simpler constraints structures and it is outperformed by LAny in the uninteresting CSSPs. COMMIP has a similar trade-off as i^2 -dual-det; however, it is unable to scale up to large problems since, unlike i^2 -dual-det and LAny, it does not perform heuristic search in the planning search space and it is forced to consider all states and actions.

Are anytime planners more practical than optimal planners? It would be unfair to compare the anytime algorithms only within the scope of solving problems optimally, so we look at how good their incumbent solutions are before the optimal planners fully converge in tab. 8.2. For each problem

		LAny				i ² -dual-det				
		cutoff	LMCut		POM		M		SOS1	
C-S&R (n,h)	(4, F)	1800	30	0.015 ± 0.016	30	0.020 ± 0.019	30	0.046 ± 0.065	30	0.003 ± 0.003
	(4, T)	1800	30	0.010 ± 0.016	30	0.002 ± 0.002	30	0.720 ± 0.157	22	0.153 ± 0.144
	(5, F)	229	30	0.008 ± 0.008	15	0.137 ± 0.107	26	0.775 ± 0.161	–	–
	(5, T)	1800	30	0.002 ± 0.003	30	0.012 ± 0.016	30	0.933 ± 0.091	–	–
C-Elev (e,w,h)	(1, 2, 2)	10	2	0.000 ± 0.000	10	0.009 ± 0.012	1	1.000	17	0.017 ± 0.014
	(2, 1, 1)	5	9	0.000 ± 0.000	–	–	–	–	–	–
	(2, 1, 2)	276	29	0.000 ± 0.000	28	0.000 ± 0.000	–	–	–	–
	(2, 2, 1)	196	29	0.001 ± 0.003	29	0.001 ± 0.003	–	–	–	–
C-ExBW (id,N,c)	(04, 5, 0.17)	17	30	0.005 ± 0.001	–	–	–	–	30	0.002 ± 0.001
	(04, 5, 0.2)	0	16	0.000 ± 0.000	–	–	–	–	–	–
	(05, 7, 0.1)	11	30	0.000 ± 0.000	–	–	–	–	–	–
	(07, 8, 0.48)	2	19	0.000 ± 0.000	–	–	–	–	–	–
C-PARC (c,f,u)	(1, 0.0, 1)	17	–	–	–	–	–	–	18	0.000 ± 0.000
	(1, 0.0, ∞)	17	–	–	–	–	–	–	18	0.000 ± 0.000
	(1, 0.1, 1)	70	–	–	–	–	21	0.000 ± 0.000	21	0.000 ± 0.000
	(1, 1.0, ∞)	21	14	0.000 ± 0.000	–	–	–	–	–	–
	(2, 0.0, 1)	17	–	–	–	–	–	–	18	0.000 ± 0.000
	(2, 0.0, ∞)	16	–	–	–	–	–	–	10	0.000 ± 0.000
	(2, 0.1, 1)	1800	30	0.055 ± 0.000	–	–	–	–	27	0.026 ± 0.000
	(2, 0.1, ∞)	1800	30	0.070 ± 0.000	9	0.067 ± 0.010	1	0.027	22	0.027 ± 0.000
	(2, 1.0, 1)	1800	30	0.003 ± 0.001	–	–	–	–	–	–
(2, 1.0, ∞)	45	16	0.000 ± 0.000	–	–	–	–	–	–	
C-ΔTW (n,d,c)	(4, 4, 2)	30	–	–	–	–	9	1.000 ± 0.000	17	0.000 ± 0.000
	(4, 4, 4)	45	–	–	–	–	4	0.990 ± 0.020	14	0.000 ± 0.000
	(4, 4, 6)	57	–	–	–	–	17	1.000 ± 0.000	13	0.000 ± 0.000
	(4, 5, 2)	142	–	–	–	–	12	0.997 ± 0.007	19	0.000 ± 0.000
	(4, 5, 4)	206	–	–	–	–	14	1.000 ± 0.000	13	0.000 ± 0.000
	(4, 5, 6)	246	–	–	–	–	13	1.000 ± 0.000	14	0.000 ± 0.000
	(4, 6, 2)	1064	–	–	–	–	18	1.000 ± 0.000	15	0.000 ± 0.000
	(4, 6, 4)	1800	–	–	–	–	14	0.993 ± 0.009	25	0.000 ± 0.000
	(4, 6, 6)	1800	–	–	–	–	15	1.000 ± 0.000	28	0.000 ± 0.000

Table 8.2: Results for anytime planners. The cutoff (secs) is given per problem rounded to the nearest integer. For each planner, we give the anytime coverage (number of instances for which the planner produces an incumbent before the cutoff), and the mean and 95% C.I. of the optimality gap for found solutions. Maximum possible coverage is 30.

p we consider the optimal planners will full coverage in tab. 8.1, and among these we define m_p to be the minimum mean time to solve. If no planner attains full coverage we set m_p to the deadline time. In tab. 8.2, the anytime coverage is the number of instances where LAny has an incumbent solution before m_p . We define the optimality gap of a solution as $|ub - lb|/|ub|$.

LAny often finds incumbent solutions before the cutoff with very tight optimality gaps. The important exceptions are C-ΔTW, which is linearisable but an optimal policy for its $\mathbb{S}(\lambda)$ s is very likely CSSP-infeasible; and non-trivial C-PARC problems where LAny does not find any incumbent solutions before the cutoff. On C-Elev $e = 1, w = 2, h = 2$ LAny finds incumbent solutions in time for trivial instances, and for the non-trivial instances, it finds none. This suggests that on the problems where LAny performs poorly as an optimal planner, its anytime capabilities can not redeem it. i^2 -dual-det-SOS1 has higher coverage and better optimality gaps than i^2 -dual-det- M except in C-S&R and C-ΔTW, where i^2 -dual-det- M has higher coverage but with a large optimality gap; this can be primarily attributed to i^2 -dual-det- M 's optimality gap requirement of 1, compared to i^2 -dual-det-SOS1's 0.2.

8.6 Summary

In this chapter, we presented three contributions related to deterministic-optimal policies for CSSPs. First, we identified a theoretical gap in the MIP for finding deterministic-optimal policies (COMMIP), namely, that there is no automatic method for finding a valid M constant in the big- M formulation; and we then provided multiple ways to address this gap. Second, we presented i^2 -dual-det, a novel heuristic-search algorithm for finding deterministic-optimal policies. It is constructed by modifying i^2 -dual with the COMMIP. Third, we showed a categorisation of CSSPs, based on the difficulty of their constraints.

Conclusion

9.1 Summary

The goal of this thesis was to extend connections between probabilistic planning and OR, with the aim of deepening the understanding of probabilistic planning, producing new insights, and developing new probabilistic planning algorithms that compete with the state-of-the-art. Indeed, this thesis presents four new probabilistic planning algorithms using techniques from OR:

- CG-iLAO* (chap. 4) – a new heuristic-search algorithm that is able to use its heuristic to ignore unpromising actions, as well as states; it is derived from a new connection between heuristic search and constraint and variable generation.
- CoGNe (chap. 5) – expresses policies in terms of plans and cycles using network flow theory, and then applies variable generation to find policies via solving a sequence of deterministic planning problems.
- CARL (chap. 7) – uses Lagrangian duality to efficiently solve CSSPs as a sequence of SSPs.
- i²-dual-det (chap. 8) – generalises existing approaches for CMDPs to CSSPs and uses constraint and variable generation to find deterministic-optimal policies efficiently.

All of these (except CoGNe) succeed in competing with state-of-the-art algorithms in their respective contexts.

9.2 New High-level Insights

There are two high-level insights that are recurring throughout this thesis and are general enough that they apply to the broader planning community.

1. Constraint and variable generation is equivalent to heuristic search.
2. Problems can often be broken down into simpler subproblems.

Heuristic search is constraint and variable generation. This connection is not entirely new and has been known at least since i-dual (Trevizan et al., 2016), which generalised iLAO* to CSSPs by applying constraint and variable generation to the COMLP (LP 9). This thesis has extended the existing connection by showing that iLAO* itself can also be interpreted as an implementation of constraint and variable generation, even though it does not solve any LPs and instead uses dynamic programming to compute its value function (see sec. 4.3), showing that the constraint and variable generation theory reaches beyond LPs, and reaches planning at a more fundamental level than previously understood. Interpreting iLAO* in terms of constraint and variable generation was a key step in developing CG-iLAO* because this perspective made it clear that iLAO*'s separation oracle is naïve and can be refined. Furthermore, the theory from constraint generation presented a way to implement the refined separation oracle, yielding a description of CG-iLAO* in terms of LPs with constraint and variable generation.

Reimplementing this algorithm with dynamic programming techniques from planning yielded CG-iLAO*, which outperformed its predecessor iLAO* in our experiments. The connection between heuristic-search and constraint and variable generation also informed the development of i²-dual-det (chap. 8), which follows i²-dual (Trevizan et al., 2016). This success story motivates that this connection is not only a pleasing theoretical insight that connects two disciplines but also offers practical value. SSPs generalise classical planning, so this insight immediately applies there, and may also apply to other planning models such as planning under constraints, numerical planning, etc.

Problems can be broken down into simpler subproblems. The technique of breaking problems down into simpler subproblems is well-established in the OR community and gives us multiple frameworks for breaking down planning problems. Chap. 7 used Lagrangian duality to solve CSSPs via a sequence of unconstrained SSPs, constructing an optimal (potentially stochastic) policy for CSSPs from a convex combination of greedy deterministic policies for a particular unconstrained SSP. This resulted in the novel algorithm CARL, which competes with the state-of-the-art. Chap. 5 reformulated the OMLP (LP 2) in terms of network flow and then applied variable generation to produce the algorithm CoGNe, which solves SSPs optimally via a sequence of deterministic planning problems. It is novel to construct optimal policies for SSPs out of plans and cycles, giving new insight into probabilistic planning, and new algorithmic approaches. Thus, we used the OR techniques of Lagrangian duality and variable generation to break down CSSPs and SSPs, which gives us a better understanding of how deterministic shortest path problems, SSPs, and CSSPs relate to each other, and in the case of CARL it resulted in an algorithm that outperforms the previous state-of-the-art. The OR tools we have discussed are general and can be applied to other planning problems, and we believe it is valuable to do this, motivated by the success in the context of SSPs and CSSPs.

To conclude, this thesis achieves its goal of studying connections between probabilistic planning and OR with the aim of gaining a deeper understanding of probabilistic planning, and developing new state-of-the-art algorithms. This thesis exploits the two concepts “heuristic search is constraint and variable generation” and “problems can be broken down into simpler subproblems,” demonstrating that these ideas are important in probabilistic planning; we argue that these concepts may be important to the broader planning community and should be investigated in more settings.

9.3 Future Work

This section discusses possible future research directions related to the work presented in this thesis.

9.3.1 Improving the Search for λ^* in CARL

Recall from chap. 7 that CARL finds optimal policies for CSSPs by solving the CVLP (LP 12) via a subgradient optimisation over λ . For each λ considered by the subgradient optimisation we must solve the corresponding $\mathbb{S}(\lambda)$, so it is important for the performance of CARL that the number of λ s considered is as small as possible. Our implementation follows Hong and Williams (2023) and finds λ^* with a coordinate search that considers one coordinate λ_i at a time, and each of the coordinate search’s one-dimensional optimisation problems over λ_i is solved with an exact line search. Hong and Williams (2023) point out that this approach has been effective in similar settings where the optimisation is over piecewise linear concave problems with small dimensions of λ (i.e., small values of n for $\lambda \in \mathbb{R}_{\geq 0}^n$). Indeed, the approach also proved effective for CARL. However, coordinate search has the key shortcoming that it is incomplete for non-smooth problems, which means there are problems where coordinate gets stuck and fails to find an optimal solution (see sec. 7.6). Moreover, we illustrated that coordinate search fails to scale as the number of constraints increases. Thus, it would be valuable to find an optimisation method that is guaranteed to converge, scales better with constraints, and does so with similar performance to coordinate search. We used the standard subgradient method (Shor, 1985) as a fallback when coordinate search fails because it is guaranteed to converge, but it is significantly slower and did not converge on any of our problems, so it is not a good candidate to replace coordinate search. It is possible that the subgradient method’s performance can be improved by tuning the algorithm’s parameters. Alternatively, there are other optimisation approaches such as the cutting plane method (Elhedhli, Goffin, and Vial, 2001) that may be more amenable to our problems.

9.3.2 Anytime Planning for Multi-Objective SSPs

We explained in sec. 7.5 that Multi-Objective SSPs (MO-SSPs) (Forejt, Kwiatkowska, and Parker, 2012; Chen, Trevizan, and Thiébaux, 2023) are related to CSSPs. The key difference is that a MO-SSP’s solution is a set of undominated policies, called a Pareto front; whereas a CSSP is solved by a single policy from this Pareto front (at least, one of the CSSP’s optimal policies is guaranteed to be in the Pareto front). Recall that CARL uses scalarisation in the context of single-objective search, but similar techniques called scalarisation exist in the MO literature (Ehrgott, 2006) and have been applied in the context of MO-SSPs (Forejt, Kwiatkowska, and Parker, 2012). One direction for future work is to adapt the techniques from CARL to an MO setting. Observe that in the context of MO-SSPs, for each scalarised SSP $\mathbb{S}(\lambda)$, its optimal deterministic policy represents a point on the MO-SSP’s Pareto front. Thus, we can use CARL’s mechanism for solving $\mathbb{S}(\lambda)$ s and the λ heuristics introduced in sec. 7.3.1 to solve the MO-SSP’s $\mathbb{S}(\lambda)$ s efficiently. Using this, we introduce a naïve baseline that randomly samples λ and evaluates the corresponding $\mathbb{S}(\lambda)$ to produce points on the Pareto front. Whenever two points on the Pareto front correspond to the same optimal deterministic policy we know that the interval between them must also lie on the Pareto front; whenever three points correspond to the same policy we know that their simplex lies on the Pareto front; and so on. This follows from the theory used for LAny (see sec. 6.3.3). With this randomised approach, we can construct a diverse set of Pareto solutions that cover large parts of the Pareto front. This baseline already has an advantage over the value-function-based approach, e.g., Chen, Trevizan, and Thiébaux (2023), in that, we produce valid points on the Pareto front very quickly and expand it in an anytime fashion, whereas their approach produces approximations of the Pareto front that only converge to true points at the end. In practice, users of MO planners are likely more interested in an incomplete but diverse set of Pareto solutions produced quickly, rather than all possible trade-offs produced slowly. Clearly, the next step of this research is to investigate techniques for selecting λ systematically in order to construct the Pareto front completely and efficiently. Roijers and Whiteson (2017) call the framework we have described an “outer loop approach” and present the *Optimistic Linear Support (OLS)* algorithm for selecting λ s efficiently. OLS works well for MO-MDPs, and it is likely that OLS with CARL’s oracle would work well in our setting. The techniques developed for i^2 -dual-det in chap. 8 may also be useful in the context of MO-SSPs, but that research path is less clear.

9.3.3 Solving SSPs with PLTL Constraints

SSPs with probabilistic PLTL constraints (PLTL-SSPs) can be transformed into CSSPs (Baumgartner, Thiébaux, and Trevizan, 2018), and we can use CARL to solve them. If CARL has the same performance improvement over i^2 -dual in the PLTL-SSP setting as we observed in our CSSP benchmarks (sec. 7.6), then CARL may be a significant step towards algorithms that can solve the large PLTL-SSPs required for real-world problems. Unfortunately, preliminary experiments showed that CARL equipped with the NBA projection heuristic (Baumgartner, Thiébaux, and Trevizan, 2018) adapted as a λ heuristic (defn. 44) did not compete with the PLTL-SSP variant of i^2 -dual (Baumgartner, Thiébaux, and Trevizan, 2018). We observed that CARL’s $\mathbb{S}(\lambda)$ s take a long time to solve, in some cases comparable to the time taken by i^2 -dual to solve the whole PLTL-SSP. This suggests that the heuristics are uninformative for CARL’s $\mathbb{S}(\lambda)$ s, while being comparatively informative for i^2 -dual. Consequently, to recover CARL’s strong performance we must develop heuristics that work better with CARL. There has been recent work in developing heuristics for deterministic planning with certain flavours of LTL (Middleton, Toro Icarte, and Baier, 2022), and there has also been work in compiling away certain LTL constraints to obtain classical planning problems (Bonassi et al., 2023). Potentially such techniques can be used to solve CARL’s subproblems efficiently.

9.3.4 Improvements from Constraint and Variable Generation

Constraint and variable generation are well-established techniques in OR, and there has been extensive research on how to make them more efficient. For example, stabilised variable generation is designed to make variable generation converge quicker (du Merle et al., 1999), and for constraint generation there are methods for removing constraints that will not, or are unlikely to be useful in the future (Paulraj and Sumathi, 2010; Rahmaniani et al., 2017). Constraint and variable generation have featured extensively throughout this thesis, so such techniques could offer improvements to most of the algorithms we introduced.

Recall that we derived a dynamic programming implementation of CG-iLAO* based on Bellman backups

by considering iLAO* under the lens of linear programming with constraint and variable generation, and improving the constraint generation procedure’s separation oracle. In a similar way, other techniques for improving constraint and variable generation may improve the algorithm. In a preliminary study, we investigated removing constraints from CG-iLAO*. Action elimination fits into this framework, which is interesting on its own because it establishes another connection between planning and OR techniques. However, in terms of performance sec. 4.9.7 showed that action elimination is ineffective without breakthroughs in domain-independent upper bounds. Action elimination permanently removes actions (constraints) only when it can prove that the relevant action is not needed for an optimal solution; alternatively, one can remove constraints without guarantees, e.g., remove constraints that have been inactive for a sufficiently long period of time. Since these removed constraints may be needed later, we may have to re-add them to maintain an optimal algorithm. We implemented this particular configuration, but preliminary experiments did not show improved performance. Alternatively, one can remove such constraints permanently, sacrificing the algorithm’s optimality. Based on our preliminary study, it is unlikely that the technique of removing constraints will yield improved performance for CG-iLAO*, but the other OR techniques may be more successful. From a theoretical perspective, implementing such techniques into CG-iLAO* is interesting because it may give further insights into heuristic search.

CoGNe and i²-dual-det use constraint and variable generation more directly than CG-iLAO*, in the sense that they are actually solving LPs and MIPs rather than using dynamic programming. Therefore, these techniques can be applied with less effort, and it would make sense to perform an empirical study to see whether they improve the algorithms.

9.3.5 Heuristics for Deterministic-optimal Policies

Recall from chap. 8 that i²-dual-det uses the C-POM heuristic from i²-dual, which computes a lower bound over stochastic policies. Deterministic policies are a special case of stochastic ones, with the effect that C-POM is admissible for deterministic policies. However, the gap between the costs of deterministic-optimal and optimal policies can be extreme, bearing in mind that a CSSP may have cheap stochastic policies and no deterministic feasible ones. Thus, a potential direction for future work is to design heuristics for i²-dual-det that are specialised for deterministic-optimal policies and are more informative than C-POM. We point out that it is not trivial to adapt C-POM to deterministic-optimal policies: recall that the heuristic constructs a set of subproblems called $\mathbb{C}^v$ via atomic projection, which are combined with tying constraints (see sec. 6.3.2). A solution $\mathbf{x}$ induces policies for each $\mathbb{C}^v$ that are stochastic, and additionally, due to the tying constraints these policies may also be non-stationary, which means that the policy makes its decision based not only in the current state, but the history it has encountered. We can try to make the policies over $\mathbb{C}^v$ deterministic with constraints similar to those for the COMMIP (MIP 12) and allow positive action usage for one action per state; this fails because it also forces stationarity. It is unclear how to enforce deterministic policies while allowing them to be non-stationary, which is a necessary feature of C-POM. One insight that may be helpful in designing heuristics for deterministic policies is that stochastic policies can be decomposed into constituent deterministic ones (thm. 12); with this, one can compute an optimal policy in polynomial time, decompose it, and use the constituent policies as guidance; it may even be possible to decompose heuristics for stochastic policies in this way. It is unclear if such an approach can produce admissible heuristics, but inadmissible heuristics are also valuable if a deterministic-optimal policy is not required.

Given that computing deterministic-optimal policies is harder (NP-complete) than computing optimal ones (P-complete) (Feinberg, 2000), it is intuitive that computing heuristics that are specialised for deterministic policies may also be more expensive than computing heuristics for stochastic policies; but, as we mentioned, there are cases where the gap between deterministic-optimal and optimal policies is very large. This raises a follow-up question: is there an automated way to determine whether the deterministic-optimal policy is “close to” the optimal one? If so, then using the heuristic for stochastic policies is likely more efficient; and if the deterministic-optimal policy is “far from” the optimal one, it may pay off to use the more expensive but more informative heuristic specialised for deterministic policies. There is a clear connection linking, on one side, the gap between deterministic-optimal and optimal policies, and on the other side, our definitions of constraint interestingness (sec. 8.4) and the Lagrangian dual (see sec. 6.3.3): deterministic-optimal and optimal policies coincide for CSSPs with trivial constraints and more generally CSSPs whose COMMIP satisfies strong duality; and the gap between policies increases as the duality gap grows, and as the constraints become interesting.

9.3.6 Heuristics for CoGNe's Plans and Cycles

Recall that CoGNe's pricing problem is to determine whether any plan or cycle exists with $rc(w, \mu) < 0$. If such a plan or cycle w exists it is added to the RMP, and if such w does not exist CoGNe can terminate with a guarantee of optimality (see sec. 5.3.2). The reduced cost $rc(w, \mu)$ is a bound on how much the RMP's objective can be improved by adding w , and in reality the addition of w may not reduce the RMP's objective at all. To illustrate this, consider an RMP with primal and dual solutions $\mathbf{f}$ and μ such that $\mathbf{f}$ describes the proper policy $\pi_{\mathbf{f}}$ with $C(\pi_{\mathbf{f}}) = 10$. Suppose there is a plan p whose states and actions are disjoint from the RMP's plans and cycles, it has $C(p) = 5$, and it has one unaccounted probabilistic effect that occurs with probability 0.5 and leads to a state s with $V^*(s) = 20$. This plan has $rc(p, \mu) = 5 - 10 < 0$ and should be added to the RMP, but the additional information about its probabilistic effect leading to s informs us that p would induce a policy π_p with $C(\pi_p) = 5 + 0.5 \cdot 20 = 15 > C(\pi_{\mathbf{f}})$. Thus, with information about p 's probabilistic effects, we observe that $rc(p, \mu) + \text{cost to make } p \text{ a closed policy} \geq 0$ with the consequence that p will not produce an RMP with a lower objective and should therefore **not** be added to the RMP. More generally, if we have a heuristic function $H : \mathcal{P} \cup \mathcal{C} \rightarrow \mathbb{R}_{\geq 0}$ that estimates the cost of handling a plan or cycle's probabilistic effects, it would enable us to refine the pricing problem to add plans or cycles to the RMP only if $rc(w, \mu) + H(w) < 0$. Currently, to prove optimality CoGNe must add many plans and cycles to the RMP that are not needed to describe an optimal policy, so the ability to prune such plans and cycles from search has a lot of potential. Consider the expansions of A^* (Hart, Nilsson, and Raphael, 1968) as an analogue: each state is assigned $g(s)$ (cheapest found cost to reach s from s_I) and $h(s)$ (an admissible heuristic from s); if the frontier contains a goal state **goal** then A^* must expand all states with $g(s) + h(s) < g(\text{goal})$ before termination, which can be a lot of states if $h(s) = 0$, and can be significantly fewer states if $h(s)$ is more informative. Moreover, heuristic over plans and cycles would allow CoGNe to prioritise plans and cycles with small $rc(p, \mu) + H(p)$ values, which may enable CoGNe to produce higher quality policies faster; the planning community has shown many times that using informative heuristics to guide search can yield significant speedup, so it stands to reason that an informative heuristic $H(w) > 0$ would also benefit CoGNe.

We demonstrate how such a heuristic might be defined in the special case where a plan p does not pass through any active regrouping constraints in the RMP, i.e., $C'(s, a, s) = C(s, a) \forall s \in \mathcal{S}, a \in \mathcal{A}(s), s' \in \text{succ}(s, a) : \langle s, a, s' \rangle \subseteq p$. Given an admissible heuristic over states $\bar{H} : \mathcal{S} \rightarrow \mathbb{R}_{\geq 0}$, we can define a heuristic $H : \mathcal{P} \cup \mathcal{C} \rightarrow \mathbb{R}_{\geq 0}$ for the plan p with

$$H(p) = \sum_{\substack{s \in \mathcal{S} \\ a \in \mathcal{A}(s) \\ s' \in \text{succ}(s, a) \text{ s.t.} \\ \langle s, a, s' \rangle \subseteq p, \\ s'' \in \text{succ}(s, a) : s'' \neq s'}} P(s'' | s, a) \bar{H}(s'').$$

This H must be admissible for the plan, in the sense that $rc(p, \mu) + H(p) \geq 0$ only if p can not induce a cheaper policy than the one described by the current solution. However, it is not clear how such a heuristic can be generalised to take into account existing regrouping constraints. A heuristic defined in this way would suffice for the purposes of pruning plans and cycles with $rc(w, \mu) + H(w) \geq 0$ from search. In order to exploit a heuristic over plans and cycles to prioritise promising plans and cycles, the heuristic would need to be decomposed and embedded in the pricing problem.

9.3.7 Using Machine Learning to Solve Pricing Problems Efficiently

For a variable generation algorithm, its various pricing problems share many common features between iterations and thanks to the way a pricing problems are defined, they always have the same structure for a given master problem. The OR community exploits this property and designs algorithms tailored to a particular class of pricing problem or uses relaxations to well-known problems, e.g., variable generation algorithms for the Vehicle Routing Problem may find paths for the pricing problem with a specialised algorithm, e.g., Righini and Salani (2006); or they can relax the subproblem, e.g., Laporte et al. (2000). As an alternative approach to solving pricing problems we can consider Machine Learning (ML); ML is amenable to finding solutions across problems with similar structure, which makes it a promising candidate. In particular, our focus is on CoGNe's pricing problems which are deterministic planning problems with negative costs and negative cycles; this is a relatively novel problem and there are no well-established methods for solving it, so it makes sense to explore new approaches. We note, however, that the ideas we describe may apply to pricing problems more generally.

In sec. 5.3.2, we listed features of CoGNe's pricing problem that make it easier to solve, namely:

- we need sufficiently cheap solutions with $rc(w, \mu) < 0$, not optimal ones (except for the proof of optimality);
- all pricing problems share the same transition model, i.e., they have the states and actions of the input SSP’s all-outcomes determinisation, and the only component that changes between pricing problems is the cost function;
- negative costs have associated semantics, which gives structure that can be exploited during search.

We have discussed a variety of techniques for solving the pricing problem efficiently in sec. 5.3.2, but none are sufficiently fast. Notably, none of the approaches we mentioned exploit all simplifying features of the pricing problem, e.g., Large Neighbourhood Search (LNS) produces sufficiently cheap suboptimal solutions, and it uses the property that pricing problems are similar by using previous solutions as a starting point, but LNS has no mechanism for exploiting the semantics and structure of the cost function.

To address this problem, we may be able to exploit recent developments in ML for planning. ML has been used to learn domain-specific policies (Toyer et al., 2020), domain-specific and domain-independent heuristics (Chen, Thiébaux, and Trevizan, 2024), and proxies for heuristics such as rankings (Hao et al., 2024) and search effort (Ferber et al., 2022). Thus, ML could be used to learn either a heuristic for the pricing problem or any other proxy measurement that is too hard to compute in closed form. ML is promising in this context because the pricing problems have the same structure, so learning can be specific to the problem (i.e., problem-specific models) which requires less generalisation power than domain-independent and even domain-specific models. Such ML approaches lack guarantees, e.g., an ML heuristic can not guarantee admissibility, but this is not an issue because in the context of pricing problems, we require sufficiently cheap solutions, not optimal ones. The main challenge is that the costs change with the pricing problems, which is problematic because most current ML approaches for planning assume fixed costs that are known a priori. This challenge can be potentially addressed with the new ML approaches for numeric planning such as numeric ASNs (Wang and Thiébaux, 2024) and numeric GOOSE (Chen and Thiébaux, 2024).

9.3.8 Generalising Beyond SSPs and CSSPs

Part I of this thesis focuses on SSPs, and part II considers CSSPs, which extend SSPs with constraints over secondary costs. There are many interesting and important extensions that generalise CSSPs even further, e.g., SSPs whose trajectories must satisfy PLTL constraints (Baumgartner, Thiébaux, and Trevizan, 2018), and SSPs that are constrained in a way to produce explanations (Steinmetz et al., 2024). We will consider SSPs with PLTL constraints (PLTL-SSPs) further, as a representative of CSSP generalisations. For the purposes of this discussion, a PLTL constraint is described by a condition over trajectories $\phi : \mathcal{T} \rightarrow \{\top, \perp\}$ that must be satisfied with a certain probability $\omega \in [0, 1]$, i.e., a policy π satisfies the constraint if $\sum_{T \in \mathcal{T}_{\pi, s_I}} P(T|\pi) \cdot \llbracket \phi(T) \rrbracket \geq \omega$. It is also possible to extend SSPs with constraints that are different from what we have seen so far, for example, we consider the problem of solving an SSP with a policy π that must not contain any cycles. This acyclic-policy requirement exists in the FOND setting where such policies are called strong solutions (Daniele, Traverso, and Vardi, 2000); this has not been investigated in the setting of SSPs, but the reasons for this restriction in the FOND setting also apply to SSPs. In particular, the effect of restricting to acyclic policies is that a satisfying policy π must be executable in a bounded number of steps, whereas a policy with policy will terminate in finitely many, but an unbounded number of steps. Note that finding an optimal acyclic policy for SSPs is not equivalent to finding strong solutions for FOND problems because our policies still have expected costs that we are trying to minimise.

CoGNe is a strong candidate for such extended problems: we have focused on applying CoGNe to SSPs, but its expressivity makes it suitable for more general problems. We start by motivating that CoGNe’s plan-and-cycle representation (LP 7) can express any constraints that can be formulated over the OMLP (LP 2 and LP 5). For example, CSSPs can be solved by augmenting the OMLP with the constraints

$$\sum_{s \in S, a \in A(s)} x_{s,a} \cdot C_i \leq u_i \quad \forall i \in 1, \dots, n$$

yielding the COMLP (LP 9). Similarly, the Plan and Cycle LP can be augmented to solve CSSPs with the constraints

$$\sum_{s \in S, a \in A(s)} \Phi(s, a) \cdot C_i \leq u_i \quad \forall i \in 1, \dots, n$$

with which CoGNe can solve CSSPs. Such changes are mostly trivial, and the only consequence that requires attention is that the cost function in CoGNe’s pricing problems becomes more complicated to take into account the additional constraints. Now, we argue that CoGNe is more expressive than the OMLP and can conveniently encode PLTL and acyclicity constraints.

To encode PLTL constraints in CoGNe, we restrict our attention to acyclic SSPs and assume that a plan p ’s flow represents the probability that p occurs as a trajectory when following the LP’s candidate policy from s_I , i.e., $f_p = P(p|\pi)$. This is true if we can use the canonical plan representation (sec. 5.2.3), but there are situations where we can not currently guarantee this semantic, e.g., in problems where stochastic policies are required, such as CSSPs. With this setup, we can simply encode the PLTL constraint described by $\phi : \mathcal{T} \rightarrow \{\top, \perp\}$ and $\omega \in [0, 1]$ with

$$\sum_{p \in \mathcal{P}: \phi(p)} f_p \geq \omega.$$

We point out that this encoding is general and can be used for constraints beyond PLTL: it can encode that an arbitrary expression over trajectories is satisfied with a particular probability. The main challenge of this research direction is that we must ensure the semantic $f_p = P(p|\pi)$, which is non-trivial for settings beyond finding deterministic policies for acyclic SSPs.

Finally, we consider the problem of finding acyclic policies for SSPs. To implement this with CoGNe, we disallow any cycles from entering its RMP, i.e., $\hat{\mathcal{C}} = \emptyset$, by modifying the pricing problem to search only for plans with negative reduced cost. In other words, we do not need to add any additional constraints to CoGNe, we only need to modify the pricing problem so that $\hat{\mathcal{C}}$ remains empty. In contrast, the only clear way to implement this requirement for value-function or occupation-measure formulations is by augmenting the state space so that each new state tracks the agent’s history and can avoid revisiting previous states, i.e., cycles. Augmenting the state space in this way produces a significantly larger SSP. We note that the concept of acyclic policies has not been investigated for SSPs, so CoGNe would be the first algorithm to implement it.

Bibliography

- Ahuja, R.; Magnanti, T.; and Orlin, J. 1993. *Network Flows : Theory, Algorithms, and Applications*. Prentice Hall. [Cited on page 71.]
- Altman, E. 1995. Constrained Markov Decision Processes. Technical report, INRIA. [Cited on pages 5, 6, 89, 90, and 91.]
- Archetti, C.; and Speranza, M. G. 2014. A Survey on Matheuristics for Routing Problems. *EURO Journal on Computational Optimization*, 223–246. DOI: 10.1007/s13675-014-0030-7. [Cited on page 10.]
- Baier, C.; Klein, J.; Leuschner, L.; Parker, D.; and Wunderlich, S. 2017. Ensuring the Reliability of Your Model Checker: Interval Iteration for Markov Decision Processes. In *Computer Aided Verification*, 160–180. DOI: 10.1007/978-3-319-63387-9_8. [Cited on page 122.]
- Barto, A. G.; Bradtke, S. J.; and Singh, S. P. 1995. Learning To Act Using Real-Time Dynamic Programming. *Artificial Intelligence*, 72(1): 81–138. DOI: 10.1016/0004-3702(94)00011-O. [Cited on pages 18, 40, and 42.]
- Baumgartner, P.; Thiébaux, S.; and Trevizan, F. 2018. Heuristic Search Planning With Multi-Objective Probabilistic LTL Constraints. In *Proc. of Int. Conf. on Principles of Knowledge Representation and Reasoning (KR)*, 415–424. [Cited on pages 7, 137, and 140.]
- Beale, E.; and Tomlin, J. 1969. Special Facilities in a General Mathematical Programming System for Nonconvex Problems Using Ordered Sets of Variables. *Operational Research*, 447–454. [Cited on page 124.]
- Bellman, R. E. 1957. *Dynamic Programming*. Princeton University Press. [Cited on pages 1, 5, and 16.]
- Bertsekas, D. 1995. *Dynamic Programming and Optimal Control*. Athena Scientific. [Cited on pages 5, 17, 18, 26, and 42.]
- Bertsekas, D. 2019. *Reinforcement Learning and Optimal Control*. Athena Scientific. [Cited on pages 6, 13, and 121.]
- Bertsekas, D.; and Tsitsiklis, J. 1991. An Analysis of Stochastic Shortest Path Problems. *Mathematics of Operations Research*, 16(3): 580–595. DOI: 10.1287/moor.16.3.580. [Cited on pages 5, 13, and 15.]
- Bertsimas, D.; and Tsitsiklis, J. 1997. *Introduction to Linear Optimization*. Athena Scientific. [Cited on pages 8, 19, 22, 75, and 110.]
- Bonassi, L.; De Giacomo, G.; Favorito, M.; Fuggitti, F.; Gerevini, A. E.; and Scala, E. 2023. Planning for Temporally Extended Goals in Pure-Past Linear Temporal Logic. In *Proc. of Int. Conf. on Automated Planning and Scheduling (ICAPS)*, 61–69. DOI: 10.1609/icaps.v33i1.27179. [Cited on page 137.]
- Bonet, B.; and Geffner, H. 2000. Planning as Heuristic Search: New Results. In *Proc. of European Conf. on Planning (ECP)*, 360–372. DOI: 10.1007/10720246_28. [Cited on page 46.]
- Bonet, B.; and Geffner, H. 2003a. Faster Heuristic Search Algorithms for Planning with Uncertainty and Full Feedback. In *Proc. of Int. Joint Conf. on Artificial Intelligence (IJCAI)*, 1233–1238. [Cited on pages 17 and 18.]

- Bonet, B.; and Geffner, H. 2003b. Labeled RTDP: Improving the Convergence of Real-Time Dynamic Programming. In *Proc. of Int. Conf. on Automated Planning and Scheduling (ICAPS)*, 12–21. [Cited on pages 1, 5, 17, 18, 23, 25, 40, 46, 53, and 83.]
- Bonet, B.; and Givan, R. 2006. 5th International Planning Competition: Non-deterministic track. Call for Participation. [Cited on page 46.]
- Boutilier, C.; Dearden, R.; and Goldszmidt, M. 2000. Stochastic Dynamic Programming with Factored Representations. *Artificial Intelligence*, 49–107. DOI: 10.1016/S0004-3702(00)00033-3. [Cited on page 7.]
- Boyarski, E.; Felner, A.; Stern, R.; Sharon, G.; Tolpin, D.; Betzalel, O.; and Shimony, E. 2015. ICBS: Improved Conflict-Based Search Algorithm for Multi-Agent Pathfinding. In *Proc. of Int. Joint Conf. on Artificial Intelligence (IJCAI)*, 740–746. [Cited on page 8.]
- Bryce, D.; and Buffet, O. 2008. International Planning Competition Uncertainty Part: Benchmarks and Results. [Cited on pages 46 and 85.]
- Bäckström, C.; and Nebel, B. 1995. Complexity Results for SAS+ Planning. *Computational Intelligence*, 625–655. DOI: 10.1111/j.1467-8640.1995.tb00052.x. [Cited on page 77.]
- Calliess, J.-P.; and Roberts, S. 2021. Multi-Agent Planning with Mixed-Integer Programming and Adaptive Interaction Constraint Generation (Extended Abstract). In *Proc. of Int. Symp. on Combinatorial Search (SoCS)*, 207–208. DOI: 10.1609/socs.v4i1.18304. [Cited on page 8.]
- Camacho, A.; Muise, C.; and McIlraith, S. 2016. From FOND to Robust Probabilistic Planning: Computing Compact Policies that Bypass Avoidable Deadends. In *Proc. of Int. Conf. on Automated Planning and Scheduling (ICAPS)*, 65–69. DOI: 10.1609/icaps.v26i1.13773. [Cited on page 6.]
- Chen, D.; Trevizan, F.; and Thiébaux, S. 2023. Heuristic Search for Multi-Objective Probabilistic Planning. In *Proc. of AAAI Conf. on Artificial Intelligence*, 11945–11954. DOI: 10.1609/aaai.v37i10.26409. [Cited on pages 112, 113, and 137.]
- Chen, D. Z.; and Thiébaux, S. 2024. Graph Learning for Numeric Planning. In *Advances in Neural Information Processing Systems (NIPS)*, 91156–91183. [Cited on page 140.]
- Chen, D. Z.; Thiébaux, S.; and Trevizan, F. 2024. Learning Domain-Independent Heuristics for Grounded and Lifted Planning. In *Proc. of AAAI Conf. on Artificial Intelligence*, 20078–20086. DOI: 10.1609/aaai.v38i18.29986. [Cited on page 140.]
- Cherkassky, B. V.; and Goldberg, A. V. 1999. Negative-cycle Detection Algorithms. *Mathematical Programming*, 277–311. DOI: 10.1007/s101070050058. [Cited on page 79.]
- Conforti, M.; Cornuéjols, G.; and Zambelli, G. 2014. *Integer Programming*. Springer International Publishing. DOI: 10.1007/978-3-319-11008-0. [Cited on pages 119 and 124.]
- Dai, P.; Mausam; and Weld, D. 2009. Focused Topological Value Iteration. In *Proc. of Int. Conf. on Automated Planning and Scheduling (ICAPS)*, 82–89. DOI: 10.1609/icaps.v19i1.18138. [Cited on pages 27 and 63.]
- Dai, P.; Mausam; Weld, D. S.; and Goldsmith, J. 2011. Topological Value Iteration Algorithms. *Journal of Artificial Intelligence Research*, 181–209. DOI: 10.1613/jair.3390. [Cited on page 43.]
- Daniele, M.; Traverso, P.; and Vardi, M. Y. 2000. Strong Cyclic Planning Revisited. In *Recent Advances in AI Planning*, 35–48. DOI: 10.1007/10720246_3. [Cited on pages 6 and 140.]
- d’Epenoux, F. 1963. A Probabilistic Production and Inventory Problem. *Management Science*, 98–108. [Cited on pages 7, 19, and 20.]
- Desrosiers, J.; and Lübbecke, M. 2005. *A Primer in Column Generation*, 1–32. Springer US. DOI: 10.1007/0-387-25486-2_1. [Cited on pages 1, 9, 22, 23, and 72.]
- Dolgov, D. A.; and Durfee, E. H. 2005. Stationary Deterministic Policies for Constrained MDPs with Multiple Rewards, Costs, and Discount Factors. In *Proc. of Int. Conf. on Automated Planning and Scheduling (ICAPS)*, 1326–1332. [Cited on pages 92, 119, 120, 121, 122, and 130.]

- Domshlak, C.; and Hoffmann, J. 2007. Probabilistic Planning via Heuristic Forward Search and Weighted Model Counting. *Journal of Artificial Intelligence Research*, 565–620. DOI: 10.1613/jair.2289. [Cited on page 7.]
- du Merle, O.; Villeneuve, D.; Desrosiers, J.; and Hansen, P. 1999. Stabilized Column Generation. *Discrete Mathematics*, 194(1): 229–237. DOI: 10.1016/S0012-365X(98)00213-1. [Cited on pages 10 and 137.]
- Ehrgott, M. 2006. *Multicriteria Optimization*. Springer London, Limited. DOI: 10.1007/3-540-27659-9. [Cited on pages 113 and 137.]
- Elhedhli, S.; Goffin, J.-L.; and Vial, J.-P. 2001. *Nondifferentiable Optimization*, 1705–1710. Springer US. DOI: 10.1007/0-306-48332-7_337. [Cited on page 136.]
- Feinberg, E. A. 2000. Constrained Discounted Markov Decision Processes and Hamiltonian Cycles. *Mathematics of Operations Research*, 130–140. [Cited on pages 91, 119, 126, and 138.]
- Ferber, P.; Geißer, F.; Trevizan, F.; Helmert, M.; and Hoffmann, J. 2022. Neural Network Heuristic Functions for Classical Planning: Bootstrapping and Comparison to Other Methods. In *Proc. of the Int. Conf. on Automated Planning and Scheduling (ICAPS)*, 583–587. DOI: 10.1609/icaps.v32i1.19845. [Cited on page 140.]
- Fikes, R. E.; and Nilsson, N. J. 1971. STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving. *Artificial Intelligence*, 189–208. DOI: 10.1016/0004-3702(71)90010-5. [Cited on page 77.]
- Forejt, V.; Kwiatkowska, M.; and Parker, D. 2012. Pareto Curves for Probabilistic Model Checking. In *Automated Technology for Verification and Analysis*, 317–332. DOI: 10.1007/978-3-642-33386-6_25. [Cited on pages 112, 113, and 137.]
- Frigioni, D.; Marchetti-Spaccamela, A.; and Nanni, U. 2000. Fully Dynamic Algorithms for Maintaining Shortest Paths Trees. *Journal of Algorithms*, 251–281. DOI: 10.1006/jagm.1999.1048. [Cited on page 79.]
- Geffner, H.; and Bonet, B. 2013. A Concise Introduction to Models and Methods for Automated Planning. *Synthesis Lectures on Artificial Intelligence and Machine Learning*, 1–141. DOI: 10.2200/S00513ED1V01Y201306AIM022. [Cited on page 6.]
- Geißer, F.; Haslum, P.; Thiébaux, S.; and Trevizan, F. 2022. Admissible Heuristics for Multi-Objective Planning. In *Proc. of Int. Conf. on Automated Planning and Scheduling (ICAPS)*, 100–109. DOI: 10.1609/icaps.v32i1.19790. [Cited on page 107.]
- Geißer, F.; Pováda, G.; Trevizan, F.; Bondouy, M.; Teichtel-Königsbuch, F.; and Thiébaux, S. 2020. Optimal and Heuristic Approaches for Constrained Flight Planning under Weather Uncertainty. In *Proc. of Int. Conf. on Automated Planning and Scheduling (ICAPS)*, 384–393. DOI: 10.1609/icaps.v30i1.6684. [Cited on pages 1, 2, 9, 89, 91, and 92.]
- Guestrin, C.; Koller, D.; Parr, R.; and Venkataraman, S. 2003. Efficient Solution Algorithms for Factored MDPs. *Journal of Artificial Intelligence Research*, 399–468. DOI: 10.1613/jair.1000. [Cited on page 7.]
- Guillot, M.; and Stauffer, G. 2020. The Stochastic Shortest Path Problem: A polyhedral combinatorics perspective. *European Journal of Operational Research*, 148–158. DOI: 10.1016/j.ejor.2018.10.052. [Cited on page 5.]
- Handler, G. Y.; and Zang, I. 1980. A Dual Algorithm for the Constrained Shortest Path Problem. *Networks*, 293–309. DOI: 10.1002/net.3230100403. [Cited on pages 10 and 104.]
- Hansen, E.; and Abdoulahi, I. 2015. Efficient Bounds in Heuristic Search Algorithms for Stochastic Shortest Path Problems. In *Proc. of AAAI Conf. on Artificial Intelligence*, 3283–3290. DOI: 10.1609/aaai.v29i1.9643. [Cited on pages 18, 42, and 112.]
- Hansen, E.; and Abdoulahi, I. 2016. General Error Bounds in Heuristic Search Algorithms for Stochastic Shortest Path Problems. In *Proc. of AAAI Conf. on Artificial Intelligence*, 3130–3137. DOI: 10.1609/aaai.v30i1.10410. [Cited on page 34.]

- Hansen, E.; and Bowman, T. 2020. Improved Vector Pruning in Exact Algorithms for Solving POMDPs. In *Proc. of Conf. on Uncertainty in Artificial Intelligence (UAI)*, 1258–1267. [Cited on page 9.]
- Hansen, E. A. 2017. Error bounds for stochastic shortest path problems. *Mathematical Methods of Operations Research*, 1–27. DOI: 10.1007/s00186-017-0581-5. [Cited on page 112.]
- Hansen, E. A.; and Zilberstein, S. 2001. LAO*: A Heuristic Search Algorithm That Finds Solutions With Loops. *Artificial Intelligence*, 129(1): 35–62. DOI: 10.1016/S0004-3702(01)00106-0. [Cited on pages 1, 5, 17, 18, 25, 26, 27, 34, 39, 42, 46, 53, 63, 83, 94, 98, and 125.]
- Hao, M.; Trevizan, F.; Thiébaux, S.; Ferber, P.; and Hoffmann, J. 2024. Guiding GBFS through Learned Pairwise Rankings. In *Proc. of Int. Joint Conf. on Artificial Intelligence (IJCAI)*, 6724–6732. DOI: 10.24963/ijcai.2024/743. [Cited on page 140.]
- Hart, P.; Nilsson, N.; and Raphael, B. 1968. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2): 100–107. DOI: 10.1109/tssc.1968.300136. [Cited on pages 1, 17, 23, 43, and 139.]
- Haslum, P.; Lipovetzky, N.; Magazzeni, D.; and Muise, C. 2019. *An Introduction to the Planning Domain Definition Language*. Springer International Publishing. DOI: 10.1007/978-3-031-01584-7. [Cited on page 77.]
- Helmert, M.; and Domshlak, C. 2009. Landmarks, Critical Paths and Abstractions: What’s the Difference Anyway? *Proc. of Int. Conf. on Automated Planning and Scheduling (ICAPS)*, 162–169. DOI: 10.1609/icaps.v19i1.13370. [Cited on pages 7, 46, 83, 107, 113, and 130.]
- Hoey, J.; St-Aubin, R.; Hu, A.; and Boutilier, C. 1999. SPUDD: Stochastic Planning Using Decision Diagrams. In *Proc. of the Conf. on Uncertainty in Artificial Intelligence (UAI)*, 279–288. [Cited on page 8.]
- Hoffmann, J. 2015. Simulated Penetration Testing: From “Dijkstra” to “Turing Test++”. In *Proc. of Int. Conf. on Automated Planning and Scheduling (ICAPS)*, 364–372. DOI: 10.1609/icaps.v25i1.13684. [Cited on page 13.]
- Hoffmann, J.; and Nebel, B. 2001. The FF Planning System: Fast Plan Generation Through Heuristic Search. *Journal of Artificial Intelligence Research*, 14: 253–302. DOI: 10.1613/jair.855. [Cited on pages 41, 80, and 85.]
- Hong, S.; and Williams, B. C. 2023. An anytime algorithm for constrained stochastic shortest path problems with deterministic policies. *Artificial Intelligence*, 103846. DOI: 10.1016/j.artint.2022.103846. [Cited on pages 10, 92, 97, 98, 101, 106, 107, 111, 112, 119, 125, 130, and 136.]
- Howard, R. 1960. *Dynamic programming and Markov processes*. Technology Press of Massachusetts Institute of Technology. [Cited on page 5.]
- Kaelbling, L. P.; Littman, M. L.; and Cassandra, A. R. 1998. Planning and Acting in Partially Observable Stochastic Domains. *Artificial Intelligence*, 99–134. DOI: 10.1016/S0004-3702(98)00023-X. [Cited on page 6.]
- Keller, T.; and Eyerich, P. 2012. PROST: Probabilistic Planning Based on UCT. In *Proc. of Int. Conf. on Automated Planning and Scheduling (ICAPS)*, 119–127. DOI: 10.1609/icaps.v22i1.13518. [Cited on pages 5 and 40.]
- Klößner, T.; and Hoffmann, J. 2021. Pattern Databases for Stochastic Shortest Path Problems. In *Proc. of Int. Symp. on Combinatorial Search (SoCS)*, 131–135. DOI: 10.1609/socs.v12i1.18561. [Cited on pages 5, 7, and 46.]
- Klößner, T.; Hoffmann, J.; Steinmetz, M.; and Torralba, Á. 2021. Pattern Databases for Goal-Probability Maximization in Probabilistic Planning. In *Proc. of Int. Conf. on Automated Planning and Scheduling (ICAPS)*, 201–209. DOI: 10.1609/icaps.v31i1.15963. [Cited on page 46.]
- Klößner, T.; Hoffmann, J.; Steinmetz, M.; and Torralba, Á. 2021. Code and Benchmarks of ICAPS 2021 Paper “Pattern Databases for Goal-Probability Maximization in Probabilistic Planning”. DOI: 10.5281/zenodo.4604720. [Cited on pages 46, 47, 48, and 49.]

- Klößner, T.; Pommerening, F.; Keller, T.; and Röger, G. 2022. Cost Partitioning Heuristics for Stochastic Shortest Path Problems. In *Proc. of Int. Conf. on Automated Planning and Scheduling (ICAPS)*, 193–202. DOI: 10.1609/icaps.v32i1.19802. [Cited on page 7.]
- Koenig, S.; Chan, S.-H.; Li, J.; and Zheng, Y. 2023. *Artificial Intelligence and Automation*, 205–231. Springer International Publishing. DOI: 10.1007/978-3-030-96729-1_9. [Cited on page 1.]
- Koenig, S.; Likhachev, M.; and Furcy, D. 2004. Lifelong Planning A*. *Artificial Intelligence*, 93–146. DOI: 10.1016/j.artint.2003.12.001. [Cited on page 79.]
- Kohl, N.; and Karisch, S. E. 2004. Airline Crew Rostering: Problem Types, Modeling, and Optimization. *Annals of Operations Research*, 223–257. DOI: 10.1023/B:ANOR.0000019091.54417.ca. [Cited on page 10.]
- Kolobov, A.; Mausam; Weld, D.; and Geffner, H. 2011. Heuristic Search for Generalized Stochastic Shortest Path MDPs. *Proc. of Int. Conf. on Automated Planning and Scheduling (ICAPS)*, 130–137. DOI: 10.1609/icaps.v21i1.13452. [Cited on page 5.]
- Kolobov, A.; Mausam; and Weld, D. S. 2009. ReTrASE: Intergating Paradigms for Approximate Probabilistic Planning. In *Proc. of Int. Joint Conf. on Artificial Intelligence (IJCAI)*, 1746–1753. [Cited on page 85.]
- Kolobov, A.; Mausam; and Weld, D. S. 2012. A Theory of Goal-Oriented MDPs with Dead Ends. In *Proc. of Int. Conf. on Uncertainty in Artificial Intelligence (UAI)*, 438–447. [Cited on page 90.]
- Kushmerick, N.; Hanks, S.; and Weld, D. S. 1995. An Algorithm for Probabilistic Planning. *Artificial Intelligence*, 239–286. DOI: 10.1016/0004-3702(94)00087-h. [Cited on page 7.]
- Laporte, G.; Gendreau, M.; Potvin, J.-Y.; and Semet, F. 2000. Classical and Modern Heuristics for the Vehicle Routing Problem. *International Transactions in Operational Research*, 285–300. DOI: 10.1111/j.1475-3995.2000.tb00200.x. [Cited on pages 78 and 139.]
- Lee, J.; Kim, G.-H.; Poupart, P.; and Kim, K.-E. 2018. Monte-Carlo Tree Search for Constrained POMDPs. In *Advances in Neural Information Processing Systems (NIPS)*. [Cited on pages 2, 10, and 104.]
- Little, I.; and Thiebaux, S. 2007. Probabilistic Planning vs. Replanning. In *ICAPS Workshop on IPC: Past, Present and Future*, –. [Cited on pages 49, 85, and 126.]
- Luc, D. T. 2016. *Multiobjective Linear Programming: An Introduction*. Springer International Publishing. DOI: 10.1007/978-3-319-21091-9. [Cited on page 113.]
- Lübbecke, M. 2011. Column Generation. In *Wiley Encyclopedia of Operations Research and Management Science*. DOI: 10.1002/9780470400531.eorms0158. [Cited on page 75.]
- Mausam; and Kolobov, A. 2012. *Planning with Markov Decision Processes: An AI Perspective*. Morgan & Claypool Publishers. [Cited on pages 7, 17, and 18.]
- McMahan, H. B.; Likhachev, M.; and Gordon, G. J. 2005. Bounded Real-Time Dynamic Programming: RTDP with Monotone Upper Bounds and Performance Guarantees. In *Proc. of Int. Conf. on Machine Learning (ICML)*, 569–576. DOI: 10.1145/1102351.1102423. [Cited on pages 27, 42, and 63.]
- Middleton, J.; Toro Icarte, R.; and Baier, J. 2022. Real-Time Heuristic Search with LTLf Goals. In *Proc. of Int. Joint Conf. on Artificial Intelligence (IJCAI)*, 4785–4792. DOI: 10.24963/ijcai.2022/663. [Cited on page 137.]
- Muise, C.; McIlraith, S. A.; and Beck, J. C. 2024. PRP Rebooted: Advancing the State of the Art in FOND Planning. In *Proc. of AAAI Conf. on Artificial Intelligence*, 20212–20221. DOI: 10.1609/aaai.v38i18.30001. [Cited on page 6.]
- Murray, A. 2024. *Robust Planning and Scheduling using Column Generation*. Ph.D. thesis, University of Strathclyde. DOI: 10.48730/5zav-yd92. [Cited on page 9.]
- Murray, A.; Arulselvan, A.; Cashmore, M.; Roper, M.; and Frank, J. 2023a. A Column Generation Approach to Correlated Simple Temporal Networks. In *Proc. of the Int. Conf. on Automated Planning and Scheduling (ICAPS)*, 295–303. DOI: 10.1609/icaps.v33i1.27207. [Cited on page 9.]

- Murray, A.; Arulselvan, A.; Roper, M.; Cashmore, M.; Mohalik, S. K.; Burdick, I.; and David, S. 2023b. The Cost of Quality of Service: SLA Aware VNF Placement and Routing Using Column Generation. In *Int. Workshop on Resilient Networks Design and Modeling (RNDM)*, 1–8. DOI: 10.1109/rndm59149.2023.10293056. [Cited on page 9.]
- Nebel, B. 2020. On the Computational Complexity of Multi-Agent Pathfinding on Directed Graphs. In *Proc. of Int. Conf. on Automated Planning and Scheduling (ICAPS)*, 212–216. DOI: 10.1609/icaps.v30i1.6663. [Cited on page 8.]
- Papadimitriou, C.; and Steiglitz, K. 1998. *Combinatorial Optimization: Algorithms and Complexity*. Dover Publications. DOI: 10.1109/TASSP.1984.1164450. [Cited on pages 1 and 8.]
- Paulraj, S.; and Sumathi, P. 2010. A Comparative Study of Redundant Constraints Identification Methods in Linear Programming Problems. *Mathematical Problems in Engineering*. DOI: 10.1155/2010/723402. [Cited on page 137.]
- Peterson, J. R.; Kulkarni, A.; Keyder, E.; Kim, J.; and Zilberstein, S. 2022. Trajectory Constraint Heuristics for Optimal Probabilistic Planning. In *Proc. of Int. Symp. on Combinatorial Search (SoCS)*, 153–161. DOI: 10.1609/socs.v15i1.21763. [Cited on page 86.]
- Piacentini, C.; Castro, M.; Cire, A.; and Beck, J. C. 2018. Linear and Integer Programming-Based Heuristics for Cost-Optimal Numeric Planning. In *Proc. of AAAI Conf. on Artificial Intelligence*, 6254–6261. DOI: 10.1609/aaai.v32i1.12082. [Cited on page 7.]
- Pommerening, F.; Helmert, M.; Röger, G.; and Seipp, J. 2015. From Non-Negative to General Operator Cost Partitioning. In *Proc. of AAAI Conf. on Artificial Intelligence*, 3335–3341. DOI: 10.1609/aaai.v29i1.9668. [Cited on pages 2 and 7.]
- Pommerening, F.; Röger, G.; Helmert, M.; and Bonet, B. 2014. LP-Based Heuristics for Cost-Optimal Planning. In *Proc. of Int. Conf. on Automated Planning and Scheduling (ICAPS)*, 226–234. DOI: 10.1609/icaps.v24i1.13621. [Cited on pages 2 and 7.]
- Puterman, M. L. 1990. Markov Decision Processes. In *Stochastic Models*, chapter 8, 331–434. Elsevier. DOI: 10.1016/S0927-0507(05)80172-0. [Cited on page 6.]
- Puterman, M. L. 2005. *Markov Decision Processes*. Wiley-Interscience. [Cited on pages 20 and 21.]
- Péron, M.; Jansen, C. C.; Mantyka-Pringle, C.; Nicol, S.; Schellhorn, N. A.; Becker, K. H.; and Chadès, I. 2017. Selecting Simultaneous Actions of Different Durations to Optimally Manage an Ecological Network. *Methods in Ecology and Evolution*, 1332–1341. DOI: 10.1111/2041-210X.12744. [Cited on page 13.]
- Rahmaniani, R.; Crainic, T. G.; Gendreau, M.; and Rei, W. 2017. The Benders Decomposition Algorithm: A Literature Review. *European Journal of Operational Research*, 801–817. DOI: 10.1016/j.ejor.2016.12.005. [Cited on page 137.]
- Righini, G.; and Salani, M. 2006. Symmetry Helps: Bounded Bi-directional Dynamic Programming for the Elementary Shortest Path Problem with Resource Constraints. *Discrete Optimization*, 255–273. DOI: 10.1016/j.disopt.2006.05.007. [Cited on page 139.]
- Rojijers, D. M.; Vamplew, P.; Whiteson, S.; and Dazeley, R. 2013. A Survey of Multi-Objective Sequential Decision-Making. *Journal of Artificial Intelligence Research*, 67–113. DOI: 10.1613/jair.3987. [Cited on page 92.]
- Rojijers, D. M.; and Whiteson, S. 2017. *Multi-Objective Decision Making*. Springer International Publishing. DOI: 10.1007/978-3-031-01576-2. [Cited on page 137.]
- Rovner, A.; Sievers, S.; and Helmert, M. 2019. Counterexample-Guided Abstraction Refinement for Pattern Selection in Optimal Classical Planning. In *Proc. of Int. Conf. on Automated Planning and Scheduling (ICAPS)*, 362–367. DOI: 10.1609/icaps.v29i1.3499. [Cited on page 9.]
- Russell, S.; and Norvig, P. 2016. *Artificial Intelligence A Modern Approach, Global Edition*. Pearson Deutschland, third edition. [Cited on page 43.]

- Sanner, S. 2010. Relational Dynamic Influence Diagram Language (RDDL): Language Description. [Cited on pages 7 and 46.]
- Sanner, S.; and Boutilier, C. 2006. Practical Linear Value-Approximation Techniques for First-Order MDPs. In *Proc. of the Conf. on Uncertainty in Artificial Intelligence (UAI)*, 409–417. [Cited on page 8.]
- Sanner, S.; Goetschalckx, R.; Driessens, K.; and Shani, G. 2009. Bayesian Real-Time Dynamic Programming. In *Proc. of Int. Joint Conf. on Artificial Intelligence (IJCAI)*, 1784–1789. [Cited on pages 27 and 42.]
- Say, B.; and Sanner, S. 2019. Metric Hybrid Factored Planning in Nonlinear Domains with Constraint Generation. In *Proc. of Conf. on Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR)*, 502–518. DOI: 10.1007/978-3-030-19212-9_33. [Cited on page 8.]
- Schmalz, J.; and Trevizan, F. 2023. Code, Benchmarks, and Technical Report for AAAI 2024 paper “Efficient Constraint Generation for Stochastic Shortest Path Problems”. DOI: 10.5281/zenodo.10344842. [Cited on pages 46 and 50.]
- Schmalz, J.; and Trevizan, F. 2024a. Code and Benchmarks for ECAI 2024 paper “Finding Optimal Deterministic Policies for Constrained Stochastic Shortest Path Problems”. DOI: 10.5281/zenodo.12714199. [Cited on page 130.]
- Schmalz, J.; and Trevizan, F. 2024b. Efficient Constraint Generation for Stochastic Shortest Path Problems. In *Proc. of AAAI Conf. on Artificial Intelligence*, 20247–20255. DOI: 10.1609/aaai.v38i18.30005. [Cited on pages ii, 2, 25, 34, 49, 53, and 83.]
- Schmalz, J.; and Trevizan, F. 2024c. Finding Optimal Deterministic Policies for Constrained Stochastic Shortest Path Problems. In *Proc. of European Conf. on Artificial Intelligence (ECAI)*, 4148–4156. DOI: 10.3233/FAIA240986. [Cited on pages ii, 2, 114, and 119.]
- Schmalz, J.; and Trevizan, F. 2025. Solving Constrained Stochastic Shortest Path Problems with Scalarisation. In *Proc. of European Conf. on Artificial Intelligence (ECAI)*. DOI: 10.3233/faia251372. [Cited on pages ii, 2, and 103.]
- Schmalz, J.; and Trevizan, F. 2026. Efficient Constraint Generation for Stochastic Shortest Path Problems. *Artificial Intelligence*, 354: 104505. DOI: 10.1016/j.artint.2026.104505. [Cited on pages ii and 25.]
- Schuurmans, D.; and Patrascu, R. 2001. Direct Value-Approximation for Factored MDPs. In *Advances in Neural Information Processing Systems (NIPS)*, 1579–1586. [Cited on page 9.]
- Seipp, J.; and Helmert, M. 2013. Counterexample-Guided Cartesian Abstraction Refinement. In *Proc. of Int. Conf. on Automated Planning and Scheduling (ICAPS)*, 347–351. DOI: 10.1609/icaps.v23i1.13605. [Cited on page 9.]
- Seipp, J.; von Allmen, S.; and Helmert, M. 2020. Incremental Search for Counterexample-Guided Cartesian Abstraction Refinement. In *Proc. of Int. Conf. on Automated Planning and Scheduling (ICAPS)*, 244–248. DOI: 10.1609/icaps.v30i1.6667. [Cited on page 79.]
- Sharon, G.; Stern, R.; Felner, A.; and Sturtevant, N. 2012. Conflict-Based Search For Optimal Multi-Agent Path Finding. In *Proc. of AAAI Conf. on Artificial Intelligence*, 563–569. [Cited on page 8.]
- Shi, H.-J. M.; Tu, S.; Xu, Y.; and Yin, W. 2017. A Primer on Coordinate Descent Algorithms. DOI: 10.48550/arXiv.1610.00040, arXiv:1610.00040. [Cited on page 98.]
- Shor, N. Z. 1985. *Minimization Methods for Non-Differentiable Functions*. Springer Berlin Heidelberg. DOI: 10.1007/978-3-642-82118-9. [Cited on pages 98, 107, and 136.]
- Smith, T.; and Simmons, R. G. 2006. Focused Real-Time Dynamic Programming for MDPs: Squeezing More Out of a Heuristic. In *Proc. of AAAI Conf. on Artificial Intelligence*, 1227–1232. [Cited on pages 27 and 42.]
- Steinmetz, M.; Hoffmann, J.; and Buffet, O. 2016. Goal Probability Analysis in Probabilistic Planning: Exploring and Enhancing the State of the Art. *Journal of Artificial Intelligence Research*, 57: 229–271. DOI: 10.1613/jair.5153. [Cited on page 47.]

- Steinmetz, M.; Thiébaux, S.; Höller, D.; and Teichteil-Königsbuch, F. 2024. Explaining the Space of SSP Policies via Policy-Property Dependencies: Complexity, Algorithms, and Relation to Multi-Objective Planning. *Proc. of Int. Conf. on Automated Planning and Scheduling (ICAPS)*, 555–564. DOI: 10.1609/icaps.v34i1.31517. [Cited on pages 9, 125, and 140.]
- Stern, R.; Sturtevant, N.; Felner, A.; Koenig, S.; Ma, H.; Walker, T.; Li, J.; Atzmon, D.; Cohen, L.; Kumar, T. K.; Barták, R.; and Boyarski, E. 2021. Multi-Agent Pathfinding: Definitions, Variants, and Benchmarks. In *Proc. of Int. Symp. on Combinatorial Search (SoCS)*, 151–158. DOI: 10.1609/socs.v10i1.18510. [Cited on page 8.]
- Teichteil-Königsbuch, F. 2012. Stochastic Safest and Shortest Path Problems. In *Proc. of AAAI Conf. on Artificial Intelligence*, 1825–1831. DOI: 10.1609/aaai.v26i1.8367. [Cited on pages 5 and 15.]
- Teichteil-Königsbuch, F.; Kuter, U.; and Infantes, G. 2010. Incremental Plan Aggregation for Generating Policies in MDPs. In *Proc. of Int. Conf. on Autonomous Agents and Multiagent Systems (AAMAS)*, 1231–1238. [Cited on pages 6, 10, 40, 41, 85, 86, and 126.]
- Toyer, S.; Thiébaux, S.; Trevizan, F.; and Xie, L. 2020. ASNets: Deep Learning for Generalised Planning. *Journal of Artificial Intelligence Research*, 1–68. DOI: 10.1613/jair.1.11633. [Cited on page 140.]
- Trevizan, F.; Teichteil-Königsbuch, F.; and Thiébaux, S. 2017. Efficient Solutions for Stochastic Shortest Path Problems with Dead Ends. In *Proc. of Int. Conf. on Uncertainty in Artificial Intelligence (UAI)*. [Cited on pages 5, 14, 40, 46, and 83.]
- Trevizan, F.; Thiébaux, S.; Santana, P.; and Williams, B. 2016. Heuristic Search in Dual Space for Constrained Stochastic Shortest Path Problems. In *Proc. of Int. Conf. on Automated Planning and Scheduling (ICAPS)*, 326–334. DOI: 10.1609/icaps.v26i1.13768. [Cited on pages 2, 5, 7, 9, 20, 89, 90, 91, 93, 94, 103, 113, 114, 125, 135, and 136.]
- Trevizan, F.; Thiébaux, S.; and Haslum, P. 2017. Occupation Measure Heuristics for Probabilistic Planning. In *Proc. of Int. Conf. on Automated Planning and Scheduling (ICAPS)*, 306–315. DOI: 10.1609/icaps.v27i1.13840. [Cited on pages 2, 7, 9, 46, 48, 69, 83, 94, 95, 98, 103, 107, 113, 114, 125, and 130.]
- Trevizan, F.; and Veloso, M. 2012. Short-Sighted Stochastic Shortest Path Problems. In *Proc. of Int. Conf. on Automated Planning and Scheduling (ICAPS)*. [Cited on page 10.]
- Trevizan, F.; and Veloso, M. 2014. Depth-based Short-sighted Stochastic Shortest Path Problems. *Artificial Intelligence*, 179–205. DOI: 10.1016/j.artint.2014.07.001. [Cited on page 10.]
- Vinzent, M.; Sharma, S.; and Hoffmann, J. 2023. Neural Policy Safety Verification via Predicate Abstraction: CEGAR. *Proc. of AAAI Conf. on Artificial Intelligence*, 15188–15196. DOI: 10.1609/aaai.v37i12.26772. [Cited on page 9.]
- Walraven, E.; and Spaan, M. 2017. Accelerated Vector Pruning for Optimal POMDP Solvers. In *Proc. of AAAI Conf. on Artificial Intelligence*, 3672–3678. DOI: 10.1609/aaai.v31i1.11032. [Cited on page 9.]
- Walraven, E.; and Spaan, M. T. J. 2018. Column Generation Algorithms for Constrained POMDPs. *Journal of Artificial Intelligence Research*, 489–533. DOI: 10.1613/jair.1.11216. [Cited on page 9.]
- Wang, R. X.; and Thiébaux, S. 2024. Learning Generalised Policies for Numeric Planning. In *Proc. of the Int. Conf. on Automated Planning and Scheduling (ICAPS)*, 633–642. DOI: 10.1609/icaps.v34i1.31526. [Cited on page 140.]
- Warnquist, H.; Kvarnström, J.; and Doherty, P. 2010. Iterative Bounding LAO*. In *Proc. of European Conf. on Artificial Intelligence (ECAI)*, 341–346. [Cited on pages 27 and 42.]
- White, D. J. 1985. Real Applications of Markov Decision Processes. *Interfaces*, 15(6): 73–83. [Cited on page 13.]
- Ye, X.; Han, S.-P.; and Lin, A. 2010. A Note on the Connection Between the Primal-Dual and the A* Algorithm. *Int. Journal of Operations Research and Information Systems*, 73–85. DOI: 10.4018/joris.2010101305. [Cited on pages 1 and 8.]

- Ye, Y. 2011. The Simplex and Policy-Iteration Methods Are Strongly Polynomial for the Markov Decision Problem with a Fixed Discount Rate. *Mathematics of Operations Research*, 593–603. [Cited on page 7.]
- Yoon, S.; Fern, A.; Givan, R.; and Kambhampati, S. 2008. Probabilistic planning via determinization in hindsight. In *Proc. of National Conf. on Artificial Intelligence*, 1010–1016. [Cited on pages 10 and 85.]
- Yoon, S. W.; Fern, A.; and Givan, R. 2007. FF-Replan: A Baseline for Probabilistic Planning. In *Proc. of Int. Conf. on Automated Planning and Scheduling (ICAPS)*, 352–359. [Cited on pages 6, 10, 23, 24, 41, 85, 86, and 126.]
- Yoshizumi, T.; Miura, T.; and Ishida, T. 2000. A* with Partial Expansion for Large Branching Factor Problems. In *Proc. of AAAI Conf. on Artificial Intelligence*, 923–929. [Cited on pages 43 and 44.]
- Younes, H.; Littman, M.; Weissman, D.; and Asmuth, J. 2005. The First Probabilistic Track of the International Planning Competition. *Journal of Artificial Intelligence Research*, 851–887. DOI: 10.1613/jair.1880. [Cited on pages 1, 7, 17, 46, 74, and 85.]
- Yu, J.; and LaValle, S. 2013. Structure and Intractability of Optimal Multi-Robot Path Planning on Graphs. In *Proc. of AAAI Conf. on Artificial Intelligence*, 1443–1449. DOI: 10.1609/aaai.v27i1.8541. [Cited on page 8.]